

TECHNICAL DEEP DIVE

How AArchX works

An x86-64 to arm64 binary translator for macOS, from the instruction decoder to Steam, Wine and native frameworks.

AArchX 0.3-dev (formerly Ocerz) · source at commit **9169680**, 2026-10-04

Written from the source tree, its explanation blocks, its documentation and its git history.

About this document

This is a technical deep dive into AArchX, a userspace binary translator that runs x86-64 macOS programs on Apple silicon. It is written for systems engineers who know C, can read some x86 and some arm64, and want to understand how this particular translator works: not only what each part does, but why it has the shape it has, which failures shaped it, and what it costs.

It follows a program through the translator from the command line to its exit, then takes each layer in turn: the decoder, the reference interpreter, the arm64 JIT, guest memory, the system-call layer, the Mach-O loader, the dynamic linker that binds against Apple's x86-64 shared cache, and native mode, which binds the same programs against the Mac's own arm64 frameworks instead. Later chapters cover the real software that runs (Cocoa applications, the macOS Steam client, games, Windows programs through Wine), how correctness is held, how speed is measured, the debugging tools, and what is still missing.

How it was written

Everything here describes the source tree at commit `9169680` of 4 October 2026. The material comes from four places, in this order of authority: the source itself, where every file opens with a prose block that explains what it owns and why; the commit history, whose messages state the rule each change establishes; the project's own documentation under `docs/`; and a set of engineering notes that were checked against the code. Numbers were taken from the tree on that date (line counts, test counts, knob counts) or from dated measurements recorded in the history, and each says when it was measured. Where something could not be verified, the text says so or leaves it out.

The document describes Apple's software only from the outside: public interfaces, published file formats, error messages and behaviour that can be observed by running programs. AArchX's own source is quoted freely; it is free software under the GNU Lesser General Public License, version 2.1 or later.

Names and conventions

AArchX

The product. It was called Ocerz until 4 September 2026, and the old name survives in the binary, the source paths and every environment variable.

`ocerz`

The executable you run, and the prefix of its environment variables, `OCERZ_*`, and its C symbols, `ocerz_*`.

guest

The x86-64 (or, inside Wine, i386) program being translated, and everything it sees: guest registers, guest memory, guest threads, guest addresses.

host

The arm64 process AArchX actually is, and the macOS kernel underneath it. Host code is arm64; host addresses are what the processor really uses.

cache mode, native mode

The two ways a guest's system libraries are supplied: Apple's x86-64 shared cache, translated like any other code, or the Mac's own arm64 frameworks reached through generated bridges.

Code, file names, functions and environment variables are set in `monospace`. Ratios against Rosetta are AArchX's time divided by Rosetta's, so lower is better and anything under 1.00x is faster. Sizes use KB, MB and GB in the binary sense. Callouts mark a *Key idea*, a *Note*, a *Warning*, or *How it was found*: the story of a bug whose fix explains a rule.

How to read it

- **If you want to run software:** Chapter 1, Chapter 13 and Chapter 14 for what runs, Chapter 17 for diagnosing a failure, and Appendices A and B for the switches and messages.
- **If you want to contribute:** Chapters 2 and 3 first, then the chapters for the layer you will touch, then Chapter 15 on testing; every change is expected to carry the test that pins it.
- **If you want to understand binary translation:** read Chapters 2, 4, 5, 6 and 7 in order, then Chapter 12 for the less common idea of running translated code against native libraries.

Contents

PART I • ORIENTATION

1	What AArchX is	11
	A translator written from scratch.....	11
	Why it exists	12
	What it is not.....	12
	Two ways to supply the system libraries.....	12
	What runs today	14
	How fast	14
	History.....	15
	Where to go from here	16
2	The life of a program	17
	1. The command line.....	18
	2. Reserving guest memory.....	19
	3. Mapping the shared cache.....	19
	4. Mapping, binding and protecting the program.....	20
	5. The initial frame, and the pieces the guest expects	20
	6. Initializers, in dyld's order	21
	7. The first block.....	22
	8. A system call.....	23
	9. A second thread.....	24
	10. A signal.....	24
	11. Exit.....	25
	The same program in native mode	25
	A static program	26
	Exit codes along the way.....	26
3	The source tree and its conventions	28
	The shape of the repository	28
	Every source file	28
	The headers and how they depend on each other	31
	Three structures hold the state	32
	Building.....	33
	The conventions	34
	The switches.....	35
	Documentation in the tree.....	35

PART II • THE CPU

4	The decoder	37
	One function, one format	37
	Anatomy of an x86-64 instruction	38
	The decode pipeline	38
	Prefixes.....	39
	ModRM, SIB and addressing	40
	The X86Insn record	41
	Operand and address widths	42
	548 internal operations	42

	The opcode maps.....	43
	VEX.....	44
	i386 mode	44
	What it refuses, and why that is right.....	45
	Proving a decoder correct.....	46
	Bugs that shaped it	47
5	The interpreter and x86 flags.....	48
	Three jobs for one engine	48
	The CPU state	49
	Executing one instruction	51
	x86 flags, computed eagerly	52
	The deferred record	54
	Flag liveness	55
	Integer corners	56
	The x87 unit	57
	SSE and AVX.....	58
	i386 execution.....	60
	What the JIT leaves to the interpreter	60
6	The JIT: translation, registers and control flow	62
	Where the design came from.....	62
	The code arena	63
	Finding a block.....	64
	The run loop and the dispatcher.....	64
	Building a block.....	65
	Register pinning	65
	Anatomy of a block.....	67
	Addressing guest memory	68
	Flags without computing flags.....	69
	Exits and chains.....	71
	Calls and returns	72
	Superblocks and trace inversion	74
	Lane caches and the edges of a loop.....	75
	The emitter.....	75
	When a block faults	76
	Lessons, in one table.....	76
7	The JIT: floating point, SIMD, memory ordering and changing code	78
	Exact floating point	78
	SIMD coverage	81
	Memory ordering.....	83
	Addressing in Wine's memory layout.....	86
	Code that changes	87
	When the arena fills	89
	In-place string and memory routines.....	91
	Keeping translations: the on-disk cache.....	93
	Switches for this chapter	95
 PART III • MEMORY AND THE OPERATING SYSTEM		
8	Guest memory	97
	The constraints	97

Three address maps	97
Translating an address	100
4 KB guest pages on 16 KB host pages	101
Files and shared memory	103
Armed pages and kernel writes	103
What the guest is told about its memory	104
Host memory inside the guest's world	104
Native mode	106
Footprint	107
Switches	107
9 System calls, Mach, threads and signals	108
How a guest enters the kernel	108
The return convention is Rosetta's	109
Pointers the kernel reads and writes	110
Mach messages	111
Threads	112
Suspending threads	114
Signals	114
Faults	117
The unstick monitor	117
Processes	118
Diagnostics	118
 PART IV • LINKING	
10 The Mach-O loader	121
Two kinds of program, one front door	121
What main() settles first	122
Choosing the slice	123
The static path	124
The initial stack	124
The dynamic frame	125
Where a dynamic image lands	126
From here to the mini-dyld	127
11 The mini-dyld and the shared cache	128
Why a linker of its own	128
From exec to main	128
Finding and mapping the cache	129
Resolving symbols	131
Fixups	133
Initializers	134
dlopen, dlsym and the image list	136
The dyld API table	137
Objective-C in cache mode	138
Thread-local variables	139
Smaller things that had large effects	139
12 Native mode	141
The idea	141
Synthesized libraries	142
The API database	144

The ABI engine	145
Calls from translated code	147
Calls back into guest code	147
Objective-C	149
Blocks	151
CoreFoundation and callback structures	151
Everything else a process does	152
When a crossing fails	153
What it costs	153
Steam in native mode	154
What native mode cannot run yet	154

PART V • REAL SOFTWARE

13 macOS applications, Steam, games and the GPU	157
How an application is run	157
Command-line tools	158
Applications in cache mode	159
The GPU	161
The macOS Steam client	162
Games	164
What is still open	166
14 Wine and WoW64	168
Why Wine is the hardest guest	168
The segment-register contract	169
WoW64 and 32-bit code	170
Memory: the low shadow	171
The walls, in order	174
Games through Wine	177
What is still open	179

PART VI • QUALITY, SPEED AND TOOLING

15 Testing	181
The shape of the problem	181
Unit harnesses	182
The guest suite	183
The x86-64 differential gate	184
The i386 differential gate	185
The dynamic suite	186
The native suite and its companions	186
Real software, outside the gates	187
Where expected outputs come from	187
Conventions	188
Current results	189
Adding a test	189
16 Performance	190
How it is measured	190
Results	191
What makes it fast	193
What is still slower	194

	SIMD: AVX2, FMA and MMX	194
	Exactness and its price	195
	Memory ordering	195
	Native-mode crossings	196
	Wine's layout	196
	Startup: where real applications lose	197
	Games	198
	Native mode on a real application	199
	Reading these numbers honestly	199
17	The debugging toolkit.....	200
	The method	200
	Bisecting a JIT bug	201
	Reading a crash report	202
	A hung process	203
	Where the time goes	204
	Loader and library diagnostics	205
	System calls, Mach and IOKit	205
	Case studies	205
18	Limitations and roadmap	208
	Programs that stop	208
	The GPU and Steam	208
	Slower than Rosetta	209
	Semantic deviations	209
	Loader gaps in cache mode	210
	Translator internals still open	210
	Native mode	211
	The direction: after Rosetta	211
	Open fronts	212
	The June plan, scored	213
APPENDICES		
A	Environment variables	214
	Command line	214
	Choosing what runs	214
	Process trees	215
	Keeping translations	215
	Memory ordering	215
	Turning off what makes it fast	216
	Behaviour	216
	Switches that restore old behaviour	217
	The diagnostic knobs, counted	218
B	Exit codes and messages	219
	Exit codes	219
	Starting up	220
	Loading and binding	220
	Running	220
	Native mode	221
C	Glossary	222

D	Timeline	228
	June: from static binaries to Wine	228
	July: a JIT that keeps up.....	228
	August: the pinned JIT and 32-bit code.....	229
	September: Steam, AVX, native mode and games	229
	October: games under Wine	230

PART I

Orientation

What AArchX is, how a program travels through it from exec to exit, and how the source tree is laid out.

CHAPTER 1

What AArchX is

AArchX runs Intel macOS programs on Apple silicon by translating their x86-64 machine code to arm64 as they run, the job Rosetta 2 does. This chapter says what it is and is not, the two ways it supplies a program's system libraries, what runs on it today, and how it got here in four months.

76,178LINES OF C, ASSEMBLY
AND HEADERS**698**COMMITTS, 3 JUN TO 4
OCT 2026**14 / 15**BENCHMARK KERNELS
FASTER THAN
ROSETTA**20,033**1386 CASES CHECKED
PER RUN

A translator written from scratch

An x86-64 Mach-O executable cannot run on an Apple silicon processor: the instructions mean nothing to it. Rosetta 2, Apple's answer, translates them ahead of time and at run time, and does so invisibly enough that most people never notice it is there. AArchX does the same job with none of Rosetta's code. It is one arm64 program, `ocerz`, that is given an x86-64 program on its command line and runs it inside its own process:

```
$ ./ocerz /usr/bin/sw_vers
ProductName:      macOS
ProductVersion:   26.7.1
BuildVersion:     25G241
```

Everything between that command and the output is AArchX's own: it parses the Mach-O file, reserves and lays out the guest's memory, loads and binds the program's libraries, decodes the x86 instructions, executes them in an interpreter or translates them to arm64 in a JIT, and answers every system call, Mach message, thread creation and signal the program makes. The source is about 76,000 lines of C11 and a little arm64 assembly, in 30 source files and 28 headers, built with nothing but the Xcode Command Line Tools. It is free software under the GNU Lesser General Public License, version 2.1 or later.

The project was called Ocerz until 4 September 2026, when it was renamed AArchX. The rename touched the documentation and the version string; the binary is still `ocerz`, the source lives under `include/ocerz/`, and the several hundred environment variables that control it are all `OCERZ_*`. Running `./ocerz version` prints `AArchX 0.3-dev`.

Why it exists

Rosetta 2 is closed, managed by the operating system, and on a deprecation track: the project's documentation is built around Apple ending general-purpose Rosetta after macOS 27. When it goes, so does the x86-64 copy of the system libraries that every Intel program on a Mac links against, because that copy is shipped by the Rosetta package. Intel-only software, from old command-line tools to games and their launchers, then has nowhere to run.

AArchX started for a narrower reason. Its first purpose, in June 2026, was to run Wine, and through Wine the Windows Steam client and its games, under a translator whose every decision could be read, instrumented and changed. Rosetta's behaviour can only be observed from outside; when a Wine process died inside it, there was nothing to bisect. A translator written in the open turns "Rosetta does something odd here" into "this function does this, and here is the commit that changed it". By September the project had become a general runtime for Intel macOS software, and native mode, described below, became its plan for the time after Rosetta.

KEY IDEA

AArchX is built around inspectability. Its interpreter is a complete reference implementation that every translation is checked against byte for byte; nearly every subsystem has switches that turn one optimisation off or print one kind of event; and every source file opens with a prose explanation of what it owns and why. The rest of this book leans on all three.

What it is not

- **Not a virtual machine.** AArchX emulates a processor and a user-space process, not a computer. There is one kernel, the Mac's own, and the guest's system calls are answered by AArchX on top of it, translating arguments and results where the two architectures differ.
- **Not Rosetta.** It never invokes Rosetta's translator and contains none of its code. In cache mode it does read the x86-64 shared cache file that the Rosetta package installs, because that file is where the Intel system libraries live; it maps and relocates that file itself.
- **Not a Windows layer.** Windows programs run through Wine, unmodified, and AArchX runs Wine the way it runs any other Intel program. What AArchX adds is the processor: x86-64, and i386 for 32-bit Windows code inside Wine's WoW64 process.
- **Not an ahead-of-time translator.** Code is translated when it is first run. Since 27 September 2026 those translations are kept on disk and reused by later processes, but nothing is translated before a program asks for it.
- **Not finished.** The project is experimental, version 0.3-dev, with no stability guarantees. What runs is listed below and in Chapter 13; what does not is in Chapter 18.

Two ways to supply the system libraries

Translating a program's own instructions is only half the problem. A real program calls `printf`, `malloc`, `CFRelease` and `-[NSString length]` thousands of times a second, and those functions live in system libraries. AArchX can supply them in two ways, chosen once per process by a flag.

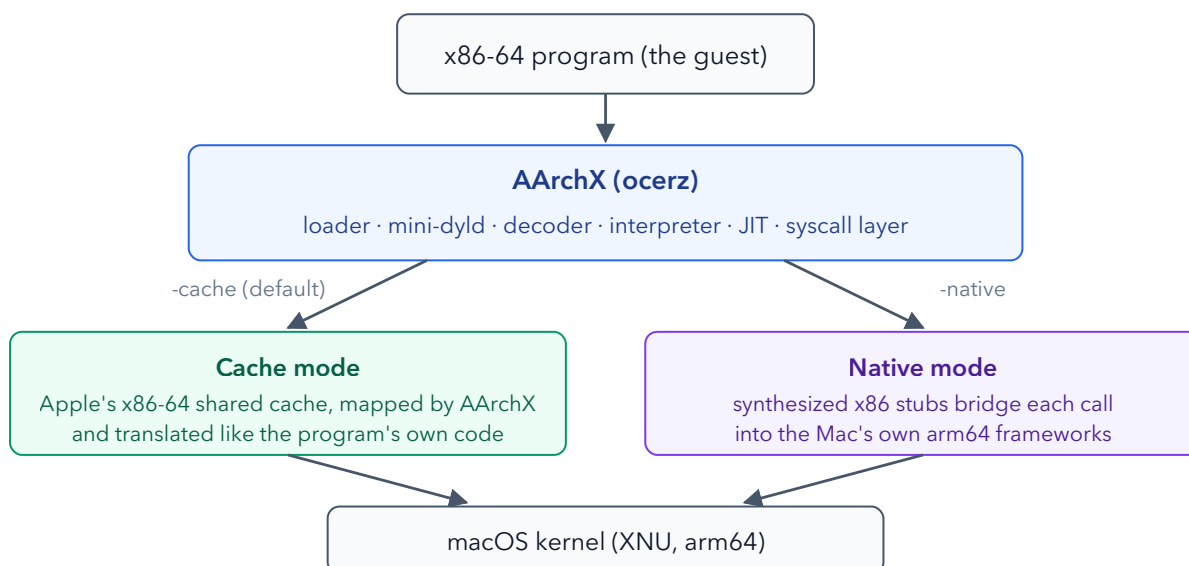


Figure 1.1. One translator, two sources of system libraries. Both modes translate the program's own code identically; they differ in what happens when it calls into the system.

Cache mode is the default. On macOS, the system libraries do not exist as separate files; they are pre-linked into one large "shared cache" per architecture, and the x86-64 one is installed with Rosetta. AArchX maps that file into the guest's address space, applies its relocations, registers its Objective-C classes and runs its initializers, and from then on the guest's `printf` is Apple's x86-64 `printf`, translated exactly like the program's own code. This is the mode that runs the most software, because the libraries are the real ones and behave exactly as they do under Rosetta. Chapter 11 describes it.

Native mode (`-native`) maps no x86 libraries at all. For each system library the guest links against, AArchX builds an x86-64 Mach-O image in memory whose exports are tiny stubs; calling one crosses into the host's real arm64 function, moving arguments between the two calling conventions on the way. A table generated from the macOS SDK says what each export's signature is. In this mode, `strlen` or `CFDictionaryGetValue` runs as native arm64 code at full speed, and no x86 system libraries need to exist on the machine, which is what makes native mode the answer to Rosetta's end. It runs a narrower set of programs so far. Chapter 12 describes it.

	Cache mode	Native mode
Flag	<code>-cache</code> , the default	<code>-native</code> , or <code>OCERZ_MODE=native</code>
System libraries	Apple's x86-64 shared cache, translated	the Mac's own arm64 frameworks, run as they are
Needs Rosetta installed	yes, for the cache file only	no
Library coverage	everything in the cache	59 libraries with a generated API database (September 2026)
Cost of a library call	translated x86 code	a crossing of about 17 ns, or 2 ns for eleven string and memory routines
Runs	the most software	command-line programs, some Cocoa apps, the Steam client

The mode is fixed before the guest starts, holds for the whole process, and is inherited by every program the guest launches through `OCERZ_MODE`.

What runs today

"Runs" here means the program did its job: a command-line tool printed what it prints natively, an application drew its main window and stayed up, a game reached its menus. Everything below was seen working on the date given; Chapter 13 and Chapter 14 tell each story in detail.

Software	Result	When
Command-line tools	16 of 16 checked tools match native output byte for byte, on macOS 26.7.1 and 27; a sweep of 74 system tools agrees with Rosetta	2026-10-03
Bundled Cocoa apps	Chess (with its engine), Calculator, Dictionary, Font Book, Grapher, Digital Color Meter, Activity Monitor, Console, TextEdit, Preview, Script Editor open their windows	2026-09-11, macOS 26.6
Safari	runs and browses	2026-09-21
macOS Steam client	full UI in cache mode with software rendering; in native mode on macOS 27 with its GPU process on hardware graphics	2026-09-12, 2026-09-21
Discord	the Electron client reaches its signed-in view	2026-09-22
Brawlhalla	an Adobe AIR game, launched from Steam, reaches its menus	2026-09-22
Ollama	command line, HTTP server and menu-bar app	2026-09-13
Wine	boots in about 14 s; 32-bit Notepad and WineMine through WoW64; Windows Steam reaches its main window	September 2026
R.E.P.O.	a Unity game under Wine with DXMT renders: roughly 45-65 frames a second on its main menu, varying from run to run, and about 50 in a level, where Rosetta manages about 178 on the menu	2026-10-03, 2026-10-04
Counter-Strike 2	under Wine with D3DMetal, reaches its own window, then stops on an error Rosetta does not hit	2026-09-26

On macOS 27 the bundled applications in the second row ship as arm64 only, with no Intel code in them, so there is nothing for a translator to run; AArchX says so and exits, and a guest that launches one gets the native program, as it would under Rosetta. Safari, Steam, Discord and the games still carry Intel code on macOS 27.

How fast

On the project's fifteen-kernel benchmark, measured on an Apple M5 under macOS 27 on 21 September 2026, the build linked against libSystem runs faster than Rosetta on fourteen of the fifteen kernels in cache mode, from 0.72x of Rosetta's time on small `memcpy` calls to 0.99x on integer division; the one loss is a loop of tiny function calls at 1.03x. A freestanding build that makes no library calls wins eleven and loses four, none by more than five percent. AVX2 and FMA loops mostly beat Rosetta too. Chapter 16 has every number and the method.

The weak spot is start-up. Everything a large program runs has to be translated the first time it runs, and a Windows Steam session under Wine is a dozen processes running the same system and Wine code. Its main window took about 33 seconds under AArchX on 27 September, against 11 to 12 under Rosetta; the on-disk translation cache added that week is the main lever on that number.

History

The first commit is dated 3 June 2026. By 4 October the history held 698 commits, 101 of them the routine follow-up that refreshes the committed unit-test binaries. Commits per month: 35 in June, 82 in July, 181 in August, 368 in September, and 32 in the first four days of October.

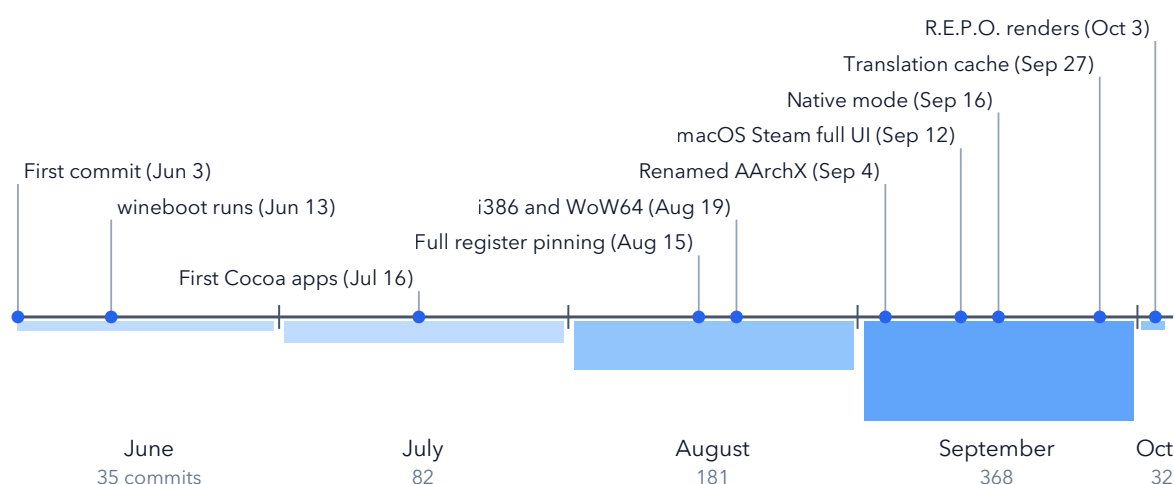


Figure 1.2. Selected milestones from 3 June to 4 October 2026. The bars under the axis show commits per month; September alone has more than half of the history.

June: from static binaries to Wine

The first version ran statically linked x86-64 programs: a Mach-O loader, a decoder, an interpreter and a small call-threaded JIT that inlined only moves and called back into the interpreter for everything else. Within days it was running dynamically linked programs against the shared cache with its own dynamic linker: real Objective-C programs on 5 June, guest threads and the C++ standard library on 6 June, `fork`, `posix_spawn`, shells and the core utilities on 7 June, thread-local variables on 8 June, and delivered guest signals on 11 June. On 13 June Wine's `wineboot` ran end to end into the GUI layer, and the next day Notepad built its GUI.

July and August: speed, threads and 32-bit code

July made the memory model honest (x86's ordering guarantees on 16 July) and ran the first real macOS applications. On 17 July the JIT gained register allocation and deferred flags. August rebuilt it: on 15 August every guest general-purpose register and every `xmm` register became permanently pinned in a host register, the next day blocks could continue past conditional branches as superblocks, and on 17 August ordered memory became cheap enough to use by default once a program has a second thread. On 19 and 20 August the decoder, interpreter and JIT learned i386, with a 32-bit differential gate, so that 32-bit Windows code inside Wine's WoW64 process could run.

September: Steam, native mode and the world outside

September is more than half the history. The project was renamed AArchX on 4 September, and its license moved from LGPL-2.1 to a proprietary non-commercial license the next day. On 8 September the Windows Steam client's login window appeared under Wine, rendered with software compositing. On 12 September the macOS Steam client came up with its full UI, and the source tree adopted its rule that each file explains itself in one prose block at the top, with no comments in the code bodies. AVX2 and FMA reached the JIT on 13 September. Native mode began on 16 September and ran the Steam client five days later, the same day the in-place string routines and hardware NaN handling put the dynamic benchmark ahead of Rosetta on fourteen kernels. On 22 September a bug in the loaded-image list was found to have hidden the GPU from every translated program, and the same day Brawlhalla and Discord ran. On 23 September the license returned to LGPL-2.1-or-later and the documentation moved into the tree, and games launched from Steam began running under AArchX. Counter-Strike 2 reached D3DMetal under Wine on 26 September, and on 27 September translations started being kept on disk and shared between processes.

October

Some time between 19 September and 2 October the project's public history was rewritten (the rewrite kept every original commit date, so it cannot be dated more closely): the old and new histories share no commit, every hash changed, and the generated API databases were taken out of all of it. On 1 October Steam ran on stock Wine Staging 11.16 with the DXMT graphics layer. On 3 October cache mode learned to find the shared cache where macOS 26 keeps it again, an initializer-order bug that crashed `sw_vers` on macOS 26.7.1 was fixed by matching the order Rosetta's dyld uses, and R.E.P.O., a Unity game, rendered under Wine. On 4 October MMX instructions joined the JIT, a full code arena started being flushed instead of overflowing, and a cheaper address translation for Wine; together they took R.E.P.O.'s menu from 28-30 frames a second to roughly 45-65.

NOTE

The project's two development machines worked on the tree in parallel through September and October, one on macOS 26 and an M5 on macOS 27. That is why several results in this book name the macOS version they were measured on, and why both versions are supported.

Where to go from here

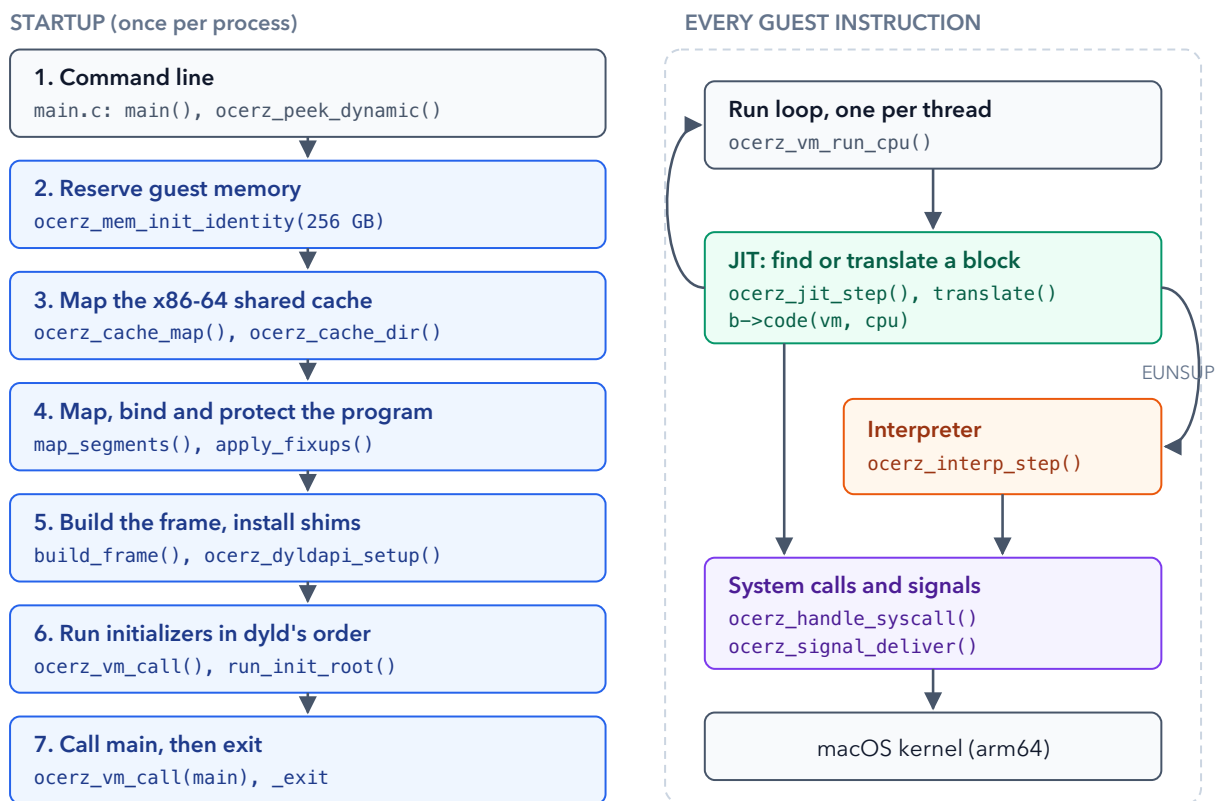
Chapter 2 follows one program from the command line to its exit and names the functions it passes through; it is the map for everything after it. Chapter 3 lays out the source tree and the conventions it is written in. From there, Parts II to IV take the layers one at a time, from the instruction decoder up to native mode.

CHAPTER 2

The life of a program

This chapter follows one program, `./ocerz /usr/bin/sw_vers` in cache mode, from the command line to its exit, and names the function that does each step. It is the map the rest of the book fills in: every later chapter is a closer look at one stop on this route. The same program in native mode, and a statically linked one, close the chapter.

`sw_vers` is a good subject because it is small and ordinary. It is a universal binary with an x86-64 slice, it links only CoreFoundation and libSystem, it reads one property list and prints three lines. Yet getting it to start exercises the whole translator: a 256 GB memory reservation, the x86-64 shared cache with several thousand libraries in it, the initializers of a few hundred of them in the right order, a few hundred thousand translated instructions, dozens of system calls, and an exit.



Steps 6 and 7 do not leave the run loop: `ocerz_vm_call()` runs guest code through it and returns when the guest function does. Threads get a run loop of their own; signals are delivered at system-call edges; faults land in the handler that `ocerz_vm_install_handlers()` installed.

Figure 2.1. The route of a dynamically linked program in cache mode. Startup runs once; the right-hand loop runs for every block of guest code, on every guest thread.

1. The command line

Everything starts in `main()` in `src/main.c`, a 419-line file whose explanation block is about the three decisions it makes before anything else runs. Before parsing a single argument, though, it deals with two pieces of the environment that would otherwise do damage.

Standard error first. If `OCERZ_STDERR_FILE` names a file, standard error is reopened onto it in append mode. Chromium and others start their children with standard error closed or pointing nowhere, and a child that dies early under AArchX would otherwise leave no trace.

Then `DYLD_INSERT_LIBRARIES`. That variable is meant for the guest: Steam sets it to its loader and overlay for every game it launches. But the host's own dyld reads it first, and would load the arm64 slices of those libraries into `ocerz` itself, which broke startup within half a second. So `reexec_without_host_insertion()` moves the value to `OCERZ_GUEST_DYLD_INSERT_LIBRARIES`, removes the original, and re-executes `ocerz` with `execv`. Because it is an exec rather than a fork, the process ID Steam is tracking survives. In the clean copy, `restore_guest_insertion()` puts the variable back before the guest's environment is captured, so the guest sees it exactly as it was set and the mini-dyld later inserts the libraries into the guest alone.

Argument parsing is plain: `-v` (repeatable), `-trace`, `-strace`, `-no-jit`, `-native`, `-cache`, `-path file` and `--`, with the first non-option argument being the program. Any other option prints the usage and exits with status 64. `./ocerz version` prints `AArchX 0.3-dev` and stops. `main()` also sets `MallocNanoZone=0` for the process before the guest's environment is taken.

The **mode** comes from the last of `-native` or `-cache` on the command line; with neither, `OCERZ_MODE` decides, and a value other than `native` or `cache` is refused with exit 64 rather than ignored. That variable is how a child inherits its parent's mode across a spawn or an exec. `ocerz_afp_enable()` then checks whether the processor reports `FEAT_AFP`, the arm64 feature that lets the hardware produce x86's NaN results (Chapter 7).

Three more adjustments can change what will run. If the program is an application bundle, `bundle_executable()` reads `CFBundleExecutable` from `Contents/Info.plist` (or falls back to the bundle's own name) and runs `Contents/MacOS/<name>` instead, with that path as the guest's `argv[0]`; that is how Steam's `ocerz %command%` launch option works for games. If the program's real path ends in `wine` or `wine64`, `apply_wine_defaults()` marks the process as one of Wine's, which fixes the base of its memory arena (Chapter 8), and sets the default list of Objective-C images to preload. And three variables single out one process of a large tree by a substring of its command line: `OCERZ_EXE_ENV` sets variables in it, `OCERZ_STRACE_EXE` traces its system calls, and `OCERZ_NOJIT_EXE` runs it under the interpreter.

Finally `ocerz_peek_dynamic()` reads just enough of the file to classify it. It returns -2 for a file with no x86-64 slice (exit 64 with "it is not an Intel program, so there is nothing to translate"), a negative value for a file it cannot read (exit 65), 0 for a static image and 1 for one that names a dynamic linker. Native mode refuses a static image (exit 64), since there is nothing for it to bridge. `sw_vers` is dynamic, so `main()` hands it to `ocerz_dyld_run()` in `src/dyld.c`, together with a private copy of the environment made by `env_snapshot()`.

HOW IT WAS FOUND

The environment is copied because of a bug that depended on which shell launched `ocerz`. In native mode, loading the guest opens the host's real CoreFoundation, whose initializer calls `setenv`, which real-locates `environ` and frees the array a pointer taken earlier still named. The guest's stack was built from that pointer afterwards, and whether the stale read crashed depended on whether the freed block had been reused, which it was when the environment happened to be the size `bash` passes and not the size `zsh` passes. The copy owns its strings and lives as long as the process.

2. Reserving guest memory

The first thing `ocerz_dyld_run()` does is `ocerz_mem_init_identity(DYN_arena_size)`, with `DYN_arena_size` defined as 256 GB. This reserves one contiguous region of host address space for everything the guest will map, starting at 12 GB (`0x300000000`), and it is an *identity* map: a guest address and the host address behind it are the same number. Every guest load and store can then use its address directly, with no base to add, which is the fastest case the JIT has.

The arena is reserved, not committed: it costs address space, not memory. 256 GB is not a typo. Chromium's `PartitionAlloc` reserves 32 GB at startup, and `JavaScriptCore`'s `Gigacage` asks for a single 128 GB mapping; the arena grew from 32 GB to its present size for those two. A `Wine` process puts the arena at a fixed base instead of wherever the host would place it, and a `Wine` image that insists on low addresses gets a second map, the low shadow, for its first 12 GB. Chapter 8 explains both, and the 4 KB-guest-pages-on-16 KB-host-pages machinery underneath.

3. Mapping the shared cache

In cache mode the guest's system libraries come from Apple's x86-64 shared cache, and `ocerz_cache_map()` in `src/cache.c` maps it. `ocerz_cache_dir()` first decides where it is: macOS 27 keeps it in the Rosetta cryptex, `/System/Volumes/Preboot/Cryptexes/Rosetta/System/Library/dyld/`, and macOS 26 in the OS cryptex, `/System/Volumes/Preboot/Cryptexes/OS/System/Library/dyld/`. The Rosetta location is tried first, and the main file and all its numbered subcaches are taken from whichever directory holds a cache. If neither does, which normally means Rosetta is not installed, the process stops with "cannot find the x86_64 shared cache".

The main file and its subcaches, `dyld_shared_cache_x86_64`, `.01`, `.02` and so on, are mapped at the cache's preferred base, `0x7FF800000000`, which lies above the arena in host space. The cache's data regions store every pointer in a packed slide-info format that has to be unpacked even when the cache is not slid; unpacking about 500 MB of it eagerly cost about 200 ms per process, so those regions are mapped with no access at all and each 16 KB page is unpacked the first time anything touches it, from the fault handler. The cache's own header then gives the image table: several thousand libraries, from `libSystem` to `AppKit`, all present in every process from this point on. Chapter 11 has the details.

4. Mapping, binding and protecting the program

`read_file()` reads `/usr/bin/sw_vers`, `select_slice()` picks its x86-64 slice out of the universal file, and the path is made absolute with `realpath`, because programs derive things from it (Steam derives its bundle root from its executable's path, and a relative one broke it). `map_segments()` lays the segments out in the arena, read-write for now so that the copy and the fixups can land. The image must have an entry point, either `LC_MAIN` or the older `LC_UNIXTHREAD`, or loading stops.

Dependencies come next. `collect_rpaths()` gathers the image's `LC_RPATH` entries and `load_disk_deps()` loads any dependency that is a file on disk rather than a library in the cache, recursively. `sw_vers` has none: `CoreFoundation` and `libSystem` are both in the cache. Then `apply_fixups()` walks the image's chained fixups, rebasing internal pointers by the image's slide and binding every import. A cache import is bound in the specific library the image named, following re-exports, rather than by searching every cache image for the name; the difference matters when two libraries export the same symbol, which is how `openssl` once reported the wrong version. An older image whose imports are described in the classic `LC_DYLD_INFO` form goes through `apply_classic_fixups()` instead. Last, `protect_ro_segments()` gives the image its final protections; a segment left writable would be treated as possibly self-modifying code (Chapter 7).

5. The initial frame, and the pieces the guest expects

`build_frame()` builds the guest's initial stack in an 8 MB stack region the way XNU's exec path lays it out: every argument and environment string, then `argv`, `envp` and the `apple` array as one contiguous ascending run, each terminated by a null pointer. The contiguity is load-bearing, because `libSystem` finds `apple[]` by walking off the end of `envp`. The frame also holds a `ProgramVars` block pointing at `libdyld`'s own `NXArgc`, `NXArgv`, `environ` and `__progname`, so the program's `environ` and `libc`'s `_NSGetEnviron()` are one variable, as on a real system.

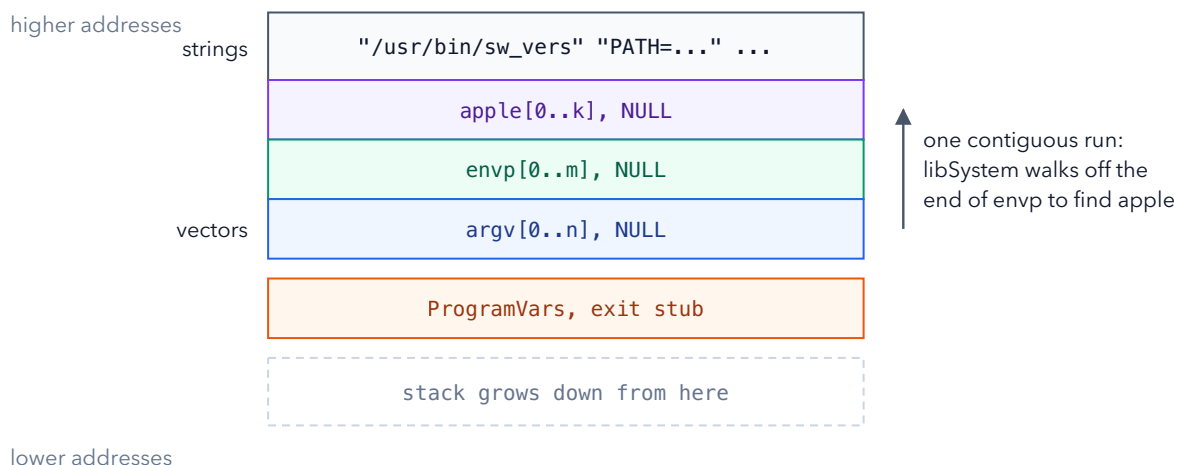


Figure 2.2. The guest's initial stack in dynamic mode. Every string and vector entry is counted and stored; an earlier version capped the environment at 60 entries, which silently dropped the marker Wine appends to stop its loader re-executing, and re-executed every Wine process forever.

Then the pieces a program expects to find already in place. A 32 KB guest thread block is mapped for the main thread, and the guest's `gs` base is pointed into it, because x86-64 macOS code reaches its thread-specific data through `gs`. `oc erz_vm_install_handlers()` installs the host signal and fault handlers that

everything later depends on (Chapter 9). `oc erz_commpage_init()` builds the emulated `commpage`, the small read-only page of system information macOS maps into every process at a fixed address. In cache mode `oc erz_dyldapi_setup()` installs AArchX's stand-in for the object that `libdyld`'s API functions call into: `_dyld_image_count`, `dlopen`, the Objective-C runtime's image callbacks and the rest reach it through a table whose entries point into a reserved window of guest addresses, `0xdda00000` to `0xdda10000`, and AArchX answers each by its position in the table (Chapter 11). Last, `libdyld`'s `environ`, `__progname`, `NXArgc` and `NXArgv` variables in the cache are pointed at the frame.

6. Initializers, in dyld's order

Before `main` can run, every library the program depends on must have run its initializers, and in the right order. This is, in the words of `dyld.c`'s explanation block, "most of the file". Each guest function runs through `oc erz_vm_call()`, which sets up a guest call with a sentinel return address, enters the same run loop that runs everything else, and returns to C when the guest function returns.

1. **libSystem first.** `find_dylib_init()` finds `libSystem`'s initializer in the cache, and it runs with five arguments: `argc`, `argv`, `envp`, `apple` and the `ProgramVars` pointer. It sets up `malloc`, `libdispatch`, the Objective-C runtime and the rest of the base system. When it returns, `init_mark_done_closure()` marks `libSystem`'s whole dependency closure as initialized, which keeps the later walk from descending through it.
2. **Thread-local variables.** `oc erz_tlv_register_closure()` registers every image's `__thread` variables.
3. **The eager set.** For a program whose closure reaches `CoreFoundation`, `compute_eager_set()` decides which images must be initialized now: those the program reaches, closed under dependencies. An image in the set whose dependency was left out would initialize on top of an uninitialized library.
4. **The walk.** `run_load_phase()` delivers the Objective-C `+load` notifications for `libSystem`'s closure, and then `run_init_root()` walks from the main image: per image, bottom-up, every dependency finished first, then that image's `+load` methods, then its own initializers. Neither global order works: running every `+load` first, or every initializer first, both broke real programs.
5. **Inserted libraries.** If `DYLD_INSERT_LIBRARIES` was set, `load_inserted_libraries()` opens each named library with the guest's own `dlopen`.

Upward dependencies are the subtle part. An *upward* link (`LC_LOAD_UPWARD_DYLIB`) is how a library declares the back edge of a dependency cycle: `CoreFoundation` links `Foundation` upward, although `Foundation` depends on `CoreFoundation`. An upward link is an ordering edge for nothing, so the walk does not follow it on the way down. But its target still has to be initialized at some point, and *when* turned out to matter.

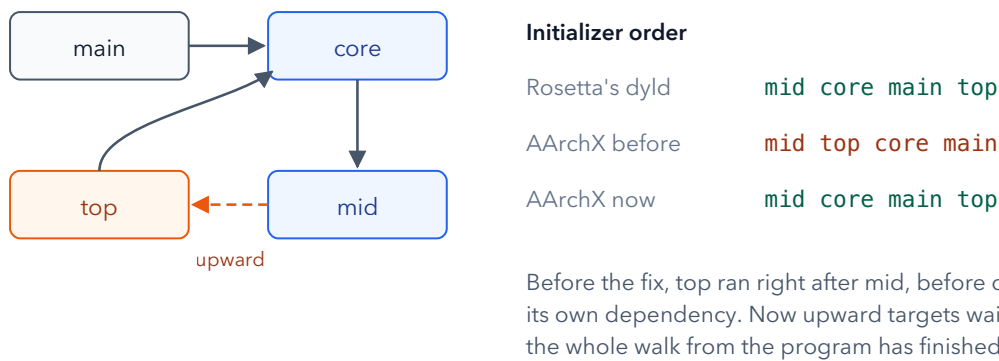


Figure 2.3. The shape tested by `tests/dynamic/upward_defer.c`: core depends on mid, mid links top upward, and top depends on core. Each initializer logs its name; the expected order was measured under Rosetta.

Until 22 September 2026 AArchX did not initialize upward targets at all, and on macOS 27 that broke `sw_vers`: Foundation's initializer never ran, `NSString` stayed an abstract class cluster, and the first `+ [NSString stringWithFormat:]` recursed until the 8 MB guest stack ran out. The fix initialized each upward target right after the library that declared it. On macOS 26.7.1 that order broke the same program the other way: there, `libobjc` links `libswiftCore` upward and `libswiftCore` links Foundation upward, and `libobjc` is itself a dependency of CoreFoundation, so Foundation's entire subtree initialized while CoreFoundation was still unfinished on the recursion stack, and an allocation through CoreFoundation's allocator recursed between `malloc` and `malloc_zone_malloc` until the stack ran out. Since 3 October each upward target goes on a list owned by the root call and is initialized after the whole walk has finished, the order Figure 2.3 shows Rosetta's dyld using.

7. The first block

With the initializers done, `oc erz_dyld_run()` calls `main` through `oc erz_vm_call()` with `argc`, `argv`, `envp` and `apple`. Like every guest function, it runs in `oc erz_vm_run_cpu()` in `src/vm.c`, the run loop. On entry the loop registers the CPU, clears the host thread's signal mask (AArchX emulates the guest's mask itself, and the host thread must never block asynchronous signals), and sets a recovery point with `sigsetjmp`. Then, until the guest exits or the call returns, it repeats one decision:

```
while (!vm->exited && !cpu->terminated && !cpu->interrupt) {
    oc erz_vm_suspend_point(cpu);
    ...
} else if (vm->jit_enabled && (vm->jit || (vm->jit = oc erz_jit_create(vm)))) {
    r = oc erz_jit_step(vm, cpu);
    if (r == OCERZ_EUNSUP)
        r = oc erz_interp_step(vm, cpu);
} else {
    r = oc erz_interp_step(vm, cpu);
}
...
}
```

`oc erz_jit_step()` in `src/jit.c` looks the guest `rip` up with `cache_lookup()`, keyed by address and by whether the code is 32- or 64-bit. The first time, there is nothing there, so it takes the translation lock and calls `translate()`. That function first checks whether this address must stay interpreted (a page that

keeps being rewritten, a range named by a debugging switch), then asks the on-disk translation cache whether another process already translated this code and the bytes it was made from are unchanged. If not, the decoder turns the bytes into `X86Insn` records until it reaches the end of the block, possibly continuing past forward conditional branches into a superblock, and the emitter produces arm64 code into the 1 GB code arena. Chapter 4 describes the decoder and Chapters 6 and 7 the translation.

A published block is a C-callable function, and running it is a single call: `b->code(vm, cpu)`. Its prologue loads the guest registers from the `0cerzCPU` struct into the host registers they are pinned to, all sixteen general registers and all sixteen `xmm` registers, for the whole time the guest runs in translated code. When the block ends with a branch to another translated block, that branch is patched into a direct arm64 branch into the other block's body, so a hot loop or call chain runs from block to block without coming back to C at all. Only when the next block is not translated yet, or something needs the dispatcher, does the epilogue store the registers back and return to the run loop, which looks up the new `rip` and goes round again. If `0cerz_jit_step()` answers `OCERZ_EUNSUP`, there is nothing it will translate here (the dyld API window is one such place), and `0cerz_interp_step()` executes one instruction instead: the interpreter is complete, so anything the JIT declines still runs.

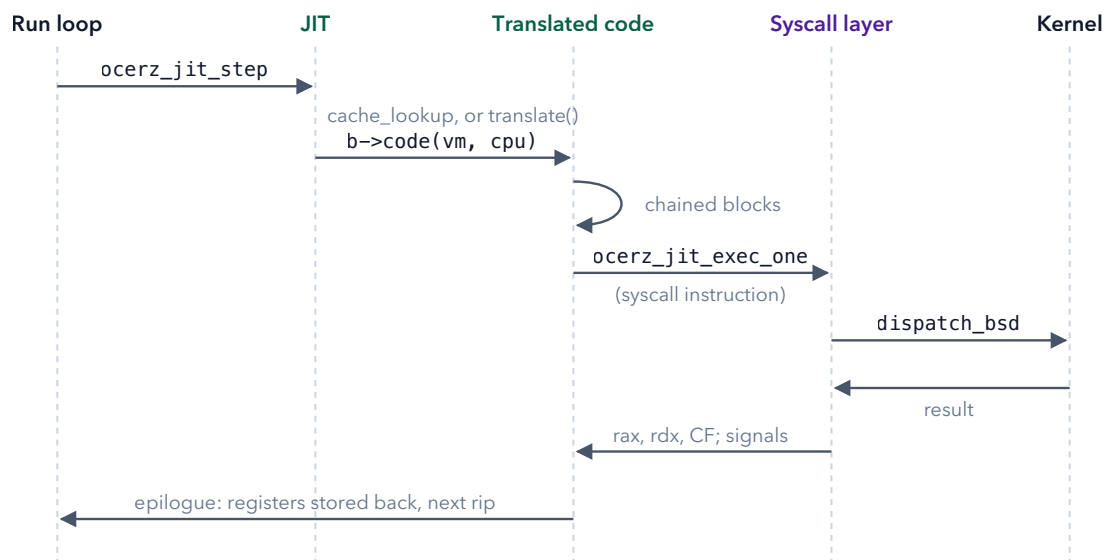


Figure 2.4. One trip through translated code. Most trips never reach the right half: blocks chain into each other and only come back for an untranslated target, an interpreter-only instruction, or a system call.

8. A system call

Sooner or later `sw_vers` opens `SystemVersion.plist`, and `libSystem`'s x86-64 `open` executes a `syscall` instruction. Translated code does not try to emit anything for it: it calls out to `0cerz_jit_exec_one()`, which runs that one instruction through the interpreter, and the interpreter's `OCERZ_OP_SYSCALL` case calls `0cerz_handle_syscall()` in `src/syscall.c`. The 8,633-line syscall layer is the guest-to-host boundary.

On macOS the syscall number in `rax` carries a class in bits 24 to 31. `0cerz_handle_syscall()` splits it: class 1 goes to `dispatch_mach()` (Mach traps), class 2 to `dispatch_bsd()` (BSD system calls such as `open`), class 3 to `dispatch_machdep()` (machine-dependent calls such as setting the thread's segment base). Most BSD calls are passed straight to the host kernel through `sys_raw.h`, the one place AArchX enters the kernel directly; the rest are intercepted because an argument or result has to be rewritten, or

because AArchX must keep its own state in step (memory mappings, signals, threads, processes). The result comes back the way Rosetta returns it, which is not quite the way the kernel does: unix calls return through `rax` and `rdx` with the carry flag set on error, and the other arithmetic flags take the fixed pattern Rosetta leaves; machine-dependent calls touch `rax` alone, because Wine keeps a frame pointer in `rdx` across one of them. Pending signals are offered both on the way in and on the way out (Section 10). Then the interpreter returns to the translated block, which carries on.

One call takes a different path. A `fork` is never performed from inside translated code: `jit_exec_one()` recognises it and returns `OCERZ_EUNSUP`, so the run loop performs it from a clean state. And a blocking call made from translated code is recorded as *parked*, so that a code-arena flush (Chapter 7) knows the thread is safely outside the arena while it waits.

9. A second thread

`sw_vers` itself is single-threaded, but libSystem's initializer and libdispatch start threads in nearly every Cocoa program, so consider one. `pthread_create` in the guest's libc ends in the `bsdthread_create` system call, and `sys_bsdthread_create()` answers it by creating a real host thread with an `OcerzCPU` of its own and its own guest stack and thread block, which then runs `ocerz_vm_run_cpu()` from the guest's start routine. Guest threads are host threads; they run in parallel on real cores.

That has one process-wide consequence. A program with a single thread cannot observe the difference between x86's strong memory ordering and arm64's weaker one, so AArchX starts every process with plain loads and stores. The first sign of a second observer (a thread, a `fork` or `spawn`, a workqueue worker, a writable shared mapping or an attached System V segment) calls `ocerz_jit_require_ordered()`, and from then on, for the rest of the process's life, scalar memory accesses are translated with acquire and release semantics. Chapter 7 explains what that costs and what it deliberately leaves plain.

libdispatch's worker threads are created by the kernel rather than by `pthread_create`; the host workqueue bridge hands those requests to the host kernel and gives each worker a guest personality when it starts running guest code (Chapter 9).

10. A signal

Suppose the guest installs a handler with `sigaction` and something sends it `SIGUSR1`. The guest's handler table, mask and alternate stacks are AArchX's own state, kept by the syscall layer; plain asynchronous signals are also mirrored onto the host, because the host kernel decides, for instance, whether `SIGPIPE` kills the process. When the host signal arrives, AArchX's handler records it as pending on the CPU of the thread it arrived on. It is delivered at the next system-call edge: on entry, with `rip` rewound so the call runs again afterwards, or on return. `ocerz_signal_deliver()` builds the x86 signal frame (`siginfo`, `ucontext` and a machine context that includes the AVX upper halves) on the guest's stack or alternate stack and enters the guest's signal trampoline, which calls the handler and then `sigreturn`. Guest masks are honoured exactly; Chapter 9 explains why getting that wrong once desynchronised Wine's server protocol.

Faults are different. A translated load from a bad address raises a real host `SIGSEGV` or `SIGBUS` in the middle of arm64 code, and the fault handler installed in step 5 decides what it was: the first touch of a lazily unpacked cache page (unpack and retry), a store into a page whose code was translated (drop the

stale translations and retry), an unaligned ordered access (patch that one access and retry), and so on. Only a fault that is none of these is turned back into a guest fault, with every guest register reconstructed from the host registers holding it, and delivered to the guest's handler, or reported as a crash.

11. Exit

`main` returns its status in `rax`, and `oc erz_vm_call()` hands it back to `oc erz_dyld_run()`, which resolves `_exit` in the cache and calls it with the low byte of the status. If the program called `exit` itself, its `atexit` handlers and `stdio` flushes have already run as guest code. Either way the guest ends in the exit system call, and `sys_exit()` calls `oc erz_vm_request_exit()`, which marks the VM exited and kicks every other guest thread out of translated code. On the main thread the run loop then stops at its next check, `oc erz_dyld_run()` returns the status and `main()` returns it; on any other host thread, which cannot unwind the main thread's C frames, `sys_exit()` flushes the profiler and the translation store and calls `_exit` on the spot. Translations of code that sits at the same address in every run (here, the shared cache's) have been streaming to the on-disk store all along, and the next process that runs the same code loads them instead of translating it again.

NOTE

A dynamic image whose entry point is `LC_UNIXTHREAD` rather than `LC_MAIN`, as older programs have, takes a slightly different last step: its initial stack is rearranged into the form the kernel would hand to start (`argc` followed by the vectors), `rip` is set to the thread entry, and `oc erz_vm_run()` runs it like a static program.

The same program in native mode

Run as `./oc erz -native /usr/bin/sw_vers`, the first two steps are identical: the same command line, the same 256 GB identity arena. From there the route diverges.

- **No cache is mapped.** `oc erz_dyld_run()` logs that native mode is independent of Rosetta and moves on.
- **The system libraries are synthesized.** When the loader reaches `/usr/lib/libSystem.B.dylib` or `CoreFoundation`, `src/vdylib.c` builds a real x86-64 Mach-O image for it in memory, with load commands, segments and an export trie, from the library's API database. Each export is twelve bytes of x86: load the export's number into `r11`, jump through a slot that points into the same reserved trap window the dyld API uses.
- **Binding is checked strictly.** An import that no synthesized library exports is collected, and if any are left after binding, they are listed and the process exits with status 71 before the guest runs a single instruction.
- **Objective-C is the host's.** `oc erz_objcbridge_fix_selrefs()` rewrites the image's selector references to the native runtime's selectors, and `oc erz_objcbridge_define_image()` realizes the guest's own classes in the native runtime.
- **Initializers are simpler.** There is no `libSystem` initializer to run, because the host's `libSystem` is already initialized in the `oc erz` process. `oc erz_objcbridge_run_loads()` runs the guest's `+load` methods, then each loaded guest image's initializers in reverse load order, then the program's own.

- **A library call is a crossing.** When `sw_vers` calls `CFStringCreateWithCString`, the stub's jump lands in the trap window. From translated code the JIT does not take the trap: it calls the bridge directly from inside the block, `src/abi.c` moves the arguments from System V registers to arm64 ones according to the function's signature, `src/abical.l.s` calls the host's real CoreFoundation, and the result comes back the same way. That costs about 17 ns; eleven string and memory routines such as `strlen` are answered in place by arm64 code in `src/leaf.s` for about 2 ns. A function with no crossing stops with status 72 and its name.

Everything below the libraries is unchanged: the same JIT translates `sw_vers`'s own code, the same syscall layer answers the system calls that do not go through a library, and the exit is the same. Chapter 12 describes native mode in full.

A static program

The test programs under `tests/guest/` are freestanding: built with `-nostdlib -static`, they make system calls directly and link nothing. For such a file `ocerz_peek_dynamic()` returns 0, and `main()` takes the oldest path in the tree:

1. `ocerz_mem_init(0x100000000, 0x900000000)` reserves a fixed arena for guest addresses from 4 GB to 36 GB. Unlike the identity map, a guest address here is translated by adding a base, because the host will not always grant the low addresses a static image asks for.
2. `ocerz_load_image()` in `src/loader.c` picks the x86-64 slice and maps the segments, reading every segment's bytes before applying any protection, because two 4 KB guest segments can share one 16 KB host page.
3. `ocerz_setup_stack()` in `src/stack.c` builds the initial stack exactly as XNU's exec path does for `start`: strings at the top, then `apple`, `envp`, `argv` and `argc` below them.
4. `rip` is set to the entry point from `LC_UNIXTHREAD` and `ocerz_vm_run()` runs it, with no dynamic linker, no initializers and no shared cache.

Native mode refuses such a program, and cache mode does not need the cache for it: a static test program runs on any Mac with nothing but `ocerz`, which is why the guest test suite, the oldest gate in the tree, still uses this path. Chapter 10 describes the loader.

Exit codes along the way

Code	Where	Meaning
64	<code>main()</code>	bad AArchX arguments, an unknown <code>OCERZ_MODE</code> , a file with no x86-64 slice, or a static image in native mode
65	<code>main()</code>	the program could not be read or loaded
70	static path	the guest arena or stack could not be set up
71	native binding	an import no synthesized library exports
72	native crossing	a bound export with no crossing was called

Code	Where	Meaning
guest's own	exit	the program ran and exited

Every one of these is preceded by a line on standard error that begins with `ocerz:` and says what went wrong. Appendix B lists the messages.

CHAPTER 3

The source tree and its conventions

AArchX is 30 source files and 28 headers, built by one Makefile with nothing but the system toolchain. This chapter lists every file with its size and its job, shows how the headers depend on each other and which three structures hold the state, and describes the build, the test gates it drives, and the conventions the tree is written in: one prose explanation per file, commit messages that state a rule, a test for every change, and several hundred environment switches.

72,277LINES IN SRC/ (28 .C,
2 .S)**3,901**

LINES IN 28 HEADERS

20,555

LINES IN JIT.C ALONE

410DISTINCT OGERZ_*
VARIABLES READ

The shape of the repository

All counts in this chapter are from commit `9169680` (4 October 2026). The repository tracks 722 files; most of them are tests.

Directory	What lives there
<code>src/</code>	the translator: 28 C files and 2 arm64 assembly files
<code>include/</code> <code>ocerz/</code>	28 headers, each opening with a prose block that states its contract
<code>tests/</code>	the gates: <code>unit/</code> (29 C harnesses that link the translator's objects), <code>guest/</code> (freestanding x86-64 programs with expected outputs), <code>dynamic/</code> (109 files of dynamically linked fixtures), <code>win/</code> (Windows programs for Wine), <code>corpus/</code> , and the runner scripts
<code>tools/</code>	<code>sdkgen.sh</code> and <code>sdkgen/</code> , which generate native mode's API databases from the SDK; <code>apptest.sh</code> , which compares real tools against native output; <code>dyldslots.sh</code> , which identifies every dyld API slot by running code; the i386 decode oracles and the corpus generator
<code>runtime/</code>	not committed: <code>apis/</code> , the API databases <code>make apis</code> generates, and <code>guest/</code> , the optional LLVM C++ runtime <code>make guest-cxx</code> builds for x86-64
<code>docs/</code>	user and contributor documentation, and the Atlas, a guided tour of the source

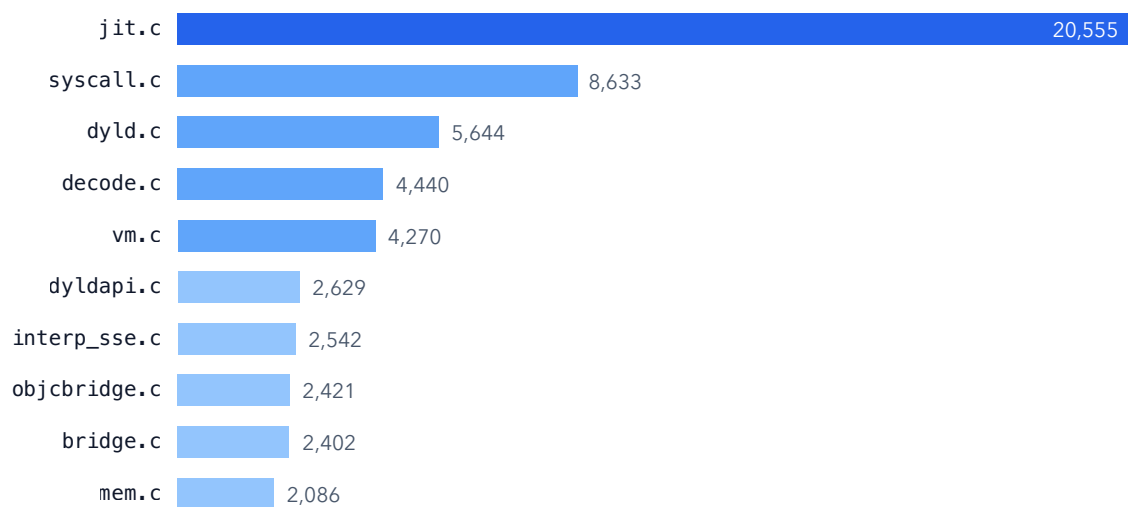
Every source file

Grouped by layer, in the order a program meets them. The line counts are the whole file, explanation block included.

File	Lines	Role
Entry and process		

File	Lines	Role
<code>main.c</code>	419	Command line, the three startup decisions (Objective-C preload list, plain memory model, mode), bundle launch, the <code>DYLD_INSERT_LIBRARIES</code> re-exec, the per-process switches for Wine's process tree.
<code>vm.c</code>	4,270	The run loop, <code>ocerz_vm_call</code> , crash containment and the fault handler, signal plumbing, guest thread suspension, the unstick monitor, native-thread attach, the <code>kill -INFO</code> thread dump and the sampling profiler.
<code>cpu.c</code>	163	CPU reset state and the register dump used by fatal paths and <code>-trace</code> ; propagates the guest's MXCSR rounding mode to the host FPCR.
<code>globals.c</code>	16	Process-wide definitions no single module should own.
The CPU		
<code>decode.c</code>	4,440	x86-64 and i386 decoding, legacy, VEX and MMX forms, into one <code>X86Insn</code> per instruction (Chapter 4).
<code>interp.c</code>	1,913	The core single-step interpreter: reference semantics and the oracle the JIT is checked against (Chapter 5).
<code>interp_sse.c</code>	2,542	SSE through SSE4.1 and the AVX forms layered on them.
<code>interp_ext.c</code>	1,197	The long tail: string instructions, CPUID, CRC-32C, x87, and the rest.
<code>flags.c</code>	230	Eager RFLAGS computation, bit for bit, which the JIT's deferred flags must agree with.
<code>flags_live.c</code>	283	Per-instruction flag definition and use, so the JIT can skip flags nothing reads.
<code>jit.c</code>	20,555	The JIT: translation, register pinning, deferred flags, chaining, superblocks, floating-point batches, memory ordering, self-modifying code, the arena and its flush (Chapters 6 and 7).
<code>a64emit.c</code>	924	The arm64 encoder: one function per instruction, no scheduling, no register allocation.
<code>leaf.s</code>	598	arm64 <code>strlen</code> , <code>memcpy</code> and nine more that translated code calls with the guest's registers left in place.
<code>tcache.c</code>	821	The on-disk store that keeps translations between processes and runs.
Memory and the system		
<code>mem.c</code>	2,086	Guest address maps, the 4 KB-in-16 KB slot machinery, write-arming for self-modifying code, the low shadow (Chapter 8).
<code>syscall.c</code>	8,633	The guest-to-host boundary: BSD calls, Mach traps and messages, threads and the workqueue bridge, signals, fork, exec and spawn (Chapter 9).
Linking		
<code>loader.c</code>	378	Static Mach-O loading: slice selection and segment layout (Chapter 10).
<code>stack.c</code>	137	The initial process stack, laid out the way XNU's exec path does.

File	Lines	Role
<code>dyld.c</code>	5,644	The mini-dyld: loading, binding, initializer order, <code>dlopen</code> and <code>dlsym</code> , in both modes (Chapter 11).
<code>cache.c</code>	1,499	Mapping the x86-64 shared cache, lazy slide, and symbol resolution out of it.
<code>dyldapi.c</code>	2,629	The dyld API object <code>libdyld</code> calls into, the image list, Objective-C image callbacks.
Native mode (Chapter 12)		
<code>vdylib.c</code>	1,228	Synthesizes the x86-64 system libraries native mode has no files for.
<code>apidb.c</code>	1,108	Reads the API database: each library's exports and signatures.
<code>bridge.c</code>	2,402	What each export does when called: crossings, the functions <code>AArchX</code> answers itself, fault attribution.
<code>abi.c</code>	1,744	Moves arguments and results between System V x86-64 and Apple arm64, both ways; the callback trampoline bank.
<code>abicall.s</code>	237	Makes an arm64 call whose argument registers arrive as data.
<code>sysbridge.c</code>	1,494	The <code>libSystem</code> calls a generic crossing cannot make: fixed-argument variadics, memory mapping, non-local jumps, processes.
<code>objcbridge.c</code>	2,421	Objective-C message sends and formatted output across the boundary.
<code>objcclass.c</code>	1,357	The guest's own classes, categories and protocols, realized in the native runtime.
<code>blocks.c</code>	909	Blocks in both directions.



lines per file, the ten largest; `jit.c` is 28% of `src/`

Figure 3.1. Where the code is. The JIT dominates, then the system-call layer and the dynamic linker; every other file is under 4,500 lines.

The headers and how they depend on each other

The 28 headers in `include/ocerz/` are the contracts between the files. Most are small; the two large ones are `decode.h` (643 lines, the instruction record and the list of operations) and `abi.h` (456 lines, the signature language native mode's crossings are written in). The include graph is shallow and acyclic: almost everything rests on `types.h` and `cpu.h`.

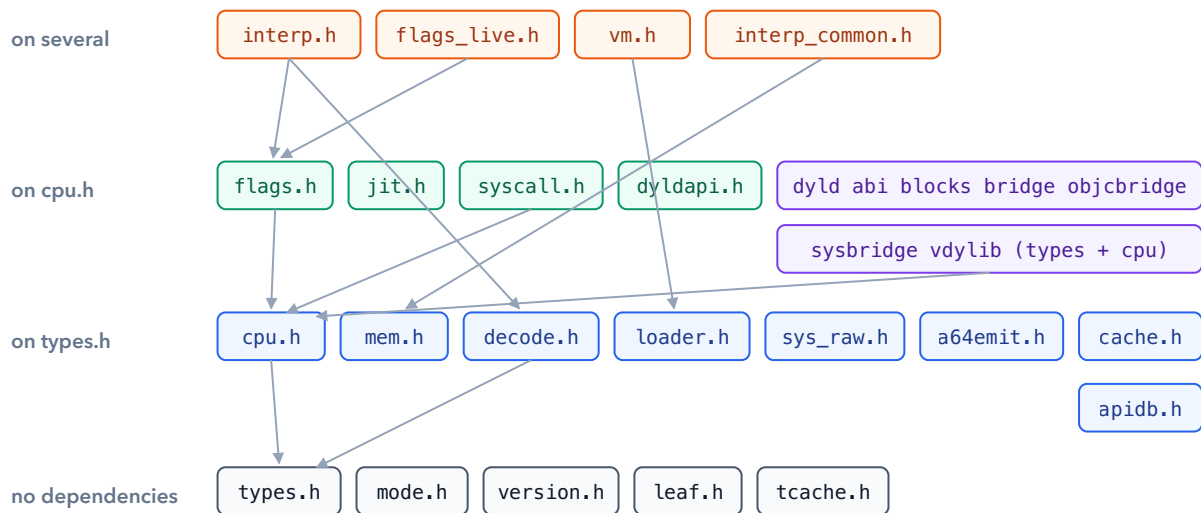


Figure 3.2. The header include graph, simplified: each row depends only on rows below it. `interp_common.h` pulls in `cpu.h`, `decode.h`, `flags.h`, `mem.h` and `interp.h`; `vm.h` needs `cpu.h` and `loader.h`; `dyldapi.h` includes only `cpu.h`.

Header	Lines	Contract
<code>types.h</code>	36	Fixed-width types, error codes, logging macros, the 128-bit vector type.
<code>cpu.h</code>	177	<code>0cerzCPU</code> , one guest thread's architectural state; its field order is load-bearing for the JIT.
<code>decode.h</code>	643	<code>X86Insn</code> , the only format the decoder, interpreter and JIT share, and the list of 548 internal operations.
<code>mem.h</code>	241	The guest address maps and sized guest loads and stores; the commpage window.
<code>vm.h</code>	102	<code>0cerzVM</code> , one guest process; the run loop and <code>0cerz_vm_call</code> .
<code>jit.h</code>	85	The JIT's interface, and the retirement counter callers use to tell whether code they will return to is still current.
<code>syscall.h</code>	116	The system-call entry and the helpers other layers call.
<code>abi.h</code>	456	Native mode's signature language and the calling-convention engine.

Header	Lines	Contract
<code>objcbridge.h</code>	370	Objective-C sends, selectors and guest classes across the boundary.
<code>a64emit.h</code>	294	The arm64 encoders.
<code>apidb.h</code>	177	The API database format and where its files are read from.
<code>interp_common.h</code>	166	Helpers shared by the three interpreter files.
<code>bridge.h</code> , <code>vdylib.h</code> , <code>blocks.h</code> , <code>sysbridge.h</code> , <code>dyld.h</code>	164, 135, 129, 99, 94	The rest of native mode and the dynamic linker.
<code>flags.h</code> , <code>leaf.h</code> , <code>cache.h</code> , <code>tca</code> <code>che.h</code> , <code>sys_raw.h</code>	72, 62, 53, 49, 39	Flag semantics, the in-place routines, the shared cache, the translation store, the one direct path into the kernel.
<code>interp.h</code> , <code>dyldapi.h</code> , <code>mode.h</code> , <code>l</code> <code>oader.h</code> , <code>flags_live.h</code> , <code>version.h</code>	31, 30, 29, 25, 17, 10	Small interfaces; <code>version.h</code> defines "AArchX" and "0.3-dev".

Three structures hold the state

Nearly everything AArchX does reads or writes one of three structures.

OcerzVM (`vm.h`) is one guest process: the main thread's CPU, the loaded image, the JIT, the exit flag and code, the trace switches, and the two memory-model flags, `jit_plain_mem` and `jit_ordered_required`, that record whether the process has acquired a second observer yet. There is one per ocerz process.

OcerzCPU (`cpu.h`) is one guest thread. It holds the sixteen general registers in x86 encoding order (`rax`, `rcx`, `rdx`, `rbx`, `rsp`, `rbp`, `rsi`, `rdi`, then `r8` to `r15`), `rip`, the flags with the deferred-flag record beside them (`cc_op`, `cc_src`, `cc_dst`), the `fs` and `gs` bases, the sixteen `xmm` registers, the AVX upper halves, the x87 registers held as 64-bit doubles, the MMX registers, MXCSR, the signal mask and pending set, the alternate stack, and the segment selectors 32-bit code needs. The field order matters because the JIT addresses this struct with immediate offsets: `xmm[]` is 16-byte aligned for scaled vector loads, and the 256-entry return-address stack, 4 KB by itself and most of the struct, sits immediately after it so that every entry stays within reach of a single `ldp` or `stp`. New fields are appended at the end so nothing that exists moves by a byte. A fair part of the rest is per-thread instrumentation: the block-entry trace ring, the fields the unstick monitor reads, the thread-suspension handshake, a ring of the last system calls, and the form of the instruction the interpreter is running, for the sampling profiler.

OcerzJit (private to `jit.c`) is the translator: the code arena, 1 GB by default (`OCERZ_JIT_CODE_MB` or `OCERZ_JIT_CODE_KB` change it, and the test suite runs some programs in a tiny one on purpose), the hash table of translated blocks, the live-block array, per-granule block lists, and the tables that chaining, return prediction and invalidation use. There is one per process, shared by every thread under a translation lock.

Between them sits **X86Insn** (`decode.h`), the 96-byte record the decoder produces for each instruction: its address, length, a 16-bit operation number, size and prefix fields, the condition code, the mode, the VEX fields, and up to three operands. It is the only representation the decoder, interpreter and JIT share. The

operation field was once eight bits wide while the operation list had 364 entries; operations above 255 truncated into the wrong interpreter unit, every consumer honoured the flawed header identically, and only the end-to-end differential exposed it. The field is 16 bits now and there are 548 operation values.

Building

The Makefile compiles every `src/` file with `clang -arch arm64 -std=c11 -O2 -g -Wall -Wextra` and links `ocerz` against `libcompression`, which the translation store uses for its LZ4 records. There are no other dependencies. A full build takes well under a minute.

Target	What it does
<code>make</code> or <code>m</code> <code>ake</code> <code>ocerz</code>	Builds the translator.
<code>make</code> <code>unit</code>	Builds the 29 unit harnesses, which link every object except <code>main.o</code> , and runs each with <code>OCERZ_NO_ARM_EXEC=1</code> , because they drive the JIT without a run loop and the self-modifying-code trap would have no fault recovery to land on.
<code>make</code> <code>guest</code>	Builds the freestanding x86-64 test programs in <code>tests/guest</code> with <code>-arch x86_64 -nostdlib -static</code> .
<code>make</code> <code>apis</code>	Generates native mode's API databases under <code>runtime/apis</code> from this machine's SDK, with <code>tools/sdkgen.sh</code> ; a stamp named for the SDK version keeps it from rerunning until the generator or its inputs change. They are derived from the SDK and so are never committed.
<code>make</code> <code>check</code>	Depends on <code>ocerz</code> , <code>unit</code> , <code>guest</code> and <code>apis</code> , then runs ten gates in order: the guest suite interpreted, the guest suite translated, the x86-64 differential, the i386 differential, the dynamic suite, the native suite, the guest-library tests, the native C++ tests, the native framework tests and the native format tests.
<code>make</code> <code>guest-</code> <code>cxx</code>	Builds LLVM's <code>libc++</code> , <code>libc++abi</code> and <code>libunwind</code> for x86-64 into <code>runtime/guest</code> , for native mode.
<code>make</code> <code>i386diff</code>	32-bit decode conformance against the capstone disassembler, reported as a coverage percentage; deliberately not part of <code>check</code> .

The Makefile exports `OCERZ_TCACHE=off` unless the environment says otherwise, so no test loads a translation an earlier test stored and none writes to `~/Library/Caches`; the translation-store tests turn it on themselves against a directory of their own.

The test binaries are **committed**: 29 unit harnesses under `tests/unit/bin` and the guest programs and benchmarks under `tests/guest/bin` and `tests/guest/benchbin`, 104 files. A change to the translator is then measured against exactly the binaries the previous run used, not against whatever the local compiler produces today. Because the unit harnesses link the translator's objects, they change whenever it does, and they are refreshed by a separate commit with the fixed subject `tests: rebuild unit binaries`; 101 of the 698 commits are that. Chapter 15 describes every gate.

The conventions

One explanation block per file

Since commit 47d311c of 12 September 2026, which changed 154 files and removed 5,501 lines of comments, every source file and header opens with one prose block explaining what it owns and why it is shaped the way it is, and the code bodies carry no comments at all. The blocks are written in the present tense, in full sentences, and they record the failures that shaped each rule: why the environment is copied, why initializers run in a particular order, why vector accesses stay plain in ordered mode. They are the deepest documentation in the project, and they are long where the subject is: 519 lines in `dyld.c`, 517 in `jit.c`, 377 in `syscall.c`, 302 in `vm.c`.

```
/*
 * Command-line parsing and the entry point.
 *
 * Three startup decisions are made here rather than lazily. Some shared-cache
 * images' Objective-C categories must be visible before any class is realized,
 * because a dlopen registers them too late for classes that already exist:
 * ...
 */
```

The rule is that a block changes in the same commit as the behaviour it describes. It is not always kept: at 9169680, the blocks of `decode.c` and `cpu.h` still say the JIT declines every VEX instruction, which stopped being true on 13 September.

Commit messages that state the rule

A subject line has the form `area: what is now true`, not a description of the activity: `jit: a chain that cannot reach goes through a veneer instead of being dropped`, `dyld: an upward dependency is initialized after the whole walk from the program`, not right after the library that declares it. The body is prose: the symptom, what was measured, the mechanism, the fix and the test. These messages are long, often several paragraphs, and together with the explanation blocks they are the project's design record; this book quotes from both. By area, the most frequent subjects are `jit` (181 commits), `tests` (123, mostly binary refreshes), `syscall` (64), `dyld` (34), `vm` (25) and `mem` (14).

A test that pins every change

Every behavioural change carries the test that would have failed before it: a freestanding guest program with an expected output, a dynamically linked fixture, a unit harness, or a native-mode case. Expected outputs come from a real run, under Rosetta or natively, never from memory. The project goes further for its hardest rules: a test is expected to fail on the build just before the fix and pass after it, and several commit messages record that check.

Sign-off, and nothing else

Commits are signed off under the Developer Certificate of Origin (`git commit -s`); 83 commits carry a `Signed-off-by` line, the first on 23 September 2026. No other trailers are used. `CONTRIBUTING.md` writes all of this down, along with two more rules: `make check` must be green before a change is sent, and a claim in the README that a change invalidates is corrected in the same change.

The switches

AArchX is unusually configurable at run time, by design. At 9169680 the source reads 410 distinct `OCERZ_*` environment variables through `getenv` or its two caching wrappers, 162 of them in `jit.c` alone, and 124 are `OCERZ_NO_*` switches that each turn one optimisation or one behaviour off. They fall into a few families:

- **`OCERZ_NO_<FEATURE>`** turns one optimisation off: `OCERZ_NO_FLIP` (superblock retranslation), `OCERZ_NO_AFP` (hardware NaN results), `OCERZ_NO_LEAF_INPLACE` (the arm64 string routines). If a program misbehaves and one of these fixes it, the bug is in what that switch controls.
- **`OCERZ_<AREA>LOG`** and their relatives print one kind of event: `OCERZ_DLPATH`, `OCERZ_EXCLOG`, `OCERZ_JITLOCKLOG`, `OCERZ_PERFSTAT`, `OCERZ_GUESTPROF`.
- **Ranges** select part of a program: `OCERZ_INTERP_LO` and `OCERZ_INTERP_HI` keep an address range in the interpreter, which is how a miscompiled block is bisected in about an hour.
- **Per-process** switches pick one process of a tree by its command line: `OCERZ_NOJIT_EXE`, `OCERZ_STRACE_EXE`, `OCERZ_EXE_ENV`.

Each is read once, at first use, and cached. Most exist because one real bug needed exactly that view, and they stay because the next bug usually needs it too. `docs/reference.md` documents the ones meant to be used; Appendix A lists them, and Chapter 17 shows how they are combined.

Documentation in the tree

Since 23 September 2026 the documentation lives in `docs/`: getting started, the two modes, native mode in depth, the command-line and environment reference, architecture, performance, compatibility, troubleshooting and testing. The **Atlas** (`docs/atlas/index.html`, also published at `mont127.github.io/AArchX/atlas/`) is a guided tour of the source for contributors that assumes no C: the vocabulary first, then the life of a process, executing x86, the run loop, memory, the kernel boundary, cache mode's loader, native mode's bridge, how correctness is held, how to read a debugging session, and seven case files of real fixes. `CONTRIBUTING.md`, `CODE_OF_CONDUCT.md` (the Contributor Covenant 2.1) and `SECURITY.md` complete the set, with issue and pull-request templates under `.github/`.

PART II

The CPU

Turning x86 bytes into behaviour: the decoder, the reference interpreter, and the arm64 JIT that does the real work.

CHAPTER 4

The decoder

Every x86 instruction AArchX ever runs, interpreted or translated, 64-bit or 32-bit, first passes through one function that turns raw bytes into one fixed-size record. This chapter takes that function apart: the byte layout it parses, the prefix rules that are easy to get wrong, the record it produces, the 548 internal operations it maps onto, and the machinery that proves a change to it moved nothing it should not have.

4,440

LINES IN DECODE.C

548

 INTERNAL
OPERATIONS

96 B

 PER DECODED
INSTRUCTION

2²⁴

 OPENINGS COMPARED
PER CHANGE

One function, one format

The decoder lives in `src/decode.c` (4,440 lines) with its contract in `include/ocerz/decode.h` (643 lines). Its whole public surface is two functions:

```
int ocerz_decode(const uint8_t *code, size_t avail, uint64_t rip, X86Insn *out);
int ocerz_decode_mode(const uint8_t *code, size_t avail, uint64_t rip,
                      X86Insn *out, int mode32);
```

`ocerz_decode` is the 64-bit long-mode decoder and always has been; `ocerz_decode_mode` takes the mode explicitly, with `mode32 = 1` selecting i386 semantics for the 32-bit code Wine runs inside a WoW64 process. Both are pure: they read the byte buffer and nothing else. They never look at CPU registers, never touch guest memory through a side door, and never consult global emulator state, which is what makes it possible to test them exhaustively from a harness that links `decode.o` and `libc` and nothing more.

The result is an `X86Insn`, and that struct is the single exchange format between the decoder, the interpreter and the JIT. The interpreter executes an `X86Insn`; the JIT reads a run of them to plan a block; the diagnostic printers format them. Because both engines consume the same decode, they cannot disagree about what an instruction is, only about what it *does*, and the second kind of disagreement is exactly what the differential gates catch (Chapter 15).

A call returns one of four results:

Result	Meaning
<code>OCERZ_OK</code>	Decoded; <code>out->len</code> is the full length including prefixes.
<code>OCERZ_EUNDEF (-1)</code>	The byte string is not an instruction in this mode, or is one AArchX deliberately refuses.
<code>OCERZ_ETRUNC (-2)</code>	The bytes ran out before the instruction ended and fewer than 16 were available.

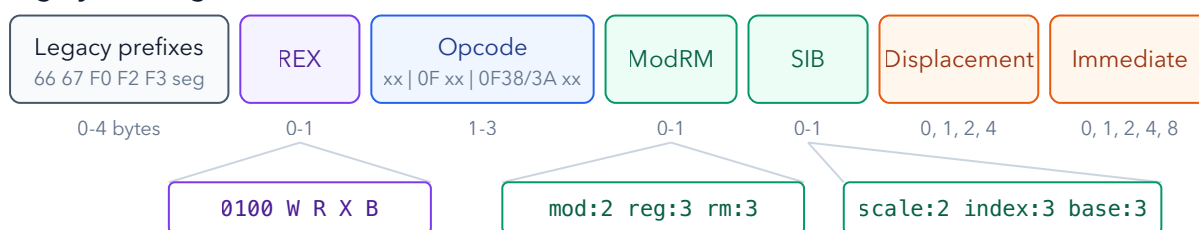
Result	Meaning
OCERZ_ET00 LONG (-3)	The instruction would be longer than 15 bytes, the architectural ceiling. A truncation with 16 or more bytes available is reported as this rather than as ETRUNC, because more bytes would not have helped.

Every read goes through four fetch helpers (`fetch8/16/32/64`) that stop at the end of a window of at most 16 bytes, so no path can read past what the caller said was there. A `NULL` code pointer is treated as what it is, an emulator bug upstream (every caller obtains the pointer through `ocerz_g2h`, the guest-to-host translation of Chapter 8): the decoder says so once on standard error with the address-map bases, and fails the decode instead of faulting.

Anatomy of an x86-64 instruction

An x86-64 instruction is a variable-length sequence of up to 15 bytes in a fixed order: optional legacy prefixes, an optional REX byte, one to three opcode bytes, and then, depending on the opcode, a ModRM byte, a SIB byte, a displacement and an immediate. AVX instructions replace the REX byte, the escape bytes and the mandatory prefix with a two- or three-byte VEX prefix. Figure 4.1 shows both forms.

Legacy encoding



VEX encoding (long mode only)



Figure 4.1. The two shapes of an x86-64 instruction. A VEX prefix folds the REX bits (stored inverted), the opcode map, the mandatory prefix, a third register operand (vvvv, also inverted) and the vector length L into two or three bytes.

The decode pipeline

A call proceeds in a fixed order, shown in Figure 4.2. A `DecState` carries the cursor and everything learned so far: which legacy prefixes were seen, the mandatory-prefix choice, the REX bits, the address-size mode, the VEX fields, and a pointer to the output record, which is zeroed first so that every field a path does not set has a defined value.

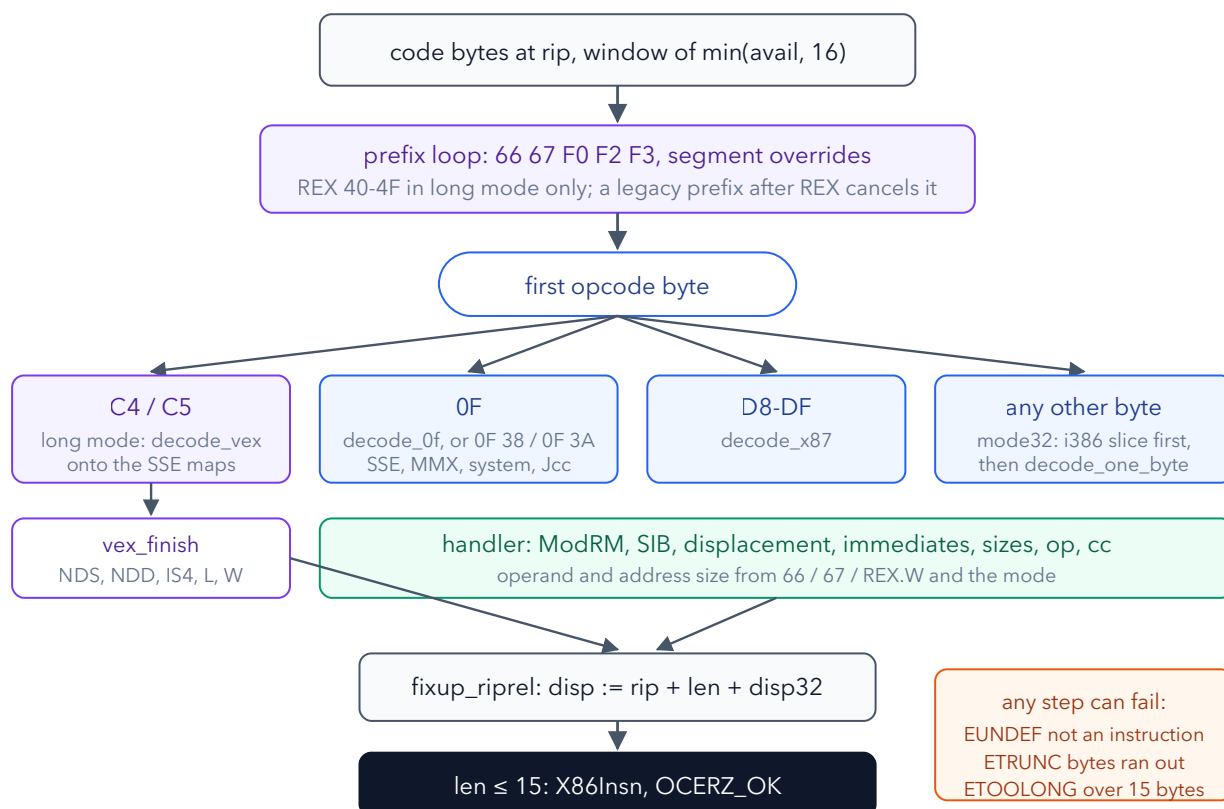


Figure 4.2. The path of one decode. Every opcode handler parses its own ModRM and immediates, because whether an operand exists, how wide it is and which register file it names all depend on the opcode.

Two design choices show in the figure. First, there is no table-driven generic decoder: each opcode or opcode group has a handler that knows its operand shapes, built from a small set of helpers (`alu_rm_r`, `alu_acc_imm`, `branch_rel`, `decode_sse_rr`, `decode_mmx_rr` and so on). The integer ALU block at `0x00–0x3D` is the one place where the encoding is regular enough to compute: the operation is `op >> 3` (add, or, adc, sbb, and, sub, xor, cmp) and the form is `op & 7` (register/memory, direction, byte or full width, accumulator with immediate). Second, the RIP-relative fix-up happens once, at the very end, because only then is the instruction length known.

Prefixed

The prefix loop is short, but nearly every line encodes a rule that has bitten someone.

- **REX must be last.** In long mode, bytes `0x40–0x4F` are REX prefixes and set the W, R, X and B bits. Hardware honours a REX only when it immediately precedes the opcode, so when a legacy prefix follows a REX, the loop clears the REX state it had recorded. In 32-bit mode the same bytes are `INC` and `DEC` of a 32-bit register, so the loop leaves them alone and the opcode dispatcher sees them.
- **Mandatory prefixes.** For SSE instructions, `66`, `F3` and `F2` are not modifiers but part of the opcode: `0F 58` is `addps`, `66 0F 58` `addpd`, `F3 0F 58` `addss`, `F2 0F 58` `addsd`. The loop records the choice in `mand`. An `F2` or `F3` always wins over `66`, and once one of them has been seen a later `66` no longer changes the mandatory prefix (it still sets the operand-size flag). Between `F2` and `F3` the last one seen wins.

- **Segment overrides.** In both modes only FS and GS carry a base that applies to addresses. 2E, 36, 3E and 26 (CS, SS, DS, ES) are consumed and recorded as no override; 64 and 65 set `insn.seg` to FS or GS. Thread-local storage on macOS uses GS, and Wine on macOS uses GS for the Windows TEB.
- 67 swaps the mode's default address size for the other size the mode can name: 64 and 32 in long mode, 32 and 16 in 32-bit mode. The result is `insn.addrsize`: 8, 4 or 2.
- F0 (LOCK) is recorded in `insn.lock`. The decoder does not check that the instruction is one LOCK may legally prefix; the engines act on the flag.
- REP state is held in `DecState` and copied into `insn.rep` only by the string instructions that use it, so an F3 that served as a mandatory prefix does not also make an SSE instruction look repeated.

ModRM, SIB and addressing

The ModRM byte splits into `mod` (2 bits), `reg` (3) and `rm` (3). REX.R extends `reg` to four bits and REX.B extends `rm`; `mod = 3` means `rm` names a register, and every other `mod` describes a memory operand. `decode_modrm` handles the special cases in one place:

Encoding	Long mode	32-bit mode
<code>rm = 100</code>	a SIB byte follows: <code>scale</code> , <code>index</code> , <code>base</code>	same
SIB <code>index = 100</code> , no REX.X	no index register	no index register
SIB <code>base = 101</code> , <code>mod = 00</code>	no base, <code>disp32</code>	same
<code>mod = 00</code> , <code>rm = 101</code>	RIP-relative <code>disp32</code>	absolute <code>disp32</code>
<code>mod = 01 / 10</code>	<code>disp8 / disp32</code> , sign-extended	same
under 67 in 32-bit mode	(cannot occur)	16-bit table: no SIB; bases BX/BP with optional SI/DI; <code>disp16</code> at <code>mod = 00</code> , <code>rm = 110</code>

The scale is stored as its two-bit logarithm, so the effective address is `disp + base + (index << scale)`. A RIP-relative operand is **pre-resolved**: after the whole instruction has been parsed, `fixup_riprel` rewrites the operand's displacement into the absolute guest address `rip + len + disp32`, clears base and index, and leaves the `riprel` bit set. No consumer ever re-derives the program counter, which matters to the JIT: a RIP-relative load is a load from a constant address, and Chapter 7 shows how translation-time constants let it skip run-time checks entirely.

The 16-bit addressing table is a different beast from the 32- and 64-bit one, with a fixed set of eight base/index pairs:

rm:	000	001	010	011	100	101	110	111
	BX+SI	BX+DI	BP+SI	BP+DI	SI	DI	BP*	BX
	(* <code>mod = 00</code> , <code>rm = 110</code> is <code>disp16</code> with no base instead)							

It exists only for 32-bit code under a `67` prefix, which real WoW64 programs rarely execute. The JIT leaves it entirely to the interpreter.

The X86Insn record

Figure 4.3 shows the record a decode produces. It is 96 bytes: a 24-byte header and three 24-byte operands.

X86Insn, 96 bytes



Header (one cell = 1 byte)



X86Operand (one cell = 1 byte)



kind: NONE, REG, XMM, ST, MEM, IMM, MMX. base / index 0xff = none. scale is log2. vex: PRESENT, L, W, NDS, NDD, IS4.

Figure 4.3. The decoded-instruction record. The header holds what applies to the whole instruction; each operand carries its own kind and size, which consumers trust over the header's operand size.

The conventions behind the record:

- **Each operand carries its own size** in bytes: 1, 2, 4, 8, 10 for an x87 extended-precision memory operand, 16 for an xmm register or a 128-bit memory operand. Consumers read `ops[i].size`, not `insn.opsize`, because the two differ in exactly the instructions that matter: `movzx` has a 2-, 4- or 8-byte destination and a 1- or 2-byte source.
- **The high-byte registers.** An 8-bit register encoding 4–7 with no REX prefix names AH, CH, DH or BH; it is stored as the base register (RAX, RCX, RDX, RBX) with `high8 = 1`. With any REX prefix the same encodings name SPL, BPL, SIL and DIL instead.
- **Immediates** arrive sign-extended to a 64-bit pattern, so a consumer can use them without knowing which immediate form was encoded. Only `mov r64, imm64` and the `moffs` forms carry raw 64-bit values, and the `moffs` offset width follows the `address` size, not always 64 bits (a bug fixed on 2026-08-19).
- **Branches carry their target.** A direct `jmp`, `jcc`, `call` or `loop` has a single immediate operand holding the absolute target, already masked to 32 or 16 bits when the operand size calls for it. Conditional instructions (`jcc`, `setcc`, `cmovcc`, `fcmovcc`) store the four-bit condition in `insn.cc`.
- **Segment registers have no operand kind.** `mov Sreg, r/m` (`MOVSEG`) encodes its destination as a one-byte immediate holding the segment index, `PUSHSEG` and `POPSEG` follow the same convention,

and the width actually moved on the stack is `insn.opsz`. The reverse direction, `mov r/m, Sreg`, is its own operation, `MOVFROMSEG`.

Twenty-four bytes per operand is generous, and the JIT does not keep it. A published translation stores 16 bytes per instruction (address, length and operation) and copies a full `x86Insn` only for the few instructions it still runs through the interpreter or must inspect during fault recovery. A GUI Wine process once carried about 870 bytes of decoded instructions per block across 215,000 blocks, 180 MB in all, before that change (Chapter 6).

Operand and address widths

The default operand size is 4 bytes; `66` makes it 2 and `REX.W` makes it 8, with `REX.W` winning when both are present. Stack and branch widths do not follow that rule, and they split three ways. The split exists because the i386 work had to be done without moving 64-bit decoding by a single bit (see the `decodiff` gate below):

Class	Examples	Long mode	32-bit mode
Explicit stack operand	<code>push / pop r</code> , <code>push imm</code> , <code>push / pop r/m</code>	8; <code>66</code> gives 2	4; <code>66</code> gives 2
Implicit stack traffic	<code>ret</code> , <code>leave</code> , <code>pushf / popf</code>	always 8 (the 16-bit forms are not implemented, and the gate requires that)	4; <code>66</code> gives 2
Near branches	<code>jmp / call rel</code> and <code>r/m</code> , <code>jcc</code> , <code>loop</code> , <code>jrcxz</code>	forced to 8, <code>66</code> ignored, as the SDM specifies	4; <code>66</code> gives 2

548 internal operations

The decoder does not hand its consumers x86 opcodes. It maps every instruction onto an internal operation, an `0cerzOp` enum value in a 16-bit field, and the engines dispatch on that. The enum's final member, `0CERZ_OP_COUNT`, is 548. New operations are always appended rather than inserted, so the enum reads as a history of the instruction set's arrival in AArchX:

Region	Values	Contents
Integer	1-102	moves, ALU, shifts and rotates, bit operations, control flow, far transfers, string operations, system and miscellany
x87	103-172	from <code>0CERZ_OP_X87_FIRST</code> : loads, stores, arithmetic, compares, transcendental functions, control
SSE	173-394	from <code>0CERZ_OP_SSE_FIRST</code> : SSE through most of SSE4.1, then AES-NI and PCLMULQDQ
i386-only	395-409	<code>pusha / popa</code> , <code>push / pop sreg</code> , the BCD adjusts, <code>bound</code> , <code>les</code> , <code>lds</code> , <code>into</code> , <code>salc</code> (August 2026)
SSE4.1/4.2 completion	410-419	<code>pmuldq</code> , <code>mpsadbw</code> , <code>dpps</code> , <code>crc32</code> , the four <code>pcmpXstrY</code> (2026-08-25)
Segment read	420	<code>MOVFROMSEG</code> (2026-09-05)

Region	Values	Contents
AVX and AVX2	421-470	VEX-only operations with no SSE twin: broadcasts, 128-bit inserts and extracts, permutes, <code>vzeroupper</code> , masks, gathers
FMA block	471-530	<code>OCERZ_OP_VFMA_FIRST</code> to <code>VFMA_LAST</code> : sixty reserved values, one per fused multiply-add form and operand order
Late extensions	531-547	BMI1/BMI2 (<code>andn</code> , <code>bsr</code> , <code>bzhi</code> , <code>pdep</code> , <code>pext</code> , <code>mulx</code> , <code>rorx</code> , <code>sarx</code> , <code>shlx</code> , <code>shrx</code> and the rest), <code>movbe</code> , <code>xsave</code> / <code>xrstor</code> , <code>rdrand</code>

Two rules govern how the enum grows, and both protect existing behaviour. The i386-only one-byte opcodes were appended at the end of the enum as it then stood, so no pre-existing operation was re-numbered and none of them is reachable from 64-bit decoding. AVX-only operations are always placed above `OCERZ_OP_SSE_FIRST`, because the interpreter's last dispatch layer, the SSE unit, takes everything at or above that value that the integer layers did not claim. The field holding the operation has been 16 bits wide since the enum passed 255: in June, when it held 364 values in an 8-bit field, every operation above 255 was silently truncated and dispatched to the wrong handler.

Several encodings that differ only in ways no program can observe collapse onto one operation, which keeps both engines smaller:

Encodings	Decoded as	Why it is safe
<code>andpd</code> , <code>orpd</code> , <code>xorpd</code> , <code>andnpd</code>	the <code>ps</code> forms	bitwise operations are identical on any element type
<code>movapd</code> , <code>movupd</code>	<code>movaps</code> , <code>movups</code>	moves do not interpret their data
<code>lddqu</code>	<code>movdqu</code>	an unaligned 16-byte load either way
<code>movntps</code> , <code>movntpd</code> , <code>movntdq</code>	ordinary stores	the non-temporal hint has no observable semantics
<code>endbr64</code> , the <code>0F 1F</code> multi-byte NOPs, <code>fnop</code> , every <code>prefetch*</code>	<code>nop</code>	no architectural effect

The opcode maps

x86 has four opcode maps reachable from legacy encoding: the one-byte map, the `0F` two-byte map, and the `0F 38` and `0F 3A` three-byte maps. AArchX implements all four, plus the x87 escape bytes `D8–DF`, which select a further eight-way table by the ModRM `reg` field and, for register forms, by the whole ModRM byte.

The integer SIMD opcodes in the `0F` map are shared between MMX and SSE2: with no mandatory prefix, `0F FE` is `paddb mm, mm/m64`; with `66` it is `paddb xmm, xmm/m128`. AArchX decodes both onto the same operation and lets the operand kind tell them apart: an MMX form has `OCERZ_OPK_MMX` operands of size 8, an SSE form `OCERZ_OPK_XMM` operands of size 16. That was done on 2026-09-13, and since 2026-10-04 most of those MMX forms are translated by the JIT as well. `punpcklqdq` and `punpckhqdq` exist only in the SSE2 form, so they are the two exceptions to that sharing.

VEX

VEX prefixes exist only in long mode for AArchX's purposes: in 32-bit mode `C4` and `C5` are `LES` and `LDS`. In long mode, `decode_vex` reads the prefix bytes and turns them into the state the legacy decoder already understands, then calls into the ordinary `0F`, `0F 38` or `0F 3A` handler:

- The inverted R, X and B bits become the REX extension bits (the two-byte `C5` form has only R; X and B are zero), and REX is marked present.
- `mmmmm` selects the map: 1, 2 and 3 are `0F`, `0F 38` and `0F 3A`; anything else is undefined.
- `pp` becomes the mandatory prefix, exactly as if `66`, `F3` or `F2` had been written.
- W becomes REX.W, L the vector length, and the inverted `vvvv` the extra register operand.

Because the legacy handlers do the work, a VEX instruction decodes into the same operation as its SSE twin. `vex_finish` then tags the record in `insn.vex` with bits saying how its operands relate:

Flag	Meaning	Examples
<code>PRES</code> <code>ENT</code>	encoded with VEX: write the full register and zero what lies above the result	every VEX instruction
<code>L</code>	256-bit (ymm) form	<code>vaddps ymm</code>
<code>W</code>	VEX.W	gather and permute variants
<code>NDS</code>	non-destructive source: the first source is <code>v vvv</code> , not the destination	arithmetic, logic, compares, unpacks, shuffles, FMA
<code>NDD</code>	the destination is <code>vvvv</code> ; the ModRM register is a source	shifts by an immediate (<code>vpsllw ymm, ymm, imm</code>); the decoder moves the operands so <code>ops[0]</code> is the destination
<code>IS4</code>	a fourth register operand in the immediate's high nibble	<code>vblendvps</code> , <code>vblendvpd</code> , <code>vpblendvb</code>

A 256-bit register operand is still an `OCERZ_OPK_XMM` of size 16: the width lives in the L bit. That is deliberate. The interpreter executes most VEX.256 instructions as two 128-bit halves (Chapter 5) and the CPU state keeps the upper halves of the ymm registers in a separate array, so "xmm register n, plus its upper half when L is set" is exactly the shape both engines want. Broadcasts and half-precision converts are the exception that needs a size fix-up: their memory operand is narrower than the register, so `vex_finish` sets it to 1, 2, 4, 8 or 16 bytes as the instruction requires.

i386 mode

Wine runs 32-bit Windows programs inside a 64-bit process by switching the CPU into 32-bit compatibility mode with a far transfer to a 32-bit code selector, and back again for system calls. AArchX therefore decodes i386 too, and the way it was added is the most instructive design decision in the file.

The i386 decoding had to be added without changing long-mode decoding by a single bit. Two earlier i386 proposals each shipped a silent 64-bit regression that the entire test suite missed, so the rule became structural: **the whole i386-only slice lives in one function, `decode_i386_only`, which the one-byte dispatcher calls only when `mode32` is set.** Every byte it owns is undefined in long mode, and it sets a

handled flag even for the architecturally invalid forms (`BOUND` , `LES` and `LDS` with a register operand), so those report `OCERZ_EUNDEF` from there instead of falling through to a generic path. Where a width differs between the modes, the 64-bit arm keeps the literal it always had and only the 32-bit arm is new.

Difference	Long mode	32-bit mode
<code>40 – 4F</code>	REX prefixes	<code>inc / dec r32</code>
Default operand / address size	4 / 8	4 / 4
<code>mod = 00, rm = 101</code>	RIP-relative	absolute <code>disp32</code>
<code>67</code>	32-bit addressing	16-bit addressing table
<code>06 0E 16 1E / 07 17 1F</code>	undefined	<code>push / pop ES, CS, SS, DS</code>
<code>C4, C5</code>	VEX	<code>les, lds</code> (far pointer: a 6-byte memory operand for the 32-bit form)
<code>27 2F 37 3F, D4 D5</code>	undefined	BCD adjust: <code>daa, das, aaa, aas, aam, aad</code>
<code>62</code>	undefined (EVEX prefix, not supported)	<code>bound</code>
<code>82</code>	undefined	alias of <code>80</code> (group 1, byte immediate)
<code>60, 61, CE, D6</code>	undefined	<code>pusha, popa, into, salc</code>
<code>9A, EA</code>	undefined	far <code>call / jmp ptr16:32</code> , recorded selector first

Segment registers are the subtle part. In long mode, reading a segment register is uninteresting, and for a long time `mov r/m, Sreg` was decoded as a load of a constant: `0x2B` for CS, `0x23` for SS, the values macOS uses. That is only true in long mode. In a WoW64 process, 32-bit code runs under a code selector from the local descriptor table, and Wine's `RtlCaptureContext` stores what it reads there into the context record that later decides which mode an `iretq` returns to. With the constant folded in, every 32-bit exception was dispatched as 64-bit code, overflowed its stack, and killed the process within a second of starting, which is what made Steam relaunch itself in a loop. Since 2026-09-05 selectors are CPU state that the interpreter reads, and `MOVFROMSEG` is a real operation.

What it refuses, and why that is right

An emulator that guesses is worse than one that stops. A byte string `AArchX` does not implement decodes to `OCERZ_EUNDEF`, and the interpreter then stops the process with the address, the mode and the raw bytes. Decoded and rejected, as of this writing:

- **EVEX and AVX-512** (`62` in long mode). Rosetta does not run AVX-512 either.
- **SHA extensions** and `rdseed`.
- **Privileged and I/O instructions**: `cli, in, out, lgdt` and the rest of `0F 00` and `0F 01`, which a user-mode program cannot execute anyway.
- **The BCD adjust instructions in long mode**, where they do not exist (they decode in i386 mode).

Decoding is also separate from *advertising*. CPUID (Chapter 5) reports SSE3, SSSE3, CX16, SSE4.1, SSE4.2 and POPCNT, every one implemented in full, and the synthesized `hw.optional` sysctls agree. AVX2, FMA, BMI1, BMI2, F16C, LZCNT, MOVBE and XSAVE decode and execute but are not advertised, which is exactly how Rosetta runs x86-64-v3 code on macOS 15 and later. Programs that probe CPUID see a conservative processor; programs compiled for x86-64-v3, or that use AVX2 without asking (Go does), still run. The project's rule since SSE4.2 became advertised is never to set a CPUID bit without the whole family behind it.

Proving a decoder correct

Length is the property that matters most. If one instruction's length is wrong by a byte, every later instruction in the block is decoded from the wrong place, and the result is garbage that may still look plausible for a while. Four layers of checking hold the decoder to account:

1. **Hand-written cases.** `tests/unit/test_decode.c` asserts length, operation and operand fields for 246 encodings (last counted in September 2026; it was 190 in June).
2. **The assembler as a length oracle.** `tools/gen_corpus.sh` assembles a list of instructions with the host's own `clang -arch x86_64 -c`, reads the bytes back out of the object file, and writes `tests/corpus/corpus.inc`: 511 rows of the form `X("add r/m32 r32", 2, 0x01, 0x08)`. `tests/unit/test_corpus.c` decodes each byte string and requires the decoder's length to equal the assembler's. Nothing is executed; the assembler is trusted only for how long an instruction is.
3. **decodiff, the no-change gate.** `tools/decodiff.c` links `decode.o` alone and sweeps every one of the 2^{24} three-byte openings, each followed by the same 13-byte tail so 16 bytes are always available. That covers every opcode, every prefix combination up to three deep and every ModRM and SIB pairing. Each decode is rendered as canonical *text*, with the operation by name rather than by number, and hashed. `tools/decodiff-cmp.sh` builds the harness against two trees and compares the digests: it prints `IDENTICAL -- 16777216/16777216 64-bit decodes match` or lists every difference. Recording names rather than numbers makes the digest immune to renumbering the enum, which is a legitimate change, while any change in what a byte string decodes to still shows. The script always rebuilds `decode.o`, because a stale object silently produces a false `IDENTICAL`, the worst failure a gate can have.
4. **dec32, the correctness gate for i386.** `decodiff` proves 64-bit decoding did not change, but is blind to whether new 32-bit answers are right. `tools/dec32probe.c` and `tools/dec32-oracle.py` feed the same byte strings to capstone 5.0.7 in 32-bit mode and compare field by field, including exhaustive 2^{24} length sweeps. The unit tests `test_decode32.c` and `test_decode32_addr.c` pin 102 decodes, 26 rejections and 122 addressing cases.

WHERE THE ORACLE IS WRONG

capstone is the reference for the i386 tests, and it is documented as wrong in two places. With a 16-bit operand size the SDM has a near branch clear the upper two bytes of EIP, which capstone does not model, so the capstone-checked rows keep their targets below 64 KB. And `LES / LDS` load a far pointer, so their memory operand is two bytes plus the operand size, six bytes for the 32-bit form, while capstone reports four and prints no size keyword. A reference tool is a witness, not a judge.

Bugs that shaped it

Date	Bug	What it broke
June 2026	The operation field was 8 bits wide while the enum held 364 values	Every operation above 255 was dispatched as a different one.
2026-08-19	<code>moffs</code> offsets were always read as 64 bits	32-bit <code>mov eax, [moffs]</code> consumed four bytes too many.
2026-09-05	<code>mov r/m, Sreg</code> folded to a long-mode constant	32-bit exceptions in WoW64 ran as 64-bit code; every Steam process died at once.
2026-09-06	<code>lddqu (F2 0F F0)</code> was not decoded	Chromium's renderer uses it to scan UTF-8; every renderer Steam launched died with "decode failed".
2026-09-08	<code>movntps / movntpd (0F 2B)</code> missing	SDL3 stopped on them.
2026-09-08	<code>0F 6D</code> decoded as <code>punpcklq dq</code>	Every Cocoa window was squashed to 74 pixels high (below).

HOW IT WAS FOUND: THE 74-PIXEL WINDOWS

For a long time every titled window in every translated Cocoa process came up 74 pixels high, and Steam's login window logged `AfterCreated ... 700x74`. The `0x60–0x6D` case in the `0F` map mapped `0x6C` to `punpcklq dq` and let `0x6D` fall through a default arm to the same operation: a bug present since the initial commit. With the high quadword never moved down, a geometry calculation inside the window system came out wrong, the screen's visible frame became a 74-pixel strip, and AppKit constrained every window to it.

It was found with three five-second probes instead of six-minute Steam runs: a small native AppKit program that printed the visible frame (wrong, and identically wrong under the interpreter, so not a JIT bug), then `OCERZ_MSGHEX` showing that the kernel's reply to the window server request was correct. If the data arrived right and the interpreter was wrong too, only the shared front end was left. The guest test `punpck_qdq` now pins every `punpck[h]dq` form, with the low and REX-extended registers.

The pattern in that table is the reason the decoder is held to such a strict standard. A decoding bug is shared by both engines, so the differential gate cannot see it; it shows up as a wrong result far away from the instruction that caused it. The defences are the ones described above: an assembler that knows the lengths, a disassembler that knows the 32-bit fields, an exhaustive sweep that proves nothing else moved, and, for meaning, guest tests whose expected output comes from running the same binary under Rosetta.

CHAPTER 5

The interpreter and x86 flags

The interpreter is slow on purpose and correct on purpose. It defines what every x86 instruction means inside AArchX, it runs whatever the JIT declines, and it is the oracle every translation is checked against. This chapter covers the CPU state it operates on, how one instruction is executed, the exact flag formulas and the deferred form the JIT stores instead, and the corners of the x86 architecture where getting the semantics right took a real program failing first.

5,652

 LINES ACROSS INTERP,
EXT AND SSE

7,968 B

 PER GUEST CPU
(OCERZCPU)

513

 LINES OF FLAG
SEMANTICS AND
LIVENESS

20,033

 1386 SEQUENCES
CHECKED AGAINST IT

Three jobs for one engine

A translator could exist without an interpreter, by translating everything. AArchX keeps one, and gives it three distinct jobs.

1. **Reference semantics.** For every instruction there is exactly one C implementation of what it means, in `src/interp.c`, `src/interp_ext.c` or `src/interp_sse.c`. The JIT's emitted arm64 must produce the same architectural state, and the differential gate checks it: every guest test binary is run with `-no-jit` and with the JIT, and the outputs must match byte for byte. Because the interpreter is the reference, any divergence is a JIT bug by construction (Chapter 15).
2. **Fallback.** The JIT does not translate everything. x87 floating point, the SSE4.2 string instructions, string operations, `cuid`, system calls, far transfers, misaligned locked atomics and a list of rarer forms are executed by the interpreter, one instruction at a time, from inside translated code. The JIT calls these *slow calls*.
3. **Vantage point.** Because it is the slow path, the interpreter is the right place to watch from. Trace switches that need to see each instruction live here, and `OCERZ_INTERP_LO / HI` can force any address range out of the JIT and into the interpreter, which is how a JIT miscompile is narrowed to one block (Chapter 17).

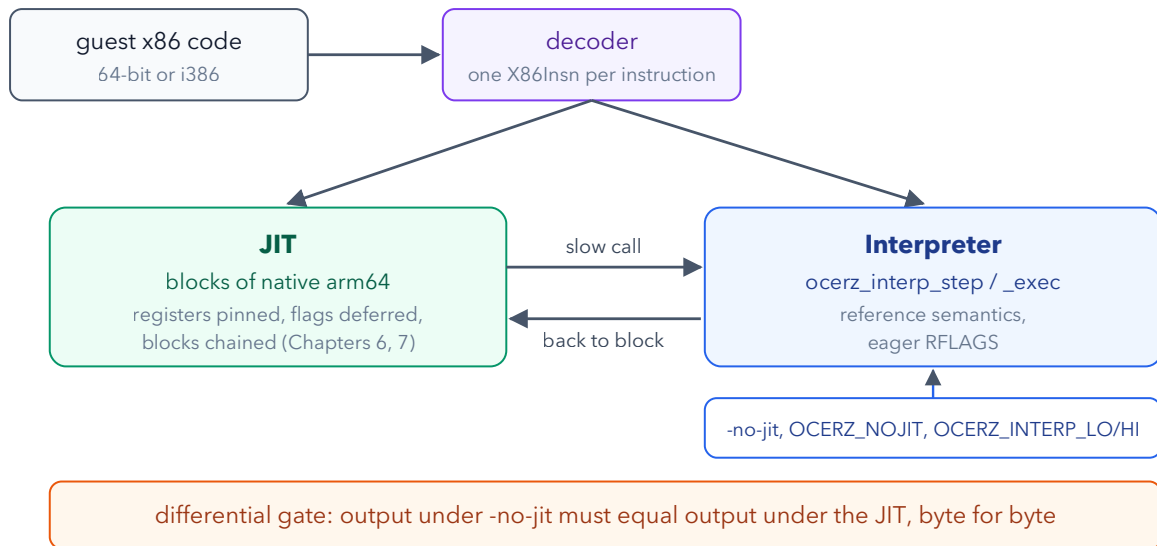
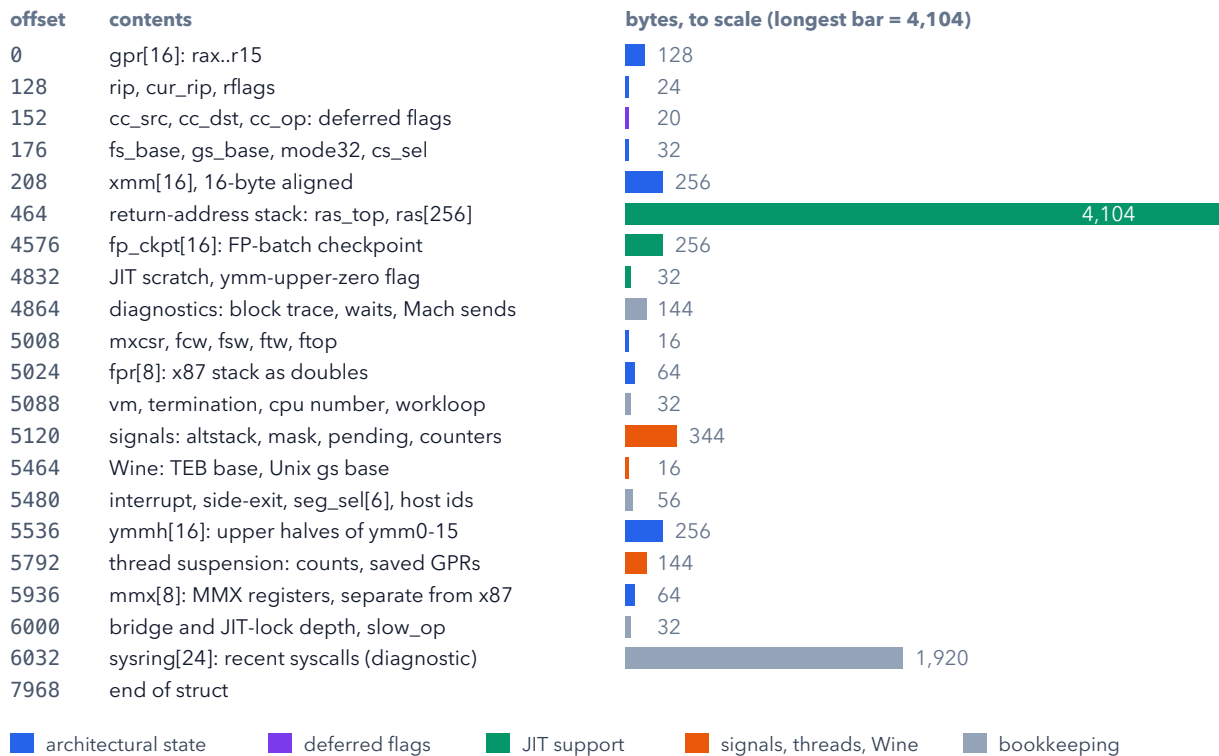


Figure 5.1. One decoder, two engines. The JIT runs what it can translate and calls the interpreter for single instructions it cannot; the interpreter can also run a whole process or chosen address ranges, which is what the differential gate and JIT bisection rely on.

The CPU state

Each guest thread has one `OcerzCPU`, declared in `include/ocerz/cpu.h`. Measured with `sizeof` at 9169680, it is 7,968 bytes. Most of it is not architectural state at all: it is per-thread bookkeeping that has to live where the JIT can reach it with one instruction. Figure 5.2 maps it.



Field order is load-bearing: the JIT addresses these fields with immediate offsets, so nothing that exists may move.

Figure 5.2. The per-thread CPU state, by region. The return-address stack dominates; the architectural registers fit in under 900 bytes.

The field order is load-bearing, and `cpu.h` says so first. The JIT reaches every field with immediate-offset loads and stores off one pinned base register, and arm64 immediates have limited reach: `xmm[]` is 16-byte aligned so that scaled 128-bit loads and stores can address it, and the return-address stack sits right after it so its first entries stay within the ± 504 -byte reach of a load-pair or store-pair instruction on the call and return paths. When the segment selectors were needed for WoW64, they were appended at the end for the same reason: nothing that already existed, above all neither `xmm[]` nor the stack window, could move by a byte. The same discipline added the latest diagnostic, a 24-entry ring of recent system calls, at the very end on 2026-10-02.

A few conventions matter for everything that follows:

- **General registers** are indexed in x86 encoding order (RAX = 0, RCX = 1, RDX = 2, RBX = 3, RSP = 4, RBP = 5, RSI = 6, RDI = 7, then R8–R15), so a decoded register number is directly an index.
- **Vector registers** are split: `xmm[16]` holds the low 128 bits and `ymmh[16]` the upper 128 bits of each ymm register. Legacy SSE instructions leave `ymmh` alone, VEX.128 instructions zero it, and VEX.256 instructions write it, exactly as the hardware's upper-state rules say.
- **x87** lives in `fpr[8]` as C `double`s with `ftop`, `fcw`, `fsw` and `ftw` beside them, and the eight MMX registers have their own array, `mmx[8]`, rather than aliasing the x87 stack (both choices are discussed below).
- **Reset state** is what a fresh Darwin x86-64 thread sees: RFLAGS `0x202` (the always-one bit and IF), x87 control word `0x037F`, MXCSR `0x1F80`, CS selector `0x2B` and SS `0x23`.

Executing one instruction

`ocerz_interp_step` executes the instruction at `cpu->rip`. Stripped of its diagnostics, it is short:

```
int ocerz_interp_step(0cerzVM *vm, 0cerzCPU *cpu)
{
    ocerz_flags_materialize(cpu);          /* a JIT-deferred flag record becomes RFLAGS */
    if (rip is in the dyld-API trap window)
        return ocerz_dyldapi_dispatch(vm, cpu); /* Chapter 11 */

    X86Insn insn;
    rc = ocerz_decode_mode(ocerz_g2h(cpu->rip), 15, cpu->rip, &insn, cpu->mode32);
    if (rc != OCERZ_OK)
        report the bytes, the module, recent rips; return OCERZ_STEP_FATAL;

    vm->insn_count++;
    cpu->cur_rip = cpu->rip;
    cpu->rip    += insn.len;                /* rip is ALREADY the next instruction */
    if (insn.mode32) cpu->rip = (uint32_t)cpu->rip;
    rc = ocerz_interp_exec(vm, cpu, &insn);
    if (cpu->mode32) cpu->rip = (uint32_t)cpu->rip; /* the mode may have changed */
    return rc;
}
```

Three details carry weight. First, **rip advances before the handler runs**, so a non-branching handler never touches it, a branch simply overwrites it, `call` pushes it, and `syscall` publishes it in RCX as the hardware does. Second, the **decode mode is chosen in exactly one place**, from `cpu->mode32`; no handler ever asks which mode it is in, because the decoded instruction already says. Third, **EIP is re-wrapped after execution** keyed on the mode the CPU is in *afterwards*, because a far transfer that just left 32-bit mode has legitimately produced a full 64-bit `rip`.

`ocerz_interp_exec` is the dispatch half. It is layered by file: the integer core in `interp.c` handles what it owns; anything else falls to `ocerz_interp_ext` (string operations, `cpuid`, `rdtsc`, CRC-32C, x87, FXSAVE and XSAVE, MXCSR access, the bit-manipulation extensions); operations at or above `OCERZ_OP_SSE_FIRST` fall to `ocerz_interp_sse`; and anything left reaches `ocerz_unimpl`, which stops with the operation name, the address and the raw bytes. The JIT's slow calls enter at `ocerz_interp_exec` with an already-decoded instruction, so for every instruction the JIT hands over there is exactly one implementation.

A step returns one of five results, two of which are not ordinary control flow:

Result	Meaning
<code>OCERZ_STEP_OK</code>	executed; continue
<code>OCERZ_STEP_EXIT</code>	the guest exited
<code>OCERZ_STEP_FATAL</code>	undecodable or unimplemented; the process stops with a report
<code>OCERZ_STEP_REDIRECT</code>	a non-branching instruction changed <code>rip</code> because a guest signal was delivered; the caller must re-dispatch
<code>OCERZ_STEP_PROFILER</code>	a profiled JIT side exit came back to C only to be counted; continue

Reading and writing operands

No handler touches an operand directly. `include/ocerz/interp_common.h` holds the inline accessors, and it is the one place the easy-to-get-wrong rules live:

- A **32-bit register write zero-extends** into the upper half, including `cmovcc` with a false condition, which still writes its destination. 8- and 16-bit writes merge, and `high8` addresses bits 8–15.
- `ocerz_ea` computes `disp + base + (index << scale)`, truncates to 32 bits under a 32-bit address size, and then adds the FS or GS base. `lea` has its own path, because it must never add a segment base.
- **The stack pointer is RSP in long mode and ESP in i386 mode**, so in 32-bit mode every push and pop wraps at 2^{32} . The mode comes from the decoded instruction, never from a test in the handler.
- **Wine's TLS slot.** The absolute form `gs:[0x58]` is special-cased. Wine on macOS runs 64-bit PE code with GS pointing at the Darwin thread-specific data and mirrors the TEB fields PE code reads into Wine's reserved slots, but `ThreadLocalStoragePointer` is the one field that changes after the mirror is taken, so a thread whose snapshot preceded its TLS setup read zero and MSVC `__declspec(thread)` code crashed. Slot 6 holds the TEB's self pointer on Wine threads and zero elsewhere, so reading through it yields the live field and selects itself.

x86 flags, computed eagerly

x86 arithmetic instructions write six status flags in RFLAGS as a side effect. AArchX's definition of them is `src/flags.c` (230 lines): one helper per instruction family, each taking the operand width in bytes and the inputs and result, and each rewriting the six arithmetic flags while leaving the control bits (TF, IF, DF) alone.

Flag	Bit	Meaning
CF	0	carry out of (or borrow into) the most significant bit
PF	2	even parity of the <i>low byte</i> of the result
AF	4	carry or borrow between bits 3 and 4 (used by BCD arithmetic)
ZF	6	the result, at the operation's width, is zero
SF	7	the result's most significant bit
OF	11	signed overflow

The formulas, with all values masked to the operation's width first:

- **ZF, SF, PF** come from the result alone. PF looks only at the low eight bits, whatever the width.
- **ADD and ADC:** CF if `res < a`, or `res == a` with a carry in. OF from the top bit of `~(a ^ b) & (a ^ res)`: two operands of the same sign produced a result of the other sign. AF from bit 4 of `a ^ b ^ res`.
- **SUB, SBB, CMP, NEG:** CF if `b > a`, or `b == a` with a borrow in. OF from the top bit of `(a ^ b) & (a ^ res)`. AF as for add.
- **AND, OR, XOR, TEST** set ZF, SF and PF from the result and clear CF, OF and AF.

- **INC and DEC never touch CF.** OF is positional: INC overflows exactly when it lands on the minimum signed value, DEC on the maximum. AF when the low nibble wrapped to 0 (INC) or to F (DEC).
- **Shifts** are never called with a count of zero, because a zero-count shift leaves every flag unchanged and the caller skips the helper. SHL: CF is the last bit shifted out of the top, OF is CF xor the result's top bit. SHR: CF is the last bit out of the bottom, OF the original top bit. SAR: CF from the sign-extended value, OF zero.

The worked examples below are 8-bit, where the flags are easy to check by hand:

Instruction	Result	CF	PF	AF	ZF	SF	OF	Why
add 0x7F, 0x01	0x80	0	0	1	0	1	1	positive plus positive gave negative; nibble carried
add 0xFF, 0x01	0x00	1	1	1	1	0	0	unsigned wrap; zero has even parity
sub 0x00, 0x01	0xFF	1	1	1	0	1	0	borrow; eight one-bits is even parity
sub 0x80, 0x01	0x7F	0	0	1	0	0	1	most negative minus one overflows
cmp 0x05, 0x05	(0x00)	0	1	0	1	0	0	equal
and 0xF0, 0x0F	0x00	0	1	0	1	0	0	logic clears CF, OF, AF
inc 0x7F	0x80	=	0	1	0	1	1	CF unchanged; landed on the minimum
shl 0x81, 1	0x02	1	0	0	0	0	1	bit 7 shifted out; OF = CF xor top bit
mul 0x10, 0x10	0x0100	1	0	0	0	0	1	high half non-zero; the other four cleared

Conditions

`ocerz_cc_eval` turns RFLAGS and a four-bit condition code into true or false. x86 arranges its sixteen conditions as eight pairs that differ only in bit 0, so the function computes a base predicate from `cc >> 1` and inverts it when bit 0 is set:

cc >> 1	Conditions (bit 0 = 0 / 1)	Predicate
0	O / NO	OF
1	B / AE	CF
2	E / NE	ZF
3	BE / A	CF or ZF
4	S / NS	SF
5	P / NP	PF
6	L / GE	SF ≠ OF
7	LE / G	ZF or SF ≠ OF

Undefined flags follow Rosetta

The architecture leaves some flags undefined after some instructions. "Undefined" does not mean a program cannot observe them, and AArchX's tests compare against the same binary run under Rosetta, so the rule since 2026-09-03 is to produce what Rosetta produces:

- **MUL and IMUL** define only CF and OF, set when the high half carries information (non-zero for MUL, not the sign extension of the low half for IMUL). SF, ZF, AF and PF are undefined, and Rosetta leaves all four clear whatever the result and the flags going in, so the helpers clear them. The June code had derived them from the low half.
- **CMPXCHG** sets its flags as for `dest - accumulator` and writes the accumulator in both outcomes, so the 32-bit form zero-extends RAX on a match as well.
- **DIV and IDIV** leave flags architecturally undefined, but the interpreter produces specific values and the differential gate compares them, so the liveness table treats them as defined (below).
- **System-call returns** reproduce Rosetta's fixed patterns for the arithmetic flags, probed on 2026-09-03: PF and AF after a successful Unix call, CF, ZF and PF after a failed one, AF after a Mach trap. Only CF carries meaning (libSystem's stubs branch on it), but a program that reads the rest sees what it would see under Rosetta. Machdep calls leave the flags alone (Chapter 9).

The deferred record

The interpreter computes flags eagerly, after every instruction that writes them, because it is the reference and simplicity is the point. The JIT cannot afford to: most flags are never read. It computes flags only where something reads them, usually by fusing a compare directly into an arm64 conditional branch (Chapter 6). When it cannot fuse but also cannot prove the flags dead, it stores a **deferred record** instead: three fields in the CPU state.

```
uint64_t cc_src;      /* first input                */
uint64_t cc_dst;      /* second input, result, or count */
uint32_t cc_op;       /* kind | size << 8 | carry_in << 16 */

/* kinds: OCERZ_CC_ADD, _SUB, _LOGIC, _INC, _DEC, _SHL, _SHR, _SAR, _MUL, _IMUL */
```

`ocerz_flags_materialize` turns a pending record into RFLAGS by calling the very same helpers the interpreter uses, then marks the record empty. That gives one formula per flag in the whole project, and it is why the interpreter's step function starts by materializing: when execution falls from translated code into the interpreter, the record left behind becomes ordinary RFLAGS before the next instruction can read them. A record is also materialized for `pushf`, for a signal frame, for a fault, and for any condition the JIT did not fuse.

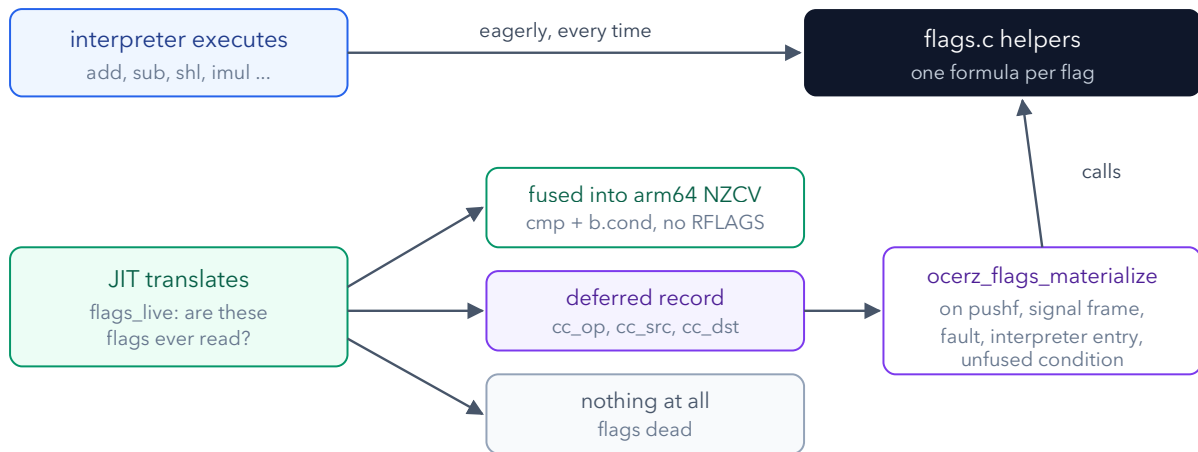


Figure 5.3. Three fates for a flag in translated code, one formula for all of them. Whatever the JIT does, a flag that is finally read is computed by the same `flags.c` helper the interpreter uses.

Flag liveness

Whether a flag is ever read is decided by `src/flags_live.c` (283 lines), a table of what each instruction *defines* and *uses*. The JIT runs a liveness pass over each block with it, and a flag that no later instruction uses before the next definition is dead and never computed. Every entry is a claim about what an instruction may do, so the safe direction is always "**defines less, uses more**": overstating a definition could kill a flag something reads, while overstating a use only costs a little speed.

Instruction class	Defines	Uses
<code>moves</code> , <code>lea</code> , <code>not</code> , <code>push/pop</code> , <code>call/ret</code> , <code>xchg</code> , string moves, <code>cpuid</code>	nothing	nothing
<code>ADD</code> , <code>SUB</code> , logic, compares, <code>popcnt</code> , <code>rdrand</code> , <code>xadd</code> , <code>cm</code> , <code>pxchg</code>	all six	(<code>ADC/SBB</code> use CF)
<code>bt</code> , <code>bts</code> , <code>btr</code> , <code>btc</code>	CF	nothing
<code>bsf</code> , <code>bsr</code>	ZF only	the rest stay as they were
<code>tzcnt</code> , <code>lzcnt</code>	CF, ZF	nothing
BMI: <code>andn</code> , <code>blsr</code> , <code>blmsk</code> , <code>blsi</code> , <code>bzhi</code>	CF, ZF, SF, OF	nothing
Shifts by <code>%cl</code> , rotates, <code>SHLD/SHRD</code> by a register	may define	all six
<code>SHLD/SHRD</code> by an immediate	all six, or nothing if the count masks to zero	nothing
SSE and x87 compares into RFLAGS, <code>ptest</code>	all six	nothing
<code>fcmovcc</code>	nothing	CF, ZF, PF
the rest of x87 and SSE, every VEX instruction	nothing	nothing

Two rows deserve explanation. A shift by `%cl` is a *may-define*: a count that masks to zero leaves every flag untouched while any other count writes them, and that cannot be known when translating, so the instruction is treated as both using all flags and not reliably defining them. An audit on 2026-08-18 found stale flags exactly there. And **DIV and IDIV are treated as defining** flags the architecture calls undefined, because the interpreter produces specific values and the differential gate compares them; "undefined" is not "killed".

The table can also make every memory-touching instruction a **fault barrier**, forcing all flags live across it so that a guest fault handler sees exact flags even where they are dead. That was the original design, and it cost real speed, because it forced flag producers to keep records that nothing in the program's own code ever read. Since 2026-08-15 the barrier is opt-in (`OCERZ_FAULT_FLAG_BARRIER=1`): flags that are live are always recoverable in a fault, and only flags that are dead may be stale in a signal context. The commit that made it opt-in, which also classified most x87 and SSE instructions as flag-neutral, took the `vm` benchmark kernel from 3.9 to 1.6 times Rosetta's time. (The comment at the top of `flags_live.c` still describes the barrier as unconditional.)

Integer corners

Misaligned locked atomics

x86 makes a LOCK-prefixed access atomic at any alignment, because the bus or cache-line lock covers a split access. arm64's LSE atomics, and clang's `__atomic_*` built on them, fault on a misaligned address. Wine's `CRITICAL_SECTION` inside a packed Valve structure sits at offset `+0x446`, so every `EnterCriticalSection` in Steam's CEF host is a `lock cmpxchg` on a doubleword that is 2 mod 4. The JIT sends misaligned locked accesses to the interpreter, which takes one of 256 spin locks chosen by address `((addr >> 3) & 255)` and performs the read-modify-write with plain unaligned copies. That is atomic against every other misaligned access to the same location, which is the guarantee that actually matters. Aligned locked accesses use the real atomic instructions.

Divides, fast-fail and the stack pointer

- **A divide fault is a real fault.** `rip` stays at the `div`, and the guest's handler sees SIGFPE with `FPE_INTDIV` or `FPE_INT0VF`, as on Darwin; with no handler the process dies.
- **int 0x29** (Windows' `__fastfail`) ends the process in the interpreter rather than being handed to the guest. Dispatching it as an exception so Wine could raise `STATUS_STACK_BUFFER_OVERRUN` was tried and measured worse: the thread that fast-fails is already deep, the dispatch overflowed its stack, and it wedged holding ntdll's loader lock, so the process could never exit.
- **pop into [rsp+disp]** computes its address *after* the pop has adjusted the stack pointer, as the SDM says. V8's `TailCallRuntime` trampoline moves its return address with `pop qword [rsp+0x98]`; computed with the old `rsp` it landed one slot low and V8 returned into a stale heap pointer (fixed 2026-09-08, guest test `pop_rsp_mem`). `pop rsp` takes its new value from the slot, not from the adjustment.

Strings and bits

String instructions count in rCX and step rSI and rDI at the **address size**: the 64-bit registers at 8, the 32-bit ones at 4, and the 16-bit ones at 2, the last only in 32-bit code with a 67 prefix, written back through the register accessor so only the low half changes. A whole `rep` loop runs inside one step. `cmps` and `scas` take their flags from the last iteration executed, and a zero-count `rep` leaves the flags untouched. `bt` and its relatives with a memory operand and a register bit offset address memory outside the operand: the byte at `ea + (offset >> 3)`, sign-extended. `bsf` and `bsr` on a zero source set ZF and leave the destination unchanged; `tzcnt` and `lzcnt` return the operand width with CF set.

CPUID and the clock

Leaf	Value
0	highest leaf 0xD, vendor GenuineIntel
1	EAX 0x000306A9 (family 6, model 58), EBX 0x00100800, ECX 0x00982201, EDX 0x078BFBFF
7	zero: no extended features advertised
0xD	XSAVE state: subleaf 0 EAX 7 (x87, SSE, AVX), area 0x340 bytes; subleaf 2 AVX state 0x100 bytes at 0x240
0x80000001	ECX 0x1 (LAHF/SAHF in long mode), EDX 0x28100800
0x80000002 - 4	brand string 0cerz x86_64 Emulated CPU

ECX 0x00982201 is SSE3, SSSE3, CX16, SSE4.1, SSE4.2 and POPCNT, each implemented in full. SSE4.2 has been advertised since 2026-08-25, because Steam's bootstrapper refuses to start the client on a processor without it; the rule since then is never to advertise a bit without the whole family behind it. AVX2, FMA, BMI1, BMI2, F16C, LZCNT and MOVBE execute without being advertised, matching what Rosetta does on macOS 15 and later. `rdtsc` returns `mach_absolute_time()` converted to nanoseconds: monotonic, but not a cycle counter.

The x87 unit

The x87 stack is eight C `double`s, with the stack top, control word, status word and tag word beside them; it is not 80-bit extended precision. `fld m80` converts the ten-byte format to the nearest double and `fstp m80` converts back, so precision beyond 53 bits is lost. The difference is observable: `max_double * max_double` computed in `long double` returns infinity rather than about 3.2×10^{616} . In practice almost no current macOS software uses x87 arithmetic, and `OCERZ_X87LOG` counts how many 80-bit conversions a process actually performs, which is how a Steam crash was once ruled out as an x87 problem. FXSAVE and FXRSTOR use a self-consistent layout backed by those doubles, so a save round-trips only through AArchX's own restore, and per-register tags are not modelled. The JIT translates no x87 instruction; they all run here.

The MMX registers are kept apart from the x87 stack, in `mmx[8]`, rather than aliasing its mantissas as the hardware does. Programs that use both in one stretch without `emms` are broken on real hardware anyway, and the separation keeps both units simple; the cost is that FXSAVE and signal frames do not carry the MMX registers.

SSE and AVX

`src/interp_sse.c` (2,542 lines) covers scalar and packed floating point, every compare predicate, the conversions with x86's "integer indefinite" results, the integer SIMD set, AES-NI, PCLMULQDQ, the SSE4.2 string instructions, and the AVX forms layered on top. Alignment is not enforced: `movaps` on a misaligned address behaves like `movups` instead of faulting. `rcpps` and `rsqrtps` return exact results instead of x86's 12-bit approximations, a deliberate deviation listed in Chapter 18.

NaNs the x86 way

x86 and arm64 disagree about NaNs, and programs do notice. The interpreter applies x86's rule to add, subtract, multiply, divide and square root: if the result is a NaN, then if the first source (the destination register) was a NaN the result is that NaN, quieted; otherwise if the second source was a NaN, that one quieted; otherwise the operation was invalid and the result is the *negative* default NaN, which x86 calls the real indefinite.

Operation (double)	x86 and AArchX	arm64 without FPCR.AH
<code>0.0 / 0.0</code>	<code>0xFFF8000000000000</code>	<code>0x7FF8000000000000</code>
<code>inf - inf</code>	<code>0xFFF8000000000000</code>	<code>0x7FF8000000000000</code>
<code>1.0 + SNaN 0x7FF0000000000001</code>	<code>0x7FF8000000000001</code>	<code>0x7FF8000000000001</code>
<code>QNaN 0x7FF8000000000005 + SNaN 0x7FF0000000000001</code>	<code>0x7FF8000000000005</code> : the first operand	<code>0x7FF8000000000001</code> : the signalling one wins

MIN and MAX keep x86's own rule: the second operand is returned when either operand is a NaN, or when both are zero. On processors that report `FEAT_AFP`, one bit of the arm64 floating-point control register (FPCR.AH) makes the hardware itself produce the x86 column of this table, and the JIT then emits arithmetic without software checks (Chapter 7). The interpreter's C code produces x86's answers either way.

Rounding modes

The rounding-control field of MXCSR is honoured since 2026-09-11: `ocerz_apply_mxcsr_round` drives the host's rounding mode, which both the interpreter's C arithmetic and the JIT's arm64 instructions obey. The mapping goes through the C library's `fe*` constants rather than copying bits, because the two architectures number the directed modes the other way round:

MXCSR RC	Meaning	FPCR RMode for the same mode
<code>00</code>	to nearest, ties to even	<code>00</code>
<code>01</code>	toward $-\infty$	<code>10</code>
<code>10</code>	toward $+\infty$	<code>01</code>
<code>11</code>	toward zero	<code>11</code>

Before that change, `divsd`, `sqrtsd` and friends always rounded to nearest whatever a program asked for. A second bug was found while testing it on 2026-09-21: the main thread of a dynamically linked program never received the guest's floating-point control state at all, because it enters through the guest-call loop

and only the run loop applied it. `tests/dynamic/nan_contexts.c` now checks NaN results and rounding on the main thread, a pthread, inside and after a signal handler, on libdispatch's threads, after an MXCSR write, and across `fork`.

VEX: one implementation, two halves

A VEX.128 instruction is the legacy SSE operation with its first source taken from `vvvv` instead of the destination, and the upper half of the destination ymm zeroed. So the legacy implementations are reused, with the first source chosen per encoding, and the merge forms take their untouched part from `vvvv` rather than from the destination. A VEX.256 instruction is executed twice: once on the low 128-bit halves, and once with the involved registers swapped for their entries in `ymmh[]` and memory operands 16 bytes further on. Operations that cross the 128-bit lanes cannot be split that way and are written out in full: widening and narrowing conversions, broadcasts, 128-bit inserts and extracts, permutes, half-precision conversions, and the mask and test instructions.

The SSE4.2 string instructions

`pcmpestri`, `pcmpestrm`, `pcmpistri` and `pcmpistrm` compare a 16-byte "needle" against a 16-byte "hay-stack" block in one of four ways. AArchX implements them from the architectural immediate layout directly rather than as a list of named mnemonics:

imm8 bits	Field	Values
[1:0]	element format	unsigned or signed, bytes or words
[3:2]	aggregation	equal any, ranges, equal each, equal ordered (substring search)
[5:4]	polarity	positive, negative, masked positive, masked negative
[6]	output selection	least or most significant index (<code>i</code> forms); bit or byte mask (<code>m</code> forms)

The explicit-length forms take the lengths from EAX and EDX; the implicit forms find them at the first zero element. The JIT does not translate these instructions, so this C code is their only implementation.

HOW IT WAS FOUND: STEAM'S MISSING LOCALIZATION

Under AArchX, Steam logged in and then stopped on an assertion, `strtools.cpp` (898) : `pSrc != NULL`, after logging `ILocalize::AddFile()` failed to load file `resource/vgui_%language%.txt`. Under Rosetta the same installation loaded its localization. Steam builds that file name by finding `%language%` in `public\steambootstrapper_%language%.txt` with its own SSE4.2 `strstr`, and in the buffer it searched, the needle started five bytes before the end of a 16-byte block.

In equal-ordered aggregation the architecture compares the needle against the block only while the needle position is still inside the block, so a needle that starts near the end counts as a partial match there, and SSE4.2 `strstr` relies on exactly that to find a needle that straddles two blocks. AArchX kept comparing past the end of the block, treated the missing bytes as a mismatch, and cleared the bit; `pcmpistri` answered 16 (not found) where the hardware answers 11. The fix (2026-09-26) is one loop bound, `j + i < n`. The guest test `sse42_test` gained three cases whose expected values were recorded under Rosetta.

i386 execution

The interpreter never asks per instruction whether it is in 32-bit mode; the decoded instruction carries the mode, and the step function chooses the decode mode once from `cpu->mode32`. Mode changes happen only through far transfers, and every far transfer (`jmp far`, `call far`, `retf`, `iret`) funnels through one helper, `far_transfer`, so the mode, the CS selector and EIP/RIP can never disagree. Which mode a selector means is decided from its descriptor in the local descriptor table that Wine installs through the `i386_set_ldt` call: a descriptor with the long-mode bit selects 64-bit code, one with the default-size bit 32-bit code.

The details that bite are the ones the SDM states and that are easy to miss:

- **A selector slot holds 16 bits** and hardware ignores the rest. Wine's `I386_CONTEXT` stores `SegCs` and `SegSs` as doublewords that `RtlCaptureContext` fills with a 16-bit store, so their upper halves are stack garbage by the time the WoW64 layer pushes them for an `iretq`. Taking the full 32 bits picked a garbage selector.
- **A zero CS or SS on the stack** is a frame AArchX built itself, not a mode decision, so the current selector is kept.
- **FS and GS** are the two segments whose base applies to addresses, so loading one moves its base; ES, SS and DS are flat and carry no base.
- **BCD**: `das`'s second test has no else branch, so a CF raised by the first adjustment survives when the second does not fire, and the adjust instructions leave every flag the SDM calls undefined untouched.

The i386 path is checked by its own differential gate: 20,033 generated instruction sequences run under both engines, comparing all sixteen registers at full width, the flags, the instruction pointer, the mode, the selectors, the whole floating-point and vector register file and every byte of memory touched (Chapter 15).

What the JIT leaves to the interpreter

The interpreter's share of a real program's time is small, but its list of responsibilities is long, and it changes as the JIT grows. As of 2026-10-04:

Executed by the interpreter	Why
All x87 instructions	rare in current macOS software; the double-based model is simplest in C
<code>pcmpXstrY</code>	complex, data-dependent control; rarely hot
String instructions (<code>movs</code> , <code>stos</code> , <code>lods</code> , <code>scas</code> , <code>cmps</code>), with or without <code>rep</code>	a whole <code>rep</code> loop runs in one call; the hot string routines are replaced by native ones anyway (Chapter 7)
<code>cuid</code> , <code>rdtsc</code> , <code>rdtscp</code> , <code>xgetbv</code>	answered by emulator state
<code>syscall</code> and <code>int</code>	enter the system-call layer (Chapter 9)
Far transfers	change the mode; must stay in one helper
Misaligned locked atomics	arm64 atomics cannot do them
MMX leftovers: <code>pmaddwd</code> , <code>pmovmskb</code> , <code>emms</code> , the SSSE3 MMX forms, MMX/SSE conversions, shifts by a register	most MMX has been translated since 2026-10-04

Executed by the interpreter	Why
VEX leftovers: 256-bit byte and integer shuffles, unpacks, permutes and lane inserts, <code>vptest</code> , immediate blends, mask-producing compares, gathers, <code>bextr</code> , <code>pdep</code> , <code>pext</code>	not yet translated
16-bit addressing; i386 instructions outside the JIT's 32-bit whitelist	rare

A slow call costs far more than the instruction it executes: the JIT has to write back the guest registers it keeps in host registers, call into C, and reload them. That is why the list above shrinks whenever a profile shows one of its entries in a hot loop. Most of the 2026-09-27 and 2026-10-04 additions to the JIT came from exactly such profiles, taken with `OCERZ_GUESTPROF`, which names the interpreted instruction forms a thread spends its time in (Chapter 17).

A NOTE ON THE SOURCE COMMENTS

Several explanation blocks in these files predate the JIT's growth. `interp_sse.c` and `cpu.h` still say the JIT translates no VEX instruction, and `interp_ext.c` that it does not translate CRC-32C. Both stopped being true in September 2026 (VEX on 2026-09-13; `crc32`, AES-NI and PCLMULQDQ on 2026-09-27). The code, not those sentences, is what this chapter describes.

CHAPTER 6

The JIT: translation, registers and control flow

The interpreter defines what an x86 instruction means; the JIT is what makes AArchX fast enough to run Steam and games. This chapter covers how guest code becomes arm64: the code arena, the dispatcher, how a block is built, the register layout every block shares, how flags are avoided rather than computed, and how blocks are stitched together by chains, return stacks and superblocks. Floating point, SIMD, memory ordering, code that changes under the translator and the translation cache are in Chapter 7.

20,555

LINES IN SRC/JIT.C

1 GBCODE ARENA
(MAP_JIT)**1,048,576**

HASH BUCKETS

256INSTRUCTIONS PER
BLOCK, AT MOST

Where the design came from

The first JIT, in June 2026, was call-threaded. It decoded a block once, emitted a native function for it with `vm` in `x19` and `cpu` in `x20`, inlined only the no-op family and register moves against a memory-backed register file at `[x20 + reg*8]`, and called the interpreter through `ocerz_jit_exec_one` for every other instruction. It ran about 2.9 times faster than the interpreter, with one 64 MB arena, a 65,536-bucket hash and no invalidation at all.

The real translator arrived in two waves. Between 16 and 23 July the JIT gained lazy flags, call and return inlining, chaining infrastructure, a return-address predictor and a first register allocator; the commit of 17 July recorded the `alu` kernel going from 4.46 to 2.42 times Rosetta's time, and the one of 23 July claimed rough parity. Then, on 15 August, a single day of more than twenty commits rebuilt it around the ideas this chapter describes: an in-arena dispatch stub, inline indirect calls with per-site caches, inline predicate evaluation from deferred flags, compare-and-branch fusion, xmm registers pinned in `v16-v31`, and full pinning of all sixteen general registers in fixed host registers ("pin class 3"). Superblocks followed on 16 and 17 August, the host-stack return predictor on 17 August, guest `rsp` as a host pointer on 25 August, trace inversion on 3 September. On an Apple M5 in September, the libSystem-linked benchmark build beat Rosetta on 14 of its 15 kernels in cache mode (Chapter 16).

The interpreter never went away. Anything the JIT declines to encode runs through it one instruction at a time, from inside a translated block, and every guest test runs both ways and must agree byte for byte (Chapter 15). That pairing is what made the aggressive optimizations below safe to attempt: each one can be switched off by an `OCERZ_NO_*` variable, and a miscompile can be cornered by bisecting addresses and switches against the interpreter (Chapter 17).

The code arena

`ocerz_jit_create` reserves one region with `mmap(PROT_READ|PROT_WRITE|PROT_EXEC, MAP_PRIVATE|MAP_ANON|MAP_JIT)`, 1 GB by default (`OCERZ_JIT_CODE_MB` or `OCERZ_JIT_CODE_KB` override it; the test suite runs some programs in a 64 KB arena on purpose, so that almost every block takes the slow path). If `MAP_JIT` fails, the process simply runs interpreted. On Apple silicon a `MAP_JIT` region is writable or executable per thread, never both, so every write to the arena is bracketed by `pthread_jit_write_protect_np(0)` and `(1)`, and every written range is pushed through `sys_icache_invalidate` before it can run. That applies to the one-word patches as much as to whole blocks: a chain activation writes four bytes and invalidates four bytes.

The arena fills front to back and is never compacted in place:

- **The head** holds a copy of `src/leaf.s`, the hand-written arm64 string and memory routines (Chapter 7), so that a translated block can reach them with a single direct branch.
- **Two dispatch stubs**, one for 64-bit and one for 32-bit code, are emitted the first time a block of that mode is translated.
- **Veneer pools**, 256 KB each, are carved out every 64 MB of code (up to sixteen of them) for chains that cannot reach their target with a plain branch.
- **Translated blocks** are appended at `code_cur`. A block that is retired stays where it is, on a retired list, so a thread still running inside it runs to its next exit; its bytes are only reclaimed when the whole arena is flushed, which since 4 October happens when less than an eighth of it, at most 8 MB, is left (Chapter 7).

Everything that describes a block lives outside the arena, in ordinary `malloc` memory. A `JitBlock` holds the block's key, three entry points into its code (`code`, `body_code` and `body_noreload`, explained below), its outgoing edges (at most eight), the list of blocks that chain into it, per-instruction lane and fault-recovery records, and a compact description of its instructions. Until 6 September a block kept the full 96-byte decoded `X86Insn` of every instruction; a GUI Wine process carried about 870 bytes of that per block across 215,000 blocks, 180 MB in all. A published block now keeps 16 bytes per instruction (address, length, operation) and full copies only of the instructions it still sends to the interpreter or inspects during fault recovery.

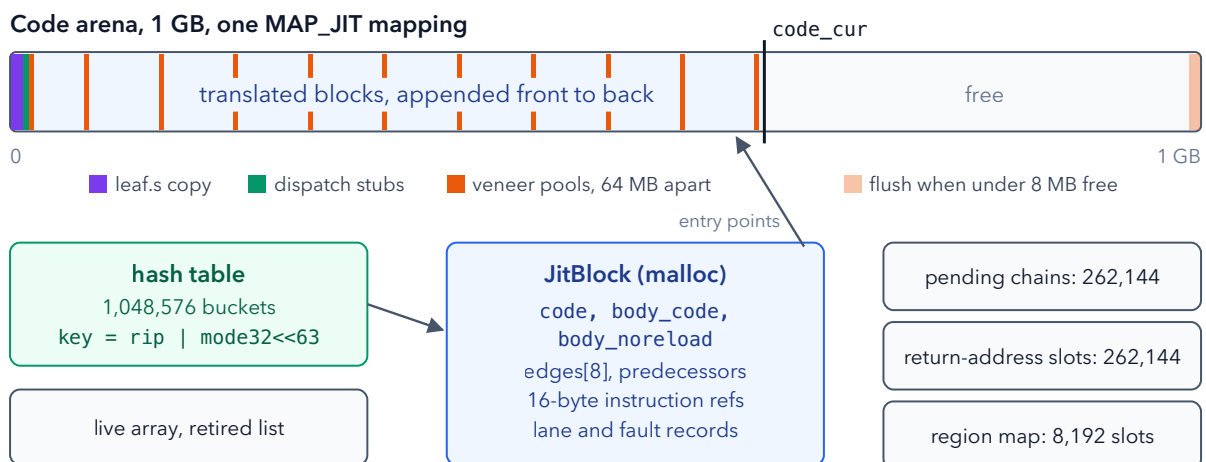


Figure 6.1. The arena holds only machine code; everything that describes a block lives in malloc memory beside it. The tables at the bottom are the ones a lookup, a chain activation, a return continuation and an invalidation consult.

Finding a block

Blocks are keyed by `jit_key(rip, mode32)`, the guest address with bit 63 set for 32-bit code. The same address in 32-bit and 64-bit mode is different code, and under Wine's WoW64 both run in one process, so the two may never share a cache entry, an invalidation mark or a commpage mark. The key is hashed with a 64-bit finalizer (shift right by 33 and XOR, multiply by `0xff51afd7ed558ccd`, shift right by 29 and XOR) into 2^{20} buckets, each a singly linked chain of `JitBlocks`.

Lookups take no lock: a bucket head is read with an acquire load, and a block is published into its bucket with a release store only after its code and metadata are complete, so a reader on another thread sees either nothing or a finished block. Translation does take a lock, the JIT lock, and `ocerz_jit_step` looks the key up a second time after acquiring it, because another thread may have translated the same block meanwhile. Since 12 September every acquisition of that lock is a `trylock` loop with yields and short sleeps that never parks the thread in the kernel: a thread parked on it could lose its wakeup and wedge translation for the whole process, which is what some Steam launches did. `OCERZ_JITLOCKLOG=1` reports any wait over three seconds with the holder's thread, the site that took the lock, the holder's guest `rip` and its kernel thread state.

The run loop and the dispatcher

Every guest thread runs a loop in `src/vm.c`. If the thread owes one interpreted instruction (`cpu->interp_once`, set by fault recovery and a few other paths), it interprets one. Otherwise it calls `ocerz_jit_step`, and if that answers `OCERZ_EUNSUP` it interprets one instruction with `ocerz_interp_step` and goes round again. That fallback is how the JIT declines things: a block that cannot be translated, an address that is kept in the interpreter on purpose, an arena flush in progress.

`ocerz_jit_step` itself does little. It refuses addresses inside the dyld API trap window (Chapter 11), credits a superblock side exit that came back to it for profiling, checks whether an arena flush has been requested, looks the block up, translates it under the lock if it is missing, and then calls the block as a C function, `b->code(vm, cpu)`. Around that call it bumps a per-thread count of translated frames, which is how the arena flusher knows when no thread is inside translated code (Chapter 7). A block that was translated with no code, because the translator gave up on it, runs through `jit_interp_block` instead.

The value a block returns is a step code: `OCERZ_STEP_OK` (zero) means "the guest `rip` in the `cpu` is where to continue", `OCERZ_STEP_PROFILE` means a probed side exit was taken, and the others mean exit or a fatal error. The common case never reaches C at all. A block's shared exit tests the code it is about to return; if it is zero, instead of returning it branches to the **dispatch stub**, a small arm64 routine in the arena (since 15 August) that does what `ocerz_jit_step` would have done for a hit. It loads the `cpu`'s interrupt word, its termination flag and the VM's exit flag and returns to C if any is set; it returns to C if the new `rip` is in the dyld API window; otherwise it hashes the `rip` exactly as `hash_key` does, walks at most six entries of the bucket, and branches straight to the matching block's full entry. Only a miss, or a block without code, goes back to the C loop. The branch from a block's exit to the stub is range-checked: until 18 August it was assumed to be in range, and after 128 MB of appended code it silently encoded a different instruction and the process died of `SIGILL`.

Building a block

`translate` does the same steps for every block. It first declines outright in a few cases: the address is in a range that has churned too often (Chapter 7), or a developer has asked for it to be interpreted with `OCERZ_INTERP_LO/HI` or `OCERZ_INTERP_RIP`, or it is the throw site an exception trap is watching. If the translation cache is on and holds a record for this key, the record is loaded instead (Chapter 7).

Decoding

The translator decodes up to 256 instructions with `jit_decode`, under a `sigsetjmp` guard so that a block running into an unmapped page simply ends there instead of crashing the translator. A block normally ends at its first terminator: any jump, conditional jump, `loop` or `jrcxz`, call, return, far transfer, `syscall`, `int`, `int3`, `ud2` or `hlt`. Three exceptions make blocks longer than the x86 basic block:

- A **direct call** to a short straight-line function may be spliced in place, its body decoded into the caller's block (see "Calls and returns").
- A **conditional branch** may be passed over, making the block a superblock: decoding continues on the fall-through path and the taken side becomes an out-of-line exit stub, up to six per block.
- A conditional branch that profiling has marked for **inversion** is rewritten while decoding: its condition is complemented and its target swapped with the fall-through, so the hot path continues inline.

Planning

Before any code is emitted, the translator decides the block's shape. It chooses the pin class (class 3, full pinning, for 64-bit code), which guest base registers are used often enough to hoist into dedicated host registers, which xmm registers are pinned (all sixteen, in practice), whether the block needs the guarded memory forms because it has faulted on the commpage or on a misaligned ordered access before, whether it should use the fixed-lane mode reserved for self-looping blocks, and where the flags each instruction defines are live. It also makes sure the dispatch stub exists and that a veneer pool has been carved if the arena has crossed another 64 MB mark.

Emitting and publishing

The body is emitted one guest instruction at a time. Each emitter either produces inline arm64 or emits a *slow call*: spill the pinned registers the C call would clobber, call `ocerz_jit_exec_one` with the decoded instruction, reload. After the body come the exit stubs and the shared exit; then the code is made executable, the instruction cache is invalidated, the block goes into the hash table, and any chains that were waiting for this address are activated (see "Chains").

Register pinning

Full pinning is the single decision that shapes everything else. Every guest general register lives in a fixed host register for the whole body of every block, and the slot equals the guest register number:

Guest	Host	Saved across C calls?
<code>rax rcx rdx rbx rsp rbp rsi rdi</code> (0-7)	<code>x21 - x28</code>	yes, callee-saved
<code>r8 - r13</code> (8-13)	<code>x3 - x8</code>	no: spilled and reloaded around every call-out

Guest	Host	Saved across C calls?
r14 , r15	x1 , x2	no: spilled and reloaded around every call-out
xmm0 - xmm15	v16 - v31	no: spilled by contract (Chapter 7)

Because every block uses the same layout, a transition between two blocks costs nothing. A chained branch can jump from the body of one block straight into the body of the next with every guest register already where the next block expects it; a return-address stack entry can be a plain pointer into a body; no block boundary pays for frame setup. The prologue and epilogue are paid only when C enters translated code and when translated code goes back to C.

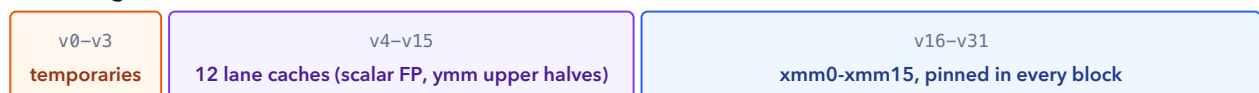
The rest of the register file is assigned too:

- **x19 and x20** hold the `vm` and `cpu` pointers for the whole body. Every access to architectural state that is not pinned (the flag record, `rip`, the interrupt word, the software return stack) is an offset from `x20`.
- **x0** holds the guest base, called `JGB` in the source (since 15 August). For a dynamically linked program, whose guest addresses are host addresses, it is zero; for a static program it is the arena's offset.
- **x9-x15** are the translator's temporaries, named `JT0`, `JT1`, `JT2`, `JTF`, `JTT`, `JTU` and `JTA`. `x15`, the computed-address temporary, is never live across a block edge, which is why veneers may use it.
- **x16, x17, x29 and x30** hold hoisted memory bases inside a body (`JMEMBASE2`, `JMEMBASE`, `JMEMAUX`, `JMEMBASE3`). `x30` is free there because a guest `ret` takes its host continuation from the shadow stack and the epilogue restores the real link register from the frame. When no base is hoisted, `x16`, `x17` and `x30` also serve as rename registers for spliced pushes and pops.
- **x18** is never touched: it is the platform register on Apple's ABI.
- **v0-v3** are vector temporaries and **v4-v15** are twelve lane registers that hold scalar floating-point results and the upper halves of `ymm` registers inside loops. Their callee-saved lower halves, `d8-d15`, are saved by the prologue.

General registers



Vector registers



□ guest register, callee-saved host register

■ state pointers and guest base

□ hoisted memory bases, push/pop renames

□ guest register, spilled around C calls

■ translator temporaries

* `rsp` is held as a host pointer: `guest_base + rsp`

Figure 6.2. Pin class 3. Every block agrees on this layout, so control passes from body to body without saving or restoring anything.

Guest `rsp` as a host pointer

Since 25 August the pinned `rsp` does not hold the guest value at all; it holds `guest_base + rsp`, a host pointer. A guest `push` then becomes one pre-indexed store, `str x10, [x25, #-8]!`, and a `pop` one post-indexed load, with no base addition and no separate update of the stack pointer. An `rsp`-relative memory operand needs no base either. The price is paid only where the value of `rsp` is read or written as data (a `mov rax, rsp`, an `lea from rsp`), where the accessor subtracts or adds the base, and at the boundaries: spilling to the cpu, filling from it, and rebuilding guest state after a fault. `OCERZ_RSP_VALUE` restores the older form.

What happens around a call into C

A slow call, a helper call or a bridged call has to respect the C calling convention, and the C callee is free to destroy `x0-x18` and the vector registers. The translator therefore spills exactly what is not callee-saved: the six pinned registers in `x3-x8` and the two in `x1-x2` go to the `cpu` struct and come back afterwards, `x0` is reloaded with the guest base, any hoisted memory bases are recomputed, and the `xmm` pins are written back and reloaded. The eight registers in `x21-x28` survive without being touched, which is why the most used guest registers were put there. Every call-out also increments a sequence number at translation time; the push/pop rename logic below uses it to know whether a call-out happened between a push and its pop.

Anatomy of a block

Each translated block is one C-callable function with three entry points, laid out as in Figure 6.3.

- **The full entry** (`JitBlock.code`) is where C, or the dispatch stub, enters. It saves `x29/x30` and `x19/x20`, moves the `vm` and `cpu` arguments into `x19` and `x20`, loads the guest base into `x0`, saves `d8-d15` and the callee-saved `x21-x28` it is about to overwrite, loads all sixteen pinned guest registers from the `cpu`, converts `rsp` into a host pointer, records the current host stack pointer in `cpu->jit_fp` as the frame base and pushes a sentinel pair `{0, 0}` onto the host stack, and loads the sixteen `xmm` registers into `v16-v31`.
- **The body entry** (`body_code`) is where chained blocks enter. It reloads the hoisted memory bases and nothing else.
- **The no-reload entry** (`body_noreload`) skips even that. A chained predecessor whose hoisted bases have the same *hoist signature* (which guest registers, which index and scale, which displacement) branches here, because its host registers already hold exactly what this block would load.

A self-looping block has its loop head aligned to 32 bytes (`OCERZ_LOOP_ALIGN`) by sliding the body forward and filling the gap with `nop`s, and it does not poll for interrupts on every iteration: as described under "Stop sites", a patched branch serves instead.

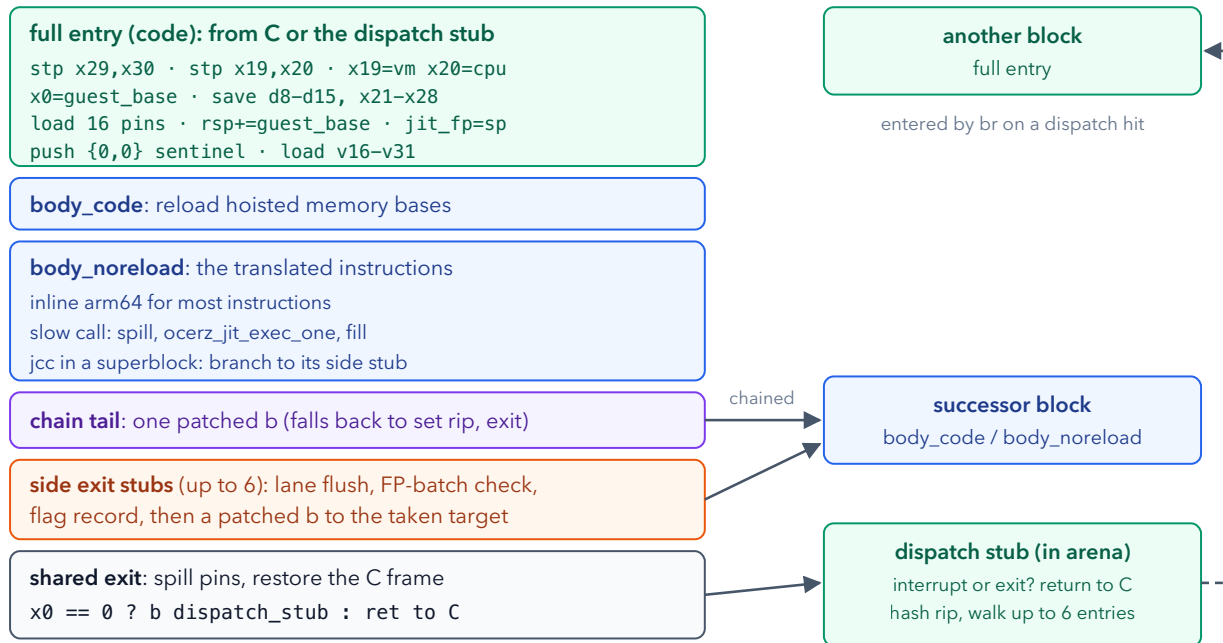


Figure 6.3. One block's code. Chained edges go body to body and never pass the prologue or the epilogue; only unchained exits reach the dispatch stub, and only misses there reach C.

Addressing guest memory

Most of what a block does is load and store, so the shape of an address computation matters more than almost anything else. How a guest address becomes a host address depends on the memory map the process uses (Chapter 8):

- **Identity map.** A dynamically linked program, the usual case, runs with a guest base of zero: every pinned register already holds a host address, and `mov eax, [rdi+16]` is a single `ldr w21, [x28, #16]`. With a zero base, the pinned register itself serves as the hoisted base.
- **Static arena.** A statically linked program has a non-zero guest base in `x0`, and an access becomes an addition and a register-offset load, or uses a hoisted base: a guest register used as a base two or more times in a block is held already offset in `x17`, `x16` or `x30`, and `base + index<<scale` can be kept in `x29` and recomputed at the loop head.
- **The low shadow.** Wine's processes keep their first 12 GB of guest memory at a different host address, so every access that is not provably to the stack needs its address translated by range; Chapter 7 covers how that went from about twenty instructions to one `orr` on 4 October.

The commpage guard

One page range breaks the identity map even in the best case: the guest's commpage at `[0x7fffffe00000, 0x7fffffe04000]`, which the host will not let a process map at that address, so AArchX keeps its contents elsewhere (Chapter 8). Guarding every load against it would be ruinous, so guarding is adaptive. Blocks are translated with plain accesses; one that touches the commpage faults on an unmappable address, the fault handler sees the address inside the commpage range, serves the access from AArchX's copy, and marks the block, which is retranslated with a guard in front of each access that might reach the range.

Since 23 September a first commpage or alignment fault retires only the faulting block, through its list of predecessors, instead of scanning every live block: Electron has about 140,000 blocks live at startup, and one scan per fault had made retiring blocks the largest single cost on Discord's main thread.

Constant addresses

For a rip-relative or absolute operand, the address is a constant of the translation, so the questions a guard asks at run time can be answered when the block is built (since 22 September). If the constant is not in the commpage, the guard is skipped; if it is, it becomes a plain redirect. In ordered memory mode, where an acquire load or release store faults if it crosses a 16-byte granule, an aligned constant goes straight to the ordered access. Measured over Adobe AIR, a rip-relative load had been 18 host instructions and a frame-relative one 17; a global load or store now takes four in either mode. A global access is the most common single form in compiled code, and Brawlhalla ran 13% more blocks per second at the same CPU. The "known address" fact is set by the guard and consumed by the very next ordered access, and building any other effective address clears it, so it can never describe the wrong address.

Flags without computing flags

Nearly every x86 arithmetic and logic instruction writes six status flags; almost none of those values are ever read. AArchX's interpreter computes them eagerly, because it is the reference (Chapter 5). The JIT computes them only when something reads them, and most of the time not even then.

The deferred record

An instruction that defines flags records what would be needed to compute them: a kind (add, subtract, logic, increment, decrement, the three shifts, the two multiplies), an operand size and a carry-in, packed into `cc_op` as `kind | size<<8 | cin<<16`, plus the result and source operand in `cc_dst` and `cc_src`. On the hot path the record is usually not even written to memory: the values sit in the registers that produced them, and the record is stored into the `cpu` struct only on an exit, a side stub or a call-out that leaves the block while the flags are still live. A consumer inside the block, a `jcc`, `setcc`, `cmovcc` or `adc`, evaluates its predicate inline from the record's operands. Anything else that needs the full RFLAGS value (a `pushf`, a signal frame, the interpreter) calls `ocerz_flags_materialize`, which computes them from the record exactly as the interpreter would have.

Liveness

Whether a flag is live is decided by a static pass over the block using the def/use table in `src/flags_live.c`, and the emitters consult the same predicates, so the pass and the code generator cannot disagree. The table errs in one direction only: an instruction may be described as defining less than it does and using more, never the reverse. Flags that x86 leaves undefined after `div` are treated as defined, because the interpreter produces specific values and the differential gate compares them. A variable shift by `cl` only *may* define flags, since a count that masks to zero leaves them alone. Any instruction that touches memory is a fault barrier: a guest fault handler must see the flags the hardware would have left, so they are live across it. To avoid paying for that on the hot path, inlined memory accesses carry a cold *fault recipe* that rebuilds the flags from the producer's operands if the access faults.

Liveness also reaches past the block (since 18 July): the flags at an exit are dead if the successor, decoded up to three blocks deep, overwrites them before reading them. That lookahead turned out to be about a fifth of all translation time during Steam's startup, because every block branching to an untranslated successor decoded it again; since 27 September its answers are memoized by address, depth and mode in 8,192 slots. An entry is trusted only while no translation has been retired since it was made and while the successor's first sixteen bytes are unchanged, which also catches code being rewritten before it was ever translated. Translation per block went from 5.78 to 4.85 microseconds over a Steam startup.

Three fusions

1. **NZCV forwarding.** When a compare-like producer (`cmp`, `sub`, `add`, `test`, `and`, `or`, `xor`, `bsf`, `bsr` or a bit test) is followed, within at most three flag-neutral "gap" instructions, by a `jcc`, `setcc` or `cmovcc`, the producer is emitted as an arm64 instruction that sets NZCV and the consumer reads NZCV directly: `cmp` then `b.ne`.
2. **Value-based conditions.** When the producer wrote a 32- or 64-bit pinned register and the condition only asks about the result (zero, sign), the condition is taken from the result register itself: `cbz`, `cbnz` or a compare against zero, with no flags at all. Additions, subtractions, logic, increments, decrements, negation and shifts by a non-zero immediate qualify.
3. **The `comis` fusion.** After `ucomisd` or `comisd`, a `jcc`, `setcc` or `cmovcc` re-derives its condition by redoing the floating-point compare with `fcmp`, mapping x86's unordered rules onto arm64's.

Deferred: the flags leave the block	
x86	arm64
<code>add rax, rbx</code>	<code>add x21, x21, x24</code>
<code>mov rcx, [rdi]</code>	<code>ldr x22, [x28]</code>
<code>jmp next</code>	<code>exit stub:</code>
	<code>cc_op = ADD 64</code>
	<code>cc_dst = x21 (result)</code>
	<code>cc_src = x24</code>
A successor that reads ZF evaluates it from the record.	

Fused: NZCV forwarded over a gap	
x86	arm64
<code>cmp rax, rbx</code>	<code>cmp x21, x24</code>
<code>lea r15, [rsi+8]</code>	<code>add x2, x27, #8</code>
<code>jne L</code>	<code>b.ne L</code>
The gap: at most 3 instructions, flag-neutral, writing only pinned registers, never a register the compare read. It is emitted into a scratch buffer first and reused there if it breaks a rule.	

Figure 6.4. Two ways the JIT avoids computing x86 flags. Left: a record that is written only when the flags leave the block. Right: the compare's own NZCV result consumed directly by the branch.

Rules written by bugs

Every legality rule around these fusions was learned from a miscompile in real software:

- **A gap may not write a register the producer read.** A byte compare through an index register that the very next instruction overwrote made the forwarding fallback re-load the byte through the new index value, and a loop spun forever.
- **A gap's emission may touch only pinned registers.** In `cmp byte [rdi+0x210], 0 ; lea r15, [rsp+0x290] ; jne`, the `lea`'s pointer-`rsp` form loaded its base through the temporary that held the compared byte, so the branch tested `rsp` instead of the byte. Steam's CEF browser process copied an unengaged `std::optional` and crashed on its first navigation (6 September). Since 7 September a gap is emitted into a scratch buffer first and reused there, instead of asserting after the compare is already in the block.

- **Flags can cross a return.** No ABI passes flags out of a function, but clang's outliner does: it turns a repeated compare into a helper that ends in a compare and a `ret`, and the caller branches right after the call. The JIT treated every `ret` as killing the flags, and every Wine window painted black (fixed 5 September, 0c03488). A pure flag producer that reaches a `ret` now keeps its flags live; a tail whose last flag writer is arithmetic (`xor eax, eax ; ret`) is returning a value, and keeps the dead seam. `tests/guest/ret_flags.c` pins it.
- **A self-loop's compare is not fused when the loop head reads the flags.** `libc++`'s partition loop starts with a `jbe` on the previous iteration's compare. Fusing the compare at the bottom with the branch back to the top skipped recording the flags the top needed, and Steam's CEF renderers died on `libc++`'s "does your comparator satisfy the strict-weak ordering requirement?" assertion (fixed 13 September, a6085f2; `tests/dynamic/cef_partition.s`).

Exits and chains

A block that ends in a direct jump or falls into another block exits through a *chain tail*. The tail ends in a single `b` instruction which initially points at its own fallback: `set cpu->rip` to the target, record the side-exit tag if profiling, and leave through the shared exit. When the target block exists, that one word is patched to branch to it directly. There are three kinds of edge. A **body edge** (pin class 3 on both sides) jumps from body to body with every guest register in place. A **cross-block edge** runs the full epilogue and enters the target's full entry. A **self-loop edge** branches back to the block's own loop head.

If the target is not translated yet, the edge is registered in a table of pending chains keyed by the target (2^{18} buckets since 21 September, 2^{12} before), and activated when the target is published. Every activation also records the source block and edge on the target's list of predecessors, so that retiring a block can unchain exactly the branches that point into it rather than scanning every block in the process (since 27 September; the scan had been 18% of a CEF process's samples while V8 rewrote its code).

Retargeting conditional branches

A conditional branch in a superblock normally goes to its out-of-line side stub, which then branches to the target. Once the target is translated, the conditional branch itself may be retargeted straight at the target's body, saving the extra branch, but only when nothing on the stub's path is still needed. Three things can be: a lane flush that writes cached scalar values back into an xmm register (Chapter 7), an FP-batch check, and the producer's flag record. Both halves of that rule came from miscompiles. On 6 September (264e647), a stub's flag record was skipped by a retargeted branch (`a cmp ebp, 0xb ; jbe L` whose target begins `L: ja`) and every SQLite open in Steam's `libcef` failed with "out of memory"; the retargeting itself dated from 15 August. On 8 September (7d7786f), a scalar SSE result still in its lane register was skipped by a side exit chained past its flush, and Cocoa's view transforms came out singular.

Veneers

An arm64 `b` reaches 128 MB in either direction. The arena is 1 GB and fills front to back, so a caller translated late could chain to a callee translated early only if less than 128 MB of code had been produced in between; beyond that the patch was silently dropped and the edge left through the epilogue on every execution. That is exactly the shape of a game, where the engine's helpers are translated in the first minute

and the script code calling them keeps being generated afterwards. An attributed profile of Brawlhalla's main thread put 26% of its time in block prologues and epilogues, with 4.5-5.8% of chains dropped, and the garbage collector's mark routine, 18% of the thread by itself, was reached that way.

Since 22 September a 256 KB veneer pool is carved out of the arena at the start of whichever translation first crosses each 64 MB mark. A patch that cannot reach its target is pointed at a 16-byte veneer in the nearest pool instead (pools within 120 MB are considered): `ldr x15, [pc+8] ; br x15` followed by the 64-bit target address. `x15` is the computed-address temporary, never live across a block edge. Brawlhalla's main thread fell from a saturated core to 71% of one while executing 60% more blocks per second; every chain activated (`veneered=39010, out_of_range=0` in `OCERZ_PERFSTAT`), and return-stack misses fell from 36 to 11 million a second because fewer exits wiped the stack. The alignment hotpatch, which puts a branch at a faulting access and a stub at the end of the arena, uses the same pools when the end is out of reach.

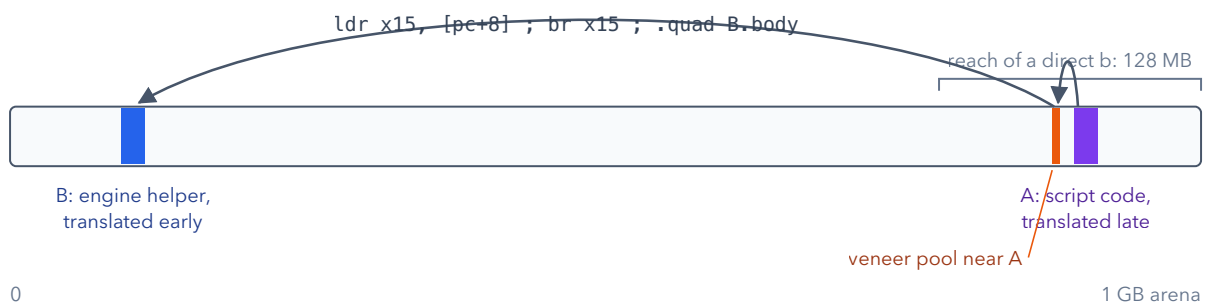


Figure 6.5. A chain that a plain branch cannot reach goes through a 16-byte veneer in the nearest pool, which loads the target's address and branches to it. Before 22 September such chains were dropped and the edge went through the dispatcher every time.

Stop sites and interrupts

Translated code has to notice when another thread wants it to stop: a signal to deliver, a process exit, an arena flush. Polling an interrupt word on every loop iteration costs a load and a branch per iteration, so steady-state loops do not poll. Instead, the first polling edge in a block keeps its chain branch as a *stop site*, and up to six more edges and indirect branches register extra stop sites. To stop the thread, AArchX patches those branches back to their fallbacks, so the next time the loop comes round it leaves through the dispatcher, which checks the interrupt word. The dispatch stub checks the same word on every unchained transition. The arena flush of Chapter 7 relies on the same mechanism.

Calls and returns

Returns are the hardest branches for a translator, because the target is data on the guest stack. AArchX makes a guest `call` / `ret` pair use the processor's own return predictor.

The host return-address stack

A direct guest `call` under full pinning (Figure 6.6) loads the guest return address into `x10`, takes the address of the host instruction after the call with `adr`, and pushes the pair `{guest return address, host continuation}` onto the *host* stack. It pushes the guest return address onto the guest stack as x86 requires, and then performs an arm64 `bl` straight into the callee's body; that `bl` starts out pointing at a fallback and is patched when the callee is translated. The callee's `ret` pops the guest return address from the guest

stack and the pair from the host stack, checks that the pair's guest address matches the one just popped and that the host continuation is not null, and executes a real arm64 `ret` through x30. Because every `bl` is matched by a `ret`, the hardware return predictor stays in step with the guest's calls. The code at the continuation is an ordinary chain tail to the block at the guest return address.

This has been the design since 17 August. Its weakness is lifetime: an entry lives only until the next trip out of translated code, because every exit resets the host stack pointer to the frame base saved in `cpu->jit_fp`. The sentinel pair `{0, 0}` pushed by the prologue marks that base, so a `ret` that finds it knows its entry is gone.

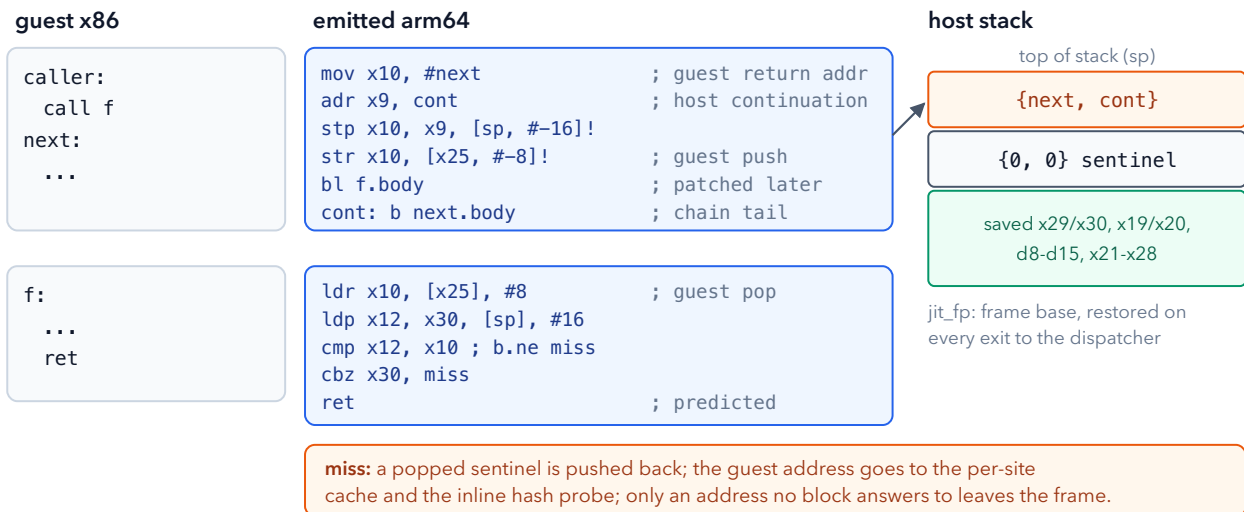


Figure 6.6. A guest call and return through the host return-address stack. Every `bl` is matched by a real `ret`, so the processor's return predictor sees the guest's call structure.

When the entry is gone

Until 22 September, a `ret` whose entry had been lost left the frame through the epilogue, and that exit reset the host stack, discarding every entry still outstanding, so the whole call chain then missed on its way up, each miss doing the same. C++ engine code makes a call every ten instructions or so and leaves translated code every few hundred thousand blocks, which is enough to keep the host stack permanently empty. On Brawlhalla's main thread `OCERZ_PERFSTAT` read 3.2 billion misses in 88 seconds, 36 million a second, one return in seven, all stale; the older software return stack (a 256-entry ring in the `cpu_struct`, `OCERZ_NO_HOST_RAS=1`), which survives exits, cut the misses to 393 thousand but cost 18-26% on the `memcpy` and `str` kernels. Now a miss stays in the body: the popped guest address goes through the same per-site cache and inline hash probe an indirect call uses, with the same body handoff, and only an address no block answers to leaves. A pop that finds the sentinel pushes it back first, because the saved frame registers sit right above it and the next call would overwrite them. Brawlhalla's translated throughput rose from 2.6-2.9 to 3.85 billion guest instructions a second; the kernels the host stack was designed for measure the same.

Indirect jumps and calls

An indirect jump or call first probes a per-site cache: 32 direct-mapped pairs of {guest rip, body address}, indexed by bits 2-6 of the target, 16-byte aligned so that the `ldp` reading a pair is single-copy atomic and a concurrent update can never produce a torn pair. On a miss it runs the same hash probe the dispatch stub uses, inline, and only then leaves to C. A hit on an indirect call uses `blr`, keeping the return predictor matched as a direct call does.

Splicing small callees

Since 25 August, a direct call to a short function is spliced into the caller's block while decoding. The callee qualifies if it is straight-line code of at most 24 instructions ending in a plain `ret`; calls inside it may themselves be spliced, two levels deep. The `call` becomes a push of the return address and the `ret` becomes a compare of the popped address against the known return address, with a mismatch leaving at the `ret`'s address so the dispatcher can run the real return. Two refinements remove most of the remaining memory traffic. Where the callee's frame is pure register work, the pushed return-address slot is provably never read, so the push becomes a bare `rsp -= 8`. And matched push/pop pairs, the store-to-load chain that dominates call-dense code, become register renames into x16, x17 and x30 when no memory base is hoisted there.

The renames had a hole. Every C call-out clobbers x16, x17 and x30, so a renamed pop whose push had been followed by a call-out read garbage. In the Wine memory map every nested spliced call took the slow path, and a two-level leaf-call loop built at `0x200000000` printed a different wrong sum on every run; xbench's `leafcall` tripped the stack protector. The bug was older than that layout (a build from 26 September fails the same way) and could have struck in any mode. Since 27 September (`3d626d5`) the renamed push still stores its slot, and the pop compares the call-out sequence number recorded at the push with the current one: if they differ, it pops from memory. `tests/dynamic/promo_callout.c` is that loop.

Returns in Wine's low shadow

Wine's stacks may lie below 12 GB, where guest and host addresses differ, so every stack fast path used to be switched off in that map and every spliced call and return went out to C: xbench's `leafcall`, built as a program at `0x200000000`, took 2.75 seconds against 0.16 outside the window. Nothing in the return-address machinery depends on the memory map; only the two guest-stack accesses do. Since 27 September, with `rsp` pinned as a pointer and a zero guest base, a call stores its return address and a return loads it through the same guarded translation as any other low-window access, and the rest (the host shadow push, the `bl`, the compare, the `ret`) is unchanged. `leafcall` went to 0.35 seconds, `icall` from 0.74 to 0.44, and Windows Steam's main window came up in about 33 seconds instead of 36-38. `OCERZ_NO_LOW_RAS` and `OCERZ_NO_LOW_SPLICE` turn the two halves off.

In native mode, a call into a synthesized system library is turned into a call made from inside the block as well, with a real `ret` afterwards; that bridged fast call is described in Chapter 12.

Superblocks and trace inversion

Since 16 August a block may run past up to six conditional branches. Decoding continues on the fall-through path and each taken side becomes an out-of-line side stub; since 17 August that includes backward branches other than a branch to the block's own start. A side stub carries everything the taken path

needs that the inline path deferred: it replays lane flushes, checks a pending floating-point batch, writes the flag record if the target reads flags, and then chains. Keeping that work in the stub is what makes the inline path cheap, and it is also why the retargeting rule above exists.

Superblocks assume the fall-through is the hot path. When a compiler laid the rare case inline (`if (rare) { ... }` with a jump over it), the hot path is the taken edge, and a loop fragments into a chain of blocks connected through side stubs. Since 3 September such a branch is probed: counters in the arena count both sides, and when the taken counter reaches 1,024 its stub returns to the dispatcher with `OCERZ_STEP_PROFILE`, which hands the edge to `flip_side_hit`. The verdict is "invert" when the taken side ran at least twice as often as the fall-through and at least 512 times. The first window of a loop is often unlike its steady state, so a verdict counts only when the next window repeats it, and the fourth window decides regardless. A decision to keep the branch turns the counters into `nop`s and chains the edge. A decision to invert retires the block, and the retranslation complements the condition while decoding, so the hot side continues inline. Decisions are remembered in a 4,096-entry table so the block is not probed again, at most 65,536 probes are made per process, and a translation shaped by a decision like this is never stored in or loaded from the translation cache. `OCERZ_NO_FLIP=1` turns inversion off, `OCERZ_FLIP_JCC` names branches to invert by hand, and `OCERZ_FLIPL0G=1` logs every verdict.

Retiring a flipped block used to find it in the array of live blocks by a linear search and close the gap with a `memmove`. A Steam webhelper holds about half a million live blocks and flips a thousand or more of them during startup, so each flip moved several megabytes and flip retirement was about 330 ms of every busy process's startup. Since 27 September each block records its index in the live array and a retired block trades places with the last one; nothing depends on the array's order.

Lane caches and the edges of a loop

Chapter 7 describes the floating-point machinery; one part of it shapes control flow and belongs here. Inside a loop, a scalar SSE result is kept in one of the twelve lane registers (v4-v15) instead of being merged back into its xmm register after every operation, and since 14 September the upper halves of `ymm` registers are kept in lane registers across a loop too. A self-looping block can use a fixed-lane mode that dedicates four lane registers for the whole body. While a value is cached, the architectural xmm register is stale. Every exit from the loop therefore has to write the cached lanes back first, which is why side stubs "replay lane flushes" and why a conditional branch may not be retargeted past such a stub. Each block also records, for every instruction, which guest registers are in which lanes and which are dirty, so that a fault in the middle of a loop can fold the lanes back into the cpu state before the guest's handler sees it.

The emitter

`src/a64emit.c` (924 lines) and its header (294) know how to spell arm64 instructions and nothing else: one function per instruction, each appending a 32-bit word to an `A64Buf`. There is no scheduling, no register allocation and no peephole pass; `jit.c` decides what to emit. An `A64Buf` carries its start, current and end pointers and an overflow flag. Writing past the end sets the flag instead of writing, and asking for a label past the end returns a pointer to a private sink word, so the translator can emit freely and check once at the end whether the block fit.

The encoders that contain logic are the ones where arm64's encoding is not a straight packing of fields. A logical immediate must be a rotated run of ones replicated at some element size; the encoder finds the smallest element that replicates the value and checks that the run, or its complement, does not wrap. `mov_imm64` offers every 64-bit constant it builds to that encoder first, and almost none qualify, guest addresses above all; the failed searches came to about a tenth of translation time. Since 27 September a value is refused at once unless the number of bit transitions going round the register is a power of two, which every valid pattern satisfies; all 6,636 valid size, rotation and width combinations still encode, and a million random values are all refused as before.

Two range limits matter to callers. `tbz` and `tbnz` carry a 14-bit offset, ± 32 KB, far shorter than the ± 1 MB of `b.cond`, so a large block can put a stub out of a `tbz`'s reach; a truncated `tbz` offset was a real bug found by audit on 18 August. And the scaled immediate forms of `ldr` and `str` cover only non-negative multiples of the access size, which is why the unscaled `ldur`/`stur` forms exist too, and why the ordered accesses are built from `ldapur` and `stlur`, their FEAT_LRCPC2 counterparts. `tests/unit/test_a64emit.c` does not compare encodings against expected words: it emits each sequence into a `MAP_JIT` page, runs it and checks the result, so an encoding bug cannot pass by agreeing with a wrong table.

When a block faults

A guest fault inside translated code arrives as a host signal with host registers. Before the guest can see it, the guest state is rebuilt: the pinned registers are read from the signal frame, `rsp` is converted back to a guest value (a push whose store faulted has already decremented it, so it is repaired by eight), elided return-address slots are written back so the interpreter can resume in the middle of a spliced frame, the xmm pins and lane caches are recovered from the frame's NEON state because the memory copy is stale, and the flags are rebuilt from the instruction's fault recipe. A faulting `ldp`/`stp` pair is re-run from its first instruction in the interpreter, so the guest sees the signal at the right one. Chapter 9 follows the signal from there to the guest's handler, and Chapter 7 covers the faults the JIT provokes deliberately: write-protected code pages, the commpage and misaligned ordered accesses.

Lessons, in one table

Symptom	Cause	Rule now	Fixed
Every SQLite open in Steam's CEF failed	retargeted <code>jcc</code> skipped its stub's flag record	never retarget past a record, lane flush or batch check	2026-09-06
Every Wine window painted black	flags assumed dead at <code>ret</code>	a pure flag producer keeps flags live across <code>ret</code>	2026-09-05
CEF crashed on its first navigation	gap <code>lea</code> clobbered the compared value's temporary	gaps touch pinned registers only; emitted to scratch first	2026-09-06/07
CEF renderers asserted in <code>libc++</code> <code>sort</code>	self-loop compare fused although the loop head read flags	no fusion when the loop head reads flags	2026-09-13
Cocoa view transforms came out singular	side exit chained past a lane flush	same retargeting rule	2026-09-08
SIGILL after 128 MB of code	out-of-range branch to the dispatch stub	range-check every long branch; veneers	2026-08-18

Symptom	Cause	Rule now	Fixed
A game's hottest edges never chained	chains beyond 128 MB silently dropped	veneer pools every 64 MB	2026-09-22
36 million return misses a second	a lost return entry left the frame and wiped the rest	a miss stays in the body	2026-09-22
A leaf-call loop summed garbage	renamed pop after a call-out read a clobbered register	compare call-out sequence numbers	2026-09-27

Each of those has a test that fails on the build before its fix (Chapter 15), and each optimization involved can be switched off individually, which is how most of them were found (Chapter 17).

CHAPTER 7

The JIT: floating point, SIMD, memory ordering and changing code

Chapter 6 showed how a block of x86 becomes arm64 that keeps every guest register in a host register and runs from one block body straight into the next. This chapter covers the places where the two architectures disagree about meaning rather than encoding: what a NaN looks like, what order memory operations become visible in, what happens when the guest rewrites code that has already been translated, and what to do when the code arena fills. It ends with the two mechanisms that make the JIT cheaper to use than to run: arm64 routines called in place of the system's string functions, and an on-disk cache that lets one process reuse another's translations.

20,555

LINES IN SRC/JIT.C

598

LINES IN SRC/LEAF.S

821

 LINES IN SRC/
TCACHE.C

1 GB

 DEFAULT CODE
ARENA

Exact floating point

Where x86 and arm64 disagree

For ordinary numbers the IEEE 754 operations on both architectures produce identical bits, and the rounding mode is the only thing that can change a result. The disagreement is entirely about NaNs, and it is visible to programs. When an SSE operation produces a NaN, x86 follows one fixed rule: if the first operand is a NaN the result is that NaN made quiet, otherwise if the second is a NaN the result is that one made quiet, and otherwise (an invalid operation such as 0/0 or infinity minus infinity) the result is the *default NaN*, whose sign bit is set. arm64 lets a signalling NaN win over a quiet one regardless of operand order, and its default NaN is positive. A program that formats, hashes or compares the bit patterns of its results sees the difference, and the test suite does too: every NaN test in `tests/guest` compares raw bit patterns against the same binary run natively under Rosetta.

The JIT therefore has to produce x86's NaN on every floating-point result, and on most processors it has to do that in software. The obvious method is a check after every instruction: compare the result with itself (a NaN is the only value unequal to itself), and on a NaN branch to an out-of-line *exact fixup arm*, emitted after the block body, that recomputes the result the x86 way. That is correct and it was the first implementation. It is also expensive: the check sits on the dependency chain of every floating-point loop, and in a packed loop it is several vector instructions on top of the few doing the work.

Deferring the check, then batching it

Two observations make most of those checks unnecessary. A NaN propagates: if an operation's input is a NaN its output is a NaN, so a check of the last result in a chain also catches a NaN created earlier in it. And a NaN is rare: real programs produce one in a tiny fraction of operations. The JIT first deferred each check

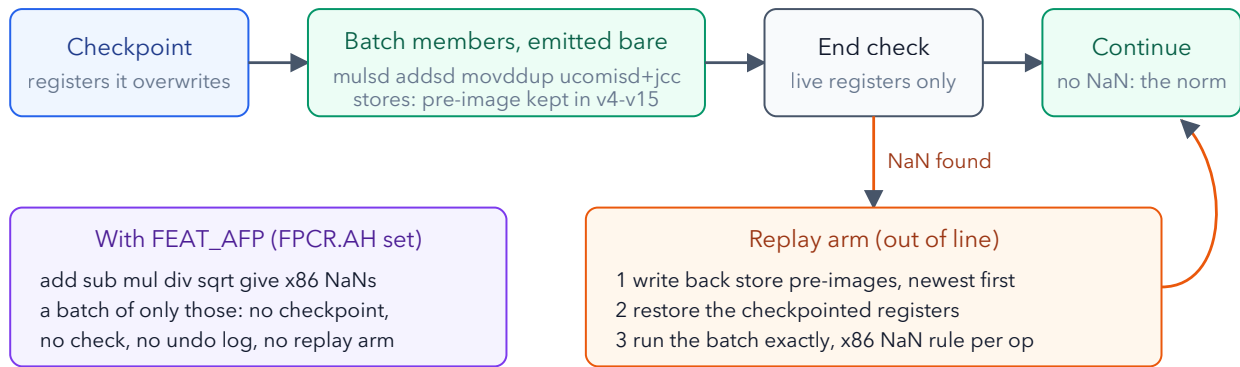
to the instruction that consumes the value, then generalised that into **FP batches**. A batch is a run of floating-point work that is checked once, at its end, with the registers it overwrites checkpointed at its start. If the end check finds a NaN anywhere it should not be, the batch is *replayed*: the checkpoint is restored and the whole run is executed again exactly, with every result correct to the bit. The fast path pays for one check per batch instead of one per instruction, and the slow path pays a replay only when a NaN actually appears.

Early batches covered only maximal runs of arithmetic and register moves, and anything else ended them. A scalar loop such as nbody's pair loop split into five or six batches per iteration, each paying a checkpoint and a check; of its 107 host instructions per iteration, 42 were NaN bookkeeping. Since 2026-09-15 (f5919cc) the scanner builds one batch per run of floating-point work. Members now include self-zeroing `xorps`, `unpckhpd` and `unpcklpd`, `movddup`, `ucomisd` and `comisd` together with the `jcc` that consumes their flags, and stores. A few rules keep that exact:

- A tainted register is checked right before it is stored, because a stored NaN escapes the batch.
- A compare is a free detector. `ucomisd` raises the unordered condition for a NaN in either operand, so the compare's unordered path replays the batch before the flags are read, and the fall-through path needs no check at all.
- The `jcc` after a compare carries its check into the side-exit stub, which runs only when the branch leaves the block.
- At the batch's end only registers still live on the path through the block are checked. A self-loop's back edge is live for the registers the next iteration reads before writing; registers dead in the loop but possibly live after it are checked once, on the loop's fall-through exit.

Stores that alias the batch's loads

A replay re-executes the batch's loads, so a store inside the batch that might overwrite memory an earlier load read used to end the batch: the replay would read the stored value instead of the original. nbody's pair loop updates both bodies' velocities in place and was three batches for that reason. Since 8fe7a18 the batch keeps an **undo log**: the memory pre-image of every such store is saved, and the replay arm writes the pre-images back, in reverse order, before re-executing. Where the pre-image lives decided whether this was worth doing at all. The first version kept it in the CPU structure and was slower than no undo log, because that loop is bound by its stores and four extra stores per iteration ate the gain. The pre-image now lives in a lane register reserved from the top of the v4-v15 pool, and when the guest itself loaded the same operand earlier, the read-modify-write shape, it costs one register move right after that load. The undo log needs addresses the JIT can form without a range check, so it is off under memory guards (the Wine layout described below) and in 32-bit code, where batches still split at such stores; 0CERZ_FPB_NUNDO=1 switches it off everywhere.



Side exits carry the check for lanes still unverified; ucomisd detects NaNs for free on its unordered path

Figure 7.1. An FP batch. The fast path checks once, at the end, and only registers that are still live. A NaN sends the batch to a replay arm that restores memory and registers and recomputes it exactly. On a processor with FEAT_AFP most batches need none of this.

Wide operations and fused multiply-add

A 256-bit packed add, subtract, multiply, divide or square root used to check each 128-bit half separately, five instructions and an out-of-line fixup per half. Since 31f7325 it takes one check for both halves (two `fcmeq`, an `and`, a `uminv` and a `cbz`), and a NaN sends the whole instruction to the interpreter from its untouched sources. fpvec's inner loop in the x86-64-v3 build went from 287 to 257 host instructions per iteration with that change. Fused multiply-add needs more care than the other operations, because its negated forms negate a NaN operand where x86 returns the operand as it came. VEX.128 packed FMA is a batch member, which made the FMA nbody loop a single batch; outside a batch every FMA keeps its own check.

Testing the slow path on a machine that never takes it

Replay arms run only when a NaN appears, so ordinary tests would leave them almost unexercised, and on a processor with FEAT_AFP most of them are never emitted at all. `tests/run_guest_tests.sh` therefore runs its twelve NaN tests (`fpb_loop_nan`, `fpb_undo_nan`, `fp_loop_nan`, `fpb_defer_nan`, `cvt_nan_batch`, `cvt_merge_nan`, `simd_misc_jit`, `avx_fp_jit`, `avx_cmpblend_jit`, `jit_sse`, `jit_sse2` and `nan_rules`) twice more after the main pass: once with `OCERZ_NO_AFP=1`, so the software checks are emitted, and once more with `OCERZ_FPB_FORCEREPLAY=1` as well, which makes every check take its replay arm whether or not a NaN appeared. A replay that is not exact then shows up as a wrong bit pattern in a golden comparison. `fpb_undo_nan` exists specifically for the undo log: six loop shapes, including aliasing through two base registers, an exact read-modify-write, partial overlap with two stores to one address, and a store whose register is rewritten afterwards.

FEAT_AFP: letting the processor do it

Armv8.7's FEAT_AFP adds a bit to the floating-point control register, FPCR.AH (alternate handling), that changes how arm64 arithmetic treats NaNs. With it set, add, subtract, multiply, divide and square root produce x86's NaN result themselves, as long as the x86 destination is the first arm64 source operand, which is how the JIT already emitted them. AArchX checks `sysctl hw.optional.arm.FEAT_AFP` once (`ocerz_afp` in `src/cpu.c`) and, where it is present, sets the bit on every guest thread in cache mode.

Those five operations then go out bare. A batch made only of them, moves, shuffles, compares and stores keeps its members' fast emission and drops the checkpoint, the checks, the undo log and the replay arms. Minimum and maximum were exact already, being a compare and a select.

The effect is largest in packed loops. In fpvec the end-of-batch check was three vector operations on top of the eight doing the work, and the four vector pipelines were the limit: fpvec went from 1.06x of Rosetta's time to 0.82x, and the x86-64-v3 build's from 0.96x to 0.58x (Apple M5, 2026-09-21). Two places keep the software checks. Fused multiply-add is not covered by the bit, for the negation reason above, so a batch that holds one keeps the whole apparatus. And native mode keeps the translated checks everywhere, because there the host's own arm64 frameworks run on guest threads and would need the bit cleared around every crossing: two FPCR writes cost about 14 ns, roughly what the crossing itself costs. That is why fpvec in native mode is still 1.05x while every other mode is below 0.85x. `OCERZ_NO_AFP=1` keeps the translated checks everywhere, which is also exactly what a processor without the bit gets.

HOW IT WAS FOUND

Writing `tests/dynamic/nan_contexts.c` for the FEAT_AFP change, which computes NaN results on the main thread, on a pthread, inside and after a signal handler, on libdispatch's threads, after an MXCSR write and across `fork`, turned up an older bug: the main thread of a dynamically linked program had never been given the guest's floating-point control state at all. It enters through the guest-call loop, and only the run loop applied that state.

Rounding modes

Until 2026-09-11 AArchX stored the guest's MXCSR but never applied its rounding-control field to the host, so every SSE result rounded to nearest whatever mode the program selected with `ldmxcsr` or `fesetround`. Both engines take their rounding from the host's FPCR (the JIT through arm64 instructions, the interpreter through C arithmetic), so setting FPCR fixes both at once. `ocerz_apply_mxcsr_round()` maps MXCSR.RC onto the host mode through the `fe*` constants rather than copying bits, because the two architectures number the directed modes differently: MXCSR 01 rounds toward minus infinity and FPCR 01 toward plus infinity. It runs wherever the guest's MXCSR changes (`ldmxcsr`, `fxrstor`, the context restore in `sigreturn`) and at run-loop entry, so a worker or a resuming thread starts in its guest's mode. The dynamic test `fp_rounding` checks the direction of rounding in each mode. Conversions follow the same mode: `cvtps2dq` rounds with `frinti`, which obeys FPCR, and then shares `cvttps2dq`'s conversion, which gives x86's `0x80000000` for NaN and out-of-range lanes where arm64 would saturate or give zero. In native mode a bridged call writes FPCR only when the guest's rounding mode is not already the default.

SIMD coverage

SSE

Most of SSE maps almost one to one onto NEON, and the interesting cases are where it does not. The explanation block at the top of `src/jit.c` lists the translations: widening multiplies followed by a narrowing unzip for the high-half multiplies and `pmaddubsw`; saturating narrows for the packs; `uabd` followed by three pairwise widening adds for `psadbw`; `addp` or an unzip pair for the horizontal operations; `ext` against a zero register for the byte shifts and `palignr`; lane inserts for the immediate blends and `pshufhw`/`pshufwb`. A shift by an xmm or m128 count clamps the 64-bit count to 64 and shifts by a duplicated register, which reproduces x86's saturation: zero for logical shifts and the sign for arithmetic ones.

AES rounds use `aese` or `aesd` against a zero key, then `aesmc` or `aesimc`, then the round key, because arm64 adds the key before the substitution and x86 after it; `aeskeygenassist` picks its words out of a zero-key `aese` with a table lookup. `crc32` is arm64's `crc32c` of the same width and `pclmulqdq` is `pmull`.

Many of these forms were emitted only from 2026-09-27 (`3eeaf45`), when sampling Steam's web helper under Wine showed it interpreting `pinsrw` from memory for 15% of its samples. The reason was general: an emitter that had only the plain memory forms handed every memory operand to the interpreter in the Wine layout, where guest and host addresses differ. Every emitter that takes a memory operand now reaches it through `emit_mem_load_any` or `emit_sse_mem_addr`, which fall back to the guarded address path rather than the interpreter. With that and the new forms the web helper's main process went from 47-67% of its samples in translated code to 78-81%. `tests/guest/simd_pack_jit.c` covers every form with register, memory and aliased sources over random and edge rows, the four rounding modes and eighteen shift counts, against hashes produced under Rosetta, and the dynamic suite builds the same source at `0x200000000` (`dsimd_low`) so every operand sits in the Wine layout.

AVX2, FMA and BMI

Rosetta on macOS 15 and later runs x86-64-v3 code, AVX2, FMA, BMI1 and BMI2, F16C, LZCNT, MOVBE and XSAVE, even though CPUID advertises none of them, and real programs depend on it: Go turns on its AVX2 paths under Rosetta without checking CPUID. AArchX decoded and interpreted VEX from 2026-09-08 and translated it from 2026-09-13. Before translation, AVX2 and FMA loops ran up to a hundred times slower than under Rosetta; after it, most beat it (Chapter 16 has the kernel table).

The JIT translates VEX moves, integer, bitwise and compare operations and broadcasts; `vpmovmskb`, most sign and zero extensions and shifts by an immediate; `vzeroupper`, FMA, 256-bit packed arithmetic and the variable blends; `vshufps`, `vshufpd`, `vinsertps` and `vmovddup`; the VEX.128 forms of every SSE instruction it translates; and `rorx`, `shlx`, `shrx`, `sarx`, `mulx` and `andn`, plus `blsr`, `blmsk`, `blsi` and `bzhi` when their flags are dead. Still interpreted: 256-bit byte and integer shuffles, unpacks, permutes and lane inserts, `vpctest`, the immediate blends, mask-producing compares, gathers, and `bextr`, `pdep` and `pext`. A `vcmpss` or `vcmpsdb` whose mask feeds a `vblendv` is fused into one compare-and-select.

Two details matter for speed. A `vzeroupper` in a block without 256-bit instructions tests a per-thread flag and skips its sixteen stores when the upper halves are already zero. And a block with VEX.128 instructions reserves a zero lane register and clears an upper half with one 16-byte store instead of two, halving the store traffic of every VEX.128 loop (`0CERZ_NO_ZEROREG=1` turns that off).

MMX

Until 2026-10-04 every MMX instruction ran through the slow path: one interpreter call per instruction with all guest state spilled around it. Old code still uses MMX, and Unity's video decoder in R.E.P.O. is written in it, which cost about 4% of the game process in the interpreter at its main menu. `emit_mmx` (`74e4300`) now translates the moves to and from memory and general registers; the lanewise adds, subtracts, saturating forms, multiplies, logic and compares; the unpacks and packs; `psadbw`, the averages and the minimum and maximum; shifts by an immediate; and `pshufw`, `pextrw` and `pinsrw`.

The eight MMX registers stay in the CPU structure's `mmx` array rather than in host registers. Each instruction loads what it reads into a scratch vector register with a 64-bit load, which clears the upper half; runs the 128-bit form of the operation; and stores the low 64 bits back. For lanewise operations that is all it takes. The others are arranged so their answer lands in the low half: a high unpack is a low zip followed

by taking the upper half, the packs join both operands into one register before narrowing, `pmulhw` and `pmulhbw` keep the odd halves of a widening multiply, and `psadbw` reuses the SSE sequence. Each translated MMX instruction leaves the x87 tag word full and TOP at zero, as the interpreter does, because MMX and x87 share state architecturally. `pmaddwd`, `pmovmskb`, `emms`, the SSSE3 forms, conversions to and from SSE and shifts by a register count still go to the interpreter. A loop of the decoder's moves, adds, multiplies and packs runs in 0.142 s instead of 1.395 s; Rosetta takes 0.053 s, which is still 2.7 times faster because every instruction here goes through memory. `tests/guest/mmx_jit.c` covers every translated form on saturating and edge values, and the dynamic suite runs it in the Wine layout too (`dmmx_low`). `OCERZ_NO_JIT_MMX=1` interprets MMX again.

Lane caches

The xmm registers are pinned in V16-V31 (Chapter 6), but a scalar SSE result has to land in lane 0 of its guest register while lane 1 keeps its old value. Done naively, every scalar operation ends with an `ins` into the pinned register and the next one reads it back out, and that insert sits on the dependency chain of every scalar loop. The **lane cache** keeps scalar results in their own registers instead. Since 2026-09-14 (`5e890e0`) it has twelve lanes, v4-v15; the block prologue saves d8-d15, which are callee-saved for the C code that calls into a block. A self-looping block keeps its lane values live around the back edge: it enters with every fixed lane mapped, a register whose value sits in another register's lane gets a move between lanes rather than a write-back, and only a register that lost its lane is reloaded. A mid-block `jcc` does not flush inline, because its side stub writes back the lanes that were dirty there.

The upper halves of the ymm registers get the same treatment (`d194edc`). They used to live only in `cpu->ymmh`, so a `vpaddq ymm` cost two loads and a store on top of its two adds. A self-loop block now keeps the upper halves it touches in the lanes the scalar cache leaves free, loaded once at the loop head and written back on every way out: the fall-through, the side-exit stubs, the out-of-line slow arms and the FP batch replay arms. In the x86-64-v3 build that took memcopy from 1.16x of Rosetta's time to 0.93x.

A value that lives only in a lane is invisible to a fault handler that rebuilds the guest's xmm state from the pinned registers, so at first both caches were switched off wherever a guest fault could be caught and resumed, which includes the whole Wine layout. Since `e170a9c` every block records, at each instruction start and at each re-emitted instruction in a replay arm, which registers sit in which lane, in what width, and which lanes are dirty; identical consecutive states share one record. Fault recovery looks up the record for the faulting host program counter and folds the dirty lanes and cached upper halves into the CPU state. That is exact for any fault inside an instruction because every emitter reads all of its memory before it writes a lane, a pinned register or a cache lane; the one emitter that did not, the 256-bit packed arithmetic with a memory source, was changed to read both halves first, low half first, so a fault also reports the address x86 would. `tests/guest/fault_lane`, a scalar SSE2 loop, its AVX2 twin and a 256-bit accumulation loop running into a protected page, caught 25 wrong lines in the build before the fix.

Memory ordering

The gap between x86-TSO and arm64

x86 promises total store order (TSO). Each core's stores become visible to other cores in program order, loads are not reordered with other loads, and stores are not reordered with earlier loads. The only reordering x86 allows is that a store may become visible after a later load from a different address. arm64 promises far less: without explicit ordering, any two accesses to different addresses may be observed in either

order by another core. Multithreaded x86 code is written against TSO, often without a single fence, and translated into plain arm64 loads and stores it breaks in the classic message-passing pattern: one thread writes data and then a flag, another reads the flag and then the data, and on arm64 the reader can see the new flag with the old data.

Plain until a second observer appears

A core always sees its own memory operations in program order, so a program with one thread cannot observe the difference. AArchX therefore starts every process in a **plain** memory model, translating loads and stores as ordinary arm64 accesses, and switches it to the **ordered** model the first time a second observer can exist. The switch is `ocerz_jit_require_ordered()`, and it is one-way: once a process is ordered it stays ordered. The syscall layer calls it from every place a second observer can come from:

- creating a thread (`bsdthread_create`) or registering the host workqueue bridge, whose workers run guest code;
- `fork` and the spawn paths, because parent and child may share memory;
- a writable shared mapping from `mmap` or `mprotect` (a read-only `MAP_SHARED` mapping no longer counts), and System V `shmat`;
- aliasing host memory into the guest, such as IOKit and IOSurface buffers that the kernel or a service also writes;
- in native mode, a host thread created by a framework calling into guest code for the first time.

Blocks translated under the plain model are not reused once the process is ordered: the switch invalidates every existing translation, and the memory model is part of a block's key in the translation cache.

Ordered mode: acquire and release, scalars only

In ordered mode a scalar load becomes an acquire load and a scalar store a release store, using the LRCP-C2 forms `LDAPUR` and `STLUR`, which take an unscaled immediate offset and so fit most x86 addressing directly. That is enough for TSO and no more than it needs. Release stores are ordered after every earlier access and acquire loads before every later one, which gives load-load, load-store and store-store ordering; the RCpc acquire (the `LDAP` family) is allowed to pass an earlier release store to a different address, and that is precisely the store-load reordering TSO permits. No `dmb` barrier is needed on the ordinary path.

Three classes of access are left plain even in ordered mode, each because ordering them costs much more than it buys:

- **Stack accesses through `rsp`.** `rsp`-relative loads and stores stay plain, on the heuristic, stated in the commit that introduced ordered mode (2ac3ae0, 2026-08-17), that a thread's stack is private to it; this also keeps `push`, `pop`, `call`, `ret` and the return-stack protocol identical across the switch to ordered mode. `OCERZ_TSO_STRICT=1` orders them too.
- **Vector accesses.** Programs do not synchronise through 16-byte accesses (x86 does not even make them atomic), and ordering them cost `memcpy` 3.3x and `fpvec` 3.0x of Rosetta's time on an M2 Max on 2026-09-05. Leaving them plain took those to 1.35x and 0.99x. What that gives up is a vector load followed by a scalar re-read of a sequence-lock counter observing newer data than the counter covers, and two vector stores becoming visible out of order. FEX ships the same default, and `OCERZ_TSO_VECTOR=1` orders them.

- **The system's own string and memory routines.** The exported string and memory routines of the platform library are handed buffers at any alignment, and an ordered access that straddles a 16-byte boundary takes a barrier (below). Translated with ordered accesses they made the dynamically linked memcpy kernel 4.6 times slower than Rosetta; translated plain it reached 0.98x.
`ocerz_dyldapi_memfn` answers whether an address lies inside one of them, and
`OCERZ_NO_MEMFN_PLAIN=1` orders them.

The narrow case: movd and movq

The 8- and 4-byte SSE forms are vector accesses too, and they are plain in ordered mode, but compilers do move pointers and counters through xmm registers. A plain `movq` store after an ordered scalar store can become visible first, and a plain `movq` load can be satisfied after the scalar load that follows it, so a reader can see a published counter newer than the data written before it. `tests/dynamic/tso_narrow.c` catches that hundreds to over a thousand times in three million handoffs, where x86 never shows it. `OCERZ_TSO_NARROW=1` (ef59a1d, 2026-10-04) orders `movd` and `movq` with two instructions and no barrier each: a store is an `fmov` to a general register and an `stlur`, and a load is a plain load followed by a one-byte acquire load of the same address. That second trick is how any ordered vector load is done: same-address reads are coherent, so the acquire copy sees a value at least as new as the vector, and everything after it is ordered behind it, which is TSO's load ordering without the `dmb ishld` that waited for every outstanding miss (40-60 ns per access on a 4 MB working set). `OCERZ_TSO_NARROW=2` orders every vector access of 8 bytes or fewer. Neither is the default: the first took R.E.P.O.'s main menu from 65 frames a second to 44, and no program is known to depend on it. The test runs with `OCERZ_TSO_NARROW=1`, in the ordinary and the Wine layout.

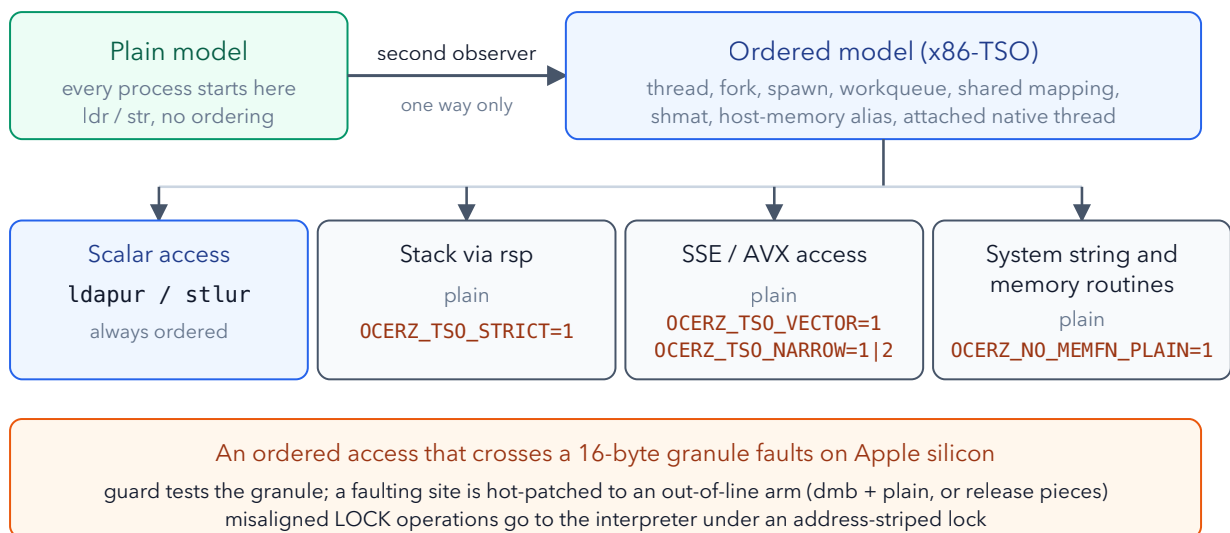


Figure 7.2. The memory model a translation uses. Every process starts plain and switches to the ordered model, for good, when a second observer can exist. Within the ordered model only scalar accesses are ordered by default; each plain class has a switch that orders it.

Granule crossings and the alignment hotpatch

Ordinary arm64 loads and stores accept any alignment, but acquire and release accesses do not. On Apple silicon an acquire or release access faults only when it crosses a 16-byte granule, which was measured rather than assumed, so the alignment guard in front of an ordered access tests exactly that and nothing

stricter: testing natural alignment instead sent three quarters of memcpy's unaligned tail accesses down the slow arm. When a site does fault, it is hot-patched: a branch replaces the access and leads to a stub that does the access a safe way. A store's stub uses `dmb ish` followed by a plain store when the block also does ordered loads, because the drain is cheap then (memcpy 0.31 s against 1.1 s with release pieces), and release-ordered pieces otherwise, which store-only loops prefer (memset 0.10 s against 0.5-1.0 s).

The patch is a `b`, which reaches 128 MB, and its stub originally had to sit at the end of the arena. When the end was out of reach, or the store's data register was 31 (in store encodings, the zero register, which is how a guest store of an immediate zero comes out), the patch used to give up and retranslate the block. Discord's V8 went through 5,391 retranslations in its first ten seconds that way, mostly the same few stores. Since 3619823 the stub goes into the nearest veneer pool when the end is out of reach and zero-register stores are patched as well, so it takes none. A first alignment or commpage fault also retires only the faulting block, through the predecessor lists, instead of invalidating its whole range, which at Electron's startup meant scanning about 140,000 live blocks per fault (`OCERZ_FAULT_INV_RANGE=1` restores the range invalidation).

A rip-relative or absolute operand has an address that is a constant of the translation, so for those both questions, whether the address is in the commpage and whether it crosses a granule, are answered when the block is built (`fabd213`). A global load or store, the most common memory form in compiled code, costs four host instructions instead of fourteen in plain mode and eighteen in ordered mode. The known-address note is set by the guard and consumed by the very next ordered access, and building any other effective address clears it, so it can never describe an address other than the one just guarded.

Locked operations at any alignment

x86 makes a `LOCK`-prefixed read-modify-write atomic at any alignment, because the bus or cache-line lock covers a split access. arm64's LSE atomics fault on a misaligned address, and so does clang's `__atomic_*`. This is not a corner case: Wine's `CRITICAL_SECTION` inside a packed Valve structure sits at offset `0x446`, so every `EnterCriticalSection` in Steam's CEF host is a `lock cmpxchg` on a doubleword that is 2 modulo 4. The JIT sends a misaligned locked access to the interpreter, which takes a lock striped by address and performs the read-modify-write with plain unaligned copies. That is atomic against every other misaligned access to the same location, which is the guarantee that matters in practice. `cmpxchg`'s flags follow Rosetta: flags as for `dest - accumulator`, and the accumulator written in both cases, so the 32-bit form zero-extends `rax` even on a match.

Addressing in Wine's memory layout

In most processes a guest address either is the host address (the identity map of dynamically linked programs) or differs from it by one constant base. Wine is different. Its main image insists on a low address the host will not hand out, so Chapter 8's *low shadow* puts the guest's first 12 GB at another host base, keeps the middle of the address space as identity, and maps a strip at the very top, which contains the commpage, at a third base. Every guest memory access in a Wine process has to go through that translation.

In its general form the translation builds three 64-bit constants and tests three ranges, about twenty instructions in front of a single load. Since 16a28b1 (2026-10-04) there is a fast form whenever the low window's host base is a single run of bits above every low address, as the usual `0x8000000000` and `0x10000000000` are. An address below 12 GB then becomes a host address with a shift, a compare and one

`orr`, since or-ing a base whose bits are all above the address is the same as adding it. An address between 12 GB and the top strip is recognised as identity with two shifts and an add. Only the top strip still takes the general form. In the Wine layout that took `memcpy` from 6.1x of Rosetta's time to 3.1x, a mixed workload from 3.0x to 1.65x and an interpreter-style loop from 1.5x to 1.0x, and R.E.P.O.'s main menu from 46 frames a second to 65. A debug build that ran both forms at every access and compared the results never saw them differ. `OCERZ_NO_FAST_LOW_GUARD=1` keeps the general form.

Guest address space of a Wine process

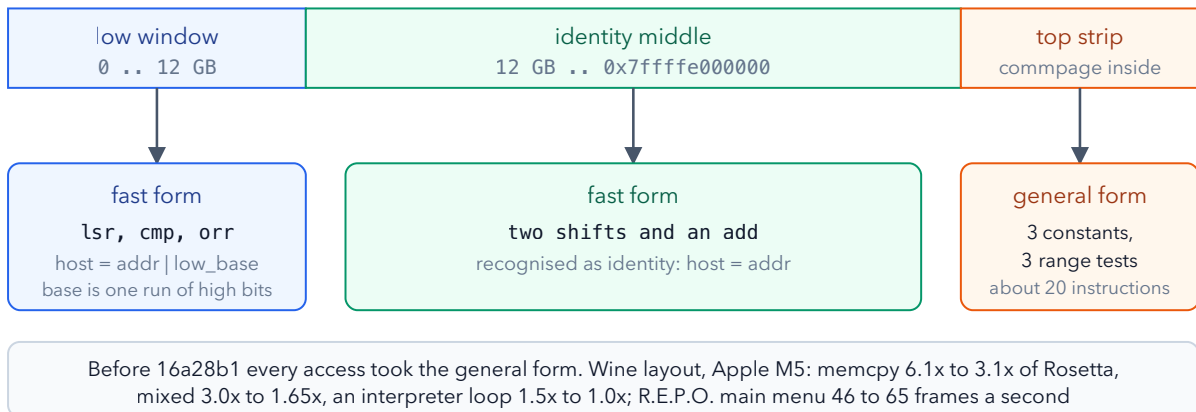


Figure 7.3. Translating a guest address in Wine's layout. Most accesses fall in the low window or the identity middle and now take a three- or four-instruction form; only the top strip with the `commpage` still takes the general twenty-instruction form.

Several of the JIT's other fast paths also needed rework to run in this layout at all. The memory-operand fallback described under SSE above is one. The call and return paths are another: the stack may live below 12 GB, so every stack fast path used to be off there, and `xbench`'s `leafcall` took 2.75 s against 0.16 s outside the window. Chapter 6 describes how calls and returns now keep the host return stack in the window (`leafcall` 0.35 s). The FP undo log, base hoisting and move pairs still need an address formed without a check, so they stay off under memory guards.

Code that changes

Catching writes to translated code

A translation is only valid while the guest bytes it came from are unchanged. Unmapping or remapping code is easy to see, because it goes through `munmap`, `mmap` or `mprotect`, and each of those asks the JIT to drop the code in its range. The hard case is a plain store into a page that is both writable and executable: JavaScript engines do exactly that, rewriting their own code in place, and until 2026-09-07 AArchX kept running the stale translation.

The fix lives in `src/mem.c` and is called **arming**. A host page holds up to four 4 KB guest pages, so its protection is the union of its slots' protections, minus write while the page is armed, meaning translations were made from code in it. A later guest store to an armed page faults; the fault handler drops the translations in the written range, unarms the page and lets the store run again, and the new bytes are translated afresh the next time they execute. Only code that sits in a guest-writable slot can be rewritten by a store, because code in a read-execute slot changes only through `mprotect` or `mmap`, which invalidate on their own. So a page is armed only when a writable slot overlaps translated bytes, which leaves Wine's PE images untouched where the tail of a `.text` section shares a host page with a writable section.

The reverse direction matters as much. A kernel copyout onto a read-only page does not fault: it fails, and with a Mach message the reply is destroyed with it. So the syscall layer unarms every armed page that overlaps a buffer the kernel is about to write, and retries once after unarming when a system call comes back `EFAULT` with armed pages present. A counter of armed pages lets the whole check be skipped when it is zero.

Finding the blocks to retire

Under Wine the live code spans PE images at 32-bit addresses and shared-cache libraries near `0x7ff800000000`, so a global minimum and maximum cannot answer "is there code here". A **region map**, one slot per 4 MB with a bitmap of the 64 KB granules in it, answers in constant time; it may say "maybe" after code is gone but never "no" while code is present. When it says "maybe", the candidate blocks used to come from a walk over every live block, and retiring them rebuilt the whole live array. Unity's Mono JIT writes code all through R.E.P.O.'s startup with about 390,000 blocks live, and those two functions were 30% of the game process. Since `8b480ea` each 64 KB granule keeps a list of the blocks in it beside its count, an invalidation looks only at the lists of the granules it covers (falling back to the walk past 256 granules or when the list table is full), and a retired block leaves the live array by swapping places with the last one. A benchmark that rewrites one small function at a time among 300,000 translated ones went from 37.9 to 24.7 microseconds per rewrite, against Rosetta's 14.7. `0CERZ_INV_SCAN=1` walks every block again.

Retirement itself was then made precise (`27833ac`). Mono patches call sites in code it has already run, over a thousand times a minute in R.E.P.O., and each patch is a write to an armed page. The fault unprotected one 16 KB page, but the invalidation used to be rounded out to the whole 64 KB granule, retiring up to four times the code that could have changed; then it cleared every return-address cell and slot, every site cache and every inline-cache slot in the process, and wiped every thread's return stack, so every thread mispredicted its returns after every patch. Now only the blocks overlapping the written range are retired, a granule's "may hold code" bit is cleared only when no block is left in it, and only the cached entries that point into a retired block are cleared, found by a binary search over the retired blocks' code. Site caches are filled by translated code, which can race a retirement, so every 256th retirement still clears them all and wipes the return stacks. The inline-cache slots turned out to be used by no translated code at all and were removed. `0CERZ_INV_GRANULE=1` restores the granule rounding.

A retired block's code is not freed. It stays on a retired list, so a thread that is still inside it runs on to its next exit; that is what makes retirement safe without stopping other threads, and it is also why the arena eventually fills (below).

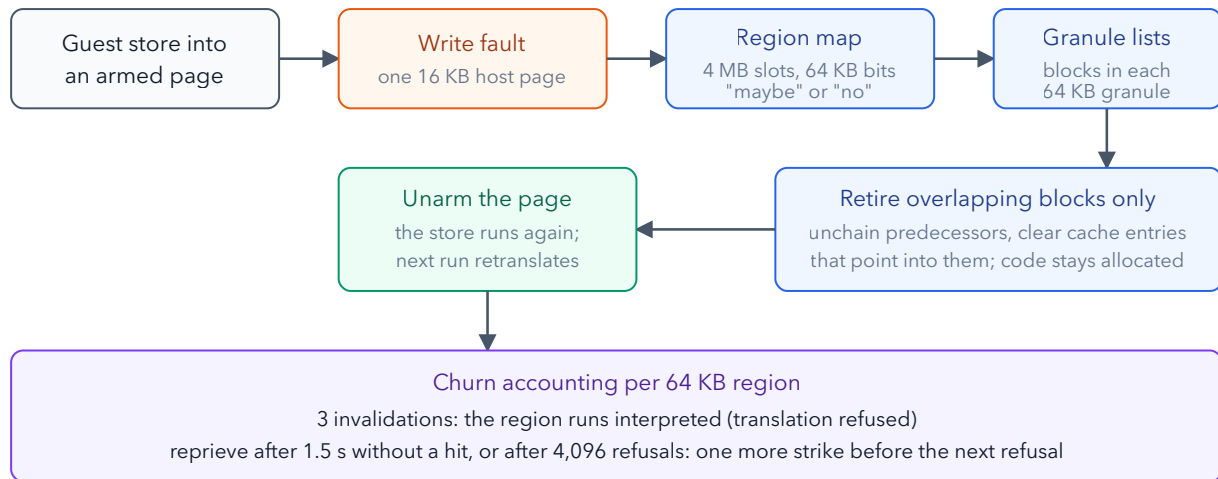


Figure 7.4. From a store into translated code to the next translation. Since October 2026 only the blocks that overlap the written range are retired, found through per-granule lists rather than a walk over every live block.

Regions that never stop changing

Some code is rewritten so often that translating it is wasted work: a JavaScript engine that flips its code space between writable and executable retires its translations faster than they pay back. The JIT counts invalidations per 64 KB region in a 4,096-slot table, and after three it refuses to translate in that region, so the code runs in the interpreter. A permanent blacklist turned out to be wrong, though. Module-load fixups also retire blocks a few times and then never again, and a blacklist made the hottest code of a freshly loaded DLL interpret forever. So a region is given a reprieve, dropping back one strike, when it has been quiet for 1.5 seconds (`CHURN_QUIET_NS`). That was still not enough for Steam: hot pages in ICU, libdispatch and Foundation never went quiet for 1.5 s, and one ICU page was refused translation about 30 million times in a two-minute run. Since 2026-09-12 a region also gets its reprieve after 4,096 refusals (`CHURN_REFUSE_MAX`), which took refusals from about 25 million to under 64 thousand per run. `OCERZ_BLACKLOG=1` prints the most-refused pages.

When the arena fills

Translated code goes into one `MAP_JIT` code arena, 1 GB by default (`OCERZ_JIT_CODE_MB` and `OCERZ_JIT_CODE_KB` change it; the test suite uses a 64 KB arena for some tests to force most blocks down the interpreted path). The arena fills front to back, and retired code is never reused in place, because another thread might still be running it. A program that keeps generating code therefore fills the arena eventually, and Unity's Mono JIT in R.E.P.O. does it within two minutes of starting. Until 2026-10-04 every block after that point ran interpreted.

There was a worse problem. R.E.P.O. also crashed in Mono's metadata code in every run whose arena filled while it was still loading. A 2 or 3 GB arena made the crash go away and a 600 MB one brought it back. Running `tests/dynamic/dlopen_cryptex.c` with a 512 KB arena reproduces the same kind of damage within a second whenever a translation overflows the arena and more translating follows. Why an overflowing translation leaves the translator in that state is not understood yet, so the arena is no longer allowed to overflow (`9226846`).

Instead, when less than an eighth of the arena, and at most 8 MB, is left, the next translation miss **flushes** it: every block is retired, every table that points into the arena (chains, return-address cells and slots, site caches, veneer pools, the dispatch stubs, the code index) is emptied, and translation starts again at the front. The same flush runs when the return-address slots run out, since only a flush gives those back. The difficulty is that no thread may still be running code in the arena when it is reused, and the flusher cannot simply stop other threads, because a thread may be blocked in a system call made from translated code.

The protocol works with two counts per thread: how many translated frames it is in, and how many of those are parked in a system call made through the slow path, `ocerz_jit_exec_one`. A thread whose two counts are equal is not executing arena code, even if arena frames are on its stack. The flusher bumps a generation number, patches every *stop site* so that a running thread leaves at its next block edge (synchronising one cache line per patched word), and waits until every thread's counts are equal. A thread that reaches the dispatcher meanwhile runs its next instruction in the interpreter rather than waiting, so a garbage collector that has frozen another thread can still get round to resuming it. A parked call that returns into a newer generation does not return into the arena: the slow path has already written the guest state back to the CPU structure, so it goes back to its run loop (`ocerz_vm_jit_escape`) with its result, which is the exit the block itself would have taken. If some thread cannot leave within 500 ms, for example one stopped by `thread_suspend` while it spins, the flush gives up, restores the stop-site words it changed, and is retried two seconds later. A thread that is itself inside translated code never flushes, and a fork child forgets its parent's other threads.

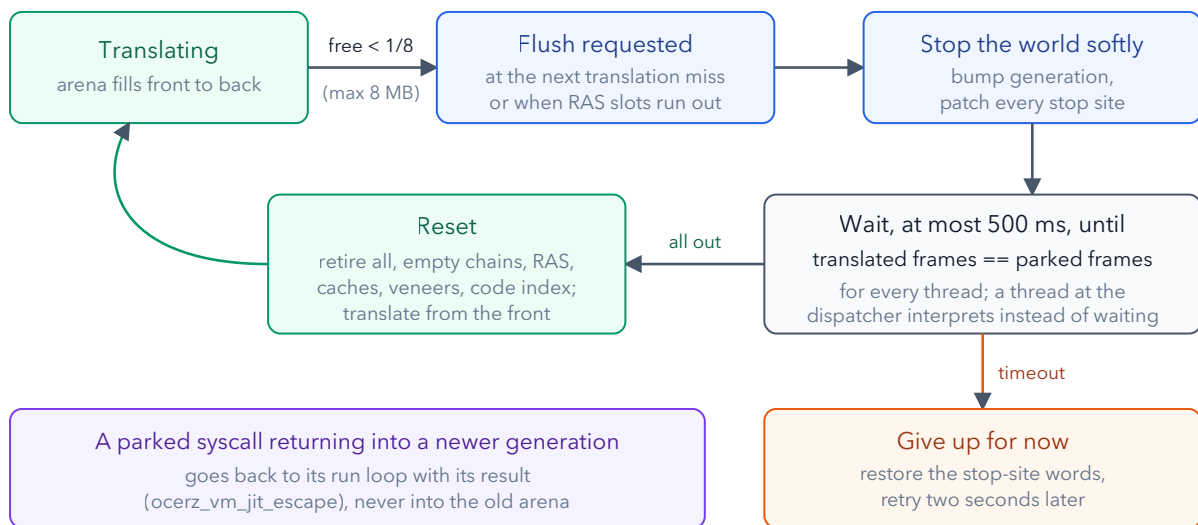


Figure 7.5. The arena flush. Threads leave translated code at their next block edge; a thread blocked in a system call made from translated code counts as out, and is redirected to its run loop when the call returns.

The first version of the flush unchained the old code and pushed the whole arena through the instruction cache, which held every thread for a third of a second; neither is needed, since nothing runs the old code again, and both were removed (27833ac). Only a system call made through the slow path is parked, because any other slow-path call returns soon enough to leave by a stop site, and parking all of them had cost R.E.PO.'s worker threads a tenth to a quarter of their busy time in atomics and thread-local lookups. With the flush in place R.E.PO.'s main menu went from 28-30 frames a second to 46 with nothing else changed, the arena is flushed about once a minute, and the Mono crash did not come back in five runs.

With a 512 KB arena the dynamic suite passes everything except the translation-cache cases, which need room to keep records, at thousands of flushes per test. `OCERZ_NO_JIT_FLUSH=1` restores the old behaviour and `OCERZ_FLUSHLOG=1` logs every flush and names the threads it waits on.

In-place string and memory routines

Eleven routines are called constantly by almost every program and do very little each time: `strlen`, `strnlen`, `strcmp`, `strncmp`, `memcmp`, `bcmp`, `strchr`, `memchr`, `memcpy`, `memmove` and `memset`. In native mode a guest call to one of them crosses into the host's implementation, and the crossing costs far more than the routine: every pinned guest register goes out to the CPU structure and comes back, the arguments are read into an argument array, the rounding mode and `errno` are swapped twice, and the block that made the call is re-entered through its return check. For a routine that reads a few dozen bytes and returns a number, nine tenths of the time went to the crossing. `src/leaf.s` holds arm64 versions of them under a contract that needs none of that.

The contract

A routine reads its first three arguments where full pinning keeps them, `rdi` in x28, `rsi` in x27 and `rdx` in x23, and writes its result to x21, which is `rax`. It changes x9 to x15, v0 to v3 and the flags, and nothing else; it makes no call, uses no stack and never writes x30. The guest base is zero wherever these routines are used, so a guest pointer is dereferenced as it arrives. A block therefore reaches one with a single direct branch and spills nothing. The routines are copied, being position-independent, to the start of the code arena, so the branch is always in range.

Never writing x30 is what keeps a fault exact. If a guest hands `strlen` a bad pointer, the fault happens at a program counter inside `src/leaf.s`, which belongs to no translated block, but x30 still names the instruction after the branch, and every pinned register is as the guest left it. `ocerz_leaf_site` maps a program counter inside the routines to that calling site, so `src/vm.c` recovers the guest state as if the stub's own instruction had faulted, and the same substitution lets a suspended thread report its registers. Because `rax` is written by the last instructions before the return, which touch no memory, a fault never observes half a result.

The scanning routines read whole 16-byte blocks at 16-byte-aligned addresses. Such a block never spans a page boundary and always holds at least one byte the caller owns, so reading past the end of a string can never touch an unmapped page. The two-string comparisons cannot align both pointers at once; they read unaligned blocks while both pointers are at least 16 bytes short of a 4,096-byte boundary and compare byte by byte for one block otherwise. A 16-byte comparison leaves 0xff or 0 in each byte, `shrn` by four packs that into a 64-bit value with four bits per byte, and the index of the first interesting byte is its count of trailing zeros divided by four.

Declining

Every routine also answers in x9: zero when it did the work, one when it declines, having changed nothing the guest can tell, in which case the caller does what it would have done without the routine. `memmove`, which also serves `memcpy`, declines by itself for one case: ranges that overlap without being identical. The reason is what a fault means here. A fault inside a routine leads, in either mode, to the whole call being made again from its original arguments. Doing part of the work and then all of it is harmless for reads, for copies between ranges that do not touch and for filling a range with one byte, but not for an overlapping

move, where the part already moved has overwritten source bytes the second run would read. Up to 64 bytes every load is made before any store; beyond that the copy runs forward in 64-byte blocks and finishes with the last 64 bytes, which the ranges being disjoint makes safe.

The routines that write (`memmove` and `memset`) may store onto a page whose code has been translated; the fault handler resolves that by retiring the translations and letting the store run again, and the caller compares the retire count from before the call with the one after and does not return through its shadow stack when they differ. In native mode these two are used only for lengths up to 16,384 bytes, above which the host's routines are faster.

Two modes, one set of routines

In native mode the routines replace the crossing: the block checks its stub's slot as before, branches to the routine, pops the return address and runs the same return. A fault inside a routine is attributed to the stub, so a guest with a `SIGSEGV` handler gets its handler with `rbx` and `r12` to `r15` holding what the caller left there, where a fault inside an ordinary crossing ends the process.

In cache mode the guest's `strlen` is Apple's x86 code from the shared cache. When a block starts at the exported entry of one of ten of those routines, which `ocerz_dyldapi_leaf_entry` recognises, the same call and return are emitted ahead of the block's first instruction, and the translation of the x86 code follows as what runs when the routine declines. No length limit applies, since the x86 routine it would fall back to is slower at every length. A fault inside a routine is not delivered to the guest from there: `src/vm.c` makes the routine decline instead, and the x86 code then takes the same fault, with every detail the guest can observe already right. A block with a branch back to its own start is left alone, because its lanes may be live where the call would leave.

Guest loop, ns per iteration (Apple M5, 2026-09-21)	trap path	default	arm64 build
<code>getpid</code> , an ordinary crossing	40.5	17.0	0.7
<code>strlen</code> of a 12-byte string	42.0	2.0	0.9
<code>memcpy</code> of 48 bytes	45.6	3.0	2.7

With `OCERZ_NO_LEAF_INPLACE=1` the `strlen` and `memcpy` rows read 17.3 and 20.8 ns, which is what an ordinary crossing costs. In the dynamically linked benchmark the routines took the `str` kernel from 6.78x of Rosetta's time to 0.82x in native mode and from 1.08x to 0.77x in cache mode, and `memcpy` from 1.06x to 0.73x and from 0.98x to 0.72x. `tests/unit/test_leaf.c` checks every routine against the C library over 6,736,902 inputs, each string placed against an inaccessible page at every alignment so that reading one byte too far dies in the test. `sys_strings` repeats that from inside a guest in both modes and both engines, `sys_strings_inplace` fails unless the translator reports that it answered the calls in place, and `sys_strings_fault` runs each routine into a forbidden page and requires all twelve faults to reach the guest's handler with its callee-saved registers intact.

Keeping translations: the on-disk cache

A short Wine program spends most of its life translating, and every process of a Wine session translates the same system and Wine code again. The translation cache (`src/tcache.c`, `20465b4` and `71b3d28`, 2026-09-27) stores a finished block, and the next process that needs the same key loads it instead of translating. It is on by default; `OCERZ_TCACHE=off` turns it off, and every test target runs with it off so that no test depends on what an earlier one stored and none writes to the user's caches.

What is kept

Only code that sits at the same address in every run is worth keeping: all of a Wine process, whose low-shadow layout and fixed image bases make addresses stable, and otherwise only the shared cache. A main program, a dylib or a JIT loaded at a random address would fill the store with records nobody loads, and when the guest's own base is chosen at random nothing is kept. A translation made because the process *learned* something, such as a flipped branch or the guard a commpage or alignment fault asked for, is never answered from the store: a key that carries such a mark, or whose block was retired or invalidated, is translated again and the new record replaces the old. The key adds the memory model to the block's ordinary key, since a block translates differently once a second thread makes loads ordered.

Making code that can move

A translation is full of values that differ between processes: the addresses of ocerz's own functions and globals, the commpage delta, the block's own `JitBlock`, instruction and profile slots inside it, return-address slots and cells, indirect-call cache tables, the leaf routines and the dispatch stub. When recording, each of these is emitted as a fixed `movz` and three `movk` or as a literal cell, and the translator records where (`TcReloc`); a leaf call and the dispatch-stub exit go through a register instead of a direct branch. A load copies the code to the same address modulo 64, because the literal pool's padding and a loop head's alignment are absolute, and fills every site from the current process, allocating slots, cells and tables afresh.

A record also names every range of guest bytes the translation read: the block, the callees spliced into it, and the successors whose flag liveness it relied on. That is why a recording translation decodes a successor itself rather than trusting a cached liveness answer, and why a memoised liveness entry keeps the ranges and a hash of the bytes it read and replays them into the record when it answers. The record carries a hash of those bytes, and a load checks the guest's memory against it before using the record, so a changed library or patched code is never answered from stale bytes.

The store

Records live in `~/Library/Caches/ocerz/tc-<fingerprint>`, or under `OCERZ_TCACHE_DIR`. The fingerprint hashes everything that changes what a translation looks like without changing the guest bytes: the ocerz binary's `LC_UUID`, the memory mode and the host bases it maps the guest at, the machine model, and every `OCERZ_` variable apart from the few that only steer logging, the loader or the cache. A different build or a different switch therefore opens a different directory, and nothing inside one needs a version check. Directories of other fingerprints are removed once untouched for a day, and sooner while they hold more than 2 GB together; one touched within the last hour is never removed, because a Wine session uses several fingerprints at once.

A directory holds one index, shared by every process, and one data file per process that has written anything, `d-<n>.td`. The index is an open-addressed table of key and location slots that every process maps shared and changes with atomic operations alone: a key is claimed with a compare-and-swap, and its location is stored only after the record is written, so a reader that finds a location finds a whole record. A process appends only to its own data file. Records are LZ4-compressed, about 1.65 times smaller for 1.6 microseconds to write and 0.3 to read, and carry a checksum checked on every load, so a record torn by a crash is refused rather than run. A put copies the record into a 256 KB buffer and nothing more; a full buffer goes to a writer thread, which compresses, appends and publishes. At exit, `exec` or a non-main-thread exit the last buffer is handed over and the writer gets up to three seconds. Nothing is ever deleted from a store; when its data files reach `OCERZ_TCACHE_MAX_MB` (4,096 by default) writing stops, and a process that finds itself the only user of a full store (an exclusive `flock` on the directory's users file succeeds) removes it and starts again, so a full store costs one cold start rather than every translation from then on.

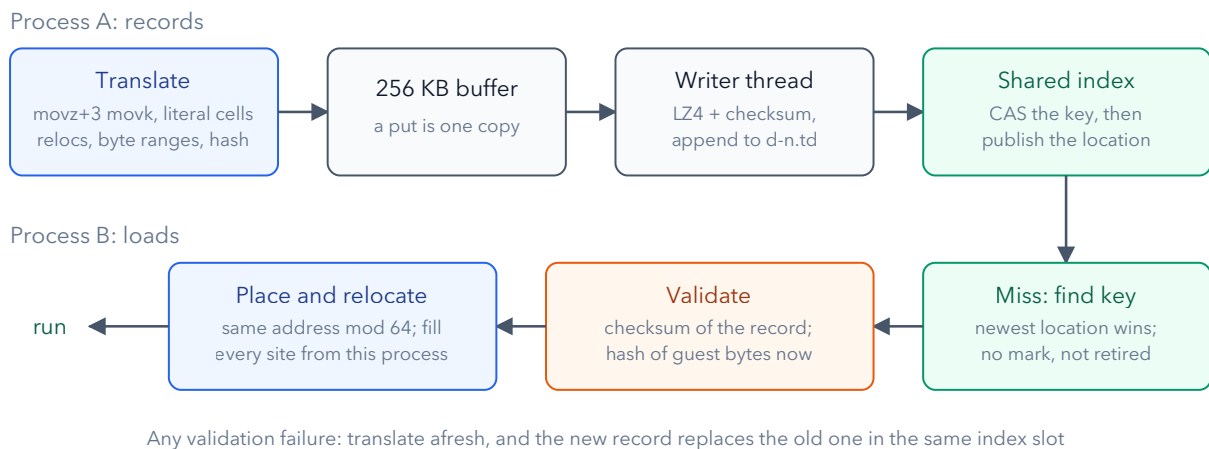


Figure 7.6. The life of a translation-cache record. One process records and publishes; any later process, in the same session or the next, validates the record against the guest's current bytes and relocates it into its own address space.

What it buys, and how it is checked

Measured back to back on the same machine, Windows Steam's window came up in 46.9 s from an empty store and in 35.2 s and 33.7 s from a warm one, and a full Steam session fills about 2 GB of store (2.9 GB before records were compressed). A cold run of `hostname.exe` under Wine, a process that lives about half a second, went from 0.78 s to 0.61 s when records began to be compressed and written by a separate thread and the dependency bookkeeping was made cheaper, against 0.58 s with nothing kept, so recording now costs a cold process almost nothing. With the cache on, the dynamic suite passes twice over one store, the second time loading 1.33 million blocks and storing 365.

Two modes exist to check the cache itself, not to use it. `OCERZ_TCACHE=verify` translates every block anyway and compares it with the record it would have loaded, counting a difference in shape as a variant and any other difference as a bug; over 4.4 million Steam blocks it found none bad. `OCERZ_TCACHE=roundtrip` writes nothing: it moves every translation to a fresh address, fills the original with `BRK`, and reports any PC-relative reference that leaves the block or any host-looking constant that no relocation covers. Across 4.6 million Steam blocks it found none, and building it turned up two older faults, both fixed with it: a fault while the translator read guest bytes used to take a path that wanted the JIT lock the translator already held, and the lane allocator now restarts for each block, so a translation no longer depends on what was translated before it.

Switches for this chapter

Variable	Effect
<code>OCERZ_NO_AFP=1</code>	keep the translated NaN checks even where FPCR.AH is available
<code>OCERZ_FPB_FORCEREPLAY=1</code>	make every NaN check take its replay arm (a test aid)
<code>OCERZ_FPB_NUNDO=1</code>	no undo log; a possibly aliasing store ends the batch
<code>OCERZ_NO_ZEROREG=1</code>	clear ymm upper halves with two stores instead of a zero register
<code>OCERZ_NO_YMMH_CACHE=1</code>	keep ymm upper halves in memory only
<code>OCERZ_NO_JIT_MMX=1</code>	interpret every MMX instruction
<code>OCERZ_NO_PLAIN_MEM=1</code>	start in the ordered memory model
<code>OCERZ_TS0_STRICT=1</code> , <code>OCERZ_TS0_VECTOR=1</code> , <code>OCERZ_TS0_NARROW=1 2</code>	also order stack accesses, every vector access, or <code>movd / movq</code> (2: every vector access of 8 bytes or fewer)
<code>OCERZ_NO_MEMFN_PLAIN=1</code>	order the system's string and memory routines like other code
<code>OCERZ_NO_FAST_LOW_GUARD=1</code>	use the general address translation everywhere in the Wine layout
<code>OCERZ_FAULT_INV_RANGE=1</code>	invalidate the whole range on a first alignment or commpage fault
<code>OCERZ_INV_SCAN=1</code> , <code>OCERZ_INV_GRANULE=1</code>	find blocks by walking every live block; round invalidations out to 64 KB
<code>OCERZ_BLACKLOG=1</code>	print the pages most often refused translation
<code>OCERZ_JIT_CODE_MB=n</code> , <code>OCERZ_JIT_CODE_KB=n</code>	size of the code arena
<code>OCERZ_NO_JIT_FLUSH=1</code> , <code>OCERZ_FLUSHLOG=1</code>	do not flush a full arena; log every flush and the threads it waits on
<code>OCERZ_NO_LEAF_INPLACE=1</code>	reach the eleven string and memory routines the ordinary way
<code>OCERZ_TCACHE=off verify roundtrip</code>	disable the translation cache, or run one of its two checks
<code>OCERZ_TCACHE_DIR</code> , <code>OCERZ_TCACHE_MAX_MB</code> , <code>OCERZ_TCACHE_LOG</code>	where stores live, when one stops growing, and per-process load and store counts

PART III

Memory and the operating system

Where guest memory lives, and how system calls, Mach messages, threads and signals are answered.

CHAPTER 8

Guest memory

An x86-64 macOS program expects to own a 47-bit address space with 4 KB pages, its images where its load commands put them, and a commpage at a fixed address. AArchX runs inside an arm64 process with 16 KB pages that the kernel lays out for its own reasons. This chapter describes how guest addresses become host addresses, how 4 KB guest pages are kept on 16 KB host pages, and what happens when the host kernel or a framework hands the guest memory it never asked for.

2,086

LINES IN SRC/MEM.C

3

ADDRESS MAPS

256 GB

IDENTITY ARENA

**4 KB / 16
KB**

 GUEST PAGE / HOST
PAGE

The constraints

Three facts about arm64 macOS shape everything in `src/mem.c` and `include/ocerz/mem.h`.

- **The host lays out the process.** arm64 macOS refuses any mapping below the slid main executable, which here is `ocerz` itself, so the classic x86-64 layout with `__TEXT` at `0x100000000` cannot exist at identical host addresses. The kernel also ignores placement hints for mappings that are not `MAP_FIXED`, which was observed on macOS 26.5 when the first arena was built.
- **Pages are 16 KB.** Apple silicon's host page is `0x4000` bytes; an x86 program believes in `0x1000`-byte pages, sizes its allocations to them, and protects memory at that granularity.
- **The guest must not see the host's view.** A program that asks about its own memory is asking about guest pages, guest protections and guest addresses. Anything the host says about its own arena, its own allocations or its own pointers has to be translated or hidden.

The answer is a set of address maps, chosen per process, plus a table that tracks every 4 KB guest page inside the 16 KB host pages that hold it.

Three address maps

In June there was one map, a 32 GB arena for static binaries. There are now three, and which one a process gets depends on what it is.

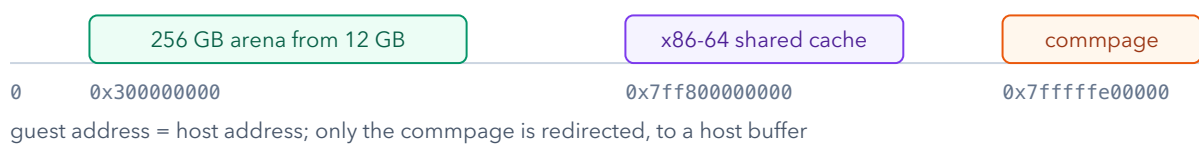
Map	Used for	Rule
Affine arena	static <code>LC_UNIXTHREAD</code> binaries	guest <code>[0x100000000, 0x900000000)</code> is one hintless <code>PROT_NONE</code> reservation; <code>host = guest + guest_base</code>

Map	Used for	Rule
Identity arena	every dynamically linked program, in both modes	256 GB reserved from host <code>0x300000000</code> upward, or at <code>0x7a000000000</code> in a Wine process; <code>guest_base = 0</code> , so a guest address is the host address
Low shadow	a program with an image below 12 GB: Wine	guest addresses below 12 GB and a 30 MB strip under the commpage live at their own host bases inside one reserved block; everything else is identity

Affine arena (static binary)



Identity arena (dynamic program)



Low shadow (Wine)

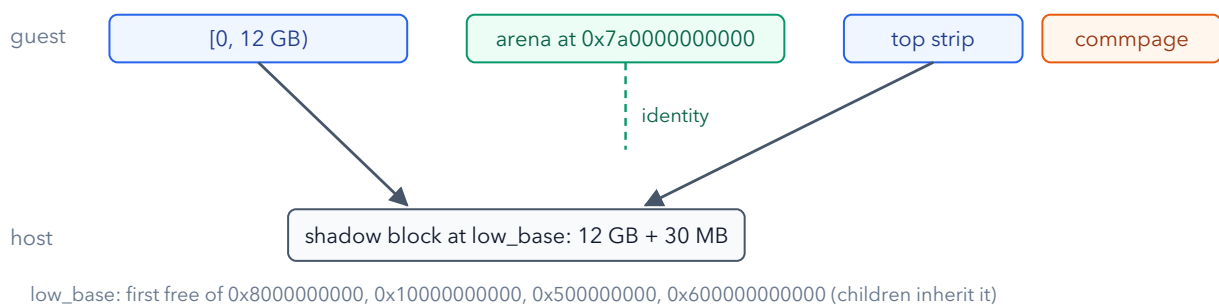


Figure 8.1. The three address maps. A static binary is offset by one constant. A dynamic program uses identity addresses, with only the commpage redirected. A Wine process keeps identity for its arena but moves the bottom 12 GB and the strip below the commpage into one shadow block, because Windows code expects those addresses and the host cannot give them out.

The affine arena

Static test binaries and other `LC_UNIXTHREAD` programs are linked at `0x100000000`, where an arm64 process can never map them. `oc erz_mem_init(0x100000000, 0x900000000)` reserves 32 GB with a plain `mmap(NULL, ..., PROT_NONE)`, takes whatever address the kernel returns, and sets `guest_base = host_base - 0x100000000`. The affine map then works for any base. Because AArchX owns the whole reservation, a `MAP_FIXED` inside it only ever replaces AArchX's own pages, and an `unmap` re-reserves the range as `PROT_NONE` instead of returning it, so the kernel can never place one of its own allocations inside the guest's address space. The first quarter of the arena is held back for the loader; anonymous allocations bump upward from there.

The identity arena

A dynamically linked program cannot use an offset map at all: its images are position-independent, but the x86-64 shared cache is pre-linked at `0x7ff800000000`, and in native mode the host's own CoreFoundation objects and malloc blocks have to be pointers the guest can hold, compare and hand back. So

`ocerz_mem_init_identity(256 GB)` reserves the arena with a hint at `0x300000000`, refuses one that lands lower, and sets `guest_base = 0`. A guest pointer is then a host pointer, and the JIT never adds a base. The loader keeps the first quarter of the arena, capped at 1 GB, and every other anonymous allocation bumps upward through a first-fit search of the slot table.

The arena has grown twice. In September 2026 Chromium's `PartitionAlloc` reserved a 32 GB region at startup and did not fit, and a day later JavaScriptCore's `Gigacage` asked for a single 128 GB mapping. Both are address-space reservations rather than memory, so a 256 GB arena costs nothing until it is touched.

HOW IT WAS FOUND

Every Wine process writes the address of its syscall dispatcher into a slot at `0x7ffe1000`, beside `KUSER_SHARED_DATA`. Under AArchX that slot shares a 16 KB host page with the file-backed shared data, so it is physically shared by every process in the Wine prefix. That is only harmless if the value is the same in all of them, which needs `ntdll.so` at the same address everywhere. With the arena wherever the host put it, each process stored an address the others had never mapped, the last writer won, and Wine's Notepad died while wineboot was still starting services. Since 2026-09-26 a process whose loader is named `wine` reserves its arena at `OCERZ_WINE_ARENA_BASE`, `0x7a0000000000`, and falls back to the usual placement only if that range is taken. `ntdll.so` now loads at the same address in every process of the session, the whole service tree starts, and wineboot finishes in about 15 seconds.

The low shadow

Windows code expects low addresses: PE images at 32-bit bases, `KUSER_SHARED_DATA` at `0x7ffe0000`, a whole 32-bit world for WoW64. Wine's macOS loader is therefore linked with a segment below 12 GB, and when the dynamic loader finds an image segment whose `vmaddr` is below `OCERZ_LOW_LIMIT` (`0x300000000`, 12 GB) it calls `ocerz_mem_init_low_shadow()`. Nothing else about Wine is special-cased here: any non-PIE program linked low gets the same map, which is how the test suite exercises it, by building ordinary tests with `-Wl,-image_base,0x200000000`.

The shadow is one host block large enough for the low 12 GB plus the top strip `[0x7ffffe000000, 0x7fffffe00000)`, 30 MB just below the commpage. It is reserved with `mach_vm_allocate(VM_FLAGS_FIXED)` at the first of four candidate bases that is free: `0x8000000000`, `0x10000000000`, `0x5000000000` and `0x600000000000`. The base chosen is exported as `OCERZ_LOWBASE`, and a child process reserves exactly that base or refuses to start, so every process in a Wine tree agrees where everyone's low memory lives. That agreement matters because the wineserver reads and writes its clients' memory with Mach VM calls aimed at their tasks (Chapter 9).

The cost is that a guest address is no longer a host address everywhere, so every memory access in a low-shadow process must decide which of the three ranges it is in. That made Wine the slowest of the three maps. Since 2026-10-04 the JIT does it with a shift, a compare and an `orr` for the common case, which took the `memcpy` kernel in that layout from 6.1 times Rosetta's time to 3.1 times; Chapter 7 has the details.

The commpage

macOS publishes kernel-maintained data, CPU capabilities among it, in a commpage at a fixed address that system libraries read directly. The guest's commpage is `[0x7fffffe00000, 0x7fffffe04000)` and must describe an x86-64 machine, not the arm64 host. AArchX allocates a 16 KB host buffer, fills its start from a fixed x86-64 template (signature, capability bits and CPU fields), and redirects that guest range to

it in `ocerz_g2h`. In identity mode the guest address itself cannot be mapped on the host, so a translated plain-form access to it faults, and the fault handler marks the block for guarded retranslation (Chapter 7). A rip-relative or absolute access whose address is known at translation time is resolved there and then.

Where the shared cache sits

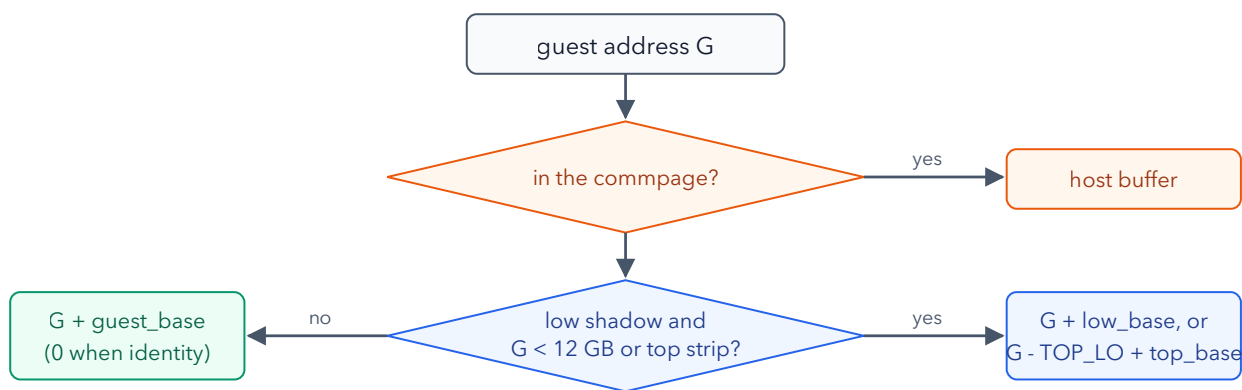
In cache mode the x86-64 shared cache is mapped by `src/cache.c` at its preferred address, `0x7ff800000000` and up, which is outside the arena and, with `guest_base = 0`, also its host address. Its data regions are mapped inaccessible and unpacked one 16 KB page at a time on first touch. Chapter 11 covers it.

Translating an address

Every access the interpreter makes, every pointer a syscall forwards, and every address the JIT cannot prove identity goes through two inline functions in `mem.h`.

```
static inline void *ocerz_g2h(uint64_t gaddr)
{
    if (ocerz_commpage && gaddr >= OCERZ_COMMPAGE_LO && gaddr < OCERZ_COMMPAGE_HI)
        return ocerz_commpage + (gaddr - OCERZ_COMMPAGE_LO);
    if (ocerz_low_base) {
        if (gaddr < OCERZ_LOW_LIMIT)
            return (void *) (uintptr_t) (gaddr + ocerz_low_base);
        if (gaddr - OCERZ_TOP_LO < OCERZ_TOP_HI - OCERZ_TOP_LO)
            return (void *) (uintptr_t) (gaddr - OCERZ_TOP_LO + ocerz_top_base);
    }
    return (void *) (uintptr_t) (gaddr + ocerz_guest_base);
}
```

`ocerz_h2g` is its inverse. Two predicates answer different questions on purpose. `ocerz_host_in_guest_space` is generous: fault attribution counts addresses below the arena as the guest's, because a NULL dereference is still the guest's fault, and in identity mode it counts the commpage range even though nothing is mapped there. `ocerz_host_in_guest_reservation` is strict and covers only memory AArchX actually reserved, which is what decides whether a pointer the kernel returned needs re-locating.



The JIT folds these tests into the emitted code: none in the identity map, a guard in the low shadow

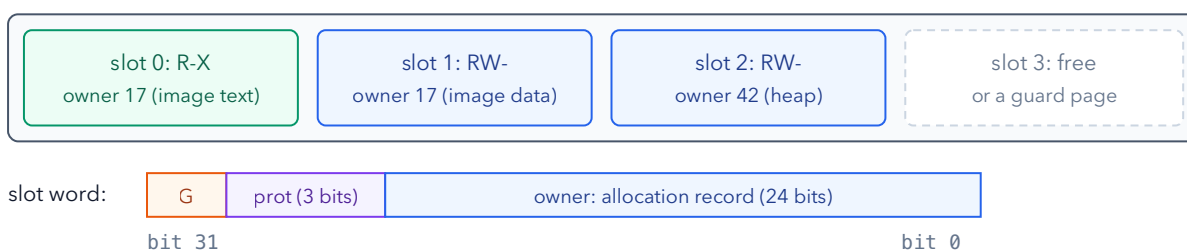
Figure 8.2. How `ocerz_g2h` decides where a guest address lives. In a dynamic program without a low shadow only the first test can succeed, which is why the identity map is the fast case.

The C accessors built on top, `ocerz_ld` and `ocerz_st`, use acquire loads and release stores for every naturally aligned access, and a fence around an unaligned copy. The interpreter therefore gives x86's ordering guarantees without help from the JIT's mode switch. The store side also carries software watchpoints: `OCERZ_WATCH` (with `OCERZ_WATCHLEN`) on an address range, `OCERZ_STVAL` on a stored value, and `OCERZ_SHADOWST` on any store of a host pointer into the low shadow.

4 KB guest pages on 16 KB host pages

A host page holds up to four guest pages, each of which can have its own protection, its own backing and its own lifetime. `mem.c` tracks guest memory in 4 KB **slots**. The address space is divided into up to 128 regions (the arena, the low window, the top strip, any extra range a fixed image needs), and each region carries four parallel arrays: a 32-bit word per slot, a bitmap, a byte per host page recording whether it is shared with another mapping, and a byte per host page recording whether it is armed.

One 16 KB host page



host protection = union of the slots' guest protections, with guest X becoming host R:

R-X | RW- | RW- | --- = RW- and minus W while the page is armed

Figure 8.3. A host page shared by four guest slots. Each slot word records its guest protection and the allocation it belongs to; a guard bit marks the uncommitted guard pages that separate allocations. The host page gets the union of its slots' protections, never guest execute, and loses write while code translated from it is live.

Several rules follow from that layout.

- **Guest execute is never host execute.** The host never runs guest bytes. Code is only read, by the decoder, the interpreter and the JIT, and translated code lives in its own `MAP_JIT` arena. A guest `PROT_EXEC` becomes host `PROT_READ`.
- **Protection changes land on 16 KB boundaries.** If one slot of a host page is read-only and its neighbour is writable, the host page is writable, so a stray write into the read-only slot is not caught. This is a documented limitation; making it exact would need a fault on every write to every mixed page.
- **Allocations carry guards.** An allocation the arena places itself ends on a host page boundary and is followed by one whole uncommitted 16 KB guard page, which its allocation record remembers, so two such allocations never share a host page and an overrun past the end of one faults.
- **The guest always sees 4 KB pages.** In cache mode `hw.pagesize` and `vm_page_size` are overridden. The override mirrors the real node's width: `hw.pagesize` by name is a 64-bit node and the legacy `{CTL_HW, HW_PAGESIZE}` MIB a 32-bit one, and writing 4 bytes into the 8 a caller offered left `sysconf(_SC_PHYS_PAGES)` returning 0, which killed `sort`. In native mode, since 2026-09-21, `getpagesize`, `sysconf(_SC_PAGESIZE)`, `host_page_size` and the page-size sysctls answer 4096 too, because V8 checks the page size against PartitionAlloc's compile-time constant.

Mapping in bulk

The general mapping path walks a range one slot at a time, several times over, and V8 reserves and releases gigabytes at startup. Since 2026-09-23 a mapping onto slots that are all free, starting on a host page, with no shared or armed page in reach, takes exactly one host `mmap` of the data pages and fills the slot array directly, because that `mmap` already leaves the memory zeroed, committed and at its final protection. A replacing mapping over nothing qualifies too, and that is every mapping at an address hint. Unmapping releases slots in runs of one owner, and a fully free unshared range is made inaccessible with one `mmap`. Reserving and releasing 1 GB went from 1.8 ms to 0.45 ms.

Zero runs

Wine's PE loader re-commits over live pages constantly, and the general path costs a `memset` and two `mprotect` calls per host page. A run of whole host pages that must all end up zeroed is replaced with one fresh anonymous mapping. Pages shared with another process are excluded, and so is one sibling slot: the dispatcher slot at `0x7ffe1000` described above. Zero-filling it for one process zeroed it for all of them, and until that process stored its pointer again every syscall elsewhere called address zero; Steam's CEF children died that way on 2026-09-06.

Mach mappings at 4 KB granularity

`mmap` goes through the slot machinery, but `mach_vm_map` is a message to the kernel, which works in 16 KB pages. A `FIXED|OVERWRITE` request at a 4 KB-aligned but not 16 KB-aligned address replaces whole host pages and takes the guest bytes on either side with it. The syscall layer copies those fragments out before the call and back after it. Safari asked for `0x4000` bytes at a `...e000` address inside the shared cache and lost the 8 KB below it, which held a constant a Swift precondition later checked; the program trapped one framework away from the cause.

Files and shared memory

Private file mappings

The host cannot map a file at every 4 KB offset, so a `MAP_PRIVATE` file mapping used to be read into anonymous memory in full. Since 2026-09-05, when the file offset and the guest address agree modulo 16 KB, which they do for nearly everything Wine maps (PE images at 64 KB-aligned bases, fonts, resources), the aligned interior is a real private mapping of the file, and its pages sit in the shared page cache rather than being copied per process. Partial head and tail pages, and anything past the end of the file, are still read in. A GUI Wine process had been carrying 280 MB of such copies, and one WineMine run read 5 GB of a single font file while enumerating fonts. `0CERZ_NO_FILEMAP=1` restores full reads.

Shared mappings

A read-only shared mapping that is not 16 KB-aligned used to become a private snapshot. Steam's UI stream is exactly that: `steam.exe` maps a ring buffer read-only while the browser process writes it. A snapshot never saw the writes, Steam re-created the stream forever, and no window appeared. The real shared overlay is now tried first, and the private copy is only the fallback. Writable shared mappings are communication channels between processes, so guest memory backing them keeps x86 ordering, and mapping one switches the JIT to ordered mode (Chapter 7).

System V shared memory

`shmat` and `shmdt` were not in the syscall table until 2026-09-22, so a guest got `ENOSYS` and continued with a null pointer. Brawlhalla, which talks to the Steam client through a System V segment, crashed at address zero. The host kernel refuses a fixed address for `shmat` even over ground the arena has reserved, so the segment is attached wherever the host wants it and `mach_vm_remap` carries that mapping into guest space as a share, not a copy, so writes from the other process show through. The host attachment is kept, so `shm_nattch` counts the process once, and `shmdt` reverses both halves. A guest-supplied address is honoured when it falls on a host page (`SHM_RND` rounds it down to one); anything else is `EINVAL`. Attaching switches the JIT to ordered mode, because a segment two processes share makes their ordering observable. The dynamic test `sysv_shm` uses a child process on purpose: a copy instead of a share would pass every check a single process can make.

Armed pages and kernel writes

A page that code was translated from is **armed**: its host protection loses write. A later guest store into it, from a JIT such as V8 rewriting its own code with plain stores into read-write-execute pages, faults; the stale translations are dropped, the page is unarmed, and the store retries. Only code in a guest-writable slot can be rewritten by a store, since code in a read-execute slot changes only through `mprotect` or `mmap`, which invalidate on their own. So a page is armed only when a writable slot overlaps the translated bytes. That leaves Wine's PE images alone where a `.text` tail shares a host page with a writable section. Chapter 7 covers the JIT's side of this.

The opposite direction matters as much. A kernel copyout onto a read-only page does not fault; it fails, and for a Mach receive the reply is destroyed with it. wineboot hung forever on exactly that on 2026-09-07. So before any syscall whose buffer the kernel writes, such as a read or a Mach receive, the sy-

scall layer unarms every armed page overlapping the buffer, and an `EFAULT` with armed pages about is re-
 tried once after unarming. A counter of currently armed pages lets the whole check be skipped when it is
 zero, which is almost always.

What the guest is told about its memory

A program that calls `mach_vm_region` on its own memory is asking about guest pages. Until 2026-09-22
 only the low-shadow map rewrote the answer; the identity map returned the host's view, in which a whole
 arena is one read-write-execute region, so an image's `__DATA_CONST` read back as writable. Electron checks
 a region it expects to be read-only and executes an `int3` otherwise, which is what killed Discord's render-
 ers. The reply is now rebuilt from AArchX's own slot table in every map, by `ocerz_guest_vm_region`; an
 address AArchX has no mapping for still gets the host's reply, since there is no other answer to give.

For the table to hold the right answer, the loader has to apply each segment's declared protection:
`initprot`, with `SG_READ_ONLY` segments made read-only. It cannot do that at load time, because Object-
 ive-C still writes into those segments, so each image is queued and the batch protected once Objective-C is
 done with it. Cache mode now reports 5, 1 and 1 for `__TEXT`, `__DATA_CONST` and `__LINKEDIT`, as a real
 system does. `max_protection` still reads 7, because AArchX keeps no per-segment maximum. Chapter 11
 has the loader's side.

Two diagnostics watch this boundary. `IMAGE-CLOBBER[pid]` is printed, always, when a guest fixed-address
`mmap` or a `munmap` lands on a segment of a loaded image, or an `mprotect` takes read or execute away
 from an executable one; a library unmapped under running code otherwise surfaces only as a wild jump
 much later. `OCERZ_MAPFAILLOG` names which test in the fixed-mapping path refused a guest
`mmap(MAP_FIXED)`, because an `ENOMEM` there is indistinguishable from real memory pressure, and Chromi-
 um's allocator treats a refused 4 KB commit as out of memory.

Host memory inside the guest's world

In the identity map, a pointer the kernel or a framework hands the guest is simply usable. Under the low
 shadow it is not: a host pointer below 12 GB names, from the guest's point of view, the guest's own
 memory at that number. The kernel and the frameworks produce such pointers constantly. An IOSurface's
 per-client page is mapped into the task by the kernel and handed over inline in an IOKit reply. Metal hands
 back buffer addresses. A Mach reply can carry an address the kernel chose. Most of the work of making
 Wine's graphics run has been in this one area.

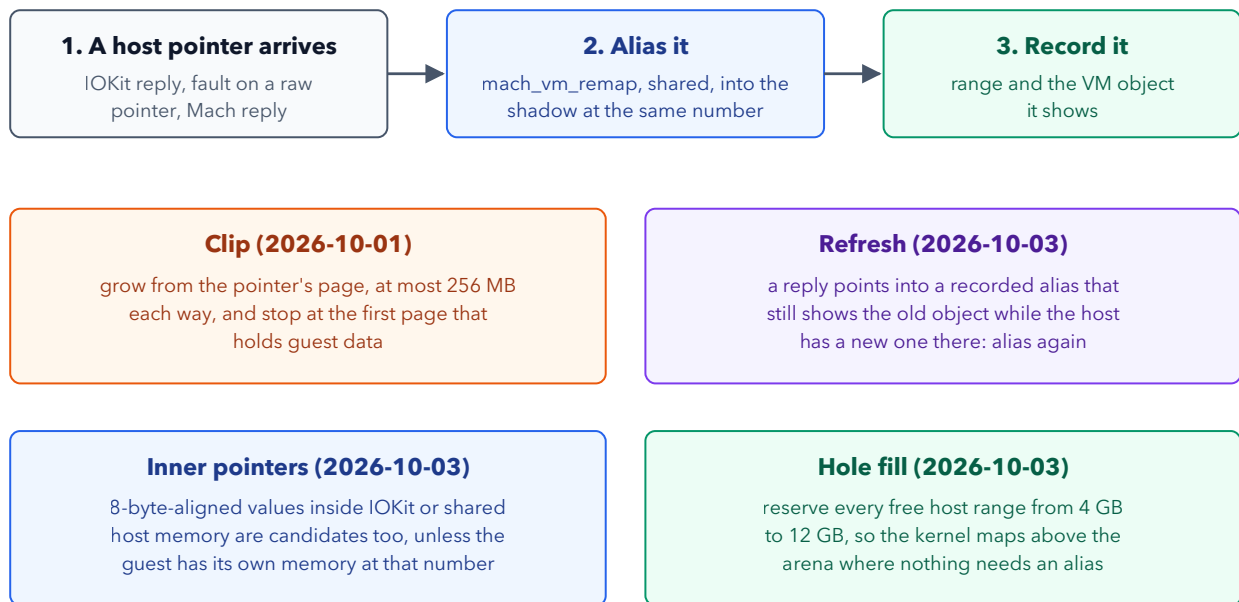


Figure 8.4. The life of an alias under the low shadow. A host region a pointer names is shared into the shadow window at the same number so the guest can read it. Four refinements, all driven by Wine graphics workloads, decide how much is aliased, when an alias goes stale, which values count as pointers, and how to stop the kernel creating the problem in the first place.

Aliasing

`alias_raw_region` in `src/syscall.c` asks the host for the region behind the pointer, maps the matching guest range, and shares the host pages into the shadow at the guest's number with `mach_vm_remap(..., VM_FLAGS_FIXED | VM_FLAGS_OVERWRITE, ..., copy = FALSE)`. The pages are shared, not copied, so the guest sees what the kernel and the GPU see. An alias also switches the JIT to ordered mode, because the other side of shared host memory is a second observer.

Clipping. The first version aliased the whole host region. A pointer into the host's arm64 shared cache once brought two gigabytes of it down on top of Wine's loader, whose header a framework then walked as four billion load commands until Chromium's GPU watchdog fired. Since 2026-10-01 the alias grows outward from the pointer's own page, at most 256 MB each way, and stops at the first page that holds guest data; a pointer whose own page holds guest data is not aliased at all. `OCERZ_NO_ALIAS_CLIP=1` restores the old reach.

Refreshing. The host does not keep its memory where it was. Metal frees a buffer and maps the next one at the same address, and an alias kept the freed buffer's VM object alive: the guest read the previous compute job's results, Metal's own completion state went stale with them, and a job could wait forever. R.E.P.O. got a nil drawable and aborted. Each alias is now recorded with the VM object it shows, and when an IOKit reply hands back a pointer into a recorded alias that still shows that object while the host has a different one there, the region is aliased again. An alias the guest has since covered with its own memory shows a different object and is left alone, and an alias of memory that has no VM object yet is not recorded, because fresh guest memory would look the same. `OCERZ_NO_ALIAS_REFRESH=1` turns it off.

Inner pointers. Creating an `IOSurface` answers with the surface's pixels and with two records inside memory the kernel shares with the process, at addresses that are not page-aligned. The reply scan took page-aligned values only, so those records were read through whatever the shadow held at their number. R.E.P.O.'s `CAMetalLayer` then fetched a drawable's properties into a buffer of garbage size, four gigabytes

with the right size in its low bits, the kernel refused with `kIOReturnVMError`, and the game aborted in about two launches of three. An 8-byte-aligned value inside IOKit or shared host memory is now a candidate as well, skipped when the guest has its own memory at that number. In fifteen launches after the change none failed. `OCERZ_NO_IOKIT_INNER_PTR=1` restores page-aligned values only.

Keeping the kernel out. The identity arena starts at 12 GB, so the kernel placed its own mappings in the free host space below it, which is where Wine and Unity keep their memory too. IOSurface's shared records landed on numbers the guest already used, where they could not be aliased, and R.E.P.O. failed to start in three launches of five. Since 2026-10-03 a low-shadow process reserves every free host range between 4 GB and 12 GB, at startup and again after each alias, so the kernel maps above the arena instead, where guest and host addresses are equal and nothing needs an alias. Eight launches of ten then reached the menu with no wild pointer at all. `OCERZ_NO_LOW_HOLE_FILL=1` turns it off.

Mach VM requests

Three more rules keep Mach memory calls honest under the low shadow.

- A `mach_vm_map(ANYWHERE)` is placed by hand at a guest-visible identity address. Relocating the kernel's answer afterwards would leave dangling host pointers wherever the kernel recorded the original, as IOSurface's bulk-attachment page showed. When AArchX does relocate an `ANYWHERE` reply, the requested alignment is honoured.
- A `FIXED` request without `OVERWRITE` is how Wine asks whether a range is free. The kernel only knows the host's numbers, so a range the guest already used looked free, and Wine mapped over its own loader at `0x200000000`. Since 2026-10-01 such a reply that lands on committed guest pages is given back and answered `KERN_NO_SPACE`, as the kernel would have answered (`OCERZ_NO_VMMAP_TAKEN=1` turns it off).
- A Mach reply must never land in the shared cache: moving it would deallocate a range every image is linked against, and Safari died one `mach_msg` later when the next routine in it faulted.

Requests aimed at another task, the way the wineserver reads and writes its clients, are covered in Chapter 9.

Native mode

In native mode the guest's heap is the host's: native `malloc` and native Mach calls hand the guest memory that no slot table has ever seen. `ocerz_mem_overlaps` answers whether a range touches memory AArchX tracks, and a range wholly outside it goes to the host kernel, after any translations of it are dropped, instead of being refused. `mach_vm_map` and `mach_vm_remap` cross too, with translations dropped first for a fixed target in the process's own task, and `mach_vm_map` never asks the kernel for execute.

The guest also sees views of the host's malloc zones rather than the zones themselves. Chromium writes its own hooks over the default zone's function pointers; those hooks are x86 code, and if they landed in the real zone the next native `calloc` would jump into x86 code. `malloc_default_zone` and zone enumeration therefore hand out views, and the host's real zones are never written. Chapter 12 covers the rest of native mode.

Footprint

Steam under Wine is nineteen emulated processes, so per-process memory matters. In early September a GUI Wine process cost about 1.1 GB: roughly 360 MB of JIT metadata, 300 MB of arena, and 280 MB of private copies of files it had mapped. Two changes took most of that away. The file interiors described above now come from the shared page cache, and a published translated block keeps a 16-byte reference per instruction instead of a full 96-byte decode, plus exactly the branch edges it has rather than a fixed table of eight. A WineMine process went from 1.1 GB to 714 MB and the Wine explorer from 860 MB to 611 MB, with the benchmark results unchanged. The test watchdog measures a process group's physical footprint rather than resident size, because resident size counts the shared cache again in every process.

Switches

Variable	Effect
<code>OCERZ_NO_FILEMAP=1</code>	read every private file mapping into anonymous memory
<code>OCERZ_NO_ALIAS_CLIP=1</code>	alias a host region's whole extent
<code>OCERZ_NO_ALIAS_REFRESH=1</code>	never re-alias a stale host region
<code>OCERZ_NO_IOKIT_INNER_PTR=1</code>	take only page-aligned pointers from IOKit replies
<code>OCERZ_NO_LOW_HOLE_FILL=1</code>	leave free host space below 12 GB to the kernel
<code>OCERZ_NO_VMMAP_TAKEN=1</code>	let a non-overwriting fixed <code>mach_vm_map</code> land on guest pages in use
<code>OCERZ_NO_VMMAP_STEER=1</code>	let <code>mach_vm_map</code> place mappings anywhere in host space
<code>OCERZ_NO_TEXT_R0=1</code>	leave image segments at the protection they were mapped with
<code>OCERZ_MAPFAILLOG=1</code>	name the test that refused a fixed mapping
<code>OCERZ_SYSFAIL=1</code>	log every guest <code>ENOMEM</code> and <code>EFAULT</code> , the signature of a missing pointer translation
<code>OCERZ_MIGTRACE=1</code>	log aliases, IOKit reply scans and collisions

CHAPTER 9

System calls, Mach, threads and signals

Everything the guest believes about the operating system is answered by AArchX. Most guest system calls cannot be passed to the kernel as they are: they carry guest pointers, structures whose layout differs between the architectures, thread ports that must name guest threads, or results the guest has to see the way Rosetta presents them. This chapter covers the boundary in `src/syscall.c` and the process machinery in `src/vm.c`: dispatch, the return convention, Mach messages, threads and the workqueue bridge, thread suspension, signals, faults, and processes.

8,633

LINES IN SRC/
SYSCALL.C

4,270

LINES IN SRC/VM.C

375

BSD TABLE ENTRIES,
59 INTERCEPTED

53

MACH TRAP NUMBERS
HANDLED

How a guest enters the kernel

A guest `syscall` instruction ends its translated block, and the JIT, or the interpreter, calls `ocerz_handle_syscall` with the guest's register file. On macOS the x86-64 convention puts a class in the top byte of `rax` and the call number in the low 24 bits, so the first thing the handler does is split them.

Class	Meaning	Handler	Returns through
1	Mach traps	<code>dispatch_mach</code> , a switch over 53 trap numbers	<code>rax</code> , a <code>kern_return_t</code>
2	BSD (unix) calls	<code>dispatch_bsd</code> , a table of 375 entries (room for 600), 59 with their own handler	<code>rax</code> and <code>rdx</code> , carry flag on error
3	machdep calls	<code>dispatch_machdep</code> : <code>thread_set_tsd_base</code> , and <code>i386_set_ldt</code> / <code>i386_get_ldt</code> for WoW64	<code>rax</code> only
other	not a macOS call: a Windows system call issued directly by PE code	delivered to the guest as <code>SIGSYS</code>	the guest's handler

The last row is how Wine works on macOS: PE code that issues a raw Windows system call is caught by Wine's own `SIGSYS` handler. Before the signal is delivered, a thread whose segment base is not on its Wine thread block is switched to it, because that handler runs on the Windows side.

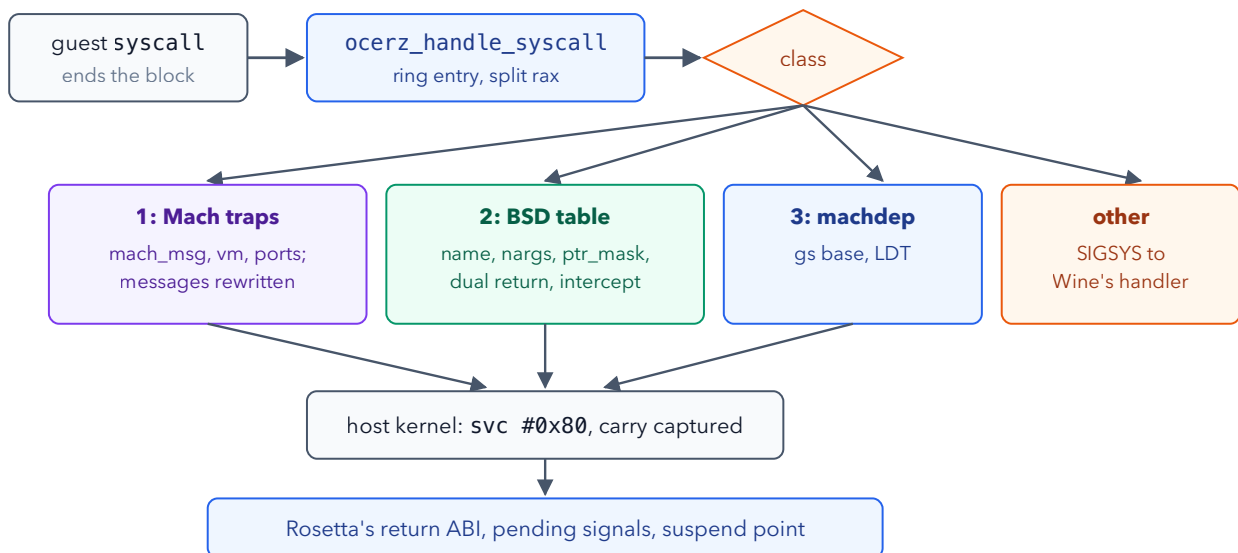


Figure 9.1. A guest system call. The class in the top byte of `rax` picks a dispatcher; most calls end in a raw `svc #0x80` from `sys_raw.h`; on the way back the result is written the way Rosetta writes it, signals the guest mask allows are delivered, and a pending suspension is honoured.

The raw kernel entry

`include/ocerz/sys_raw.h` is the one place AArchX enters the arm64 kernel without `libSystem`. It loads eight arguments into `x0` to `x7`, the number into `x16`, issues `svc #0x80`, and reads the carry flag with `cset ..., cs` in the same asm block, so nothing can change the flags between the trap and the test. A Mach trap passes its number negated, as it does natively.

The BSD table

Each entry in `bsd_table` carries the call's name, its argument count, a `ptr_mask` whose bits name the arguments that are guest pointers, a flag for calls that return a second result in `rdx`, and an optional intercept function. Arguments come from `rdi`, `rsi`, `rdx`, `r10`, `r8` and `r9` (`r10` rather than `rcx`, because `syscall` overwrites `rcx`), and a seventh and eighth are read from the guest stack. Call number 0, the indirect `syscall(2)`, shifts every argument down one register and re-dispatches. A plain entry is forwarded with its pointers translated through `ocerz_g2h`; an intercepted entry runs AArchX's own implementation. 76 calls became plain pass-throughs on 2026-09-13, when real applications started asking for them.

In the identity map, translating a pointer changes nothing, which is why a few calls whose structures hide further pointers that `ptr_mask` cannot reach (`connectx`, `sendfile`, `recvmsg_x`) are correct only there. Outside identity mode they return `ENOSYS` rather than let the kernel read or write the wrong memory.

The return convention is Rosetta's

The guest's `libSystem` stubs were written against what XNU does on an Intel Mac, and the applications AArchX runs were tested under Rosetta. So results are returned the way Rosetta returns them, measured with probes on 2026-09-03:

- A unix call returns through `rax` and `rdx`, on success and on error alike, with `rdx` zero unless the call fills it. A caller whose wrapper forgot to declare `rdx` clobbered sees zero there, as on Rosetta.

- The arithmetic flags come back in a fixed pattern whatever the caller had: `PF|AF` on unix success, `CF|ZF|PF` on unix error, `AF` after a Mach trap. Only `CF` is meaningful, since the stubs branch on it, but a program that reads the rest sees what it would see under Rosetta.
- Machdep calls are the exception and return through `rax` alone, leaving `rdx` and the flags untouched. Wine's syscall dispatcher keeps its frame pointer in `rdx` across `thread_set_tsd_base`, and applying the unix rule there gave every Wine process a NULL dereference at its first return to PE code (fixed 2026-09-04, guest test `machdep_rdx`).

Failure policy

An unimplemented call reports itself once and returns `ENOSYS`; an unknown Mach trap returns an error. In June the layer aborted instead, which killed the guest thread where it stood, and under Wine that is often inside the loader holding its lock, which deadlocks every other thread. `OCERZ_STRICT_SYSCALL=1` restores the abort for bring-up work. One more rule lives here: a contended `os_unfair_lock` whose owner died is broken the way a robust mutex is, because the system's reaction to the kernel's `EOWNERDEAD` is to crash the process, and that took Steam's web helper down mid-startup in August.

Pointers the kernel reads and writes

Three kinds of argument need more than `ptr_mask`.

Buffers the kernel writes. A page that code was translated from has lost write on the host (Chapter 8). A kernel copyout onto such a page fails instead of faulting, and a Mach reply is destroyed with it. Before a call whose buffer the kernel writes, every armed page overlapping that buffer is unarmed, and an `EFAULT` with armed pages about is retried once.

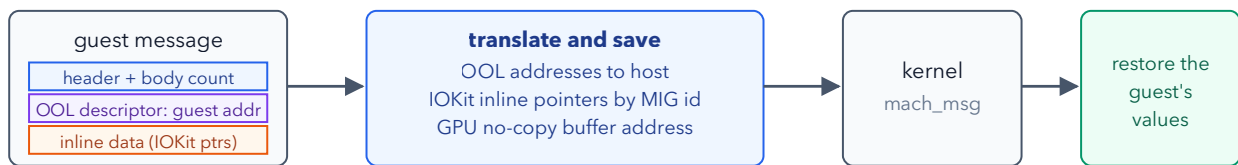
Structures whose layout belongs to someone else. `__mac_syscall` carries a policy name and an argument structure whose layout belongs to the policy (Sandbox, Quarantine and others), and that structure points at more memory, usually the caller's out-structures and locals on its stack. In the identity map that works by accident. Under Wine the Unix-side code runs on a stack in the low-shadow window, the kernel received guest addresses, and `sandbox_check` failed with `EFAULT`. The Metal OpenGL renderer took that as no access to the GPU, every `CGLChoosePixelFormat` from a Wine thread returned 10002, `wined3d` found no GL adapter, and Counter-Strike 2 stopped at an error box. Since 2026-09-26 each of the structure's first twelve words that points from 64 KB below to 1 MB above the caller's `rsp` is rewritten to its host address for the call and put back before the guest runs again. Any other word is left alone, because an integer that happens to look like a low address must not change. `dxdiag` under Wine then reported the real GPU with an OpenGL 4.1 core context. The dynamic test `mac_syscall_low_stack` pins it.

Information the host has and the guest must not see. `sysctl` is answered with the guest's view: 4 KB pages, at the width of the node the caller asked through, and an x86 CPU description synthesized to agree with AArchX's own CPUID (SSE through SSE4.2, `CX16`, `POPCNT`). The arm64 kernel has no `machdep.cpu` nodes at all, so passed through they failed with `ENOENT`, and code that asks `hw.optional.sse4_1` before taking a fast path took none (dynamic test `x86_sysctl`). `proc_pidpath` and `proc_name` of the process itself name the guest executable, not `ocerz`.

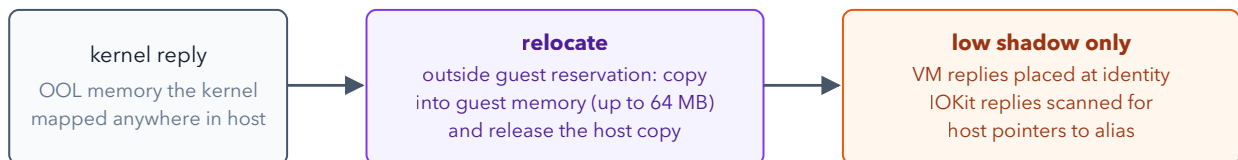
Mach messages

A Mach message is a header followed, in a complex message, by descriptors for ports and out-of-line memory, then inline data. The kernel copies out-of-line memory by address, so every address in a message has to be one the kernel can use, and every address the kernel puts in a reply has to be one the guest can use. In the identity map most of this is free. Under the low shadow, and for a handful of IOKit calls that hide addresses inside their inline data, it is most of the work.

Send



Receive



Never rewritten: MACH_RCV_INTERRUPTED on a combined send and receive (turning it into a timeout made CFRunLoop exit). Never placed: a reply inside the shared cache.

Figure 9.2. Translating a Mach message. On send, every guest address the kernel will dereference is replaced with its host address and put back afterwards. On receive, memory the kernel mapped outside the guest's reservation is moved where the guest can see it.

On send

`ocerz_send_xlate_descriptors` walks a complex message's out-of-line descriptors and replaces each guest address with its host address, saving the original so it can be restored after the call. Several IOKit calls hide addresses in their inline data, where no descriptor walk can see them, and are recognised by their MIG message id:

- `io_registry_entry_get_property_bin_buf` and `..._properties_bin_buf` (2888, 2889) carry an inline out-buffer pointer. Untranslated, the call failed with `kIOReturnVMError`, and GPU enumeration under Wine failed with it.
- `io_connect_method` and `io_connect_async_method` (2865, 2866) carry `ool_input` and `ool_output` addresses the kernel copies from directly. Untranslated, `IOSurfaceClientCopyValue` failed in every DXMT swap chain (fixed 2026-09-08).
- Metal's `newBufferWithBytesNoCopy` hands the GPU memory the caller owns, by address, in the structure input of one `io_connect_method` selector. The kernel wires the host pages at that address; under Wine it was a low-shadow guest address, so the GPU read and wrote whatever host memory lay at the guest's number, and R.E.P.O.'s first frame died on a GPU address fault. The call is recognised by the structure's shape, since IOKit will not name the class behind a connection port, and the address is translated for the call (2026-10-03). It need not be page-aligned: DXMT's ring allocator hands Metal memory from the Windows heap.

Mach trap 70, `host_create_mach_voucher_trap`, takes its recipes and the place for the new voucher by address and went to the kernel untranslated. From a Wine thread both live on a low-shadow stack, the kernel answered `KERN_MEMORY_ERROR`, and `libdispatch`, which creates vouchers for the work it queues, crashed on purpose. Its sibling trap 72 already translated its pointers; trap 70 does now too.

On receive

Out-of-line memory in a reply is mapped by the kernel wherever it likes in the host. `ocerz_reply_relocate_oof` checks each descriptor: memory outside the guest's reservation, and not in the shared cache, is copied into a fresh guest mapping (up to 64 MB) and the host copy released, and the descriptor is rewritten to the guest address. Replies to Mach VM calls that return an address (`mach_vm_allocate`, `mach_vm_map`) go through `mig_vm_reply_relocate`, and under the low shadow an IOKit reply (ids 2900 to 2999) is scanned for host pointers to alias into the shadow, as Chapter 8 describes. One reply is deliberately never rewritten: `MACH_RCV_INTERRUPTED` on a combined send and receive. Rewriting it to a timeout made `CFRunLoop` exit (2026-08-24).

Requests aimed at another task

The wineserver reads and writes its clients' memory with `mach_vm_read`, `mach_vm_read_overwrite`, `mach_vm_write`, `mach_vm_protect` and `mach_vm_msync` aimed at their tasks. Those name the other task's guest addresses. Since 2026-10-01 a low address is moved by the shadow base the whole Wine tree inherits in `OCERZ_LOWBASE`, and the top strip by `OCERZ_TOPBASE`, before the kernel sees it. Untranslated, Chromium's sandbox read a child's thread block from the wrong memory and every launch of its GPU process failed with sandbox error 43. `OCERZ_NO_PEER_VM=1` turns it off.

Thread identity

`thread_info(THREAD_IDENTIFIER_INFO)` answers with the thread's TSD base, which the kernel reports as the host's. It is replaced with the guest TSD of the guest cpu whose host thread owns that base, the pthread one even while Wine has `gs` pointing at its Windows thread block, found through the host TSD base each cpu records. Wine saves this value and switches `gs` back to it on every system call, and a host thread whose stack happened to land above 12 GB used to pass its own TSD through untouched, so that Wine thread lost its thread block and died at its first server call.

Threads

Every guest thread is a real host pthread running a guest cpu: an `OcerzCPU` structure holding the x86 register file, the segment bases, the signal mask and pending set, and bookkeeping for the JIT. The host thread runs the master loop in `vm.c`, which executes translated code until something needs the C side.

Creating threads

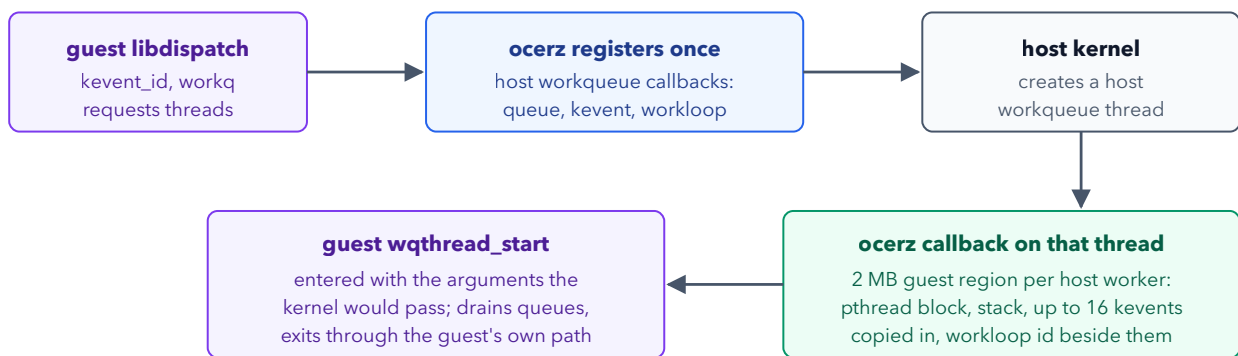
The guest's `libpthread` starts by calling `bsdthread_register` with the entry points for new threads (`pthread_start`) and for workqueue threads (`wqthread_start`). `AArchX` records both and answers with a feature word that includes workloop support whenever the host workqueue bridge is on, so that the guest's `libdispatch` chooses the model the bridge actually delivers.

`bsdthread_create` starts a host pthread and gives it a copy of the creator's cpu, taken inside the creator's own system call. Nothing about the creator's own wait or suspension comes along: a creator suspended by a garbage collector at that moment would otherwise hand the child a suspend count of one, and the child would park at its first safe point with nobody left to resume it. The signal mask is inherited, as POSIX requires. The call blocks until the new thread's kernel port is in its pthread structure, as the real kernel does; without that, a `pthread_join` right after `pthread_create` can read an empty port slot, conclude the thread has exited, and free a live thread's stack. The guest's segment base is set by the machdep call `thread_set_tsd_base`, which updates the cpu's `gs` base.

A process whose main thread AArchX builds by hand has that thread's cached thread id filled in. Without it, `pthread_threadid_np` returned 0, and the mutex protocol that stores that id as the lock owner treated owner 0 as unlocked, so mutexes taken by the main thread had no mutual exclusion at all. That was the Wine explorer freeze of 2026-08-18.

The host workqueue bridge

libdispatch does its work on workqueue threads that the kernel creates on demand, and AppKit hands work to them and waits. Without anything servicing those queues, the first `dispatch_sync` parks the main thread forever: Calculator, TextEdit and Safari all registered as foreground applications without ever reaching the window server. The bridge was opt-in until 2026-09-10 and is on by default now.



Registering retires the plain memory model, since a worker is a second observer of guest memory, and starts the unstick monitor. Native mode leaves the bridge off: the host's own libdispatch needs the process's single workqueue registration.

Figure 9.3. The host workqueue bridge. AArchX takes the process's workqueue registration for itself, and when the kernel starts a worker it builds a guest thread around it and enters the guest's own `wqthread_start`.

On the first guest request that needs it, AArchX registers its own callbacks with `_pthread_workqueue_init_with_workloop`, the libpthread entry point that claims the process's workqueue. When the kernel then starts a host worker and calls back, the callback gives that host thread a 2 MB guest region holding a guest pthread block, a stack and an event buffer, copies in up to 16 kernel events and the workloop id, and enters the guest's `wqthread_start` with the arguments the kernel would have passed. The region is kept for that host thread and reused. A `REQTHREADS` request is always forwarded to the host when the bridge is on (since 2026-09-19). Workqueue threads leave through the guest's own thread-exit path since 2026-09-13; WebKit's allocator could not suspend a thread that had ended without it, which stopped both Safari and the Ollama menu-bar app.

Native mode turns the bridge off outright. Registering claims the process's single workqueue slot, and in native mode the host's own libdispatch is what needs it.

Suspending threads

JavaScriptCore's garbage collector suspends each mutator thread, reads its registers with `thread_get_state` and scans its stack. Chromium's stack sampler does much the same. Handed to the kernel, `thread_suspend` froze an AArchX thread wherever it was, including inside the translator holding the JIT lock, after which every other cpu, the collector included, blocked on that lock for good. And an x86 `thread_get_state` on an arm64 thread fails outright.

So in cache mode `thread_suspend`, `thread_resume` and `thread_get_state` on guest threads are emulated (since 2026-09-11, the main thread since 2026-09-13). A suspend reports done only once the target holds no AArchX lock: parked at a safe point, inside a raw host wait, or stopped by the kernel with its program counter in a translated block, where the pinned host registers are its guest registers. Anywhere else, in the translator, a C helper or the interpreter, it is let go and caught a moment later. `thread_get_state` reports x86 registers reconstructed from the arm64 registers holding them. `OCERZ_NO_THREADACT=1` hands these calls back to the kernel.

In native mode, since 2026-09-21, the calls are bridged and the kernel call is real, because a cooperative stop would leave the suspend count somewhere the caller can tell. What makes it safe is refusing to leave a thread stopped where it would deadlock the caller: while its per-thread critical depth is non-zero, which means it holds the JIT lock or the memory-map lock, or while its program counter, link register or any of twelve frames up its stack is inside the host's allocator, since the suspender's next `malloc` would wait for a lock the target holds. Such a target is resumed and tried again every 100 microseconds, up to 50,000 times.

Signals

AArchX emulates the guest's signal state completely: the handler table, the per-thread mask, the pending sets, the alternate stack, and the frames handlers run on. The host's own signal machinery is used only to receive signals and wake threads.

The host side

Because AArchX emulates the guest's mask itself, the host thread must never block asynchronous signals. The wineserver suspends guest threads and kicks them for asynchronous procedure calls with `SIGUSR1` sent to the host thread, and a host mask that blocked it left the thread suspended server-side forever; Steam's CEF browser deadlocked that way on 2026-09-06. XNU starts libdispatch workqueue threads with all asynchronous signals blocked, and a pthread created from one inherits that mask, so the mask is cleared whenever a host thread starts running guest code.

A guest `sigaction` for a plain asynchronous signal is also mirrored onto the host, because the host kernel decides what `SIGPIPE` does to the process, and its default is to kill it silently. The wineserver ignores `SIGPIPE` and relies on `EPIPE`; without the mirror the whole server vanished the first time a client died with a reply in flight.

When signals are delivered

A host signal handler only records the signal. Delivery to the guest happens at well-defined points.

- **On the way out of a system call**, every pending signal the guest mask allows is delivered.
- **On the way in**, pending signals are offered too, with `rip` rewound so the call re-executes after the handler. The return edge alone left a signal unnoticed until some later call returned, and if the next call blocked, it parked with the wakeup undelivered.
- **After a bridged call** in native mode, the equivalent of a system call return.

Asynchronous signals obey the guest mask; synchronous faults never come through here and are never gated. Enforcing the mask is what keeps Wine's server protocol in one piece. Wine blocks a signal set around each request and reply to the wineserver, so that a suspend kick lands only between calls; delivering one in the middle nested a second round trip inside the first on the same strictly ordered pipe, until every client parked in `read` forever (2026-08-25).

Since 2026-10-01 a signal that arrives on a thread running a guest cpu stays pending on that cpu, because Wine aims `SIGUSR1` and `SIGQUIT` at one thread and its handler expects to run there. Before, it went to whichever thread reached a system call first. Only a signal that reaches a thread with no guest cpu goes to the process-wide pool, and a thread takes from the pool only the signals its mask accepts. `OCERZ_NO_THREAD_SIGNALS=1` pools them all again.

A self-directed `pthread_kill`, which is how `abort()`, `raise()` and every Chromium CHECK end, is delivered to the guest rather than forwarded. Forwarding raised a real `SIGABRT` on the AArchX process, the system crash reporter started and pegged a core, and under Steam's GPU-process crash loop that starved the emulation past Steam's own watchdogs.

The limit of this design: an asynchronous signal reaches a guest thread only when that thread next makes a system call or, in native mode, a bridged call. A thread spinning in its own code without calls never sees one.

The signal frame

`ocerz_signal_deliver` builds the frame the kernel would build on an Intel Mac, on the alternate stack if the handler asked for it and one is set, otherwise below the interrupted stack pointer after skipping its 128-byte red zone.

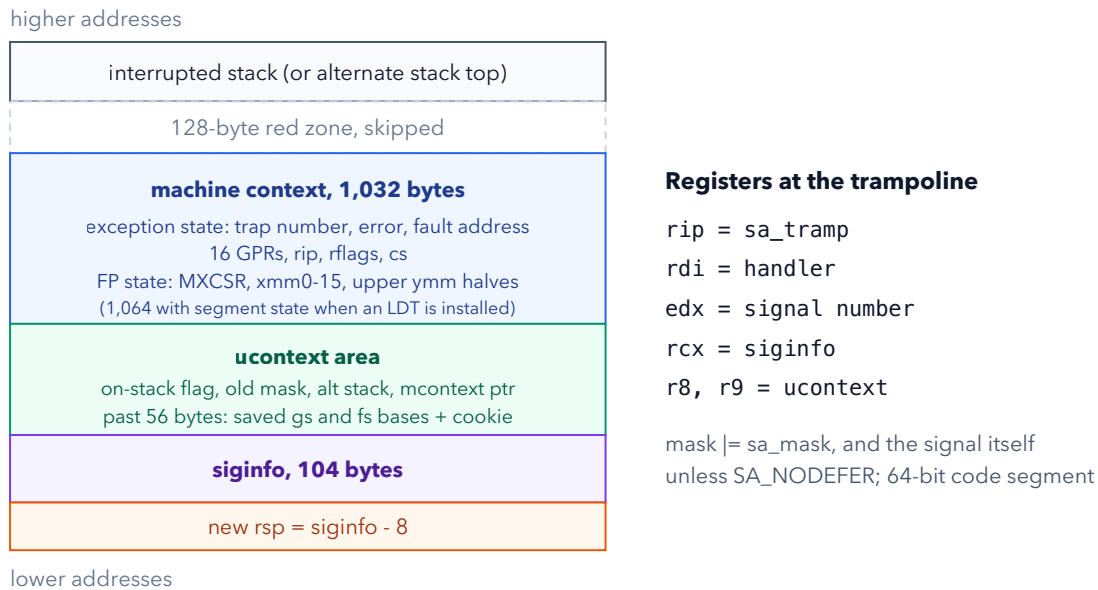


Figure 9.4. The signal frame AArchX builds, in the layout an x86-64 macOS handler expects. The machine context carries the upper halves of the `ymm` registers, so a handler that returns leaves AVX code intact.

The frame layouts are the ones in the public macOS headers: an AVX machine context of 1,032 bytes, or 1,064 with segment state when a 32-bit LDT is installed, a `ucontext`, and a 104-byte `siginfo`. The trampoline the guest's libc registered as `sa_tramp` is entered with the handler in `rdi`, the signal in `edx`, the `siginfo` in `rcx` and the `ucontext` in `r8` and `r9`, and calls the handler and then `sigreturn`. The cpu switches to 64-bit mode for the handler, since a signal can interrupt 32-bit WoW64 code. Frames carry the upper `ymm` halves since 2026-09-13.

Several details have their own history.

- **Two `sigaction` layouts.** The kernel takes a `struct __sigaction`, 24 bytes with the trampoline at offset 8, and returns a `struct sigaction`, 16 bytes without one. Writing the inbound layout to `oldact` overran the caller's buffer by 8 bytes; in `sort` those bytes were a saved `rbx`.
- **`sigreturn` leaves the `gs` base alone.** The real kernel saves and restores the `gs` base across a signal only for a task with an LDT, so a base a handler sets survives the return, and Wine's handler relies on that to install its thread block before it goes back to PE code. AArchX stashes the interrupted bases past the 56-byte `ucontext` of frames it builds, behind a cookie, and had been putting both back, which undid Wine's switch: PE code ran on the pthread base until something read `%gs:0x30`. Since 2026-09-26 only `fs` comes back from the stash.
- **Alternate stacks.** `sigaltstack` reports `SS_DISABLE` when no alternate stack is set, which Go needed, and refuses a stack smaller than 32 KB with `ENOMEM`, as the kernel does.
- **Waiting for signals.** `sigsuspend` and `__sigwait` are emulated against the guest's pending set; passed to the host they waited for a signal the host kernel could never deliver.

Native mode has no x86 libc in front of these calls, so `sigaction`, `signal`, `sigprocmask`, `sigaltstack`, `raise` and `pthread_kill` arrive as bridged calls and run the same bodies as the system calls, with the register plumbing peeled off, against the same handler table and mask. A raw system call and a bridged call

can never disagree. What native mode lacks is the trampoline, so AArchX writes a small x86 one of its own into a guest page and makes it read-execute; it moves the arguments into place, keeps the `ucontext` in callee-saved registers across the handler, and issues `sigreturn` as a system call.

Faults

The host fault handler in `vm.c` is where most of the emulation's hard edges meet. In order, it tries the recoveries that resume the faulting instruction:

1. A first touch of a lazily rebased shared-cache data page: unpack the page and retry (Chapter 11).
2. A fault while the translator reads guest bytes, for a decode or a stored translation's dependency check: abandon the read before anything takes the JIT lock the translator already holds.
3. A raw host pointer the guest was handed inline: alias it into guest space and retry (Chapter 8).
4. An alignment fault in translated code, meaning an ordered access crossed a 16-byte boundary: hot-patch that one access into an alignment-checked form and re-execute (Chapter 7).
5. A host-stack overflow of the return-address shadow: interpret the call and abandon the JIT frame.
6. A store into a page armed because code was translated from it: this is the self-modifying-code notification, so grant write, drop the stale translations and retry.
7. A plain-form access to the emulated commpage: mark the block for guarded retranslation.

Only then does it look for a bridged call in flight. In native mode a guest call into a system library runs the host's own arm64 code on a guest thread, so a bad pointer handed to `strcpy` faults with no guest instruction to blame. Such a thread is several native frames deep in code AArchX can neither resume nor unwind, so the process stops with a report that names the library, symbol and signature, and says whether the fault address was in guest space (the guest passed a bad pointer) or outside it (AArchX marshalled the call wrongly). The order matters: a guest writing into a page it has already executed must still be recovered even when the store came from a bridged `memcpy`.

Anything not recognised is delivered to the guest as an access violation at the faulting instruction, rather than killing the thread. Killing a thread leaves every lock it held taken forever: a V8 background job died that way holding a mutex, and the renderer's garbage-collection safepoint waited on it until the end of time. `SIGILL`, `SIGTRAP` and `SIGSYS` go through the crash reporter too, since a jump to a garbage code pointer used to end the process silently. A fatal fault in 32-bit guest code takes the whole process down on purpose: it usually lands on a WoW64 thread, and a half-alive process wedges its parent, which waits for the child forever. The crash report's first line names AArchX's own load address, so a host program counter in it can be handed to `atos -l`.

The unstick monitor

A guest thread can park in a blocking host wait whose wakeup was lost, for example through a kernel event edge or an alias between waiter and waker. During bring-up a manual `EINTR` nudge always revived such a session, so the monitor automates it: any cpu inside one blocking host call for more than 800 ms is sent a host signal deliberately installed without `SA_RESTART`. Only waits whose callers already loop on a spurious return are kicked: `psynch`, `kevent`, `workqueue`, `unlock` and `Mach waits`. A cpu in `read`, `recvmsg`, `poll` or an `fcntl` lock is marked `block_nokick` and left alone, because those never return `EINTR` on their own and applications rightly do not expect it. Semaphore waits are not kicked, and a timed

`__semwait_signal` waits out a kick (2026-09-13). The same no-op handler also serves as a context-synchronisation event, so a thread spinning in translated code observes the stop-site patches the JIT makes when it needs every thread out of the code arena (Chapter 7). `OCERZ_NO_UNSTICK=1` disables the monitor and `OCERZ_UNSTICK_ALL=1` kicks every blocking call, as it once did.

Processes

fork

A fork child keeps the JIT. In August children dropped to the interpreter, which looked like a Steam network hang and was really a child at 96% CPU interpreting. The child does abandon the inherited `MAP_JIT` arena, whose pages read fine but raise `SIGBUS` when executed, and fork is never executed from inside compiled code. The child forgets the parent's other threads. The JIT's, the memory map's, the shared cache's and the initializer gate's locks are all taken around the fork, the way a careful threaded program does, so the child never inherits a lock that nobody will release.

exec and spawn

`execve` and `posix_spawn` run the new program under AArchX, keeping the spawn's file actions and attributes. A universal binary runs its x86-64 slice. Since 2026-09-23 a binary with an arm64 slice and no x86-64 slice runs natively in every mode, as Rosetta runs it: Steam spawns `/usr/bin/open` three times a launch, and on macOS 27 `open` ships arm64 only, so every one of those children had died on "cannot read" (dynamic test `spawn_arm64_only`). In native mode a system tool with an arm64 slice runs as itself too, unless it is a shell or launcher whose own children must still be translated. The mode travels to children as `OCERZ_MODE` in the environment the spawn and exec paths build, even for a child started with an empty environment.

launchd

The macOS Steam client starts its `ipcserver` with `launchctl load -S Background` on a property list it writes, which would have launchd run the server natively. AArchX rewrites that plist on the way through, putting itself in front of the program, and enables the job's label first: a legacy `launchctl unload` leaves the label disabled, and every later load failed with an I/O error while the client reported `ipcserver init failed`. Chapter 13 covers the rest of Steam.

System V IPC

Semaphores, shared memory segments and message queues have been implemented since 2026-09-11, because Steam's threading library uses System V semaphores; the bootstrapper ran for the first time that day. `shmat` and `shmdt` followed on 2026-09-22 and are described in Chapter 8.

Diagnostics

This layer has more diagnostics than any other, because a missing translation looks, from inside the guest, like an ordinary failure.

Switch	What it shows
<code>-strace</code> , <code>OCERZ_STRACE_EXE=<text></code>	every call with its guest arguments; the second limits it to processes whose command line matches
<code>OCERZ_SYSFAIL=1</code>	every <code>ENOMEM</code> and <code>EFAULT</code> , which is what a missing pointer translation looks like from inside
<code>kill -INFO <pid></code>	every guest thread: host thread id, <code>rip</code> and <code>rsp</code> , the call it is blocked in, its Windows-side stack, and its last 24 system calls
<code>OCERZ_SYSRING=<n></code>	one process-wide ring of that many system calls, each tagged with its thread; on a wineserver it shows every request, reply and wakeup
<code>OCERZ_SIGMASKLOG=1</code>	every change of a guest mask to a wide one, with the caller chain
<code>OCERZ_MACHMSG=1</code> , <code>OCERZ_MSGHEX=<id></code>	Mach message results, and full request and reply bytes for one id
<code>OCERZ_IOKITERR=1</code> , <code>OCERZ_IOKITSEL=<sel></code>	<code>io_connect_method</code> calls that fail with a VM or argument error, or every call with one selector
<code>OCERZ_MSGNEEDLE=<value></code>	every outgoing message that carries a given 64-bit value
<code>OCERZ_MACSYSLOG=1</code>	each <code>__mac_syscall</code> with its policy, number and argument words
<code>OCERZ_HOSTSIG_PROBE=1</code>	each thread's real host mask and pending set, and the last 512 host signals received

HOW IT WAS FOUND

`OCERZ_SIGMASKLOG` exists because of eight threads spinning in R.E.P.O. with every signal blocked. The log showed that each had been through libc's `abort()`, called from Metal's validation layer, whose `SIGABRT` Wine had turned into an exception while the thread kept `abort`'s mask. The wide mask was a symptom; the cause was the validation failure that called `abort`.

PART IV

Linking

Loading Mach-O images and binding them, against Apple's x86-64 shared cache or against the Mac's own arm64 frameworks.

CHAPTER 10

The Mach-O loader

Before a single x86 instruction runs, AArchX has to decide what kind of program it was handed, pull the right slice out of the file, lay its segments into guest memory and build the stack the kernel would have built. This chapter covers that front door: `src/main.c`, the static loader in `src/loader.c` and `src/stack.c`, and the parts of the dynamic path that shape the process before the mini-dyld of Chapter 11 takes over.

378

LINES IN LOADER.C

137

LINES IN STACK.C

419

LINES IN MAIN.C

8 MB

INITIAL GUEST STACK

Two kinds of program, one front door

AArchX runs two very different kinds of Mach-O. A **static** program carries an `LC_UNIXTHREAD` command with its initial register state and links against nothing: the freestanding binaries in `tests/guest` are the main examples, built with `-nostdlib -static`. A **dynamic** program names a dynamic linker and libraries, and that is every real macOS program, from `/bin/ls` to Steam. The two take different paths through the code, and the decision is made once, in `main()`, before the virtual machine is set up.

The question is answered by `ocerz_peek_dynamic`, which reads the file, selects the x86-64 slice and scans its load commands without mapping anything. The rule is short:

- an `LC_MAIN` command (0x80000028) makes the program dynamic, and the scan stops there;
- an `LC_UNIXTHREAD` together with `LC_LOAD_DYLINKER` is also dynamic, which is how binaries linked for old releases of macOS start;
- an `LC_UNIXTHREAD` alone is static;
- a file with no x86-64 slice at all is not an Intel program, and AArchX says so and exits with 64 rather than pretending it cannot read the file;
- a file it cannot read exits with 65.

Native mode (Chapter 12) adds one more refusal. Its whole mechanism is standing in for the libraries a program links, so a static image has nothing to bridge into and is refused with exit 64 before the VM starts.

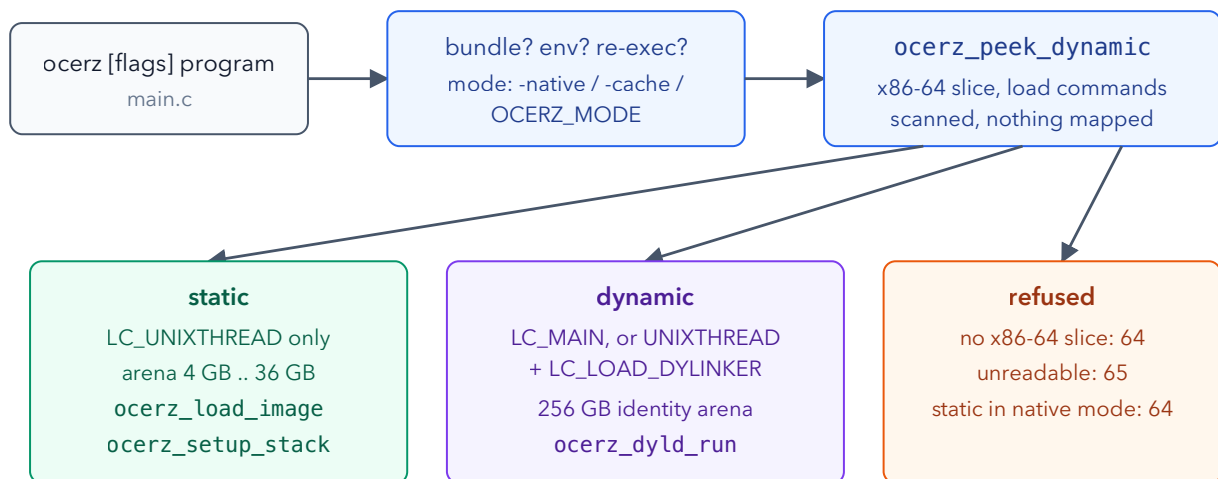


Figure 10.1. How `main()` routes a program. The decision is made from load commands alone, before any guest memory exists.

What `main()` settles first

`main.c` is small, but several of its decisions shape everything after them, and each exists because something broke without it.

Standard error, before anything else

`OCERZ_STDERR_FILE`, when set, is opened for appending and installed as file descriptor 2 before any other line runs. Chromium and Wine start children with standard error closed or pointed at nothing, and a child that died early under AArchX otherwise left no trace at all. With the variable set, every process of a tree writes into one file.

The host's dyld must not act on `DYLD_INSERT_LIBRARIES`

Steam launches every game with `DYLD_INSERT_LIBRARIES` naming its loader and overlay. The variable is meant for the guest, but the host's dyld reads it first and loads the arm64 slices of those libraries into AArchX itself, which broke startup within half a second. So `main()` moves the value into `OCERZ_GUEST_DYLD_INSERT_LIBRARIES`, removes the original and `execv`s itself. Re-executing keeps the process ID, which matters because Steam tracks the game by it. The clean copy puts the variable back before the guest's environment is taken, so the guest sees exactly what Steam set, and the mini-dyld opens those libraries in the guest after the startup initializers (Chapter 11).

The mode

`-native` and `-cache` choose outright, and the last one on the command line wins. With neither, `OCERZ_MODE` decides, and that is how a child inherits its parent's mode across a spawn or `exec`. An `OCERZ_MODE` value that is neither `native` nor `cache` is refused with exit 64 rather than ignored, because a silently ignored mode is a test that measures the wrong thing.

Application bundles

A path that is a directory is treated as an application bundle. `bundle_executable` reads `Contents/Info.plist` and takes the string after the `CFBundleExecutable` key; if the file is not readable as XML, it falls back to the bundle's own name without `.app`. The result must exist as a regular file under `Contents/MacOS`. The guest's `argv[0]` becomes that executable's path, as it is when the system starts a bundle. This exists because Steam's launch options substitute the bundle for `%command%`: a game launched as `ocerz %command%` used to end at once with "cannot read".

Wine, Steam Helper and the per-process switches

A process whose program, after `realpath`, is named `wine` or `wine64` is marked as one of Wine's. That fixes the base of its memory arena (Chapter 8) and, unless the user chose otherwise, sets `OCERZ_PRELOAD_OBJC` to AppKit, QuartzCore, HIToolbox and CoreSpotlight, the Objective-C images whose categories must be visible before any class is realized. A program named `Steam Helper` gets `--use-mock-keychain` inserted after `argv[0]`, unless it is a crash-pad handler or already has the flag, so that macOS does not ask for the login password once per helper process (Chapter 13).

A short command-line summary, the last path component of every argument, is kept in `ocerz_cmdline_summary`. Three switches match against it so that one process of a large tree can be singled out: `OCERZ_NOJIT_EXE` interprets only matching processes, `OCERZ_STRACE_EXE` traces only their system calls, and `OCERZ_EXE_ENV="cs2.exe:K=V,K=V;other.exe:K=V"` sets variables only in them, before any of those variables is read.

A private copy of the environment

The dynamic loader is handed `env_snapshot(envron)`, a copy that owns its strings and lives for the whole process, never `envron` itself. Loading a guest in native mode opens the host's own frameworks, and CoreFoundation's initializer calls `setenv`, which reallocates `envron` and frees the array an earlier pointer still named. The guest's initial stack was built from that pointer only after loading, so it read freed memory, and whether that crashed depended on whether the block had been reused yet. It was reused when the environment had the size `bash` passes and not the size `zsh` passes, which made the bug look like a shell problem.

Choosing the slice

A universal (fat) binary starts with a big-endian header listing one record per architecture. `select_fat_slice` accepts both the 32-bit (`FAT_MAGIC`) and 64-bit (`FAT_MAGIC_64`) forms, rejects a count of zero or more than 64 records as implausible, byte-swaps each record and takes the first whose CPU type is `CPU_TYPE_X86_64`. Only the slice's file offset is kept: every later read adds it, and nothing maps the file.

That is a deliberate choice. All reads go through `read_exact`, a loop over `pread` at explicit offsets that turns a short read into `OCERZ_EFORMAT`. A truncated or malformed image therefore fails while it is being read, with a message naming what was missing, instead of faulting later inside guest memory. The header must then say `MH_MAGIC_64`, CPU type `x86-64` and file type `MH_EXECUTE` or `MH_DYLINKER`; `sizeofcmds` must be non-zero and at most 16 MB; and every load command's size must be at least a load command and must not run past the end of the commands. Each check names the failure.

The dynamic path has its own slice selector in `src/dyld.c`, which reads the whole file into a buffer once and works from that buffer for mapping, binding and symbol lookup; Chapter 11 picks up there.

The static path

A static program gets the oldest and simplest memory layout AArchX has: `oc erz_mem_init(0x100000000, 0x900000000)` reserves an arena from 4 GB to 36 GB of guest address space, and guest addresses translate to host addresses by adding a single base (Chapter 8). The loader then makes two passes over the load commands.

First pass: measure

Each `LC_SEGMENT_64` contributes its `[vmaddr, vmaddr + vmsize)` to one overall range. `__PAGEZERO` is recognized by having a zero `vmaddr` and no protection, and is skipped: it reserves the bottom 4 GB on a real system and has nothing to map. The segment whose file offset is zero contains the Mach header, and its `vmaddr` becomes the image's header address. An image with no mappable segment, or none at file offset zero, is refused.

The entry point comes from one of two commands. `LC_UNIXTHREAD` carries one or more flavor blocks of thread state; the loader walks them, checking that each block fits inside the command, and for flavor 4, `x86_THREAD_STATE64`, takes register index 16, which is `rip`. `LC_MAIN` carries an offset, and the entry is the header address plus that offset. A thread state wins if both are present. A static image's slide is always zero: it is placed exactly where it was linked.

Second pass: map, copy, protect

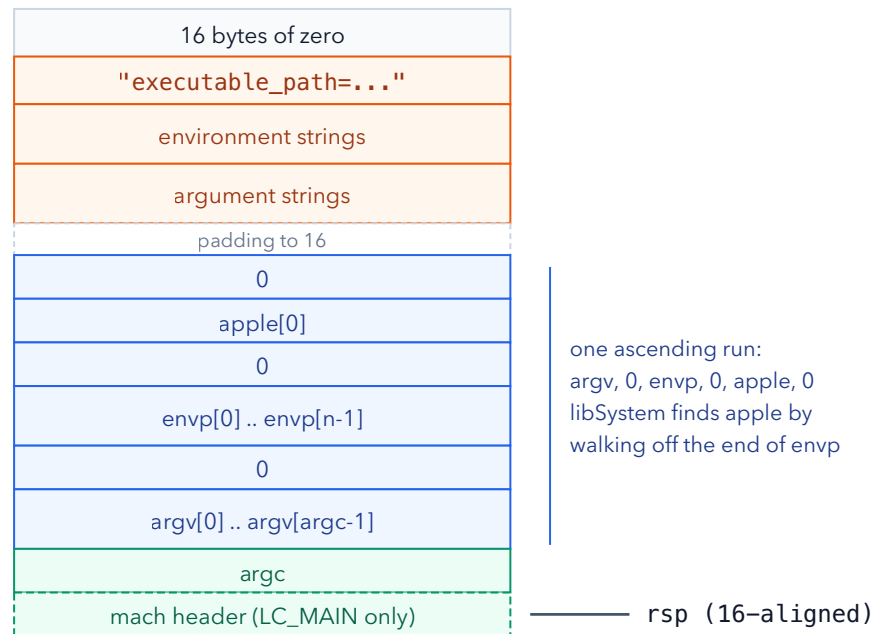
The whole range is mapped read-write with `oc erz_map_fixed`, each segment's file bytes are read straight into place, and only then are protections applied. They are applied in two rounds, read-only and executable segments first and writable ones second. Writing everything before protecting anything means the loader never has to write into a page it has already made read-only. Note that "executable" here describes the guest's view only: guest code is never executed by the host, so the host pages stay readable and the JIT reads them (Chapter 6).

The initial stack

The kernel's exec path leaves a precise structure on the new process's stack, and both C runtime start code and dyld read it positionally. So the order and alignment are a contract, not a detail. `oc erz_setup_stack` builds it in an 8 MB stack mapped anywhere in the arena:

1. sixteen zero bytes at the very top;
2. the strings, pushed downwards: the single `apple[0]` string `executable_path=<path>`, then every environment string, then every argument string;
3. below the strings, aligned down to 16 bytes, the pointer area, in ascending order: the Mach header address (only when the image entered through `LC_MAIN`, where dyld's convention puts it), `argc`, the `argv` pointers and a null, the `envp` pointers and a null, the `apple` pointer and a null;
4. `rsp` pointing at the first word of that area, itself 16-byte aligned, and `rbp` zero.

higher addresses



lower addresses

Figure 10.2. The initial stack as `ocercz_setup_stack` builds it for a static program. The dynamic frame (Figure 10.3) has the same vector layout with five more `apple` strings.

The three vectors must form one contiguous ascending run. `libSystem` never receives the `apple` array as an argument: it finds it by walking past the null that ends `envp`. An earlier version of the dynamic frame stacked the vectors downwards instead, which put the `argv` vector where `apple` belongs. `libSystem`'s walk then ran past `argv`'s terminator into the string area and handed `strlen` the bytes of `"th_port=0x..."` as a pointer, and `python3` died there.

The dynamic frame

A dynamic program's frame is built by `build_frame` in `src/dyld.c`, not by `stack.c`, because it has more to carry. Its stack is also 8 MB, and it holds:

- **Six `apple` strings** instead of one: `executable_path=`, `stack_guard=`, `ptr_munge=`, `malloc_entropy=`, `main_stack=` with the stack's top and size, and `th_port=` with the main thread's Mach port. `libSystem`'s initializer reads these.
- **A `ProgramVars` block**, the structure handed to `libSystem`'s initializer and to every image initializer, pointing at the real `NXArgc`, `NXArgv`, `environ` and `__progname`: `libdyld`'s in cache mode, the host's in native mode. It used to point at cells of its own beside the vectors, and then `_NSGetEnviron()` and the `environ` a program reads came apart at the first `setenv`: a program that called `setenv` and then `execve` with `environ` handed its child an environment without the variable.
- **A return stub at the top of the stack.** In cache mode `AArchX` bypasses `dyld`'s own start routine and calls `main` directly, so `main` returns into nine bytes `AArchX` writes 64 bytes below the stack top: `mov edi, eax; mov eax, 0x2000001; syscall`, the exit system call with `main`'s result. In native mode the same place calls C's `exit` (the symbol `_exit`) in the synthesized `libSystem` instead, so that `atexit` handlers run and `stdio` is flushed, as `start` would do; the exit system call stays behind that call only for the case where the export is missing.

Every argument and environment entry is counted first and the frame is sized to fit. The arrays used to be fixed at 64 entries with the environment cut at 60, which silently dropped the last variables of a large environment. Wine's loader marks its one-time re-exec by appending `WINELOADERNOEXEC=1` as the last variable, so a launch with one variable too many re-executed every Wine process forever. It looked like an intermittent hang and was a counting bug (2026-09-06, found under Steam).

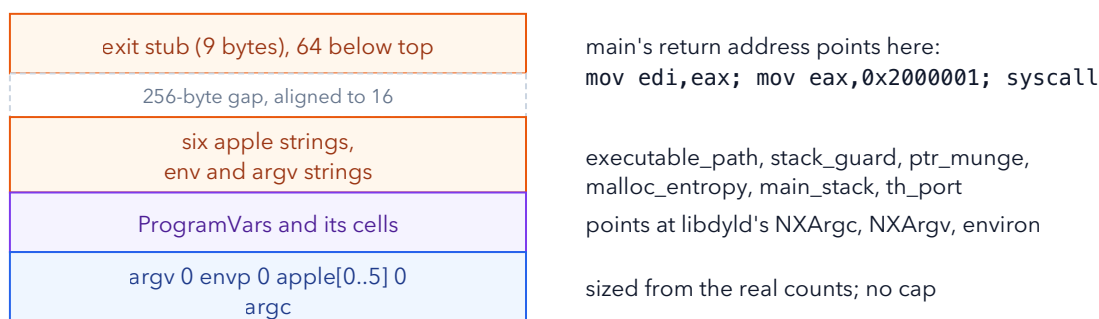


Figure 10.3. What the dynamic frame adds to the static layout, top of stack first.

The executable's own path

The dynamic loader `realpath`s the program's path before anything else uses it, so the guest always sees an absolute executable path, in `executable_path=`, in the image list and in `_NSGetExecutablePath`. A real system resolves it the same way. Without it, a program started as `./steam_osx` got a relative path back from `_dyld_get_image_name(0)`, and Steam's tier1 library built `"../Steam.AppBundle/..."` out of it and asserted. The dynamic test `exec_abspath` pins it.

Where a dynamic image lands

The dynamic path does not use the static arena. `ocerz_dyld_run` first reserves a 256 GB **identity** arena, where guest addresses are host addresses, which is the fast case for the JIT. It is that large because JavaScriptCore's Gigacage asks for a single 128 GB mapping and Chromium's PartitionAlloc reserves 32 GB at startup. Then:

- A **position-independent** main image is mapped anywhere inside the arena, and the difference from its linked address becomes its slide. Every library is placed the same way.
- A main image that is **not** position-independent goes exactly where it was linked. If any of its segments lies below 12 GB (the constant `OCERZ_LOW_LIMIT`, `0x300000000`), the process switches to the low-shadow layout, where those low guest addresses live at a separate host base, because the host will not hand out the bottom of its own address space. Wine's preloader is such a program, which is why Wine processes run in that layout. A segment elsewhere outside the arena is registered as an extra guest range and mapped fixed.
- A process whose loader is Wine reserves its arena at a fixed base, so that `ntdll.so` lands at the same address in every process of a Wine session (Chapter 8 and Chapter 14 explain why that matters).

From here to the mini-dyld

For a static program the story ends here: `vm.cpu.rip` is set to the entry point and `oc erz_vm_run` starts the run loop, which is Chapter 2's subject. For a dynamic program, `main()` only routes. It hands `oc erz_dyld_run` the path, the guest's arguments (after the Steam Helper rewrite) and the environment snapshot, and the mini-dyld takes over: mapping the shared cache in cache mode, loading every dependency, applying fixups, running `libSystem`'s initializer and the rest in dependency order, and finally jumping to the program's entry point with the return stub on the stack. Chapter 11 covers all of that.

WHAT THE LOADER DOES NOT DO

It reads only 64-bit x86 Mach-O; a standalone i386 Mach-O is not something current macOS can run either, and i386 code reaches AArchX only through Wine's WoW64 (Chapter 14). It does not check code signatures, since the guest's code is never handed to the host to execute. And it maps nothing from the file: every byte is read and copied, which costs a little at startup and means the image in guest memory never depends on the file staying unchanged.

CHAPTER 11

The mini-dyld and the shared cache

Every real macOS program is dynamically linked, and on a current Mac its system libraries do not exist as files: they live inside the dyld shared cache. AArchX never runs Apple's dyld. It brings its own dynamic linker, the mini-dyld, which maps the x86-64 shared cache, loads and binds every image, runs initializers in the order dyld would, and answers the API that libdyld exposes to programs. This chapter is about that linker in cache mode; Chapter 12 covers how the same loader serves native mode.

5,644

LINES IN DYLD.C

2,629

LINES IN DYLDAPI.C

1,499

LINES IN CACHE.C

59DYLD API SLOTS
ANSWERED

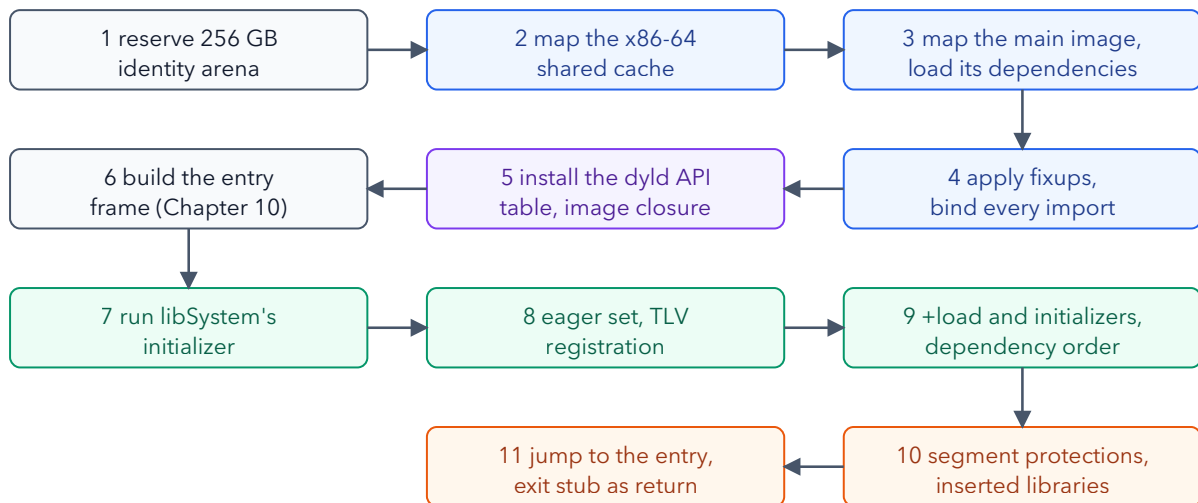
Why a linker of its own

The obvious design is to load Apple's x86-64 `/usr/lib/dyld` as a second image and let it do everything, the way the kernel would. AArchX does not, and the reason is control. A translator has to know which code is which image, when an image appears, what its segments are, when its initializers run and on which stack, and it has to answer the questions programs ask about images, because some of those answers differ under translation. Owning the linker means every one of those facts is ocerz's own data rather than something reconstructed from the outside, and it means each bug in this area could be fixed where it lived. The cost is that the linker must reproduce dyld's observable behaviour exactly, and most of this chapter is about where that turned out to matter.

The mini-dyld lives in three files. `src/cache.c` maps the shared cache and resolves symbols out of it. `src/dyld.c` loads images, applies fixups, orders initializers and implements `dlopen`. `src/dyldapi.c` answers the dyld API that libdyld's exported functions call into, and builds the image list that libobjc and every other reader of loaded images walk.

From exec to main

`ocerz_dyld_run` is the whole startup sequence, and its order is the order dyld uses where order is observable.



Native mode skips 2, 5 and 7: no cache, no libdyld, no libSystem initializer. Its images come from Chapter 12's synthesizer.

Figure 11.1. The startup sequence of `ocerz_dyld_run` in cache mode.

Two details of the sequence are worth stating now. `libSystem`'s initializer runs first and alone, called through `ocerz_vm_call` with five arguments, `argc`, `argv`, `envp`, `apple` and the `ProgramVars` block (Chapter 10), and when it returns, its whole dependency closure is marked initialized. And the program's entry point is entered with the nine-byte exit stub as its return address, because dyld's own `start` routine, which would normally call `main` and then `exit`, is never run.

Finding and mapping the cache

Where the cache is

The x86-64 shared cache is shipped by the Rosetta package, not by the base system, which is why cache mode needs Rosetta installed even though Rosetta's translator never runs. Its location moved. macOS 27 keeps it in the Rosetta cryptex, `/System/Volumes/Preboot/Cryptexes/Rosetta/System/Library/dyld/`; macOS 26 keeps it in the OS cryptex, `/System/Volumes/Preboot/Cryptexes/OS/System/Library/dyld/`. `ocerz_cache_dir` tries the Rosetta location first and the OS one only when the first holds no cache, so a macOS 27 machine maps exactly what it always did. Between 2026-09-19 and 2026-10-03 only the Rosetta location was tried, and every dynamic program on macOS 26 stopped with "cannot map shared cache". The unit test `test_cache` used to skip whenever mapping failed, which is how that went unnoticed; it now checks that the first directory holding a cache is the one in use, and fails rather than skips when a cache exists but will not map.

Main file and subcaches

The cache is a main file, `dyld_shared_cache_x86_64`, and numbered subcaches beside it, `.01`, `.02` and so on, mapped from the same directory until one is missing (at most sixteen). Everything ocerz reads from them is from the published format, `dyld_cache_format.h`: each file's header lists its mappings in an array

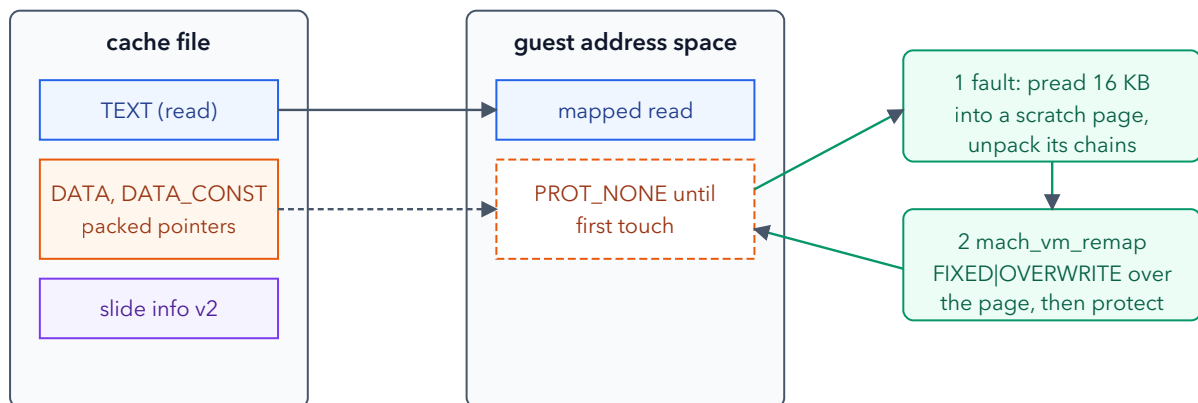
of `dyld_cache_mapping_and_slide_info` records, 56 bytes each, giving the address, size, file offset, slide-info offset and size, flags and maximum and initial protections. The image table (`imagesOffset` and `imagesCount`) lists about 3,600 images, each with its address and path.

Every mapping is mapped `MAP_FIXED` at its preferred address, starting at `0x7ff800000000`; the cache is never slid. Execute permission is turned into read: guest code is never executed by the host, only read by the decoder (Chapter 4). A mapping that carries slide information is mapped writable, because its pointers have to be rewritten.

Lazy slide unpacking

The slide information (version 2 in this cache) stores every pointer in the data regions as an offset plus a chain of deltas to the next pointer in the same page. So even at slide zero the DATA and DATA_CONST regions, about 500 MB, must be unpacked before anything reads them. Doing it eagerly touched and copied every page at process start, about 200 ms per process, and Steam under Wine starts dozens of processes.

So those regions are mapped `PROT_NONE`, and each 16 KB host page is unpacked on its first touch, from the fault handler, then given its final protection. The interesting part is how the unpacked page appears.



Other threads touching the page meanwhile fault, wait on the lock, and retry a

Figure 11.2. Lazy slide unpacking. The real page is never writable while half unpacked.

The first lazy version did `mprotect(RW)` on the page and rebased it in place. Every other thread could then read raw pointer chains mid-unpack, and `libswiftCore` inside a Steam process dereferenced one of those half-built pointers. Now the page is read from the file into a private scratch mapping, its chains are unpacked there, and the finished page is installed over the real one with `mach_vm_remap(VM_FLAGS_FIXED | VM_FLAGS_OVERWRITE)` while the real page is still `PROT_NONE`. A concurrent reader simply faults, waits on the unpack lock and retries against the finished page. A per-thread note of the last page retried tells a genuine second fault, an alignment or protection fault on an already unpacked page, apart from a lazy one, so those still reach the real handler. The lock is held across a whole page rebuild, so `fork` takes it like the other emulator locks, or a child could inherit a lock nobody will release. `OCERZ_EAGER_SLIDE=1` unpacks everything up front, `OCERZ_NO_LAZY_REMAP=1` goes back to in-place rebasing, and `OCERZ_LAZYCHECK=1` unpacks every page twice and compares.

Pages a program patches

Some engines make a `libSystem` page writable and patch it in place. So every cache mapping is recorded (which also tells an address in the cache apart from a wild one), along with write-watch state for any page a guest has made writable. A lazily slid page must be unpacked before such an `mprotect`, because once it is accessible the fault that would have unpacked it never comes. When code has been translated out of a patched page, write permission is taken back, so the next patch faults and the translations made from the old bytes can be retired (Chapter 7).

Resolving symbols

Export tries and the two-level namespace

Every image publishes its exports as a trie, a prefix tree of names whose terminal nodes carry the symbol's offset and flags. The walker has one subtle case: a terminal of size zero with the name fully consumed is not a miss, because the node carries an empty edge to a terminal child whenever a symbol is a strict prefix of others, as `_libiconv` is of `_libiconv_open`.

macOS links in a **two-level namespace**: each import records which library it came from, as an ordinal into the importing image's list of dependencies. The mini-dyld honours it. A cache dependency is resolved in the specific library the image linked, following re-exports, rather than by a search of every image. This matters more than it sounds: `openssl` links `libcrypto.46.dylib` (LibreSSL 3.3.6) while the cache also carries `libcrypto.44` (2.8.3) exporting the same names, and a flat search bound `OpenSSL_version` in whichever came first, so `openssl version` printed the wrong version. A library lookup follows `LC_REEXPORT_DYLIB` as well as re-export entries in the trie, because an umbrella such as `libSystem` answers for its members only through those load commands.

The per-process index

All resolution runs on an index built once per process: install path to image and header to image in open-addressed tables, and, filled per image on first use, its export trie and its linked images in ordinal order with their kind. The index is published lock-free with a compare-and-swap, so there is no lock for `fork` to catch half-held. A breadth-first search from one image walks the index with a visited bitmap and memoizes its answer per image and name.

It replaced a 512-slot direct-mapped memo of paths. Under Wine, which resolves thousands of OpenGL and AppKit names through `dlsym`, colliding paths fell back to a `strcmp` over all ~3,600 images, and a Steam web helper spent a quarter of its startup in the resolver. A benchmark of 4,500 framework lookups through `dlsym` went from 8.17 s to 0.12 s; Rosetta takes 0.21 s. A miss on a framework handle went from 2 ms to about 10 μ s.

Paths the image table does not list

The image table holds each library once, under its install name. Every other path for it, `/usr/lib/libz.dylib` for `libz.1.dylib` or `libgcc_s.1.dylib` for `libSystem`, is only in the cache's dylibs trie, which the published header locates. ocerz failed 108 of the SDK's 111 aliased `/usr/lib` names that Rosetta loads until all three path lookups fell back to that trie, and Steam's `steamloader.dylib`, which links `libgcc_s.1.dylib`, could not load. Of the 10,595 paths the macOS 27 Intel cache answers to, 6,677 are reached that way.

A dependency is also looked for in the cache both before and after its symlinks are followed. Eleven of the twelve dylibs in `/usr/lib/swift` are symlinks into frameworks whose binaries exist only in the cache, so following the symlink first led to a file that is not on disk: Tailscale lost `Network.framework` through Sparkle that way, with a "Library not loaded" error naming a path the cache was holding all along.

Libraries without a trie, and imports that may be missing

A dylib built by an old enough toolchain carries its exports only in the classic symbol table. Resolving through the trie alone found nothing in it at all: 21 of the 60 dylibs Photoshop ships are of that kind, and its OpenCV could not find a single Intel IPP entry point. Such an image now gets an index, built once, from the symbols `LC_DYSYMTAB` marks as external definitions. Only an image with no trie is indexed, because where there is one it is the whole truth.

A symbol imported with `-undefined dynamic_lookup` is resolved from the flat namespace and is allowed to be absent; dyld does not refuse to load over one. ocerz called each a fatal unresolved import, and Intel MKL's `libmkl_core` has 2,161 of them, because MKL puts its kernels in whichever ISA-specific library it loads at run time; one Photoshop launch printed fifteen thousand lines. They are now counted quietly (`OCERZ_DYNLOOKUPLOG=1` lists them). A genuinely unresolved import names the symbol, the image that wanted it and the library it was looked for in, and the list stops after 24 unless `OCERZ_ALLMISS=1`.

Weak coalescing

A weak-coalescing bind (ordinal `-3`) asks whether any loaded image already defines the name, so that every copy of a C++ inline function or template instantiation resolves to one definition. dyld answers it only from images that define weak symbols. ocerz walked all $\sim 3,600$ cache images, and Electron Framework's $\sim 2,560$ such binds, all its own C++ names that no system library has, cost 1.8 ms each: 4.6 s in every Discord process. Now the walk covers the ~ 300 weak-defining images, and after 128 lookups a 1 MB bloom filter of everything they and their re-exports export turns a miss into one probe. The names are hashed along the trie's edges rather than rebuilt, which keeps the build near 40 ms, and the filter is published with a release store because readers test it without a lock.

The newest rule (2026-10-03) is that only images *already loaded in the guest* count, again as dyld does: it coalesces among the images loaded so far, so a weak definition in a cache image nobody loaded never wins. Searching every cache image let a library that carries its own copy of a C++ template bind to Apple's copy instead. DXMT's `winemetal.so` statically links its own LLVM, and among its 4,481 weak binds is an LLVM template function that an LLVM inside Apple's GPU compiler framework also exports; DXMT's shader compiler then called the other LLVM's code on its own LLVM's objects, failed every shader, and R.E.P.O. aborted. The main program and its startup dependencies are bound before the set of loaded images exists, so until then every cache image still counts. `OCERZ_WEAK_ALL_CACHE=1` keeps the old rule.

The order of a lookup

Step	When it applies	Where it looks
1	positive ordinal	the named dependency: a loaded disk image by install name or path (with <code>@-paths</code> expanded), otherwise that library in the cache
2	ordinal <code>-3</code> (weak)	the weak-defining cache images already loaded
3	otherwise, still unfound	the cache's flat namespace

Step	When it applies	Where it looks
4	flat, weak or main-executable ordinals	the importing image itself
5	still unfound	every loaded disk image
6	ordinal -1 , -2 , -3	the main executable (how a plug-in binds to its host program)
7	native mode only	synthesized libraries opened on demand
miss	not weak	dynamic-lookup count, native-mode report (exit 71), or a named fatal

Fixups

Binding writes the resolved addresses into the image; rebasing adjusts the image's own pointers by its slide. The mini-dyld handles all three generations of the format.

- **Chained fixups** (`LC_DYLD_CHAINED_FIXUPS`), what every modern linker emits. Each page's chain starts from a per-page offset and links pointer to pointer; a 64-bit entry is either a bind, with an import ordinal and an addend, or a rebase, with a target. Formats 2 (`DYLD_CHAINED_PTR_64` , target is a virtual address) and 6 (`DYLD_CHAINED_PTR_64_OFFSET` , target is an offset from the image base) are supported, which is everything x86-64 uses; anything else is refused by name. An import is named by ordinal at every location that uses it, so each ordinal is resolved once and the answer reused.
- **Classic opcodes** (`LC_DYLD_INFO`): rebase and bind streams of byte-coded instructions. Binds repeat a symbol across consecutive locations, so the last answer is reused while the symbol, library and weakness stay the same. Lazy binds are applied eagerly, so no lazy-binding helper ever runs.
- **Pre-10.6 relocations**. A dylib built for macOS 10.5 or earlier has no compressed fixup information at all: local relocations are rebased by the slide, external relocations add a symbol's address to the value in place, and the indirect symbol table fills the lazy and non-lazy pointer sections. On x86-64 a relocation's address counts from the first writable segment. ocerz applied none of this, so Steam's `steamloader.dylib` , which Steam injects into every game it launches, jumped to zero on its first call.

Protections after binding

Every image is mapped writable so the copy and the fixups can land; then each segment gets its declared protection. That used to be done for `__TEXT` alone, and everything else stayed writable and said so: `mach_vm_region` reported read-write for `__DATA_CONST` and `__LINKEDIT` . That was not cosmetic. Electron asks about a region it expects to be read-only and executes a trap instruction when the answer is anything else, which is why Discord's renderers died at startup. Now the protection read is `initprot` at offset 60 of the segment command (not `maxprot` at 56, which differs in old images), and a segment flagged `SG_READ_ONLY` , which is how `__DATA_CONST` is spelled, ends up read-only whatever its `initprot` says. It cannot happen at load time, because Objective-C has not finished writing selector references into those segments, so images are queued and protected in a batch after the initializer phase and after every `dlopen` ; a `dlopen` that fails takes its images out of the queue.

Initializers

Getting initializer order right is most of `dyld.c`, and it has produced more application-level bugs than any other part of the linker. The rule dyld follows is simple to state: each image's Objective-C `+load` methods and then its own C and C++ initializers run after every one of its dependencies has finished, bottom-up. Neither global ordering works. Running all `+load` methods first runs one framework's `+load` ahead of `libc++`'s initializer; running all initializers first recurses `libsystem_malloc` into its own zone setup.

Pruning, and who counts as a Cocoa program

`libSystem`'s closure is marked done the moment its initializer returns, so a library whose only dependency is `libSystem` stops the walk right there instead of descending through `libxpc` into Foundation and dragging half the system into its subtree. Whether a program needs the full dependency-ordered phase at all cannot be decided from the main executable's own load commands: an application is often a small stub linking one umbrella framework, and Safari, a Cocoa application by any measure, once read as "not a CoreFoundation program" and skipped the phase entirely. But `libSystem`'s closure cannot be followed either, because `libxpc` weak-links a library that links Foundation, which made every program, `/usr/bin/sort` included, look like a Cocoa application. The test is made on the program's closure with `libSystem`'s left out.

The set of images whose initializers run eagerly is closed under dependencies, because the walk recurses into every dependency but only runs the initializers of images in the set. Without that, an image in the set could be initialized on top of an uninitialized dependency, which is how Safari aborted in a static initializer that called `operator new` before `libc++abi` had been initialized.

Upward links

An upward link (`LC_LOAD_UPWARD_DYLIB`) is how a library declares the back edge of a dependency cycle: CoreFoundation links Foundation upward, because Foundation depends on CoreFoundation. It is an ordering edge for nothing, and it took three attempts to treat it exactly as dyld does.

1. **Followed on the way down.** Treating upward links as ordinary dependencies made CoreFoundation's upward link to `CoreServicesInternal` a real edge, which put several unrelated frameworks inside `libc++abi`'s subtree. `libc++abi` then sat unfinished on the recursion stack while another framework's initializer called `operator new` into a `libc++` that had not been initialized.
2. **Never followed.** Leaving upward targets out entirely broke `sw_vers` on macOS 27. Foundation's code was reachable, since the whole cache is mapped, and its classes registered, but its own initializer never ran. `NSString` therefore stayed an abstract class cluster, and the first `+[NSString stringWithFormat:]` fell into the method that reports an abstract class, which builds its message with `+[NSString stringWithFormat:]`, until the 8 MB guest stack ran out.
3. **Followed right after the declaring image.** Better, and wrong on macOS 26.7.1, where `libobjc` links `libswiftCore` upward and `libswiftCore` links Foundation upward. `libobjc` is a dependency of CoreFoundation, so Foundation's whole subtree, `SkyLight` among it, initialized while CoreFoundation was still unfinished on the recursion stack, and `SkyLight`'s first allocation through CoreFoundation's allocator recursed between `malloc` and `malloc_zone_malloc`.

The rule now (e705796, 2026-10-03) is that an upward target is initialized after the *whole* walk from the root has finished. Each target goes on a list the root call owns, and the list is worked through after the root's own initializers, including targets it adds on the way. That rule was not derived from reading dyld: it was measured. `tests/dynamic/upward_defer.c` builds three libraries of its own in the same shape and logs the order in which their initializers run.

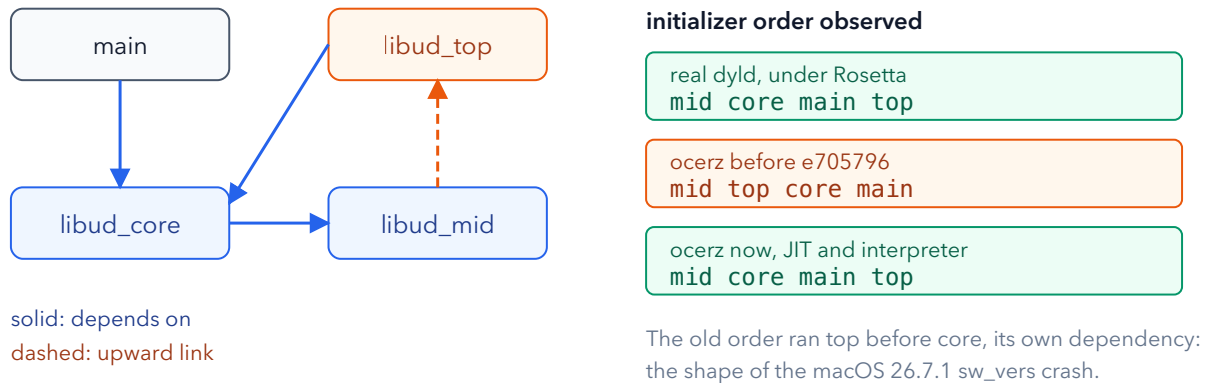


Figure 11.3. The experiment behind the upward-link rule. The libraries are the project's own, so the rule comes from observed behaviour, not from reading Apple's loader.

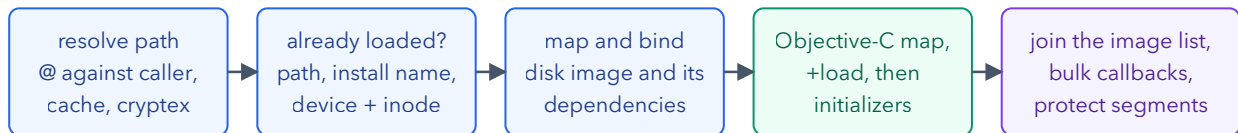
The dlopen closure

A `dlopen` runs the initializers of the images it adds, through `init_closure`, and three rules apply there that the startup phase did not need:

- **Post-order, never sorted by address.** A dlopened image is mapped below the dependencies it pulls in, so an address sort ran a dependent before its dependency. Steam's `steamui` initialized before `libtier0_s`, and its first memory-container growth called through tier0's allocator singleton while its vtable pointer was still null: a call through offset 0x28 of a null pointer, at exactly address 0x28.
- **+load before initializers, per image.** The closure used to run initializers only, so a cache framework first pulled in by a `dlopen`, Foundation under Wine's `ntdll.so`, never had its `+load` run, and `+ [NSString alloc]` kept returning the abstract class instead of its placeholder. Wine's `mountmgr` answers Steam's DHCP query through SystemConfiguration; CFBundle built a string on an abstract `NSMutableString`, and the same recursion as above ran `winedevice`'s stack out. Steam then saw no network adapters. (`OCERZ_NO_CLOSURE_LOADS=1` restores the old behaviour; the test is `dlopen_load_phase`.)
- **Upward links followed only to the Objective-C core.** The dlopen closure follows an upward link only when it points at Foundation, CoreFoundation or libobjc, so a program that dlopens SystemConfiguration still gets Foundation initialized. Following every upward link there brought in some 360 more initializers in every Wine process, and Steam's browser process then died in `objc_msgSend`. (`OCERZ_NO_DLOPEN_UPWARD=1`; test `dlopen_objc_core`.)

Initializers run under a `dlopen` are *tolerant*: if one stops with a fatal fault, ocerz prints `init-skip` with the initializer and the faulting address and carries on, because the process had been running fine and a crash inside one framework's initializer, which a real system might never reach, should not end it. An initializer that fails during startup is fatal, with a backtrace naming each frame's image and offset, and the process exits with 125.

dlopen, dlsym and the image list



A cache image skips mapping and binding: it is already mapped. It still goes through the last two steps, and joining the list must come after its Objective-C mapping, or `libobjc` takes it as handled and the framework loses its categories. A disk image's bulk callback fires before its initializers.

Figure 11.4. A `dlopen` in cache mode, left to right.

Resolving the path

A path beginning with `@` is resolved against the image that called `dlopen`, as `dyld` does: `@rpath` through that image's `LC_RPATH` entries and then the main executable's, `@loader_path` against its directory. The caller is the return address of the call into the `dlopen` slot (0x68), or `dlopen_from`'s third argument (slot 0x2e0). Before 2026-09-26 the path went to the loader as written, and the Game Porting Toolkit's `libd3dshared.dylib`, which `dlopens` `@rpath/D3DMetal.framework/D3DMetal` through its own `@loader_path` `rpath`, failed and aborted every D3DMetal game at its first Direct3D call (test `dlopen_caller_rpath`).

Some x86-64 libraries exist only in the Rosetta cryptex, and Intel code names them by their system path: Metal opens `/usr/lib/libMTLHud.dylib` for its performance HUD, and there is no such file on disk. So a cache-mode `dlopen` of an absolute path found neither in the cache nor on disk retries it under the Rosetta cryptex.

The same image, never twice

A `dlopen` of the main executable's own path returns the main image rather than a second copy. Steam's bootstrapper `dlopens` its own `steam_osx`, the copy gave duplicate Objective-C classes, and `steamui` crashed on a null vtable (test `dlopen_self`). Loaded images are also matched by device and inode, not only by path: Chromium's sandboxed helpers `dlopen` the 386 MB Chromium Embedded Framework again through a `Contents/Frameworks` symlink, `realpath` fails inside the sandbox, and each helper loaded the framework twice.

The image list

In cache mode the `dyld` APIs `_dyld_image_count`, `_dyld_get_image_name` and `_dyld_get_image_header` answer from the image closure `dyldapi.c` keeps. Until 2026-09-22 a `dlopen` of a disk dylib added to that list and a `dlopen` of a cache image did not. The library was loaded, its symbols resolved and `dlsym`

knew where they lived, while every walk of the list said it was absent; the count stayed at 45 where a real process went to 348. libobjc keys its per-image queries off that list, and Metal finds its GPU driver through them, so `MTLCreateSystemDefaultDevice` returned nil and OpenGL had no accelerated renderer: every translated program saw no GPU at all, and Brawlhalla drew a window and never drew into it. A cache image now joins the list at the end of its `dlopen`, after its Objective-C mapping and initializers. The order matters: libobjc calls back into the dyld APIs while it maps an image, and an image already in the list when that callback arrives is one it takes as handled, which cost the newly loaded framework its categories (test `dlopen_image_list`).

The initial closure itself was once walked through a fixed 1024-entry queue, which dropped dependencies once full, and it filled early because a library every image links was pushed once per dependent. Safari died in its startup on a method a category should have added. The closure array is now its own work list, deduplicated through an open-addressed set, so each image is walked once; Safari's closure is about 1,500 of the cache's 3,600 images.

Callbacks registered for bulk image loads are kept, not just driven once. libxpc registers one and learns from it which frameworks carry XPC services; driven only at startup, every framework dlopened later had services that launchd answered with "No such process", so a Wine process could not reach the Metal shader compiler service, and `steam.exe` retried a view service a thousand times a second (`OCERZ_NO_BULK_NOTIFY=1` turns this off).

dlsym

`dlsym` on a handle searches the image and then everything it links, breadth first, skipping upward links, which is what dyld answers: a symbol two dependencies down is found, one behind an upward link is not. Measured under Rosetta with the project's own dylibs. The disk-image path used to search the image alone, so `dlsym(handle, "malloc")` failed for every dylib; the cache path followed upward links, so CoreFoundation's handle found an OpenGL function by way of Foundation (test `dlsym_deps`; `OCERZ_NO_DLSYM_DEPS` and `OCERZ_DLSYM_UPWARD` restore the old behaviours).

The dyld API table

libdyld's exported functions, `_dyld_image_count`, `dlopen`, `dladdr` and the rest, are thin trampolines: each calls through a table of function pointers on an object that dyld provides at startup. In cache mode the mini-dyld provides that object itself. Its table has one entry per eight-byte slot, and entry *n* points at `OCERZ_DYLD_API_LO + n`, inside a trap window from `0xdda00000` to `0xdda10000` that the decoder, interpreter and JIT all watch. A call through slot 0x68 therefore arrives at `0xdda00068`; the run loop sees the address in the window and hands the call to `ocerz_dyldapi_dispatch`, which reads the arguments from the guest's registers, answers, and returns as a function would. ocerz answers 59 slots.

How a slot is known

Which slot is which function is established by running code, never by reading Apple's. `tools/dyldslots.sh` takes every name the SDK's public `libdyld.tbd` stub file exports, calls each one under ocerz from a small probe program with all-zero arguments, and runs with `OCERZ_DYLD_API_TRACE=1`, which prints every arrival in the table with its arguments and caller. The first slot a call reaches is that export's

slot. A slot no export reaches first is named by its caller in a traced run, through `dladdr`: slot 0x358 is called from `_dyld_objc_register_callbacks`. The `--check` mode runs inside the dynamic tests and fails for any slot the dispatcher answers that has no such witness, and for any change in the pinned pairs.

Export	First slot reached	Export	First slot reached
<code>_dyld_image_count</code>	0x10	<code>dladdr</code>	0x60
<code>_dyld_get_image_header</code>	0x18	<code>dlopen</code>	0x68
<code>_dyld_get_image_vmaddr_slide</code>	0x20	<code>dlclose</code>	0x70
<code>_dyld_get_image_name</code>	0x28	<code>dlerror</code>	0x78
<code>_dyld_register_func_for_add_image</code>	0x30	<code>dlsym</code>	0x80
<code>_NSGetExecutablePath</code>	0x50	<code>dyld_image_path_containing_address</code>	0x168

These twelve pairs are pinned in `tools/dyldslots.pinned`. Note that slot 0x30 is witnessed but is not among the 59 the dispatcher answers.

What an unanswered slot returns

A slot ocerz does not implement answers zero in `rax` and in `rdx`. The second register matters because a dyld API can return a pair. Slot 0x450, which libobjc's initialization asks for a table of message-send offsets and a count, is exactly that shape: with `rdx` left alone, the count was whatever the guest had in the register, libobjc read a table at address zero, and the damage surfaced much later as a recursive-lock abort inside class realization. That slot is now answered explicitly with an empty table.

Two answers worth knowing

`_NSGetExecutablePath` behaves as dyld's does: a buffer the path fits in gets the path and a size left exactly as the caller set it, and only a too-small buffer has the size rewritten, with `-1` returned. It used to write the length back on success too, which an arm64 build of the same program never sees.

Which image holds an address is asked by `_dyld_find_unwind_sections` once for every frame an exception unwinds through, and by the image-containing-address slots. It used to walk the load commands of each image and then of all $\sim 3,600$ cache images per call. It is now answered from a sorted table of every mapped segment, searched by bisection with a running maximum of segment ends so overlapping ranges are still found. The table is rebuilt when the closure changes and published with release and acquire ordering, and `__PAGEZERO`, which has no protection, is left out.

Objective-C in cache mode

In cache mode the translated libobjc from the cache is the Objective-C runtime. ocerz's job is to tell it the truth about images, through the callbacks libobjc registers with dyld: the `map_images` batch at startup is the dependency closure, and each image's `+load` methods are delivered before its initializers. Two refinements came from real failures.

Categories of late images. The cache has already merged every startup image's categories into the classes they extend, so startup images never report their category lists. An image mapped after startup does report them, because the classes it extends may already be realized with copied method lists that only a category list can reach (`OCERZ_NO_LATE_CATLIST=1` turns that off).

Categories that must exist before any class is realized. Some images' categories have to be visible before the classes they extend are first used, and a later `dlopen` registers them too late. CoreSpotlight adds coding methods to Foundation's collection classes, and without them the indexing AppKit starts about 20 seconds into a session threw an unrecognized-selector exception inside a dispatch block and ended the process. `OCERZ_PRELOAD_OBJC` moves named cache images' closures into the startup batch; Wine processes get AppKit, QuartzCore, HIToolbox and CoreSpotlight by default, and `@cat` preloads every image that defines categories, at a cost of 3 to 4 s per boot.

Selector canonicalization uses the selector table the cache's published Objective-C optimization header points at. It was once indexed by hand, which took thirteen seconds of startup out of every process that canonicalized a selector.

Thread-local variables

Two descriptor layouts share the same 24 bytes. A static linker emits the classic `tlv_descriptor` of a thunk, a 64-bit key and a 64-bit offset, so the offset is at `+0x10`. The shared cache ships the packed form, a thunk, a 32-bit key, a 32-bit offset, a signed delta to the initial content and its size, where the offset is the 32-bit word at `+0xc` and `+0x10` is the delta. ocerz always writes the packed form, and it once read `+0x10` unconditionally, which took the delta (zero) for a cache image and stored it over the real offset: every thread-local in the image collapsed onto offset zero and they aliased each other. SwiftUI reads a thread-local holding its current property list that way, got a small integer instead, and crashed casting it to a class. A key is small enough that a classic descriptor's 64-bit key leaves `+0xc` zero, so a non-zero `+0xc` now means the packed form is already there.

Smaller things that had large effects

The main thread's thread ID. `libpthread` caches the kernel thread ID next to the thread's TSD, and fills it in only when a thread sets itself up, which the hand-built main thread never did. So `pthread_threadid_np()` returned 0 on the main thread. The mutex protocol stores that ID as the lock owner, and owner 0 looks unlocked, so a mutex the main thread held gave no mutual exclusion at all against other threads: lost wake-ups, corrupted flags, and a Wine explorer that froze. Since 2026-08-18 the loader fills the main thread's cached ID in itself.

Inserted libraries. In cache mode `DYLD_INSERT_LIBRARIES` is honoured: once the startup initializers have run, each library it names is opened as the guest's `dlopen` would, before the program's entry point (Chapter 10 explains how the host's dyld is kept from acting on it first). This, the pre-10.6 relocations and bundle launching together are what let games launched from Steam run under AArchX.

A probe inside the real process. `OCERZ_DLOPEN_PIGGYBACK` loads a chosen dylib, initializers and all, right after the first successful `dlopen` whose path matches: a probe placed inside the real process, on the same thread, at the same point in its life. It fires for cache images too, and it is how a graphics probe walked through Wine's process start showed that the only thing that mattered was the stack the graphics library first ran on (Chapter 14).

KEY IDEA

Almost every rule in this chapter is "do what dyld does", and almost every one was learned the hard way, from an application failing far from the cause. The project's discipline for these rules is to pin each with a dynamic test whose expected output comes from running the same program under Rosetta, and to give each a switch that restores the old behaviour, so that the next regression can be bisected rather than guessed at.

CHAPTER 12

Native mode

Cache mode depends on Apple's x86-64 shared cache, which ships with Rosetta, and Rosetta ends after macOS 27. Native mode is AArchX's answer: the program stays x86-64, but every system library it links is a synthesized x86 image whose exports lead straight into the Mac's own arm64 frameworks. This chapter explains how a call crosses from System V x86-64 into Apple's arm64 ABI and back, how Objective-C, blocks, callbacks, threads and signals cross with it, what it costs, and what still stops it.

12,900LINES ACROSS NINE
FILES**59**LIBRARIES WITH AN
API DATABASE**65,536**CALLBACK
TRAMPOLINES**87/87**

NATIVE-MODE GATE

The idea

A program compiled for an Intel Mac imports `printf` from `/usr/lib/libSystem.B.dylib` and `CFRelease` from `CoreFoundation`. In cache mode those names bind to x86 code inside the shared cache, which is translated like the program's own. In native mode there is no x86 system code anywhere: ocerz builds, in memory, an x86-64 Mach-O for each library the program links, with a real header, real segments and a real export trie, but where each exported function is a twelve-byte stub that traps into ocerz. ocerz then performs the call for real, against the host's arm64 implementation of the same function, moving arguments and results between the two calling conventions on the way.

So the program's own code is translated exactly as in cache mode (Chapters 4 to 7), and everything the program asks of the system runs as native arm64 code at native speed. Native mode is selected with `-native` or `OCERZ_MODE=native`; cache mode remains the default because it runs more software. It was built quickly: the first synthesized `libSystem` landed on 2026-09-16, `CoreFoundation`, the API database, structures by value, Objective-C and the guest's own classes on 2026-09-17, `NSApplicationMain`, `dlopen` and processes on 2026-09-18, and on 2026-09-21 the x86-64 Steam client came up in native mode on macOS 27, with its GPU process running hardware-accelerated graphics through the Mac's own OpenGL.

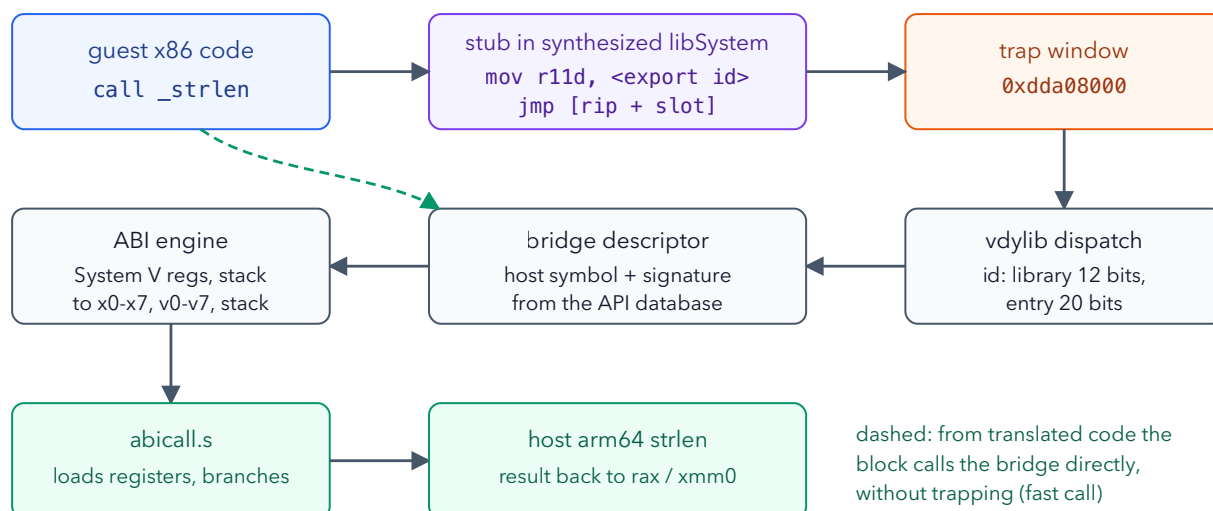


Figure 12.1. A call out, from guest code into a native function. Every synthesized stub jumps through a slot holding the same trap address; `r11` says which export it was.

Synthesized libraries

`src/vdylib.c` builds the images. The design rule is that the loader must not be able to tell a synthesized image from a file. The loader already works out of a host buffer, copying segments out of it and walking the export trie inside it, so the builder produces exactly what a linker would lay out on disk and the buffer is the file. Mapping, import resolution, `dlopen` and `dldaddr` then handle synthesized images with no special case anywhere.

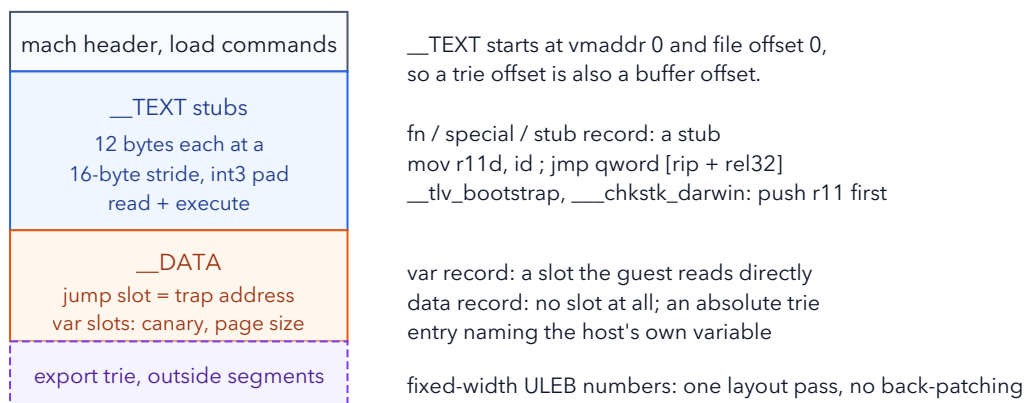


Figure 12.2. A synthesized image. It needs no rebases, binds or chained fixups: the stub's jump is `rip`-relative and its slot holds a constant.

The stub

Every function export is `41 bb <id32>, mov r11d, <export id>, then ff 25 <rel32>, jmp qword [rip + rel32]`. The slot the jump reads lives in `__DATA` and holds one address for the whole process, `0CERZ_DYLD_API_LO + 0CERZ_BRIDGE_OFF`, which is `0xdda08000`, inside the trap window the decoder, the JIT and the interpreter already watch for the dyld API (Chapter 11). So a call into a synthesized

framework needs no new instruction and no new range check: the guest jumps to an address that traps, and `r11` says which export it was. `r11` is free to use, because it is a scratch register in System V and every linker stub on the platform clobbers it.

Stubs are twelve bytes placed every sixteen, so a stub's address is its index shifted, which keeps the trie address, the jump displacement and the slot all derivable from one number. The four bytes of slack are `int3`, so a stray branch into the padding stops rather than running zeros.

Two exports cannot spend `r11`. `__tlv_bootstrap`, the thunk thread-local variable descriptors call, must preserve every register but `rax`, and clang does keep live values in `r11` across a thread-local access; a program that loaded one thread-local into `r11` and then touched another read back garbage. `___chkstk_darwin`, the stack probe, has the same contract. Their stubs push `r11` before loading the ID, still within the sixteen-byte stride, and their handlers restore it.

Export IDs

An export ID is the library's ordinal in its top twelve bits and the entry's index in its database file in the low twenty. The ordinal is the position of the library's file among the version directory's files sorted by name. Numbering libraries in load order would have given the same libSystem different IDs, and different bytes, depending on whether a program named CoreFoundation first; numbered by the directory, an ID means the same export in every process that reads the same database. The last ordinal belongs to no file: it numbers ocerz's own trampolines, such as the x86 entry of a native block handed to guest code.

The export trie

Each child edge in an export trie carries the absolute offset of the child node, so a node's size depends on how wide its children's offsets encode, which depends on the sizes of the nodes before them: minimal encoding needs a fixed-point iteration that is not guaranteed to settle. The builder instead writes every offset and terminal address as a padded ULEB128 of fixed width, which the format allows. Offsets and stub terminals are five bytes, carrying 35 bits; a native data terminal carries a host address and takes ten bytes, every bit a 64-bit value has. With every width known up front, layout is one pass and emission a second, with no back-patching. The trie is a real radix trie, because the walker takes the first child whose edge matches as a prefix: with a flat root, the edge `_write` would swallow a lookup of `_writev`.

Exports that are not functions

A **var** export is a slot in the image's own `__DATA`, filled when the image is built by a named filler, and the trie points at it exactly as it points at a stub. The main one is `___stack_chk_guard`, the canary every stack-protected function copies in its prologue and compares in its epilogue; clang turns the protector on by default, so almost every program imports it. The canary is drawn from `arc4random_buf` with its lowest byte zero, so a string copy climbing out of a buffer, which can write zero only as its terminator, cannot rewrite it and continue. The page-size fillers write 4096, 4095 and 12 behind `vm_page_size` and its relatives, because x86 programs are compiled knowing 4 KB pages and this host's are 16 KB: Chromium's allocator works in 4 KB pages by constant while V8 asks `sysconf`, and stops when the two disagree.

A **data** export takes no slot at all. Its trie entry is absolute, with the host variable's own address as its value, so the guest's reference binds straight to the native object. That matters for identity: a `CFSTR` literal's first word is bound to `___CFConstantStringClassReference`, the isa by which CoreFoundation and the Objective-C runtime recognize it as a string at all, and a guest may hand `&kCFTypingArrayCallbacks` to a CoreFoundation that compares the pointer with its own. A copy would be a second object at a second ad-

dress that nothing recognizes. It works because native mode runs a position-independent guest in the identity map, where a host address is a guest address. The same applies to `_environ`, `__progname`, `__stdoutp`, `__stderrp` and `__DefaultRuneLocale`: each is a variable native code also reads or writes, so a guest copy would go stale at the first write by either side. A name the host does not have is left out of the trie, so a program importing it fails to bind and says which name it wanted, instead of running with a reference to address zero.

The API database

What each library exports, and how each export crosses, is data rather than code. Each library has one text file, named for the last component of its install name, under `runtime/apis/macos/<version>/`, found beside the `ocerz` binary or under `OCERZ_APIDB`. The version directory is chosen once per process from the minimum macOS version the program's main image declares, compared as packed numbers so that 10.13 sorts above 10.9. The files are generated from the builder's own SDK by `make apis` and are not committed: they are derived from Apple's SDK.

Record	Meaning
<code>fn <export> <host-symbol> <signature></code>	a bridged function: calls cross to the host symbol under the signature
<code>data <export> <host-symbol></code>	an absolute export naming the host's own variable
<code>var <export> <bytes> <filler></code>	a slot of the image's own, filled by a named filler
<code>special <export> <handler></code>	a function <code>ocerz</code> answers itself
<code>stub <export> <reason></code>	binds, but names itself and stops (exit 72) when called; the reason is one word such as <code>variadic</code> or <code>long-double</code>
<code>shape, struct</code>	the layout of a structure of function pointers, and which argument of which function carries one
<code>inplace <export> <arg> <offset> <sig></code>	a function pointer inside a guest-owned structure the native function uses where it lies, converted for the length of one call (zlib and bzip2 allocators)

A file that breaks any rule is refused whole, naming its path, line and the rule, because a library half described would bind some imports to the wrong thing. Signatures are checked by the same parser the ABI engine uses, so a signature the file accepts is one the crossing can take.

`tools/sdkgen.sh` generates a file by reading the library's `.tbd` stub for the exact list of x86-64 exports, then parsing its headers with the Command Line Tools' `libclang` twice, once for x86-64 and once for arm64. An export whose declaration maps onto the engine's classes on both architectures crosses; anything else becomes a stub with a reason. `tools/sdkgen/overrides` holds what no header can say: the functions `ocerz` answers itself, callback structure shapes, and calls a generic crossing would get wrong inside an emulator. Each run writes its coverage to `tools/sdkgen/baseline` and fails if a crossing was lost.

Library (macOS 27 SDK)	Exports	fn	data	special
<code>libSystem.B.dylib</code>	9,653	2,774	505	187

Library (macOS 27 SDK)	Exports	fn	data	special
CoreFoundation	3,098	786	1,233	3
CoreGraphics	5,407	751	569	0
Foundation	16,755	117	1,548	1
AppKit	12,438	82	4,033	0
All 59 libraries	132,102	15,048	30,151	209

Foundation and AppKit have few `fn` records because their API is mostly Objective-C methods, which cross through message sends rather than through C functions. Most of the roughly 62,000 stubs are exports no public header declares at all.

The ABI engine

`src/abi.c` exists because the two calling conventions disagree in ways that make the n th argument rarely land in the n th register on either side. x86-64 System V passes integers in `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9` and floating-point values in `xmm0` to `xmm7`; Apple's arm64 passes integers in `x0` to `x7` and floating-point values in `v0` to `v7`. Each side counts its integer and floating-point arguments in separate sequences, so one `double` in the middle of a signature shifts nothing on one side and everything on the other, and the two disagree again about which arguments overflow to the stack and how. So the mapping is computed from a signature.

The notation

A signature is a result class followed by argument classes in parentheses: `i(pp)` for `strcmp`, `p(pLL)` for `memcpy`, `d(d)` for `sin`.

Class	Meaning	Class	Meaning
<code>v</code>	nothing (result only)	<code>p</code>	pointer, converted between guest and host views
<code>b B</code>	8-bit signed, unsigned	<code>f d</code>	float, double
<code>h H</code>	16-bit signed, unsigned	<code>{...}</code>	a structure by value, members in braces
<code>i u</code>	32-bit signed, unsigned	<code>c{sig}</code>	a callback: a function pointer with its own signature
<code>l L</code>	64-bit signed, unsigned	<code>k{sig}</code>	a block, with its declared signature

The disagreements that matter

- **Narrow integers.** System V leaves the bits above a narrow argument undefined; Apple's arm64 makes the caller extend an argument narrower than 32 bits and the callee extend a narrow result. CoreFoundation passes a `Boolean` (`B`) and a `UniChar` (`H`) almost everywhere. So a narrow value is only ever read as its own 8 or 16 bits and extended to all 64, by sign or by zero, before it is written anywhere. `i` and `u` are treated the same way at 32 bits, which is why a signature must say `int` where the header does, not `long`.

- **The stack.** System V gives every stacked argument an eightbyte; Apple packs a stacked argument at its own size and alignment. Eight `int`s and then `char`, `short`, `char`, `int` land at stack offsets 0, 2, 4 and 8.
- **Floating point** travels as raw bit patterns, never as C doubles, so nothing is converted on the way. A float is the low 32 bits of its slot, which is where the arm64 `s` register lives inside its `v` register.
- **Rounding mode.** A guest may set a rounding mode through MXCSR, which the JIT honours by driving the host FPCR. A host function compiled for the default mode must not inherit it, so the crossing restores the default for the call and puts the guest's back afterwards, writing FPCR only when the bits actually differ.

Most crossings need none of the machinery: a signature of scalars only, within six integer and eight floating-point arguments, goes through `ocercz_abi_perform_registers` straight from guest registers to host registers with no stack block.

Structures by value

Objective-C passes and returns structures by value constantly: `-frame` returns a `CGRect` (`{{dd}{dd}}`), `-rangeOfString:` an `NSRange` (`{LL}`). A structure crosses as bytes: gathered from wherever one ABI put it into a buffer laid out as the structure, pointer members converted in place, then scattered wherever the other ABI wants it. The classifications differ often enough that nothing may be assumed:

Structure	x86-64 System V	Apple arm64
<code>{LL}</code> <code>NSRange</code>	<code>rdi</code> , <code>rsi</code> ; returned in <code>rax</code> , <code>rdx</code>	<code>x0</code> , <code>x1</code>
<code>{ff}</code>	both floats in one <code>xmm</code>	two <code>v</code> registers (homogeneous aggregate)
<code>{{dd}{dd}}</code> <code>CGRect</code>	MEMORY: on the stack, returned through a pointer in <code>rdi</code>	<code>d0</code> to <code>d3</code>
<code>{df}</code>	<code>xmm0</code> , <code>xmm1</code>	two <code>x</code> registers (not an aggregate)
<code>{ffffff}</code> , <code>{LLL}</code>	MEMORY	pointer to a caller-made copy; returned through <code>x8</code>

A spilled structure leaves the registers free for later arguments on x86 but closes them on arm64. Over sixteen scalar members (`CATransform3D`'s count) or eight levels of nesting is refused, and a union has no notation at all.

Making the call

`src/abical1.s` is the last step, and the one no C prototype can express: it receives two arrays of eight words (the `x` and `v` register contents), a block of stacked bytes, an address for `x8` and two arrays for the result. The registers it must load are the registers the call arrived in, so everything still needed is moved into callee-saved `x19` to `x23` first, and the eight register pairs then load in one run with nothing live in `x0` to `x7`. Done in halves, the call would be right for the arities whose registers happen not to overlap and quietly wrong for the rest. The stacked block is copied to a freshly cut, 16-byte-aligned area at `sp`, and after the call `x0`, `x1` and `d0` to `d3` are stored, which covers every place a result can come back.

Variadic calls

A signature cannot say where a function's named arguments stop, and that is exactly where the ABIs part company. System V passes a variadic argument wherever a fixed one of its class would go; Apple's arm64 passes every variadic argument in its own eight-byte stack slot, doubles included, with no floating-point register used. A variadic function bridged by an ordinary signature would be quietly wrong, so the database carries generic variadics as stubs, and the ones a format string or a sentinel describes are special records with veneers: the `printf` family, `NSLog`, `NSStringCreateWithFormat`, `syslog`, `warn`, the wide `printfs` and the `scanf` family's main members. A veneer reads the format, gathers the variadic arguments from System V's registers and stack through `ocerz_abi_va_start` and `ocerz_abi_va_arg`, packs them into eight-byte slots and calls the function's `v` form with a pointer to the slots as its `va_list`, because an Apple arm64 `va_list` is nothing more than that. Positional arguments, `long double` and `%n` are refused by name, because a guess at any of them reads the wrong slot for every argument after it.

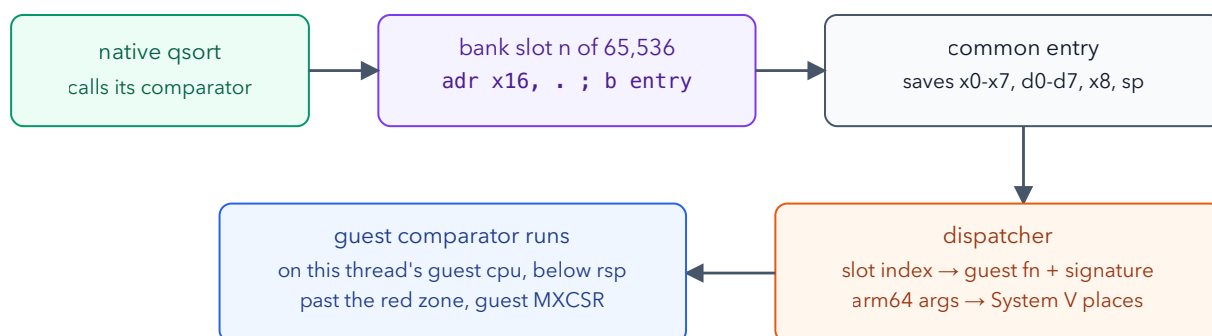
Calls from translated code

The trap path costs a round trip through the run loop: the block exits, the run loop finds `rip` in the trap window, the bridge runs, and the return comes back in through the dispatcher with the host return-address stack abandoned. So when a translated block ends in a synthesized stub's `mov r11d / jmp` pair and the slot holds the trap address, the JIT turns the jump into a call to `ocerz_vdylib_fastcall` made from inside the block, followed by the `ret` the export would have ended with. The fast call notes `rsp` and the JIT's retirement count, performs the dispatch, and lets translated code carry on only if the result was an ordinary return: `rip` is the word just below the new `rsp`, `rsp` rose by eight (sixteen for the `r11`-keeping stubs), nothing asked the thread to stop, and no translation was retired meanwhile. A delivered signal, among other things, sends it back through the dispatcher exactly as the trap path would. Only the `xmm` registers the signature reads or writes are spilled around the call, which the API database record supplies.

Eleven routines do not cross at all. `strlen`, `strnlen`, `strcmp`, `strncmp`, `memcmp`, `bcmp`, `strchr`, `memchr`, `memcpy`, `memmove` and `memset` are called constantly and do very little, so `src/leaf.s` holds arm64 versions that read their arguments from the host registers full register pinning keeps `rdi`, `rsi` and `rdx` in, and leave the result in `rax`'s register (Chapter 7). A short `strlen` costs 2 ns that way, against 17 ns for a crossing. A fault inside one of these routines is attributed to the branch that reached it, so a guest that hands `strlen` a bad pointer gets its own `SIGSEGV` handler with its callee-saved registers intact.

Calls back into guest code

Native functions take function pointers and call them: `qsort` its comparator, a run loop its observer, `pthread_create` its start routine. A pointer the guest supplied names x86 code native code cannot jump to. So an argument of class `c` carries the callback's own signature, `qsort` being `v(pLLc{i(pp)})`, and the guest function is **interned**: bound, with that signature, to one slot of a fixed bank of 65,536 arm64 trampolines. Native code receives the slot's address. The same function with the same signature always gets the same slot, so a library that compares callback pointers still sees one function, and a program that submits the same work a million times uses one slot.



A thread with no guest cpu (a libdispatch worker, an audio thread) is given one on its first callback and keeps it. The result goes back in x0/x1, d0-d3, or through the buffer the native caller passed in x8.

Figure 12.3. A call back, from native code into a guest function.

The bank is ordinary code assembled into ocerz, not generated at run time. Generating a trampoline from inside a native callback would race the translator for the one writable JIT arena, a fork child abandons that arena while native code may still hold pointers into it, and a real-time audio callback cannot wait for code generation at all. Each slot is two instructions, an `adr` of its own address into `x16` and a branch to the common entry, so the bank is one repeated block, 512 KB of text, and a slot's index is its distance from the start divided by eight. Exhausting the bank is a named refusal, never a silent reuse.

Not every function pointer a guest passes is guest code. A guest that copies `kCFTypingArrayCallbacks` copies native retain and release functions, and interning one of those would make the translator decode arm64 instructions as x86. So every `c` argument is tested by address: inside the guest reservation is guest code, inside the host's shared cache is not, and anywhere else is guest code exactly when host dyld does not know the address, because ocerz's own loader maps every guest image and never registers it with dyld.

When a slot is called, the dispatcher reads the signature against the native caller's registers and stack, packed Apple's way, places each value where System V puts it, and runs the guest function on the calling thread's guest cpu, below its stack pointer and past its 128-byte red zone, with the guest's MXCSR rounding mode applied for the duration. While it runs, the thread's bridge frame (below) is saved and cleared: a comparator that follows a bad pointer is taking an ordinary guest fault, and reporting it as a fault in `qsort` would kill a process that was fine. A call back costs about 430 ns, against about 17 ns for a call out, and most of it is three system calls that save and clear the signal mask and alternate-stack state for the callback's fault-recovery point.

Threads the guest never created

Native frameworks call back on threads of their own: libdispatch runs work on its workers, an audio device has a render thread. Such a thread has no x86 registers and no guest stack, and borrowing another thread's cpu would overwrite a guest thread that is running. So the first time such a thread calls a guest function, `ocerz_thread_attach` gives it a guest personality of its own, a cpu, a guest stack and a guest thread block behind `gs`, registered like any guest thread and reused on every later call, and torn down when the host thread exits. A second thread running guest code is a second observer of guest memory, so attaching one also retires plain memory mode, exactly as starting a guest thread does (Chapter 7).

That machinery makes guest threads free. `pthread_create`'s start routine is a callback held past the call and run on a thread the guest never created, so the native `pthread_create` starts a host thread, which is given a guest personality on its way into the start routine. Mutexes and condition variables are forwarded unchanged, which is sound only because `pthread_mutex_t`, `pthread_cond_t` and the rest have the same size, alignment and initializer values on x86-64 and arm64, checked by compiling both rather than assumed. Thread-specific data keys are ocerz's own, numbered 256 to 767 as the real library's dynamic keys are, with values in the guest thread block that `gs` points into, because Chromium's code reads them with inline `gs`-relative loads, about eighty of them in CEF. `errno` is the same kind of data: `__error` answers a slot of the guest thread block, copied to and from the host's `errno` at the edges of every native call, because between two guest instructions ocerz's own code runs on the same host thread and leaves its own errors behind.

Stopping a thread

`thread_suspend`, `thread_resume` and `thread_get_state` are bridged, because garbage collectors use them: JavaScriptCore's allocator and Chromium's samplers stop every other thread and read its registers. The kernel call is the real one, since a cooperative stop leaves the suspend count where the caller can see it. What makes it safe is refusing to leave a thread stopped where that would deadlock the caller: holding the JIT lock or the map lock, which a per-thread critical depth records, or anywhere inside the host's own allocator, which is read off its stack, since the suspender's next `malloc` would wait for a lock the stopped thread holds. Such a target is resumed and tried again. `thread_get_state` answers the guest's x86 registers, reconstructed from the arm64 registers the stopped thread holds them in.

Objective-C

In native mode there is no x86 Objective-C runtime at all. The guest's classes are the host's: `_OBJC_CLASS_$_NSString` binds to the native class object, and any object a native method returns is an address the guest can hold. What the guest cannot do on its own is send a message, because `objc_msgSend` jumps into whatever implementation the selector finds, and that implementation's arguments are in x86 registers. So every send is a crossing whose signature is not known until it arrives.

Selectors

A selector is compared by address. The guest's `@selector(length)` is a word in its `__objc_selrefs` pointing at the string inside its own image, while the native runtime's selector for the same name is a different address. So after an image's fixups and before any guest code runs, every selector reference is rewritten to `sel_registerName` of its string, which makes every selector the guest holds the native one.

A send

x86-64's `objc_msgSend` takes the receiver in `rdi` and the selector in `rsi`, and the method's own arguments after them in the usual places. The signature comes from the native runtime: `object_getClass` of the receiver (which understands tagged pointers, because it made them), then `class_getInstanceMethod` of that class and selector, which also runs a lazy method resolver, and `method_getTypeEncoding`. A class receiver's class is its metaclass, so class methods are found the same way. The call then goes to the host's `objc_msgSend` under that signature: native `objc_msgSend` leaves every argument register untouched and

jumps to the implementation, which is what makes one engine crossing correct for every method. The encoding is turned into the notation once per class and selector and parsed once per distinct notation; most of `NSString` is `p(pp)`, `L(pp)` or `p(ppp)`, so parsed signatures are shared heavily.

The encoding converts class by class: `q` to `l`, `Q` to `L`, `l` and `L` (32-bit in an encoding) to `i` and `u`, any pointer to `p`, a structure to braces. `B` becomes `b`, because arm64's `B00L` is `bool` while an x86 guest's is a `signed char`; they agree on every canonical value. A union, bitfield, `long double`, complex number or 128-bit integer stops the send with the class, selector, encoding and reason.

The special cases each have a rule. A structure result larger than sixteen bytes is sent by an x86 compiler through `objc_msgSend_stret`, with the hidden result pointer first; Apple's arm64 has no `_stret` form and returns through `x8`, so a `_stret` send is an ordinary send under the same signature, checked for agreement. `objc_msgSendSuper` and `objc_msgSendSuper2` pass their `objc_super` structure through, since it is guest memory the native runtime can read. A message to nil zeroes `rax`, `rdx`, `xmm0` and `xmm1`. A selector with no method is forwarded natively: the receiver is asked `forwardingTargetForSelector:` (up to eight targets deep) and otherwise `methodSignatureForSelector:`, never cached, since a proxy's answer can depend on the instance. Foundation's variadic methods, `stringWithFormat:`, `arrayWithObjects:` and the rest, are listed by selector and each checked against the SDK's declaration, since no encoding says a method is variadic.

An Objective-C exception cannot unwind through guest frames, because the unwinder walks the host stack and the guest's frames are x86 frames on the guest's own stack. So ocerz installs an uncaught-exception handler that prints the exception's name and reason and the crossing it escaped from, then calls the handler it displaced, so the process ends as it would natively.

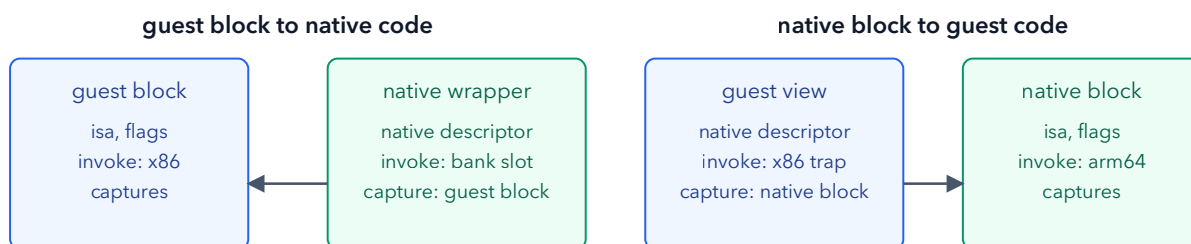
Classes of the guest's own

`src/objcclass.c` makes the guest's classes, categories and protocols the native runtime's own, after the image's fixups and selector rewrite and before any guest code runs. `objc_readClassPair` realizes a compiler-emitted class pair in writable memory without moving it, so a guest class keeps its address and no reference to it is rewritten; the runtime slides the instance variables past the native superclass's arm64 size and writes the offsets the guest's code reads. An `NSView` subclass whose ivar offset held 152 had it slid to 536, arm64 `NSView`'s size, and the view's `drawRect:` was called. Class and method structures are laid out identically on x86-64 and arm64, both being LP64, so the runtime reads the guest's own.

Three words of each class and metaclass are written first (the cache word, the word after it, and the data pointer, which is pointed at a host-heap copy of the read-only class data). Every method list is copied into an absolute list whose implementations are callback slots bound to the guest's x86 methods under the notation of their type encodings, so native code calling a guest `-hash` or `-drawRect:` lands in guest code. A method whose types cannot cross is installed as a function that names it and stops if it is ever called, so one odd method does not keep its class out. Swift classes, a class whose superclass the host lacks, and a root class are refused by name. Categories are applied with `class_replaceMethod` in list order, protocols are matched to native ones by name or built and registered, and `+load` methods are queued per image and run before `main`, dependencies first.

Blocks

A block is an object whose third word is the function that runs it, called with the block itself as its first argument, and whose descriptor holds its size, copy and dispose helpers and signature. The layout, flags and signature strings are the same on both architectures, and in native mode the guest's block classes bind to the native ones, so a guest block is a well-formed native block in every word except its functions, which are x86 code. A block therefore crosses as a pointer that is converted, class `k`, with the block's declared signature in braces: `dispatch_async` is `v(pk{v()})`.



Either wrapper handed back to the side that made the block turns into the block again. Both are heap or global blocks with native helpers, so `Block_copy`, `Block_release` and `objc_retainBlock` are correct on them from either side. A guest stack block is first copied to the heap guest-side, its x86 copy helper running as guest code.

Figure 12.4. The two block wrappers. Each holds only a reference to what it wraps.

A guest block handed to native code becomes a **native wrapper**: a block whose invoke is a bank slot bound to the guest block's own function under the guest block's own signature, whose first argument the slot turns back into the guest block, and whose dispose helper releases the guest block. A global guest block gets one global wrapper for the life of the process; a heap block gets the same wrapper for as long as that wrapper lives. A native block handed to guest code, a completion handler say, becomes a **guest view**: a heap block whose invoke is a twelve-byte x86 trampoline in a guest page, so guest code calls it like any block, and the trampoline calls the native block under its own signature.

The guest's block runtime is `libclosure`'s own except where `libclosure` would run a guest helper as arm64 code: `_Block_copy`, `objc_retainBlock` and `_Block_object_assign` copy a guest stack block, or a `__block` variable still on the stack, guest-side, and give the heap copy helpers that are callback slots, so the native `_Block_release` that finally frees it runs the guest's dispose helper as guest code. An `id` argument that holds a guest block where the receiver's method signature says `@?` crosses as a block, which is what an `NSXPCConnection` proxy's methods look like, and a guest protocol carries its extended method types into the native runtime, which `NSXPCInterface` needs to describe a reply block. Stickies, which used to stop at its first `dispatch_once`, runs because of this.

CoreFoundation and callback structures

A `CTypeRef` crosses unchanged in both directions, and that is a consequence of the address map, not a shortcut: in the identity map the native object a function returns is an address the guest can hold, compare and hand back, and the same object fetched twice is the same pointer. No handle table stands between the two.

What needs converting is a structure of function pointers passed by address: an array's or dictionary's callbacks, a timer's, observer's or source's context. CoreFoundation copies each of them when it creates the object, so the bridge copies the guest's structure first, converts every function-pointer word, and passes the copy's address for the length of the call. Which argument holds such a structure is a `struct` record, and what each word of each version is, a `shape` record; a version no shape describes stops by name, because guessing a context's layout would put a guest pointer where native code will call it. CoreGraphics' structures start with a 32-bit version and four bytes of padding that a structure filled field by field leaves as stack garbage, so a word that matches no version whole is matched again on its low half.

Everything else a process does

Signals. A guest signal handler is x86 code, so a native `sigaction` cannot be handed one. `sigaction`, `signal`, `sigprocmask`, `pthread_sigmask`, `sigaltstack`, `raise`, `kill` and `pthread_kill` are answered against the same handler table, masks and alternate stacks the system calls change in cache mode (Chapter 9), with a trampoline in guest memory in place of an x86 libc's `_sigtramp`. A handler runs on the state after the call that raised it, so `raise` returns with its handler already run. A signal from outside is delivered when the receiving thread's next crossing returns, the way cache mode delivers one at the next system call; checking costs a thread-local load, two loads and a branch per crossing.

Files, memory, jumps and processes (`src/sysbridge.c`). `open`, `fcntl`, `ioctl`, `sem_open` and the like are declared variadic but take one optional argument whose presence the named arguments decide; each handler reads it where x86-64 left it, only when the call uses it, and calls the host function through its real variadic prototype. Every structure those calls point at, `struct flock`, `struct termios`, `struct winsize` and the rest, was checked to have the same layout on both architectures. `mmap`, `munmap`, `mprotect` and the Mach VM calls run the same code the cache-mode system calls run, so guest memory comes out of the guest arena in 4 KB pages, and code written into a mapping and made executable is translated afresh. `setjmp` and `longjmp` save and restore the guest's registers, signal mask and MXCSR in Apple's x86-64 `jmp_buf` layout; a jump out of a native callback to a `setjmp` taken before it would leave native frames behind and is refused by name. `fork`, the `exec` family, `posix_spawn`, `system` and `popen` go through cache mode's own `fork`, `exec` and `spawn`, so every program a guest starts runs under ocerz in native mode, except a system tool with an arm64 slice that is not a shell or launcher, which runs as itself (`OCERZ_NO_NATIVE_CHILDREN=1` changes that).

Loading code at run time. Host dyld has never heard of an image ocerz mapped, and the host's `dlopen` would load arm64 code, so `dlopen`, `dlsym`, `dladdr`, `dlerror` and the image-list calls are answered by ocerz's loader. A load binds its whole closure before anything in it runs, and one that cannot bind fails with dyld's own wording, "Symbol not found" or "Library not loaded", leaving nothing mapped: a class handed to the native runtime cannot be taken back, so the Objective-C definitions wait until the bind has succeeded. `dlopen(NULL)` answers `RTLD_DEFAULT`, the value an arm64 program on the host printed.

The guest's identity. Native frameworks ask the process who it is, and the process is ocerz. So before any framework is opened, the host's `NXArgc`, `NXArgv` and program name become the guest's, and the first framework opened opens CoreFoundation with `CFProcessPath` naming the guest's executable, which CoreFoundation honours and keeps; the variable is removed again the moment that initializer returns, so no child inherits it. `[NSBundle mainBundle]` is then the application's bundle, and `NSApplicationMain` crosses as an ordinary call: it reads the guest's `Info.plist`, finds the principal class by name, loads the

main storyboard instantiating the guest's own classes, and runs the application with every delegate method a call back into guest code. Image Capture's x86-64 slice runs this way and quits with status 0 when asked; Stickies opens a note and stays up.

Malloc zones. Chromium saves the default zone's function pointers and writes its own over them. Written into the host's zone, those x86 pointers would be the next thing the host's own `calloc` jumps to. So the guest never holds a host zone: every call that answers one answers a view of it in a page of the guest's own, whose pointers are thinks the guest can call. What the guest writes into a view changes only its own calls.

When a crossing fails

A crossing is the one place a thread the guest drives runs native code, so for its duration the thread carries a **bridge frame**: a thread-local record naming the library, the export, the signature and the host address, raised immediately before the call and lowered immediately after, by one small function with no other way out. A fault during a crossing is then reported as such, naming the call and saying whether the faulting address was in guest space (the guest passed a bad pointer) or outside it (ocerz marshalled the call wrong). The process stops there, because a native frame can be neither resumed nor unwound:

```
ocerz: BRIDGE-FAULT[35366] SIGBUS inside a bridged call, not in guest code
ocerz:  call=/usr/lib/libSystem.B.dylib:_strlen sig='L(p)' host_fn=0x18980eac0 depth=1
ocerz:  cause: the fault address is in guest space, so the guest passed a bad pointer to
        _strlen
```

Two exit codes keep the other failures apart. An import that never bound stops the process with **71** before the guest runs, after a report naming every missing (library, symbol) pair at once. An export that bound but has no crossing names itself and stops with **72** when called. In both cases the process stops rather than running with a guessed answer; that is native mode's central rule.

What it costs

Guest loop, ns per iteration (Apple M5)	Trap path	Default	arm64 build
<code>getpid</code>	40.5	17.0	0.7
<code>strlen</code> of a 12-byte string	42.0	2.0	0.9
<code>memcpy</code> of 48 bytes	45.6	3.0	2.7
<code>-[NSString length]</code>	51.3	28.3	3.6
<code>qsort</code> comparator, per call back	426	430	3.1

The trap path is `OCERZ_NO_BRIDGE_FASTCALL=1`. With `OCERZ_NO_LEAF_INPLACE=1` the `strlen` and `memcpy` rows read 17.3 and 20.8, which is what an ordinary crossing costs. Against cache mode the cost shows only in code that crosses in a tight loop: on the benchmark suite's `str` kernel native mode measures 1.03x of cache mode, `memcpy` 0.99x, and kernels that make no calls are at parity. One kernel is slower for a different reason: `fpvec` runs at 1.05x of Rosetta in native mode, against 0.80x in cache mode, because native

mode keeps the translated NaN checks rather than set the processor's alternate floating-point mode, since the host's own code runs on guest threads and two FPCR writes per crossing would cost about as much as the crossing (Chapter 7).

Steam in native mode

On 2026-09-21, on macOS 27, the x86-64 Steam client came up in native mode: the login window, then the 1280 by 800 Steam window with its menus and store view, the GPU process running hardware ANGLE over the host's own OpenGL, and two renderers, stable for minutes. From the web helper starting to the main window took 23 s in native mode and 34 s in cache mode on the same machine; the arm64 Steam takes 2 to 4 s. Reaching it needed, in the order the failures appeared: x86 thunks for native method implementations handed to the guest, since Chromium swizzles `allocWithZone:`; the guest-owned `errno` and thread-specific data keys described above; `bzero` answering its destination in `rax`; system tools with an arm64 slice running as themselves; the SystemConfiguration exports IOKit still carries; every API database loaded before the sandbox closes, because from then on the kernel may refuse ocerz the read of its own data files; a libGL database; Steam's rewritten `ipcserver` job started in its parent's mode; the `syslog`, `warn` and wide `printf` veneers; the malloc zone views; a page size of 4096 everywhere; the guest's own stack bounds for V8's stack scanner; and legacy CoreGraphics display and CoreServices calls that are exported but gone from the headers.

What native mode cannot run yet

Each of these stops a program with a named message, rather than running it wrongly:

- **Swift.** Swift classes and the frameworks' Swift overlays are refused. Calculator and many newer system applications are written in Swift.
- **Frameworks without a database.** 59 libraries have one; WebKit and GameKit are among the notable gaps. `dlopen` of a native library without a database answers NULL with "native library without an API database".
- **C++ across the boundary.** An optional LLVM `libc++`, `libc++abi` and `libunwind` built for x86-64 (`make guest-cxx`) runs guest C++, exceptions between guest frames included, but guest C++ objects are not interchangeable with native arm64 C++ objects, and an exception cannot unwind through a bridge frame.
- **Some variadic and runtime calls.** Most of the `scanf` family beyond `scanf`, `fscanf` and `sscanf`, `err`, `asl_log`, and several method-replacement functions (`method_setImplementation`, `imp_implementationWithBlock`) are stubs. Runtime class creation with `objc_allocateClassPair`, `class_addMethod` and lazy method resolution does work.
- **Odd argument types.** `long double`, `va_list`, and unions or bitfield structures by value are stubs.
- **Bundled XPC services.** A service inside the application's own `Contents/XPCServices` is not reached, most likely because `launchd` looks it up for the executable that asks, which is ocerz.

KEY IDEA

Native mode is a translator for the program and a foreign-function interface for everything else. The FFI is driven entirely by data generated from the SDK's own declarations, so supporting another framework is a regeneration, not a rewrite, and every export that cannot be crossed correctly is a named refusal rather than a guess.

PART V

Real software

Cocoa applications, the macOS Steam client, games, the GPU, and Windows software through Wine.

CHAPTER 13

macOS applications, Steam, games and the GPU

The earlier chapters describe mechanisms; this one describes what they add up to when real software is pointed at them. It walks through the command-line tools, the Cocoa applications, the GPU, the macOS Steam client and the first games, and for each one says what broke on the way and what that taught the translator. Every result here was run by hand and dated; none of it is an automated gate.

16 / 16

 CLI TOOLS BYTE-
IDENTICAL TO NATIVE

74

 SYSTEM TOOLS
AGREEING WITH
ROSETTA

14

 FIXES TO BRING UP
MACOS STEAM

0.54 s

 ROSETTA'S FIRST
BRAWLHALLA FRAME
(AARCHX: 5.3 S)

How an application is run

AArchX runs the x86-64 slice of a Mach-O executable. For a bundled application that used to mean naming the binary inside the bundle, for example `./ocerz /System/Applications/Chess.app/Contents/MacOS/Chess`. Since 2026-09-23 (f430371) the bundle itself can be given: AArchX reads the bundle's `Info.plist`, runs the executable it names, and passes that full path as the guest's `argv[0]`, which matters because applications derive their resource paths from it. That change came from games, which Steam launches by bundle (see *Games* below).

"Works" in this chapter means a command-line tool printed exactly what it prints natively, or an application drew its main window on screen and stayed up. `tools/apptest.sh cli` automates the first check by diffing each tool's output against the native run; `tools/apptest.sh gui` automates the survival check. Neither is part of `make check`, because both depend on the machine's installed software.

macOS 27 changes what there is to run

On macOS 27 Apple ships the bundled applications, Chess, Calculator, Dictionary, Font Book, Grapher, Digital Color Meter, Activity Monitor, Console, TextEdit, Preview and Script Editor among them, as arm64 only. There is no x86-64 slice to translate. AArchX says so and exits, and since 2026-09-23 (9d90155) a guest that spawns or execs an arm64-only program gets the native program, exactly as it would under Rosetta. That rule came from Steam, not from the bundled apps: the Steam client spawns `/usr/bin/open` three times per launch, `open` is arm64-only on macOS 27, and every one of those children used to die on "cannot read" before doing anything. The application results below therefore say which macOS they were measured on. Safari, Steam, Discord and Brawlhalla still carry Intel code and can be run on macOS 27.

Command-line tools

Command-line tools are the most reliable case and the cheapest regression check. Sixteen are diffed against native output by `tools/apptest.sh cli`, and all sixteen match on macOS 26.7.1 and on macOS 27 since 2026-10-03:

Tools	What they exercise
<code>uname</code> , <code>sw_vers</code> , <code>id</code> , <code>echo</code> , <code>basename</code>	process identity, <code>sysctl</code> s, CoreFoundation and, on macOS 27, Swift
<code>ls</code> , <code>wc</code> , <code>sort</code> , <code>uniq</code> , <code>head</code> , <code>grep</code>	file I/O, locale, <code>sysconf</code> , the C library's string and memory routines
<code>file</code> , <code>xxd</code> , <code>nm</code> , <code>plutil</code>	Mach-O and property-list parsing, larger dependency graphs
<code>openssl version</code> , <code>openssl dgst -sha256</code>	two-level namespace binding, vector crypto code

Two of them earned their place by failing. `openssl` links `libcrypto.46.dylib` while the shared cache also carries `libcrypto.44.dylib`, which exports the same names; a flat search bound `OpenSSL_version` to whichever image came first and printed the wrong version. Imports are now resolved in the specific library the binary named (two-level namespace binding, Chapter 11). `sort` needed the `sigaction` structure translated between its two layouts and the 64-bit width of the `hw.pagesize` `sysctl`: writing four bytes into an eight-byte answer had made `sysconf(_SC_PHYS_PAGES)` return zero.

Beyond the sixteen, a wider sweep of 74 system tools run with read-only arguments in cache mode was compared against each tool's own x86-64 slice under Rosetta, and every one agreed: `sed`, `awk`, `otool`, `shasum`, `curl`, `sqlite3`, `perl` and the rest. The only disagreements were tools whose output changes between two runs anyway, such as `ps` and `vm_stat`. One tool cannot be run on a Mac without Xcode: `/usr/bin/python3` there is a stub that asks `xcrun` for the real interpreter, and `xcrun` has no x86-64 slice.

HOW IT WAS FOUND: SW_VERS, TWICE

On macOS 27 `sw_vers` is a Swift program, and from 2026-09-21 it crashed under AArchX with a stack overflow, briefly making the CLI score 15 of 16. CoreFoundation links Foundation as an *upward* dependency, the way a library declares the back edge of a dependency cycle, and AArchX left upward links out of its initializer walk entirely. Foundation's code was reachable anyway, because the whole shared cache is mapped, and its classes registered, so nothing looked missing; what never ran was Foundation's initializer, so `NSString` stayed an abstract class cluster and the first `+[NSString stringWithFormat:]` recursed until the 8 MB guest stack ran out. `bcb6dd` (2026-09-22) initializes an upward dependency after the image that declares it.

On macOS 26.7.1 the same tool crashed again, in a different place: memory allocation recursing into itself. There `libobjc` links `libswiftCore` upward and `libswiftCore` links Foundation upward, and `libobjc` is a dependency of CoreFoundation, so following each upward link right after the declaring library initialized Foundation's whole subtree while CoreFoundation was still unfinished. A small experiment with three libraries of the same shape showed Rosetta's own order, `mid core main top`: an upward dependency waits until the entire walk from the program is done. `e705796` (2026-10-03) does exactly that, and `tests/dynamic/upward_defer.c` pins the order.

Ollama

Ollama's command-line binary (`Contents/Resources/ollama`, a Go program using `cgo`) has worked since 2026-09-13. `ollama --version` prints what it prints natively, `ollama serve` answers its HTTP API (`/api/version`, `/api/tags`, `/api/show`), and `llama-server --list-devices` lists the same devices as natively. It took five fixes, each general:

- **AVX2.** Go's runtime turns on its AVX2 code paths under Rosetta without checking CPUID, so AArchX has to implement them whether or not it advertises them (Chapter 4).
- **sigaltstack reporting `SS_DISABLE`** when no alternate stack has been set, which Go checks before installing its own.
- **dlopen refusing a thin Mach-O of another architecture**, rather than mapping arm64 code it could never run.
- **Bare `@loader_path` and `@executable_path` rpaths**, with nothing after the token.
- **Constructors in programs that do not link `CoreFoundation`**, which the initializer phase had only been run for.

The Ollama menu-bar application runs too: in a 30-second run it started its own server, served its settings page into its window and shut down cleanly on SIGTERM. Before that, WebKit's allocator stopped it within ten seconds, because `thread_suspend` returned `MACH_SEND_INVALID_DEST`: workqueue threads that AArchX started ended without running the guest's thread-exit path, so the allocator was suspending threads that no longer existed (Chapter 9). Nobody has yet run a model under AArchX.

Applications in cache mode

Application	Result	Measured
Chess	works and plays: board window, and the <code>sjeng</code> engine subprocess answers moves	macOS 26.6, 2026-09-11
Calculator, Dictionary, Font Book, Grapher, Digital Color Meter	main window on screen	macOS 26.6, 2026-09-11
Activity Monitor, Console	window on screen; each logs one optional library missing from the x86-64 cache	macOS 26.6, 2026-09-11
TextEdit, Preview, Script Editor	run, and open a document window when given a file	macOS 26.6, 2026-09-11
Safari	runs and browses	2026-09-21, owner-confirmed
Steam (x86-64 macOS client)	full UI with <code>-cef-disable-gpu</code>	2026-09-12
Discord	the Electron client reaches its signed-in view in a 1280 by 720 window, seven processes, as under Rosetta	2026-09-22, owner-confirmed
Brawlhalla	the Adobe AIR game reaches its menus with a Steam client running	2026-09-22, owner-confirmed
Photoshop	stops during startup, later than before	2026-09-22
Photos	aborted on a graphics call; all three causes fixed by 2026-09-22, not run again	2026-09-13

In native mode (Chapter 12) the list is shorter: Image Capture opens its main window and quits cleanly, Stickies opens a note and stays up, and the x86-64 Steam client comes up as described below.

Safari

Safari had the longest bring-up of any application and is the best map of what a large Cocoa program needs. Its executable is only a launcher that links one umbrella framework, so the dependency-ordered initializer phase was at first skipped for it entirely. Behind that came, in turn: the Objective-C image closure capped at 1024 entries, so a method on `NSBundle` defined in a late image was missing; an initializer in one framework calling `operator new` before `libc++abi` had been initialized; `mach_vm_map` taking the 8 KB below a request that was 4 KB-aligned but not 16 KB-aligned; a Mach reply landing on top of the shared cache; and JavaScriptCore's Gigacage, a single 128 GB reservation, which is why the guest arena is 256 GB.

The last wall was quieter. On 2026-09-13 Safari started and never showed a window, and the cause was WebKit's allocator failing to suspend a thread and stopping the process. The guest's main thread was missing from AArchX's emulation of `thread_suspend` and `thread_get_state`, and the workqueue threads AArchX starts were leaving without running the guest's thread-exit path. Both were fixed that day; the macOS 27 work continued through `fdfab7d` (2026-09-19), and Safari was confirmed browsing on 2026-09-21.

Discord

Discord's renderers died the moment they started. Electron asks `mach_vm_region` about a region it expects to be read-only, and stops deliberately when the answer says otherwise. AArchX answered with the host's view of its own memory, where a whole arena is one read-write-execute reservation. Since `857a88e` (2026-09-22) the query is answered from AArchX's own map of the guest in every memory layout, and the loader applies each segment's declared protection, its `initprot`, with `SG_READ_ONLY` honoured for `__DATA_CONST`, so cache mode reports the same three protections for `__TEXT`, `__DATA_CONST` and `__LINKEDIT` that a real system reports. Protecting at load time turned out to be too early, because the Objective-C runtime still writes selector references into `__DATA_CONST`, so images are queued and protected in a batch once Objective-C is done with them.

Three follow-ups made Discord fast rather than merely working. Electron's framework performs about 2,560 weak-coalescing binds for its own C++ names; each one walked every one of roughly 3,600 cache images at 1.8 ms, 4.6 s per process, until `d1a74c8` limited the walk to the roughly 300 images that define weak symbols and put a 1 MB Bloom filter in front of it. Alignment faults always hotpatch now (`3619823`), which took Discord's first ten seconds from 5,391 retranslations to none. And reserving and releasing a gigabyte, which V8 does at startup, went from 1.8 ms to 0.45 ms when a mapping onto free slots became a single host `mmap` (`c2080ed`).

Photoshop and Photos

Photoshop still stops during startup, but finding where produced four general fixes in one commit (`bc6c6dd`, 2026-09-22). Upward dependencies are initialized (the `sw_vers` story above). A dylib built by an old enough toolchain has no export trie at all, only a classic symbol table, so resolving through the trie found nothing in it; 21 of the 60 dylibs Photoshop ships are of that kind, which is why its OpenCV could not find a single Intel IPP entry point. Such an image now gets an index built once from the symbols its `LC_DYSYMTAB` marks as external. Imports declared with `-undefined dynamic_lookup` are allowed to be

missing, as dyld allows; Intel's MKL has 2,161 of them, and a single launch printed fifteen thousand lines of false alarms. And every dyld API entry `AArchX` does not implement now answers zero in both result registers, because a caller that expected a pair read a table through a register it had never been given. Photoshop now stops later, when an initializer runs while the Objective-C runtime holds its lock and the recursive acquisition aborts, which is re-entrancy and wants its own change.

Photos aborted because `CGLChoosePixelFormat` returned error 10002 for every attribute set. The cause was first blamed on a Rosetta-only GPU service, and it was in `AArchX`'s loader three times over: `_dyld_shared_cache_contains_path` rejected the software renderer's plugin path because it runs through a symlink, `dlsym` on a cache image searched the whole cache instead of that image (both fixed 2026-09-13), and the third cause is the subject of the next section.

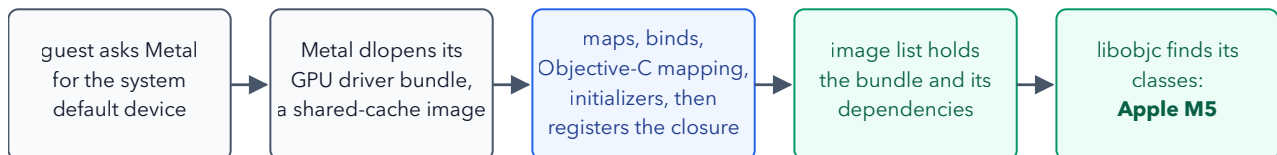
The GPU

Until 2026-09-22 a translated program saw no GPU at all. `MTLCreateSystemDefaultDevice` returned nil and `MTLCopyAllDevices` found no device, on an Apple M5 that Rosetta and a native arm64 build both report. OpenGL had only its software renderer left to offer, so `CGLChoosePixelFormat` failed for every accelerated attribute set, the same 10002 that had already been blamed on two other causes. Brawlhalla put a window on screen, ran its ActionScript and never drew into it.

Before 2026-09-22



Since 69590ca



Registering before the Objective-C mapping would be wrong in the other direction: libobjc calls back into the dyld APIs while mapping, and an image already listed then is treated as handled and loses its categories.

Figure 13.1. Why a translated program saw no GPU until 2026-09-22, and the order in which a `dlopen`ed shared-cache image is now registered.

One bug caused all of it. In cache mode the dyld APIs that walk the loaded images, `_dyld_image_count`, `_dyld_get_image_name` and `_dyld_get_image_header`, are answered from the image list `AArchX` keeps. A `dlopen` of a dylib on disk added to that list; a `dlopen` of an image that lives in the shared cache returned its Mach header and added nothing. The library was loaded, its symbols resolved and `dldaddr` knew where they lived, while every walk of the list said it was not there: the count stayed at 45 where a real system goes to 348. The Objective-C runtime keys its per-image queries off that list, so asking it for a bundle's classes returned nothing for a bundle whose classes `objc_getClass` could already find, and Metal, which loads its GPU driver bundle exactly that way, concluded there was no device.

69590ca (2026-09-22) registers a cache image and its dependency closure at the end of the `dlopen`, after the Objective-C mapping and the initializer phase. The order is the whole fix: registering earlier made `libobjc`, which calls back into the `dyld` APIs while it maps an image, treat the new image as already handled, and the framework lost its categories, which are usually what a framework is `dlopened` for. `tests/dynamic/dlopen_image_list.c` opens two cache libraries and checks the list before and after each, comparing the header the list reports against the image a symbol really lives in rather than against the opaque handle. Metal, `CGLQueryRendererInfo`, `CGLChoosePixelFormat` and `CGLCreateContext` now answer as under Rosetta, with `GL_RENDERER` reading `Apple M5` and `GL_VERSION` reading `2.1 Metal - 91.7`.

A smaller GPU-adjacent fix followed the next day. Some x86-64 libraries are in neither the shared cache nor on disk but only in the Rosetta cryptex, and Intel code names them by their system path; Metal opens `/usr/lib/libMTLHud.dylib` when `MTL_HUD_ENABLED=1`. Since 53c6096 a cache-mode `dlopen` of an absolute path found nowhere else retries it under `/System/Volumes/Preboot/Cryptexes/Rosetta`, so the Metal performance HUD appears under AArchX too.

The macOS Steam client

The x86-64 macOS Steam client is the largest macOS program AArchX runs, and it is a family of programs rather than one. It came up with its full UI on 2026-09-12 in cache mode: the bootstrapper, `ipcserver`, the client, and the CEF web helper with its GPU, utility and renderer processes, all translated, with the Steam window drawn through `SwiftShader`, Chromium's software renderer. After its first self-update the client lives in Application Support:

```
./ocerz "$HOME/Library/Application Support/Steam/Steam.AppBundle/Steam/Contents/MacOS/steam_osx" -cef-disable-gpu
```

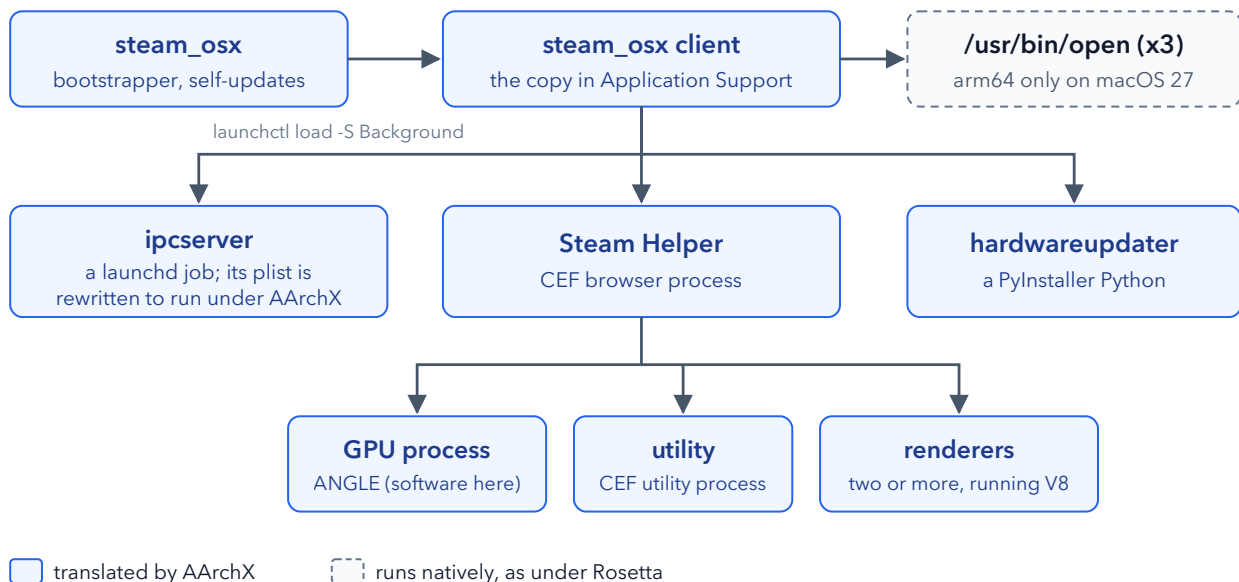


Figure 13.2. The macOS Steam client's processes under AArchX in cache mode. Every Intel process in the tree is translated; the arm64-only `/usr/bin/open` children on macOS 27 run as themselves.

What it took, in order

Each step below was found because the previous one let Steam get further. Read in order, they are a fair cross-section of what a large real program asks of a translator that a benchmark never does.

#	Date	Symptom	Fix
1	09-11	bootstrapper stops before updating	System V semaphores, which Steam's tier0 threading code uses (<code>05d42d6</code> , with the rest of SysV IPC); with them it runs and self-updates
2	09-12	an assertion on a path beginning <code>../Steam.AppBundle</code>	the guest gets an absolute executable path, as the kernel gives one; Steam derives its bundle root from it
3	09-12	duplicate Objective-C classes	<code>dlopen</code> of the executable's own path returns the image already loaded instead of loading a second copy
4	09-12	a crash at address <code>0x28</code> after 38 s	<code>dlopen</code> initializers run in dependency order (<code>ad5f0e8</code>); see the callout
5	09-12	Chromium's allocator cannot reserve its region	a guest arena big enough for PartitionAlloc's 32 GB reservation, later 256 GB for JavaScriptCore
6	09-12	<code>ipcserver</code> init failed	<code>ipcserver</code> run under AArchX: the launchd plist Steam writes is rewritten on the way through, and the job's label re-enabled first
7	09-12	whole-process stalls during translation	the JIT lock is taken without ever parking, so a lost wakeup cannot wedge translation
8	09-12	the web helper never reports ready	a page refused translation for churning is retried after 4,096 refusals as well as after 1.5 quiet seconds
9	09-12	a fixed translation thrown away again	a fault from an already retired translation just recovers instead of invalidating its page again
10	09-12	<code>hardwareupdater</code> crashes	upward links in <code>dylib_use_command</code> form stop counting as initializer-order edges
11	09-12	sandboxed helpers load the 386 MB CEF framework twice	loaded images are matched by device and inode, as dyld does, because <code>realpath</code> fails inside the sandbox
12	09-13	renderers die on a libc++ comparator assertion	a self-loop compare whose loop head reads the flags is no longer fused (<code>tests/dynamic/cef_partition.s</code>)
13	09-13	macOS asks for the login password once per helper	Steam Helper runs with <code>--use-mock-keychain</code>
14	09-23	three children die per launch on macOS 27	arm64-only children run natively

HOW IT WAS FOUND: THE CRASH AT 0X28

Step 4 is the clearest example of an initialization-order bug. `steamui.dylib` depends on `libtier0_s.dylib`, Valve's low-level runtime library, which owns the global allocator, but AArchX sorted a `dlopen`'s initializer closure by load address, and `steamui` happened to be mapped below `tier0`. Its first container growth therefore called through Valve's global allocator object before tier0's initializer had set the object's vtable pointer: a call through a null vtable at offset `0x28`, a fault at exactly that address. The fix collects the closure in dependency post-order and never sorts it. It was found independently by two agents on two machines the same day, line for line identical, and with it Steam ran its full 100-second test window instead of dying at 38 seconds, its peak memory going from 0.64 GB to 2.62 GB as it got further.

Two of those steps deserve more detail. `ipcserver` is started by `steam_osx` with `launchctl load -S Background` on a plist it writes into Application Support, which would make `launchd` run it natively, outside the translator. AArchX rewrites that plist as it passes through, putting itself in front of the program in `ProgramArguments` and in `Program` (Steam's own plist only has `Program`, so the array is built from it), and enables the job's label first: a legacy `launchctl unload` leaves the label disabled, and every later load then fails with an I/O error while the client reports `ipcserver init failed`. The Mach service lookup itself was never the problem, since Mach traps go straight to the host kernel.

The **mock keychain** is a trade-off rather than a fix. Chromium reads its "Steam Safe Storage" item from the login keychain at startup; that item does not trust the `ocerz` binary, so macOS asked for the password once per helper process, and each helper blocked until the prompt was answered. With `--use-mock-keychain`, which AArchX adds whether `Steam Helper` is launched by Steam or directly, cookies CEF stores under AArchX are encrypted with the mock key, so they are not readable by a native Steam, nor the reverse. `OCERZ_NO MOCK_KEYCHAIN=1` turns it off.

Steam in native mode

On 2026-09-21, on macOS 27, the same client came up in native mode (`./ocerz -native ...steam_osx`), where the guest's system libraries are synthesized stubs into the Mac's own arm64 frameworks (Chapter 12). It reached the login window, then the 1280 by 800 Steam window with its menus built and the store view loading, with the GPU process running **hardware ANGLE over the host's own OpenGL** and two renderers, stable for minutes with nothing reported. From the web helper starting to the main window took 23 s in native mode and 34 s in cache mode on the same machine; the arm64 Steam takes 2 to 4 s. The work that got it there (`0f810e8`) is a list of places where Chromium assumes it owns the process:

- x86 thinks for native Objective-C implementations handed to the guest, because Chromium swizzles `allocWithZone:` and then calls the original through a pointer;
- a guest-owned `errno` slot and guest-owned thread-specific-data keys, because AArchX's own code changes the host's between any two guest instructions, and CEF makes eighty inline `gs`-relative reads;
- views of the host's malloc zones, so that Chromium writing x86 hooks over the default zone cannot send the next native `calloc` into x86 code;
- a page size of 4,096 everywhere, which V8 checks against `PartitionAlloc`'s compile-time constant, and the guest's own stack bounds for V8's stack scanner;
- bridged `thread_suspend` and `thread_resume` that refuse to leave a thread stopped while it holds the JIT lock or the map lock or is anywhere inside the host's allocator.

`hardwareupdater` still fails to run its script in native mode, and several imports Steam has not reached yet are stubs. Neither run is part of a gate.

Games

Launching a game from Steam

Steam runs a game through a tool when the game's launch options are set to `/path/to/ocerz %command%`. It then passes the game's `.app` bundle as the argument and puts its loader and overlay libraries in `DYLD_INSERT_LIBRARIES`. Three things stood in the way, all fixed in `f430371` (2026-09-23):



Figure 13.3. What happens when Steam launches a game through `ocerz %command%`.

- **A bundle as the program.** Given a bundle, AArchX runs the executable the bundle's `Info.plist` names, with that path as the guest's `argv[0]`.
- **DYLD_INSERT_LIBRARIES in cache mode.** The libraries are opened inside the guest before its entry point runs. The catch is that the host's own dyld reads the same variable when it starts `ocerz`, and would load the arm64 slices of those libraries into the translator itself. So `ocerz`, started with the variable set, moves it aside, re-executes itself without it, and restores it for the guest.
- **Libraries in the pre-10.6 format.** A dylib built for Mac OS X 10.5 has local and external relocations and an indirect symbol table instead of compressed fixups, and AArchX applied no fixups to it at all, so Steam's `steamloader.dylib` called address zero at once. Those relocations are now applied (dynamic test `dlegacy_format`, whose library is itself built for 10.5). The same library links `libgcc_s.1.dylib`, which the shared cache knows only as an alias in its dylibs trie; 108 of the SDK's 111 aliased `/usr/lib` names failed to load until `a8e2cea` made every path lookup fall back to that trie.

Steam's own frame counter in its overlay relies on dyld interposing, which AArchX does not do, so `OCERZ_FPS=1` (`1aa3db7`) puts four instructions at the head of the translated block that begins the Intel `CGLFlushDrawable` export, counting calls, and a thread prints frames per second once a second. The guest sees nothing of it.

Brawlhalla

Brawlhalla, an Adobe AIR game with an x86-64 macOS build, is the first game in the record. It needed two things that were new on 2026-09-22: the GPU fix above, without which it ran its ActionScript into a window it never drew, and System V `shmat` and `shmdt` (`815cbd9`), which were not in the syscall table at all. Steam's client talks to a running game through a System V shared-memory segment, so the game asked for one, got `ENOSYS`, carried on with a null pointer and segfaulted at address zero. The host kernel refuses a fixed address for `shmat` even over address space the arena has reserved, so the segment is attached wherever the host wants it and `mach_vm_remap` carries that mapping into guest space, sharing the pages rather than copying them (Chapter 8).

Measured the same day against Rosetta, with one Steam client serving both: **the first frame took 5.3 s against Rosetta's 0.54 s**, and the game's main thread saturated a full core where Rosetta used a quarter to half of one. Three JIT changes that evening are a good example of how a real program finds costs a benchmark suite does not (Chapter 6 has the mechanisms):

Change	What was wrong	Measured effect
A return that misses the host return-address stack stays in the body (41e2773)	Every trip out of translated code resets the host stack and so empties the return-address stack. Engine code calls every ten instructions and trips out every few hundred thousand blocks, so the stack was permanently empty: 3.2 billion misses in 88 s, each one leaving through the epilogue.	2.6-2.9 to 3.85 billion guest instructions a second
Veneer pools (5681be5)	A chain is a b patched into the exiting block, and b reaches 128 MB. The arena fills front to back, so script code translated late could not chain to engine helpers translated early; 4.5-5.8% of chains were silently dropped and a quarter of the main thread's time was in prologues and epilogues. A 256 KB pool every 64 MB of arena now holds indirect jumps that reach anywhere.	71% of a core, 60% more blocks a second; every chain activates
Constant addresses resolved at translation time (fabd213)	A global load or store paid a commpage range check and, in ordered mode, an alignment test at run time, for an address that is a constant of the translation.	14-18 host words become 4; 13% more blocks a second

Taken together the main thread went from a saturated core to about three quarters of one while doing about 75% more work per second, and at the menu it now spends most of its time waiting on the display, as it does under Rosetta. 0CERZ_TRIPSTAT=1, added on 2026-09-24 to see what remains, measured 6.6 million exits to the dispatcher a second at the menu, ten destinations making 92% of them, which is where the 14% of the main thread still spent outside translated code goes.

WINDOWS GAMES

Counter-Strike 2 and R.E.P.O. are Windows games and run through Wine; they are covered with the rest of the Wine work in Chapter 14.

What is still open

- **Steam's GPU process in cache mode.** CEF's GPU process initializes ANGLE through CGL, and every accelerated pixel format failed until the loader fixes of 2026-09-13 and 2026-09-22, so cache-mode Steam runs with `-cef-disable-gpu`. It has not been retried with acceleration since; native mode already runs the GPU process on hardware ANGLE.
- **Launch stalls.** Some launches stall before the web helper starts its child processes, with the JIT lock held inside translation; relaunching gets past it.

- **A Chromium initializer that traps.** On 2026-09-12, on both test machines and at the same address, one static initializer in the Chromium Embedded Framework hit an `int3` after about 2.3 million instructions. AArchX runs initializers reached through `dlopen` in a tolerant mode, so Steam carried on; which Chromium check fires has not been identified.
- **Startup time.** Large applications start several times slower than under Rosetta, because everything they run at startup is translated the first time it runs. The translation cache (Chapter 7) helps on later launches, but does not close the gap.
- **Photoshop** stops on Objective-C runtime-lock re-entrancy during initialization; **Photos** has not been rerun since its causes were fixed; nobody has run a model under **Ollama**.

CHAPTER 14

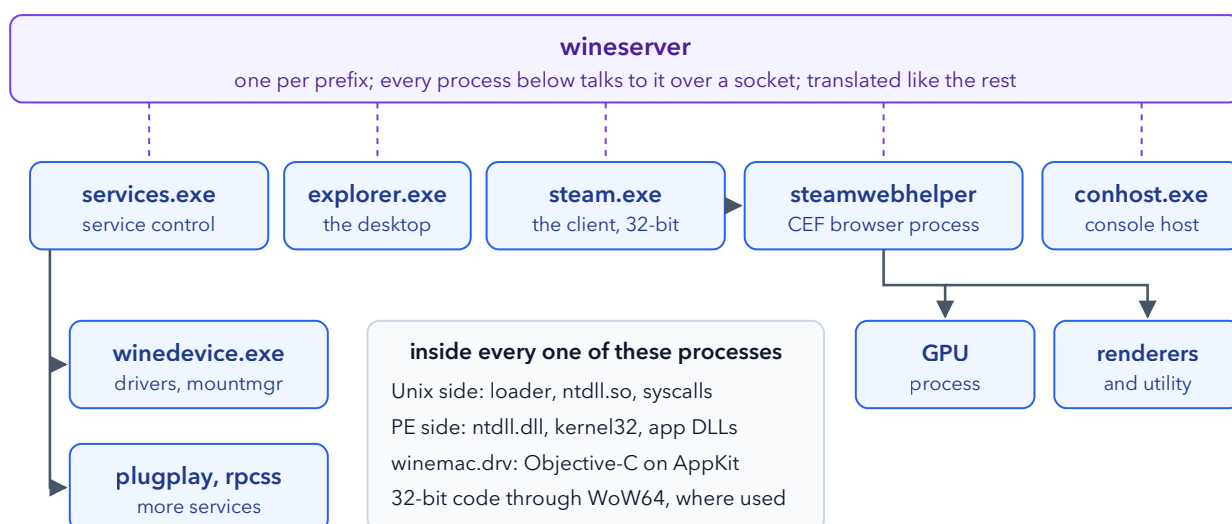
Wine and WoW64

Wine is the hardest guest AArchX runs and the one that has taught it the most. A Wine session is a dozen or more cooperating processes, each mixing Unix code, Windows PE code and Objective-C, with its own contract for the segment registers, its own exception model layered on Unix signals, and a 32-bit Windows world inside a 64-bit process. This chapter explains what Wine asks of a translator, how AArchX answers, and the long series of walls between the first Notepad window in July and R.E.P.O. rendering at roughly 45-65 frames a second in October.

~15 s WINEBOOT ON THE FIXED ARENA BASE	33 s WINDOWS STEAM TO ITS MAIN WINDOW (ROSETTA 11-12 S)	19 EMULATED PROCESSES IN ONE STEAM SESSION	45-65 fps R.E.P.O.'S MAIN MENU, 2026-10-04 (ROSETTA ~178)
---	---	--	---

Why Wine is the hardest guest

Wine is not one program. A Wine process on macOS is a Unix-side loader and `ntdll.so`, which talk to the host kernel and to the rest of the session; a set of Windows PE modules, `ntdll.dll`, `kernel32.dll` and the application's own DLLs, running as ordinary x86 code; and a display driver, `winemac.drv`, written in Objective-C against AppKit and Metal. Every process of a session talks over sockets to `wineserver`, which keeps the Windows kernel objects, and a prefix starts its own service processes at boot. All of it is x86-64 code, and under AArchX all of it is translated, `wineserver` included.



All boxes are x86-64 processes translated by AArchX. Solid arrows show which process starts which.

Figure 14.1. A Windows Steam session under Wine, simplified. On 2026-09-05 such a session was nineteen emulated processes.

That structure stresses a translator in ways a single program never does:

- **Two ABIs and two segment-register contracts in one thread.** The same thread runs Unix code, where `gs` points at the Darwin thread-specific data (TSD), and Windows code, where `gs` points at the Windows thread environment block (TEB), and switches between them on every system call.
- **Windows exceptions built on Unix signals.** A Windows access violation is a host `SIGSEGV` that Wine's handler turns into a Windows exception, walks Windows unwind data for, and resumes. The translator's signal frames, register recovery and `sigreturn` all have to be exactly what Wine expects.
- **Low addresses.** Windows programs, and the whole 32-bit WoW64 world, live below 4 GB, where a 64-bit macOS process has nothing mapped and cannot map its own code. AArchX gives Wine processes a separate memory layout for that, described below.
- **Many processes that share memory.** The processes of a prefix share files, shared-memory sections, a page of kernel data and, through `wineserver`, each other's address spaces, which `wineserver` reads and writes with Mach calls aimed at another task.
- **Cocoa and Metal underneath.** `winemac.drv` brings in AppKit, Metal and their XPC services, so every Wine GUI process is also a Cocoa application.

That is why Wine has been a better test than any benchmark: each of its parts reaches a different corner of the emulator, and the bugs it found were almost all general.

Running it

```
WINE="/path/to/Wine Devel.app/Contents/Resources/wine"
export WINEARCH=wow64
export WINEPREFIX="$HOME/.wine-ocerz"

./ocerz "$WINE/bin/wine" wineboot -u
./ocerz "$WINE/bin/wine" notepad
```

Three Wine builds have been run: Wine Devel 11.8, the build the README describes; MacNdCheese's unified Wine, which Windows Steam and the games are measured with; and, since 2026-10-01, stock Wine Staging 11.16 with DXMT. Children inherit everything they need, the translator included: a Wine process that starts another goes through AArchX's own `exec` and `posix_spawn`, so the whole tree stays translated.

ALWAYS UNDER A WATCHDOG

On 2026-08-17 an unwatched `wineboot` under AArchX went into Wine's service respawn loop, seventeen `explorer.exe` processes at 4 to 8 GB each, took the machine past 100 GB and killed WindowServer. Since then every Wine session is run under `tests/safe_run.py`, which puts the command in its own process group, checks the group's process count, physical footprint and wall clock every 0.3 s, and kills the group, plus anything that daemonized out of it, when a limit is crossed. The footprint, not RSS, is what is measured, because RSS counts the resident shared-cache pages again in every process.

The segment-register contract

Wine on macOS uses `gs` for two things. While Unix code runs, the `gs` base is the Darwin TSD, where the C library keeps `errno` and pthread data. While Windows code runs, it is the TEB, where Windows code expects `gs:[0x30]` to be the TEB's own address and `gs:[0x58]` to be the thread's TLS array. Wine

switches between the two at each of seven transitions in its code, each through the `_thread_set_tsd_base` machine-dependent call, and it saves the pthread base in the TEB at offset `0x320` so that its assembly can find its way back. Its own code caches the TEB in `r13` before switching.

In AArchX the `gs` base is ordinary CPU state, a field of the guest CPU structure that every `gs`-relative access adds in (Chapter 6), and `_thread_set_tsd_base` sets it. The difficulty is everything else that touches it:

- **Signal return.** When AArchX builds a signal frame it stashes the interrupted `fs` and `gs` bases past the end of the Darwin `ucontext`. Measured on the real kernel, a base that a handler sets survives `sigreturn` when the task has no LDT. Wine relies on exactly that: its handler installs the TEB as the `gs` base before returning to Windows code. Restoring `gs` from the stash undid that, and Windows code then ran on the pthread base until something read `gs:[0x30]`. The restore was removed in July (b5279e8), came back with a broader change in August (890fadd), and since b1957b3 (2026-09-26) only `fs` is restored.
- **The TLS array.** Wine's macOS port copies the TEB fields Windows code reads into reserved TSD slots when a thread starts, so `gs:[0x58]` under the TSD base gives the TLS array. But the TLS array is the one field that changes after that copy is taken, and a thread whose copy preceded its TLS setup read zero, which crashed MSVC `__declspec(thread)` code. TSD slot 6, at `gs:[0x30]`, holds the TEB's own address on Wine threads and zero on every other thread, so AArchX special-cases the absolute `gs:[0x58]` form: when slot 6 is non-zero it reads the live field through the TEB instead of the stale copy. The test is self-selecting, so nothing else is affected, and both the interpreter and the JIT implement it.
- **Machine-dependent calls.** The Unix-class system calls clear `rdx` on return under Rosetta, and AArchX does the same. Machine-dependent calls such as `_thread_set_tsd_base` leave `rdx` and the flags alone. Applying the Unix rule to them crashed every Wine process at its first return to Windows code, because Wine keeps its frame pointer in `rdx` across that call (fixed 2026-09-04, test `machdep_rdx`).
- **Thread identity.** `thread_info` with `THREAD_IDENTIFIER_INFO` reports a thread's TSD address. Under Wine Staging, whose `gs` moves between TSD and TEB on every system call, AArchX answered with the host TSD of whichever host thread ran the guest thread, and on a host stack above 12 GB that leaked AArchX's own TSD into Wine, which then lost its TEB. Since 654a81a (2026-10-01) the answer is mapped back to the owning guest thread's TSD.

WoW64 and 32-bit code

Modern Wine runs 32-bit Windows programs the way 64-bit Windows does, through WoW64: the process is 64-bit, the 64-bit `ntdll` and `wow64cpu` stay in charge, and 32-bit code runs in a 32-bit code segment inside it. Wine defines that segment in the process's local descriptor table, and `wow64cpu` reaches 32-bit code with a far transfer through its selector. Rosetta runs 32-bit code inside WoW64 too, so this is parity rather than novelty; standalone 32-bit Mach-O programs are not supported by current macOS at all.

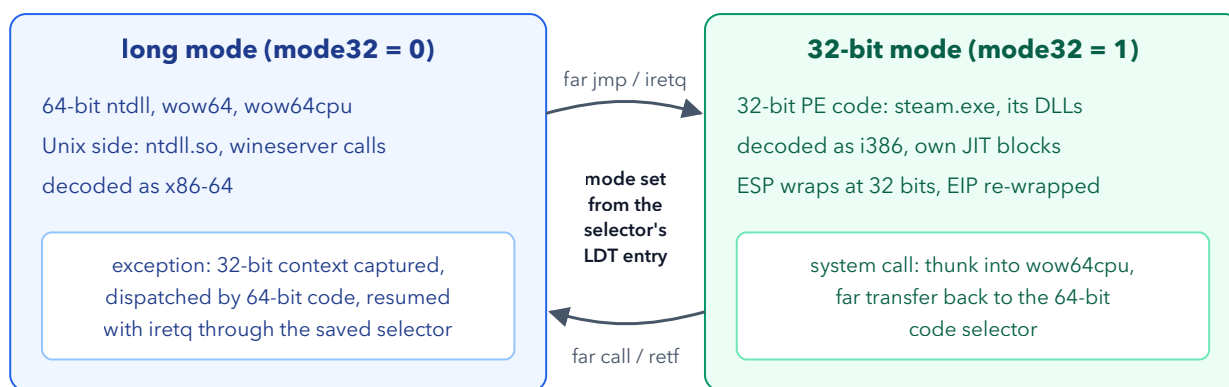


Figure 14.2. How a WoW64 process moves between long mode and 32-bit mode under AArchX. Every far transfer funnels through one helper that sets the selector, the mode and the instruction pointer together.

AArchX keeps its own copy of the LDT entries Wine installs and decides the CPU's mode from the descriptor a far transfer loads: a selector whose descriptor is a 64-bit code segment means long mode, one whose descriptor has the 32-bit default size means 32-bit mode. Every far transfer, `jmp far`, `call far`, `retf` and `iret`, goes through a single helper in the interpreter, so the selector, the mode and the instruction pointer can never disagree, and the decoder is told the mode in exactly one place. The JIT keys its blocks on the mode they were translated for, so the same bytes can have a 32-bit and a 64-bit translation. The i386 support itself, the decoder's 32-bit operand and address rules, the interpreter's BCD and segment instructions, the JIT's 32-bit forms, is covered in Chapters 4 to 6, and its differential gate of 20,033 generated cases in Chapter 15.

HOW IT WAS FOUND: THE RESTART STORM

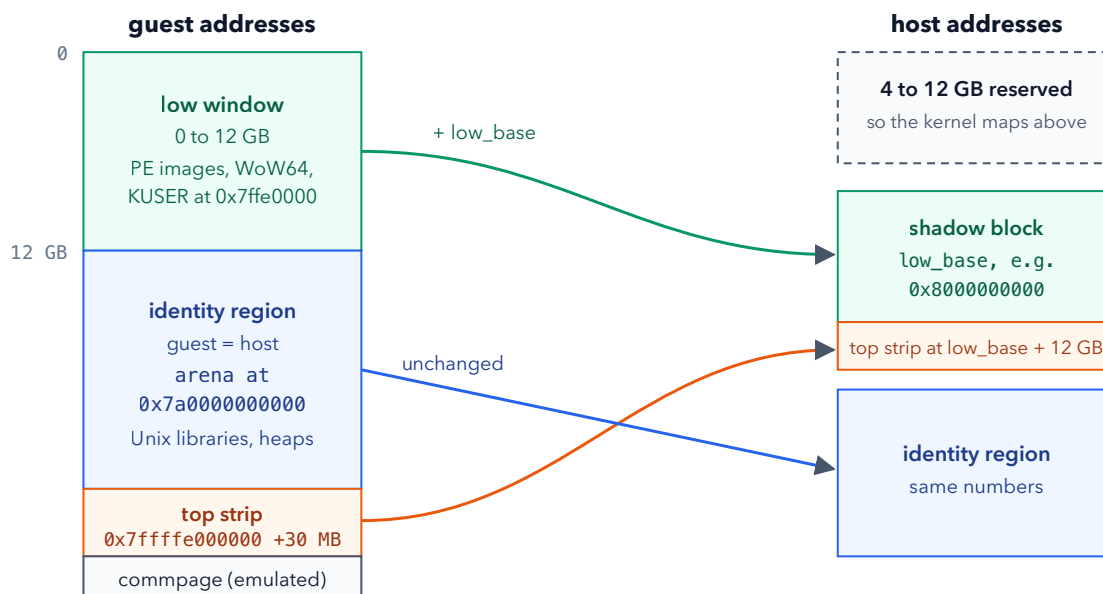
In early September every Steam process died within a second of starting, at its first `OutputDebugString`, and Steam relaunched itself in a loop. `OutputDebugString` raises a Windows exception, so the first exception was killing it. The decoder had folded `mov r/m, sreg` into the long-mode selector constants, so the 32-bit `RtlCaptureContext` recorded the 64-bit code selector in the context it captured; WoW64 copied that selector into the frame it restores with `iretq`, and the 32-bit exception dispatcher then ran as 64-bit code until its stack overflowed. Segment selectors are CPU state now (2026-09-05), read and written like any register.

A second detail came out of the same hunt. A selector slot is 16 bits, and the hardware ignores the rest; Wine's 32-bit context structure stores `SegCs` and `SegSs` as 32-bit fields that a 16-bit store fills, so their upper halves are stack garbage by the time `wow64cpu` pushes them for an `iretq`. A far transfer now takes the low 16 bits only. A 14-second MinGW reproduction and `OCERZ_MODEL0G=1`, which prints every far transfer with its selector and target mode, found it. The guest tests `mov_sreg` and `far_sel_bits` and the i386 differential case `mov-sreg pin both`.

Memory: the low shadow

A normal 64-bit macOS process reserves its first 4 GB as `__PAGEZERO`, and AArchX itself lives in the host address space too, so a guest that needs memory at low addresses cannot simply have it at the same numbers. Windows code needs exactly that: PE images at 32-bit bases, `KUSER_SHARED_DATA` at `0x7ffe0000`,

and the entire 32-bit address space of WoW64 code. When the program being loaded has a segment below 12 GB, AArchX switches the process to its third memory layout, the **low shadow** (Chapter 8 describes all three).



Not to scale. Every process of a prefix uses the same low_base (inherited in OCERZ_LOWBASE) and the same arena base.

Figure 14.3. The low-shadow layout of a Wine process: where guest addresses live on the host.

The rules, from `include/ocerz/mem.h` and `src/mem.c`:

- **The low window.** Guest addresses below 12 GB (`OCERZ_LOW_LIMIT` , `0x300000000`) are stored at `low_base` plus the address. One host block holds the 12 GB window and the top strip; AArchX tries the bases `0x8000000000` , `0x10000000000` , `0x5000000000` and `0x600000000000` in turn, or exactly the base its parent passed in `OCERZ_LOWBASE` , so every process of a prefix agrees. That agreement matters because `wineserver` reads and writes its clients' memory with Mach calls aimed at their tasks, and those calls must move a low address by the same base (`654a81a`).
- **The top strip.** The 30 MB below the commpage, `[0x7ffffe000000, 0x7fffffe00000)` , is also redirected, to `low_base` plus 12 GB, because Windows puts things there that the host cannot give at those numbers.
- **The identity region.** Everything else is at its own number, guest equal to host. The Unix side of Wine, the shared cache and the heaps live there.
- **The fixed arena base.** Since `dff4f4a` (2026-09-26) a process whose loader is named `wine` reserves its identity arena at `OCERZ_WINE_ARENA_BASE` , `0x7a0000000000` , falling back to the usual placement only if that range is taken. The callout below explains why.

HOW IT WAS FOUND: ONE PAGE, EVERY PROCESS

Every Wine process writes the address of its `__wine_syscall_dispatcher` into a slot at `0x7ffe1000`, next to `KUSER_SHARED_DATA`. Under AArchX that slot shares a 16 KB host page with the file-backed `KUSER_SHARED_DATA`, so it is physically one page shared by every process of the prefix. That is harmless only while the value is the same everywhere, that is, while `ntdll.so` loads at the same address in every process.

It was found twice. On 2026-09-06 Steam's CEF children crashed reading address zero: a new process zero-filling its fresh pages zeroed the shared slot for every process, and until it stored its own pointer a few instructions later every system call elsewhere called `NULL`. The zero-fill now leaves that slot alone. On 2026-09-26 wine notepad died while wineboot was still starting services: with the arena wherever the host put it, `ntdll.so` loaded at a different address in each process, the last process to write the slot won, and every other process sent its system calls into memory it had never mapped. With the fixed base, `ntdll.so` sits at `0x7a0041c1c000` in all of them, the whole service tree starts, wineboot finishes in about 15 s, and Notepad opens.

Host pointers into the low window

The low shadow has a cost on the boundary with the host. When the kernel or a system service hands the guest a host pointer below 12 GB, that number names the guest's own memory at that address, not the host memory it points to. AArchX handles this with **aliases**: the host region behind such a pointer is mapped again into the shadow at the same guest number, so the guest can read it. IOKit replies that carry addresses of kernel-mapped GPU memory were the first case, in July. Four refinements came in October, all from R.E.P.O.:

- **Clipping** (654a81a). An alias stops at the first page that holds guest data. Aliasing a 2 GB slice of the host's arm64 shared cache once replaced Wine's loader header, and the windowing system walked that header for ten seconds until the GPU watchdog fired.
- **Inner pointers** (28e237b). Creating an IOSurface answers with records inside memory the kernel shares with the process, at addresses that are not page-aligned. The reply scan only took page-aligned values, so those records were read through whatever the low shadow held at their numbers; a buffer size came back as four gigabytes with the right size in its low bits, and R.E.P.O. aborted on a nil drawable in about two launches of three. Eight-byte-aligned values inside IOKit or shared host memory are now candidates too, unless the guest has its own memory at that number.
- **Refresh** (0115ead). Metal frees a buffer and maps the next one at the same host address. The alias kept the freed buffer's memory object alive, so the guest read the previous compute job's results and a job could wait forever. Each alias now records the memory object it shows, and is mapped again when the host has put a different object there.
- **Keeping the window clear** (3cc5842). The identity arena starts at 12 GB, so the kernel put its own mappings in the free host space below it, exactly where Wine and Unity keep their memory: IOSurface's records landed on numbers the guest already used, where they cannot be aliased, and R.E.P.O. failed to start in three launches of five. A low-shadow process now reserves every free host range between 4 GB and 12 GB, at startup and after each alias, so the kernel maps above the arena, where guest and host numbers agree. With it, eight launches of ten reached the menu with no wild pointer.

Pointers that point the other way have the same problem. `__mac_syscall` passes the kernel a policy-defined structure that points into the caller's stack, and under Wine the Unix side runs on a per-thread stack inside the low window. The kernel received guest addresses, the sandbox check failed with `EFAULT`, and the Metal OpenGL renderer read that as no access to the GPU: every pixel-format request from a Wine thread failed, and `wined3d` found no GL adapter. Since `d82e1e7` (2026-09-26) words in the first twelve of that structure that point near the caller's stack pointer are rewritten to host addresses for the call and restored afterwards; `dxdiag` under Wine reports "GL renderer: Apple M5" with a 4.1 core context. The voucher-creation Mach trap had the same flaw from a low stack, and `libdispatch`, which creates vouchers for the work it queues, crashed on purpose until `f832152` (2026-10-03) translated its pointers too.

The cost of the low window, and how it came down

In the low shadow every guest memory access needs a translation, because the guest number may be in the window, in the identity region or in the top strip. Until October that translation built three 64-bit constants and tested three ranges, about twenty host instructions per load, and the JIT's fastest memory forms, which assume an address can be used as it is, were switched off entirely. Two rounds of work narrowed the gap. In September (`3eeaf45`, `8e84910`) memory operands, calls, returns, the host return-address stack and spliced calls learned to take the guarded path instead of falling back to the interpreter, which took Steam's web helper from 47-67% of its samples in translated code to 78-81%. In October (`16a28b1`) the guard itself shrank: when the window's host base has no bits below 16 GB and is encodable as an arm64 logical immediate, as `0x8000000000` and `0x10000000000` are, an address below 12 GB becomes a host address with a shift, a compare and an `orr`, and one in the identity region is recognized with two shifts and an `add`. In the Wine layout that took `memcpy` from 6.1 times Rosetta's time to 3.1, a mixed workload from 3.0 to 1.65 and an interpreter loop from 1.5 to 1.0.

The walls, in order

The Wine bring-up ran from June to October. Most of its walls were not where they first appeared to be, and the way each was found is as instructive as the fix.

June and July: the first windows

- **libdispatch.** `CFRunLoopRunInMode` never returned, which blocked `winemac.drv`'s Cocoa startup. Four bugs, fixed in July; bridging guest work queues to the host's real workqueue mechanism is what made dispatch work at all (Chapter 9).
- **The display configuration.** Hundreds of "Failed to read display config" lines came from `MTLCreateSystemDefaultDevice` returning `NULL`, because one dyld API, `dlopen_preflight`, was unimplemented and Metal concluded its compiler was missing. A dozen lines of code fixed it, and the real Apple GPU appeared. Display problems are logged under `WINEDEBUG=+display`, not `+macdrv`.
- **Invalid exception frames** (`78dee57`, 2026-07-31). `mach_vm_region` replies described the host's map instead of the guest's, so Wine's address-space reservation recorded garbage areas and later put a thread stack at a shadow host address, and exception dispatch rejected every frame. Region queries now answer from AArchX's own accounting.
- **The explorer deadlock** (resolved 2026-08-18). Three threads waited on a mutex owned by a thread waiting on a Win32 handle. The real cause was older: the hand-built main thread's cached thread id

had never been filled in, so `pthread_threadid_np` returned 0, mutexes recorded owner 0, which reads as unlocked, and the main thread's mutexes provided no mutual exclusion at all.

August: the Steam client starts

Date	Wall	What it really was
08-17	Wine sees no GPU	IOKit out-buffer pointers inside a request were not translated under the low shadow (<code>5bd2e42</code>)
08-19	32-bit code	the i386 decoder, interpreter, JIT and WoW64 host support
08-24	sessions ending 15-60 s after a window appeared	AArchX rewrote an interrupted Mach receive as a timeout, which CoreFoundation took as its run loop timing out, and applications exited with status 0 (<code>5c35c25</code>)
08-25	a desynchronized wineserver protocol	the guest's signal mask was never enforced, so a <code>SIGUSR1</code> landed in the middle of a <code>wineserver</code> round trip
08-25	a fast failure after 23 s	the <code>gs: [0x58]</code> TLS copy described above
08-25	Steam exits silently with status 14	the bootstrapper requires SSE4.2, so it is implemented and advertised; status 14 is the client asking to be restarted
08-25	first window	Steam's rescue dialog, rendered and clickable through <code>winemac.drv</code> ; a fresh prefix boot fell from 43 s to 17 s
08-26	the web helper times out at startup	V8 reserves a 32 GB sandbox with a fixed-address <code>mach_vm_map</code> , and AArchX's region table held 16 entries and leaked one on overflow, refusing 642 million calls in one run
08-26	two real crashes	an <code>os_unfair_lock</code> whose owner died, now broken as a robust mutex is, and guest self-kills sent to the host instead of the guest
08-26	profile database errors	813 refused fixed-address mappings that two instrumentation bugs had hidden

September: to the login window

The first week of September was a series of short, sharp bugs, most found by bisecting the JIT against the interpreter (Chapter 17):

- **Black windows** (introduced by `cf3b9ad` on 2026-08-25, fixed by `0c03488` on 2026-09-05). Every Wine window painted black. clang's outliner produces helpers that end in a compare and a `ret` and return their result in the flags, and the JIT assumed no flags survive a return. A pure flag producer that reaches a `ret` now keeps its flags live (test `ret_flags`).
- **Self-exiting windows** (`c23de6f`). Notepad quit after about 24 seconds with status 0. A category that never attached made an Objective-C method missing inside a dispatch block, and a page the kernel mapped for an IOSurface sat at a host address the guest could not see. The first is why CoreSpotlight is in the default Objective-C preload list.
- **The restart storm**, described in the WoW64 section above.
- **An endless re-exec**. The initial stack capped the environment at 60 entries and dropped Wine's trailing `WINELOADERNOEXEC=1`, so the loader re-executed itself forever.
- **A stream that never updated**. A read-only, unaligned shared mapping became a private snapshot, so `steam.exe` never saw the browser's writes to its UI stream; real shared overlays are tried first now.

- **SQLite "out of memory" in CEF.** One miscompiled block: a fused compare-and-branch whose taken side's flag record was bypassed by chaining (264e647).
- **Misaligned locked instructions.** A Windows critical section inside a packed structure made `lock cmpxchg` misaligned, and arm64's atomics fault on misalignment, so those go through an address-striped lock.
- **Self-modifying code.** V8 writes its code with plain stores, and the JIT ran stale translations until code pages were write-armed (Chapter 7).

On 2026-09-08 a last run of seven fixes, among them a V8 safepoint deadlock, `pop [rsp+displ]` computing its address before the pop, AVX decoding for Metal's shader paths and `punpckhqdq` decoded as `punpcklqdq`, which had squashed every Cocoa window to 74 pixels high, brought up Steam's 700 by 440 "Sign in to Steam" window at full size, painted, with software compositing.

Late September: to the main window

- **The fixed arena base** (dff4f4a), described above.
- **"No network adapters"** (e5cd84b). Foundation, first pulled in by a `dlopen` under Wine's `ntdll.so`, never had its `+load` methods run, so `+[NSString alloc]` returned an abstract class rather than a placeholder. Wine's mount manager answers Steam's network query through `SystemConfiguration`, a string method then recursed until `winedevice`'s stack ran out, and Steam saw no adapters and stopped at its connection dialog. `dlopen` closures now run `+load` before initializers.
- **No localization** (30ecbfa). `pcmpistri` must count a needle that runs past the end of its 16-byte block as a partial match; AArchX cleared the bit, so Steam's SSE4.2 `strscr` never found `%language%` in a file name, loaded no localization, and stopped on an assertion right after login.
- **Wine-only JIT gaps** (3eeaf45), found with the new `OCERZ_GUESTPROF` sampler, which names the instruction forms a process is interpreting: Steam's web helper interpreted `pinsrw` from memory for 15% of its samples, because every emitter that needed a plain address handed its memory forms to the interpreter in the low window.

On 2026-09-27 Windows Steam reached its **main window in about 33 s with 58 s of CPU**, where Rosetta takes 11-12 s and 17-21 s. A profile of one startup split the time: translation took 12.6 s of 54, over 2.18 million blocks; Windows code spent 14 s in translated code; and a short Wine process's startup is about 70% translation, most of it while `winemac.drv` brings in AppKit and Foundation. That is what the translation cache addresses (Chapter 7): a Wine process's code sits at the same address every run, so a block one process translated is loaded, not translated again, by the next. Windows Steam's window came up in 46.9 s from an empty store and in 35.2 s and 33.7 s from a warm one. Three and a half minutes after launch the two busiest web-helper processes used 31% and 10% of a core where, the day before, they had used 197% and 167%; once Steam settles, the whole Wine tree uses about a tenth of a core.

Stock Wine Staging 11.16 with DXMT

On 2026-10-01 Steam was run on stock Wine Staging 11.16 with DXMT, the Direct3D-to-Metal translation layer. Staging moves `gs` between the TSD and the TEB on every system call and runs Chromium's sandbox, and it found six more bugs (654a81a, 3ca606a): the TSD identity leak described above; signals aimed at one thread delivered to whichever thread reached a system call first, which broke Wine's `SIGUSR1` and `SIGQUIT`; `wineserver`'s Mach calls into another task not moving low addresses by the shadow base, which failed every GPU-process launch under the sandbox with error 43; a fixed-address

`mach_vm_map` over pages in use being moved over them instead of refused; aliases that reached into guest data; and `libxpc`'s bulk image-load callback being called only once, at registration, so XPC services of frameworks loaded later were unreachable: shaders failed to compile, text input could not reach its service, and `steam.exe` retried a view bridge a thousand times a second. All but the TSD mapping have an `OCERZ_NO_*` switch (Appendix A).

Games through Wine

Counter-Strike 2

Counter-Strike 2, launched from Windows Steam with D3DMetal from Apple's Game Porting Toolkit, was the first game run through Wine under AArchX. D3DMetal did not load at first: its bridge library `dlopen`s `@rpath/D3DMetal.framework/D3DMetal` through its own `@loader_path` `rpath`, and in cache mode AArchX passed `@-paths` to the loader as written. The bridge asserted "Failed to dlopen D3DMetal" at the first Direct3D call, Wine's abort handling spun in its unwinder, and the game sat at 100% CPU with no window. `c86567d` (2026-09-26) resolves `@rpath` and `@loader_path` against the image that called `dlopen`, as dyld does; with that and the `__mac_syscall` fix above, which cleared an error box from `wined3d`, CS2 loads D3DMetal and reaches its own window.

It then stops on a fatal "Multiple entity classes have the same designer name" error that Rosetta never reaches, which points at a translation problem rather than at D3DMetal. Finding it means running `cs2.exe` with JIT features switched off, but switching them off for the whole process tree slows Steam so much that CS2 never starts, so `OCERZ_EXE_ENV="cs2.exe:K=V,..."` (`3a09112`) sets variables only in the processes whose command line matches. The same runs print hundreds of "dropping stale oolslow arm" warnings whose source is not yet known. That is where CS2 stands.

R.E.P.O.

R.E.P.O., a Unity game, run through MacNdCheese's Wine with DXMT, is the first game to render under AArchX through Wine, as of 2026-10-03. Unity brings two things Steam does not: a JIT of its own, Mono, which writes and patches code constantly, and a renderer that pushes real GPU traffic through DXMT and Metal. Getting it to render, then to run well, took thirteen commits over 3 and 4 October:

Symptom	Cause	Commit
livelock at about nine cores; every shader failed	DXMT links its own LLVM statically, and a weak-symbol bind took the definition from Apple's LLVM in an unloaded cache image; weak binds now count only images already loaded	<code>2b344c5</code>
GPU address fault on the first frame	Metal's no-copy buffers handed the kernel a low-window guest address; recognized by the call's shape and translated for the call	<code>b347ebc</code>
nil drawable after a failed IOSurface query	the voucher-creation trap's pointers were not translated from a low stack	<code>f832152</code>
stale compute results, jobs waiting forever	aliases kept freed GPU buffers alive; refreshed now	<code>0115ead</code>
nil drawable in two launches of three	unaligned pointers in IOKit replies; aliased now	<code>28e237b</code>

Symptom	Cause	Commit
failed to start in three launches of five	the kernel's mappings landed in the low window; 4-12 GB reserved now	3cc5842
30% of the process in invalidation	Mono writes code all through startup, with about 390,000 blocks live; invalidations now use per-granule block lists	8b480ea
a crash on the profiler thread	the sampler read a cpu that had just been freed	711a94a
Mono crashes; new code interpreted after two minutes	the 1 GB code arena filled; it is now flushed and refilled	9226846
low frame rate	the low-window address guard; now a shift, compare and <code>orr</code>	16a28b1
(a test, not a symptom)	<code>OCERZ_TSO_NARROW</code> orders <code>movd</code> and <code>movq</code> ; off by default	ef59a1d
every Mono call-site patch retired four times too much	retirement touches only the blocks in the written range	27833ac
4% of the process interpreting MMX	UnityPlayer's video decoder is MMX; MMX is translated now	74e4300

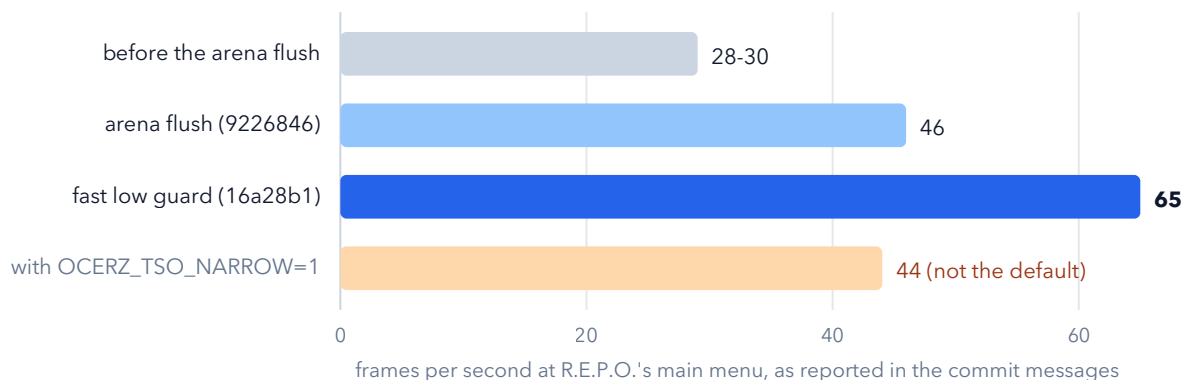


Figure 14.4. R.E.P.O.'s main-menu frame rate across the October changes. The ordered `movd` / `movq` mode is shown for scale; it is correct for a pattern no known program depends on, and is off by default. These are single runs as each commit reported them. With the build at 9169680 the menu's median varies from about 40 to 65 frames a second between runs, a singleplayer level runs at about 50, and Rosetta draws the same menu at about 178.

Two of those changes say something general about running a JIT on top of a JIT. Mono generates code for the whole of a game's run, and AArchX never reuses a retired translation's space in place, so the 1 GB code arena filled within two minutes and every block after that ran in the interpreter; worse, a translation that overflowed the arena left the translator in a state that damaged later translations, which is why Mono crashed whenever the arena filled while the game was still loading. Since 9226846, when less than an eighth of the arena, at most 8 MB, is left, the next translation miss flushes it: every thread is brought out of translated code at its next block edge, every table pointing into the arena is emptied, and translation starts again at the front (Chapter 7). And Mono patches call sites in code it has already run, over a thousand times a minute, each a write to a write-armed code page; rounding each such write up to a 64 KB

granule and clearing every cache in the process had made every patch cost four times what it should and made every thread mispredict its returns afterwards. Retirement is now limited to the blocks the write overlaps, with only the cached entries that point into them cleared (27833ac).

A MEMORY-ORDERING RESULT WORTH KNOWING

SSE loads and stores stay plain even in ordered mode, the 8- and 4-byte `movq` and `movd` forms included, and compilers do move pointers and counters through xmm registers. `tests/dynamic/tso_narrow.c` shows a reader seeing a published counter newer than the data written before it hundreds to over a thousand times in three million handoffs, which x86 never allows. `OCERZ_TSO_NARROW=1` orders those two instructions and closes the gap, at the price shown in Figure 14.4. No program is known to depend on it, so it is not the default.

What is still open

- **Startup time.** Windows Steam needs about three times Rosetta's time to its main window. Translation is the largest part, the translation cache helps on later launches, and the low window still makes translated code slower than in the other memory layouts.
- **Windows Steam with GPU compositing.** In September, without its software-rendering flags, Steam's GPU process asked DXMT for a swap chain on a window owned by another process, and `winemac.drv` refuses that ("Cross-process child window Metal swapchains are not implemented"), under Rosetta as well. Under Rosetta the GPU process then exits and Chromium falls back to software; under AArchX its main thread stayed parked and the browser waited on it. Two later fixes bear on that, signals delivered to the thread they were aimed at and sandboxed GPU-process launches no longer failing, but whether the GPU process now survives has not been recorded.
- **Counter-Strike 2** stops on the designer-name error Rosetta never reaches, and the "stale oolslow arm" warnings are unexplained.
- **Arena overflow.** Why an overflowing translation damages later ones is not understood; the flush avoids the condition rather than fixing its cause.

PART VI

Quality, speed and tooling

How correctness is held, how speed is measured, how bugs are found, and what is still missing.

CHAPTER 15

Testing

A translator that is fast and occasionally wrong is worse than useless, because its mistakes surface hours later as a crash in somebody else's code. AArchX is held to correctness by a chain of gates that all reduce to one idea: the interpreter is the reference, the JIT must agree with it byte for byte, and the interpreter itself must agree with real x86 behaviour as Rosetta or the Intel manual defines it. This chapter walks through every gate, what it can and cannot catch, where its expected outputs come from, and how a new test is added.

29

UNIT HARNESSES

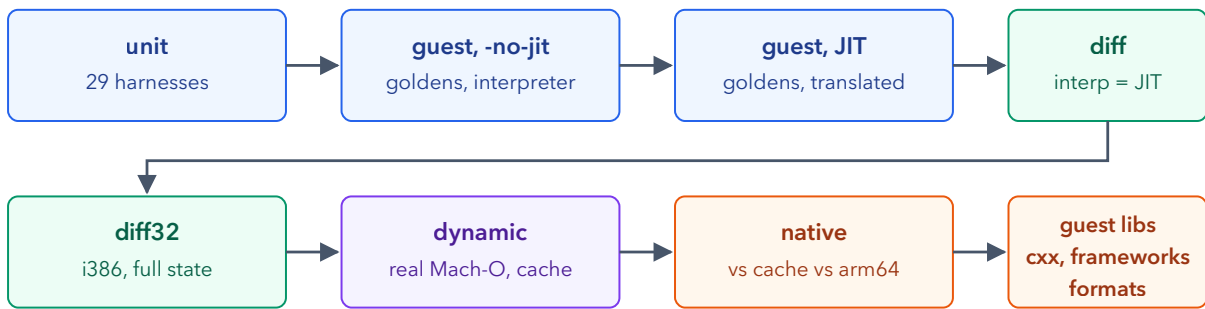
133GUEST-GATE CASES
PER ENGINE**20,033**1386 DIFFERENTIAL
CASES**91**DYNAMIC-SUITE
CASES

The shape of the problem

Every mechanism in Chapters 4 to 9 has two kinds of failure. The first is a wrong answer: a flag computed one way by the interpreter and another by the JIT, a NaN with the wrong sign, a store that becomes visible in the wrong order. The second is a wrong *environment*: a system call that returns the host's view of memory instead of the guest's, an initializer that runs before its dependency, a Mach message whose out-of-line buffer was never translated. The first kind is local and can be pinned by a few instructions. The second kind only shows up when real libraries run, and needs real dynamically linked programs to reproduce.

The gates are arranged accordingly. Unit harnesses drive one component directly. Freestanding guest programs exercise instruction semantics with nothing between them and the translator. Differential gates compare the two engines on the same input, so that a JIT bug is caught even where nobody wrote a golden for it. The dynamic suite builds real Mach-O programs against the system libraries. The native suite does the same for native mode and compares against a separately compiled arm64 build. Above all of these sit two tools that run real software: `tools/apptest.sh` and the benchmark harness, whose kernels checksum their own work.

make check builds ocerz, the unit harnesses, the guest binaries and the API databases, then:



Outside make check, run by hand

tools/apptest.sh cli: 16 system tools, stdout compared with the native run

tools/apptest.sh gui: system applications, verdict is survival for SECS seconds

tests/xbench_compare.py: 15 kernels against Rosetta, output must be byte-identical

Figure 15.1. The gates in the order **make check** runs them. Blue gates compare against stored goldens, green gates compare the two engines with each other, purple and orange gates run real dynamically linked programs. Each runs to completion and fails the build on any mismatch.

Unit harnesses

`tests/unit` holds 29 C programs. Each links the translator's own object files (everything except `main.o`) and drives one component directly, without loading a guest program at all. The `unit` target in the Makefile runs them one after another and stops at the first that exits non-zero. Their binaries are committed under `tests/unit/bin` and refreshed by separate "tests: rebuild unit binaries" commits, a convention discussed below.

What makes several of them valuable is that they are exhaustive rather than illustrative. The ABI engine (`test_abi.c`) is checked over 28,071 argument arrangements. The callback trampoline bank (`test_callback.c`) is driven through 131,974 cases. The in-place string and memory routines of `src/leaf.s` are compared with the C library over 6,736,902 inputs, and every string is placed against an inaccessible page, so a routine that reads one byte past the terminator dies inside the harness rather than in somebody's application. The memory suite runs 2,692 checks of the 4 KB slot table against 16 KB host pages.

Component	Harness	Checks (Sept. 2026)
arm64 emitter	<code>test_a64emit</code>	validated by execution
decoder corpus, assembler as oracle	<code>test_corpus</code>	511 instructions
x86-64 decode	<code>test_decode</code>	246 / 246
i386 decode and addressing	<code>test_decode32</code> , <code>test_decode32_addr</code>	102 + 26 rejects, 122
extension and SSE semantics	<code>test_ext</code> , <code>test_sse</code>	237 / 0, 246 / 0
loader, syscall layer	<code>test_loader</code> , <code>test_syscall</code>	54 / 0, 365 / 0
guest memory, shared mappings	<code>test_mem</code> , <code>test_shared_map</code>	2,692 / 0, 105 / 0

Component	Harness	Checks (Sept. 2026)
API database parser	<code>test_apidb</code>	334 / 0
synthesized images	<code>test_vdylib</code>	142,426 / 0
bridge, ABI engine, callbacks	<code>test_bridge</code> , <code>test_abi</code> , <code>test_callback</code>	1,247, 28,071, 131,974
thread attach, Objective-C, blocks	<code>test_attach</code> , <code>test_objcbridge</code> , <code>test_blocks</code>	270, 125,400, 107
in-place string routines	<code>test_leaf</code>	6,736,902 / 0
shared cache mapper	<code>test_cache</code>	15 checks

Other harnesses pin JIT behaviours that are hard to reach from a guest program: `test_jit_exit` and `test_jit_order_transition` drive the dispatcher and the plain-to-ordered memory switch, `test_jit_psc_invalidate` the invalidation of call-site caches, `test_chain_concurrency` the patching of chains while other threads run them, and `test_jit32`, `test_interp32` and `test_wow64` the 32-bit engines and the WoW64 transitions.

A TEST THAT USED TO SKIP

`test_cache` once skipped silently whenever the shared cache could not be mapped. That is how cache mode stopped working on every macOS 26 machine on 2026-09-19 without a single failing test: the cache had moved on macOS 27 and the lookup followed it. Since 2026-10-03 the harness requires the first directory that holds a cache to be the one ocerz maps, and fails rather than skips when a cache exists but cannot be mapped.

The guest suite

`tests/guest` holds 106 freestanding x86-64 sources. They are built with `clang -arch x86_64 -nostdlib -static -fno-stack-protector -O2` against a tiny runtime of their own (`crt0.s`, `libmini.c` and the syscall wrappers in `gsys.h`), so nothing stands between the test and the translator: no dyld, no libSystem, no shared cache. A guest test is the purest possible statement about instruction semantics. The binaries are static `LC_UNIXTHREAD` images that run both under ocerz and directly on the Mac under Rosetta, which is how their goldens are produced.

`tests/run_guest_tests.sh` runs every binary and compares standard output with `tests/guest/expect/<name>.out`. `make check` invokes it twice, once with `-no-jit` and once without, so the same goldens judge both engines. Inside each invocation it runs the binaries several ways, because each configuration reaches code the others cannot:

Stage	Setting	Cases	What it reaches
main pass	defaults	93	every test, plain memory model, hardware NaN rule where the processor has <code>FEAT_AFP</code>
ordered	<code>OCERZ_NO_PLAIN_MEM=1</code>	11	the LSE atomics and <code>ldapur</code> / <code>stlur</code> forms a multi-threaded program uses
translated NaN checks	<code>OCERZ_NO_AFP=1</code>	12	the software NaN checks a processor without <code>FEAT_AFP</code> needs

Stage	Setting	Cases	What it reaches
forced replay	OCERZ_FPB_FORCEREPLAY=1	12	every deferred NaN check taking its replay arm
tiny arena	OCERZ_JIT_CODE_KB=64	5	the interpreted-block path, where a taken side exit must stop the block
Wine	OCERZ_HOSTWQ=1, 512 KB arena	2	a real dynamic program from the shared-cache world; skipped when no x86-64 Wine is installed

At 9169680 that is 133 cases per engine, `mmx_jit`, added on 2026-10-04 with the MMX translation, being the newest. The full run that gated 9169680, on the MacBook Air under macOS 27 on 2026-10-04, passed 133 / 133 under each engine.

The tiny-arena stage exists because of a specific bug. On 2026-08-17 a superblock side exit taken in the middle of an interpreted block let the rest of the block run on after the branch, and it crashed roughly one Wine run in 150. A 64 KB code arena makes almost every block take the interpreted path, so the stage reaches it in every run. The NaN stages exist because the hardware NaN rule made the software path unreachable on the machines the project is developed on: on 2026-09-21 AArchX began setting FPCR.AH where the processor reports `FEAT_AFP`, and without the extra stages the translated checks every older processor depends on would have gone untested.

What the guest tests pin

Most guest tests pin one bug that real software hit, which makes the list a history of the project. A few examples, with the failure each one would show:

- `ret_flags`: helpers that end in a compare and a `ret` return their result in EFLAGS. The JIT once treated every `ret` as killing the flags, and every Wine window painted black (2026-09-05).
- `jcc_gap_fuse`: a fused compare, flag-neutral gap and branch, where the gap writes a register the compare read. The fusion once let the gap clobber the compared value, and Steam's browser process crashed on its first navigation (2026-09-06).
- `punpck_qdq`: `0F 6D` decoded as `punpcklqdq` from the initial commit until 2026-09-08, so the high quadword never moved down and every Cocoa window was constrained to a 74-pixel strip.
- `side_exit_l0`: a scalar SSE result still held in the JIT's lane-0 scratch when a hot conditional branch is taken; chaining past the side exit skipped the flush.
- `simd_pack_jit` and `mmx_jit`: every packed, multiply, shuffle, AES, `pclmulqdq` and `crc32` form translated on 2026-09-27, and every MMX form translated on 2026-10-04, on random rows, edge rows, all four rounding modes and every shift count, with outputs recorded under Rosetta.
- `tso_forms`: every load and store form under the ordered memory model, run in the ordered stage too.

The x86-64 differential gate

`tests/run_diff_test.sh` globs `tests/guest/bin/*`, 99 binaries at 9169680, and runs each under `-no-jit` and under the JIT with the same arguments and environment. The two outputs must match byte for byte, and so must the exit codes. The interpreter is the reference implementation, so any divergence is a JIT bug by construction. The run that gated 9169680 passed 99 / 99.

The gate needs no golden at all, which is its strength. A test added for one purpose exercises hundreds of other instructions along the way, and the differential checks all of them. The project's notes from early September record JIT work in which this gate caught a regression the guest suite had passed, which is why it is part of the gate list every JIT commit must clear.

WHAT A DIFFERENTIAL CANNOT SEE

A mistake the interpreter and the JIT share is invisible to a differential. The interpreter's own semantics are pinned only by goldens taken from real hardware behaviour, and by the real software in the dynamic suite and in daily use. Several bugs in this book were found exactly there: the `punpckhqdq` decode, the `pcmpistri` partial match that broke Steam's localisation, and the SSE4.2 instructions Steam needed at all.

The i386 differential gate

There is no 32-bit Mach-O for current macOS to load, so the 32-bit engines cannot be tested by running whole programs. `tests/run_diff32.sh` and its harness `tests/diff32.c` build 32-bit instruction sequences directly in guest memory instead, and run each one twice from the same architectural state, once interpreted and once through the JIT and the same dispatcher `vm.c` uses. The comparison is total: all sixteen general register slots at 64 bits wide, EFLAGS after materialisation, EIP, the mode, the segment selectors, the floating-point and SSE register file, and every byte of every memory region the sequence was allowed to touch.

The default gate is 33 hand-written cases plus 20,000 generated ones, 20,033 in all. Every sequence ends in the 32-to-64-bit far jump `Wow64` uses, so the terminator is both the stop condition and a test of the mode switch itself, and unreachable code is filled with `hlt`, which is fatal in the interpreter, so a sequence that runs off its end is loud rather than silent.

A generator that crashes is worthless, so nothing in it relies on luck. EBP is pinned to the middle of a mapped scratch region and never written; it, rather than EBX, is the pin because the 32-bit byte registers are AL, CL, DL, BL, AH, CH, DH, BH and no byte write can reach it. ESP is changed only by bounded adds and subtracts with stack traffic tracked by depth. Every memory operand is one of a fixed set of forms that provably lands inside a mapped window, and the guards that make it so are ordinary i386 instructions emitted before the operand's instruction, so they are under test too. `div` and `idiv` carry a prologue that makes a divide error impossible, and `pusha`/`popa` and `enter`/`leave` appear only in matched pairs.

Two switches keep the gate honest. `--jit-required`, which `run_diff32.sh` always passes, fails the run if the JIT translated no 32-bit blocks, so a future change that quietly declined them could not turn the gate into a second interpreter run that passes every case. `--selftest` corrupts one field of the JIT side's result for each class of compared state and requires the comparator to catch every one, so a green run cannot come from a comparator that compares nothing. The random generator is `splitmix64`, so a seed and a case index reproduce any failing sequence exactly.

The first time it was pointed at a real 32-bit JIT, on 2026-08-20, the gate found eight bugs, among them an effective-address computation that truncated at 4 GB with no case for 16-bit addressing, the BCD instructions declared to read no flags, CL-count shifts that wrote their destination when the count masked to zero, and a `mov A,B + op A,A` fusion that miscompiled 64-bit blocks too.

The dynamic suite

Freestanding binaries never reach dyld, libSystem, threads, signals delivered through the system's trampolines, fork, or the shared cache. `tests/run_dynamic_tests.sh` builds real dynamically linked x86-64 Mach-O programs with the Command Line Tools' clang, from the 109 sources in `tests/dynamic`, and runs them under ocrz in cache mode. At 9169680 it has 91 cases, most run under both engines, plus a check that every dyld API slot ocrz answers has a witness (Chapter 11). The run that gated 9169680 passed 184 / 184, counting each case under each engine separately.

The cases group by what they protect:

- **Threads, signals and processes:** `dfork_signal`, `dthread_signal`, `dsignal_wait`, `dsignal_redzone`, `dsigaltstack_query`, `dthread_suspend`, `dthread_act`, `dspawn_pipe`, `dspawn_arm64_only`, `dwq_thread_exit`, `dwq_overcommit`.
- **Memory ordering and atomics:** `drel_acq_order`, `datomic_counter`, `dbyte_atomics`, `dunaligned_atomics`, `dshmem_coherence`, `dtso_narrow`.
- **Self-modifying code and real engines:** `dsmc_io`, `dsmc_high`, and `jsc`, which runs a loop of 50,000 object allocations through the system's JavaScriptCore and checks the sum.
- **The loader:** `dupward_init`, `dupward_defer`, `dinit_order`, `ddlopen_load_phase`, `ddlopen_objc_core`, `ddlopen_caller_rpath`, `ddlism_deps`, `ddlopen_image_list`, `ddlopen_cache_alias`, `dlegacy_format`, `dapp_bundle`, `dinsert_libraries`, `dweak_unloaded`.
- **Floating point and SIMD:** `dfp_rounding`, `dnan_contexts`, `davx2_ops`, `davx_fma`, `dbmi_ops`, `dmmx_ops`, `dymm_state`.
- **Wine's address layout:** the "low" cases below.
- **The translation cache:** `dtcache` and `dtcache_low`, which run the suite's own workload twice over one store.

Low-shadow builds of guest tests

Wine's main image insists on a low address, so a Wine process runs in a third memory layout, the low shadow, where guest addresses below 12 GB live at their own host base and every memory access in translated code is checked (Chapter 8). Code paths that exist only there were long tested only by running Wine. The suite now builds selected guest tests as non-PIE mains at `0x200000000`, which activates the low shadow, and checks them against the same goldens the static gate uses: `dsimd_low`, `dmmx_low`, `dras_stress_low`, `dcallret_fault_low`, `dfault_link_low`, `dstack_test_low`, `djcc_chain_rec_low`, `drsp_ops_low`, `dfault_chain_low`, `dpromo_callout_low` and `dtso_narrow_low`. Three more, `dmac_syscall_low_stack`, `dmetal_nocopy_low` and `diosurface_low_stack`, put a thread's stack or a GPU buffer below 12 GB to pin the system-call translations that Wine depends on.

The native suite and its companions

`tests/run_native_tests.sh` is the largest gate in the tree, over fifteen thousand lines. Its central argument is in its own header: an exit code cannot prove that a bridged call worked, because a guest handed nonsense by every crossing can still exit 0. So the native cases compare *output*. Each x86-64 fixture is run under native mode with the JIT, under native mode interpreted, and under cache mode, and all three must print what a separately compiled arm64 build of the same source prints. Cache mode reaches the system

libraries through the real shared cache, so agreeing with it is agreeing with the real libSystem. `xbench_dyn` is in the suite because its kernels checksum their own work: a `memcpy` that copied the wrong bytes changes the checksum, and the checksum is all it prints.

The suite also pins refusals. Native mode stops with a named message when it cannot make a call correctly (Chapter 12), and the gate checks those messages and their exit codes, 71 for an import that cannot bind and 72 for an export with no crossing, so a refusal cannot silently become a wrong answer. Current result: 87 / 87, last counted on macOS 27.

Four companion runners extend it, all run by `make check: run_guest_library_tests.sh` (guest x86-64 libraries supplied in place of a database), `run_native_cxx_tests.sh` (the optional LLVM guest C++ runtime: containers, streams, regex, RTTI, exceptions, threads and futures, compared with arm64 fixtures and required to show positive JIT and bridge counts), `run_native_framework_tests.sh` (CFNetwork, SystemConfiguration, Security and CoreServices crossings, including a certificate check against an in-memory trust anchor) and `run_native_format_tests.sh` (the `va_list` printf veneers past both register limits).

Real software, outside the gates

`tools/apptest.sh cli` runs sixteen system command-line tools under ocerz and compares their standard output with the native run: `uname`, `sw_vers`, `echo`, `ls`, `id`, `basename`, `wc`, `sort`, `uniq`, `head`, `grep`, `file`, `xxd`, `nm`, `plutil` and `openssl`. It reads 16 of 16 on macOS 26.7.1 since the initializer-order fix of 2026-10-03, and on macOS 27. `tools/apptest.sh gui` launches system applications; a GUI application has no output to diff, so the verdict is survival: "ALIVE" means it was still running after `SECS` seconds.

The benchmark harness, `tests/xbench_compare.py`, is also a correctness check. Each of its fifteen kernels prints a checksum of its own result, and the harness fails a kernel whose output under ocerz differs from its output under Rosetta, so a kernel that is fast because it computed the wrong answer fails rather than scores. All fifteen are bit-identical (Chapter 16).

Where expected outputs come from

A golden is only as good as its source. The project uses three, in a fixed order of preference.

1. **Rosetta.** The guest binaries run directly on an Apple-silicon Mac, where Rosetta translates them, and their output becomes the golden. Rosetta is independent of AArchX, so agreeing with it is evidence about x86 semantics rather than about the interpreter. The SIMD tests store hashes produced this way.
2. **The interpreter,** for tests whose point is the JIT: the golden is what the reference engine prints, and the test fails if the translated run differs.
3. **The architectural semantics** in the Intel manual, for tests written before Rosetta could be consulted, checked by hand; `movhlpd`'s golden was taken this way and later confirmed to match Rosetta.

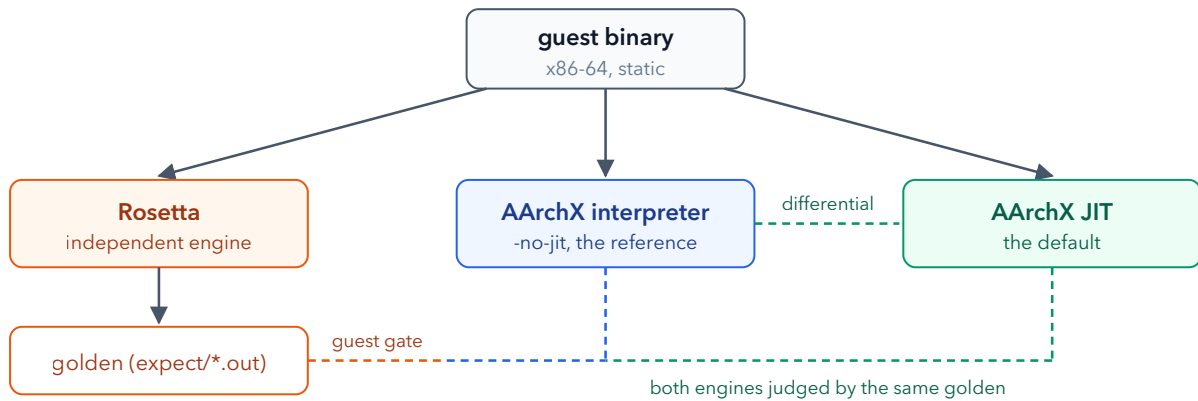


Figure 15.2. Three engines, two kinds of comparison. Goldenes come from Rosetta, an engine independent of AArchX, and judge both of AArchX's engines; the differential compares the two engines directly and needs no golden at all.

Contribution rules state the principle bluntly: take expectations from a real run under Rosetta or natively, not from memory. The loader tests follow it the same way. The test for the initializer order of upward dependencies (`dupward_defer` , Chapter 11) was written by building a few libraries of the project's own in the shape that crashed `sw_vers` , logging the order in which their initializers ran under Rosetta, and making that order the expected output.

THE 2026-10-03 REWRITE

Four guest tests and three passages of `jit.c` 's explanation block had described their bugs through code in macOS's own frameworks. On 2026-10-03 (`33bb4ae`) they were rewritten to state each rule in x86 terms: `side_exit_l0` and `xf_epilogue` became new routines built from the rule they pin, `punpck_qdq` gained the REX-register forms, and `jcc_gap_fuse` 's last loop became three scans whose flag-neutral gap writes the register the compare read. The new goldenes came from Rosetta. Each rewritten test was then run against a build of `ocerz` from just before the fix it pins, and fails there exactly as its predecessor did.

Conventions

Binaries are committed. The test binaries under `tests/guest/bin` and `tests/unit/bin` are tracked in git, so a change to the translator is measured against exactly the binaries the previous run used, and a compiler update cannot silently change what the tests exercise. Rebuilt unit binaries are committed separately, with the subject "tests: rebuild unit binaries", which is why that subject appears after nearly every change in the history.

Benchmarks live apart. `tests/guest/benchbin` holds the benchmark binaries, deliberately outside `bin` , because the differential gate globs that directory and a benchmark there would silently become an extra case with a runtime of minutes.

Tests run without the translation cache. Every runner exports `OCERZ_TCACHE=off` unless told otherwise, so no test depends on what an earlier one stored and none writes into `~/Library/Caches` . The two cases that test the cache turn it on against a directory of their own.

Do not load the machine during a run. The dynamic suite has timing-sensitive cases (thread suspension, timed waits, the unstick monitor), and the contribution guide asks that no other `ocerz` work be built or run beside `make check` .

Current results

Gate	Run that gated 9169680	Cases at 9169680
unit harnesses	0 failures	29 programs
guest suite, interpreted	133 / 133	133
guest suite, translated	133 / 133	133
x86-64 differential	99 / 99	99
i386 differential	20,033 / 20,033	20,033
dynamic suite	184 / 184	91 cases, most under both engines
native suite	87 / 87	87
CLI tools against native	16 / 16	16
xbench output against Rosetta	15 / 15	15

The "last reported run" column is the full `make check` on the MacBook Air under macOS 27 on 2026-10-03, on the tree that merged both machines' work. The case counts are taken from the runners at 9169680 ; the differences are the MMX tests added the next day.

Adding a test

Every behavioural change carries the test that would have failed before it. Where the test goes depends on what it needs:

- **Pure instruction semantics** go in `tests/guest` : add the source, its name to `TESTS` in `tests/guest/Makefile` and to `NAMES` in `tests/run_guest_tests.sh`, and a golden from a run under Rosetta in `tests/guest/expect` . If it exercises a path only Wine's layout reaches, also add a `run_low_golden_case` line to the dynamic suite.
- **Anything that needs real libraries** (threads, signals, dyld, Mach) goes in `tests/dynamic` with a case in `tests/run_dynamic_tests.sh` . A test that should print `OK` under both Rosetta and ocerz is the usual form; where the point is an ordering or a value, the program computes it and compares with the Rosetta result it embeds.
- **Component behaviour** that no guest can reach goes in a `tests/unit` harness.
- **Native-mode crossings** go in `tests/run_native_tests.sh` as an output comparison against an arm64 build.

A good test also proves it can fail. The dynamic cases for switchable features name the switch that restores the old behaviour, and record that the case fails under it: `dupward_init` fails with `OCERZ_NO_UPWARD_INIT=1`, `ddlopen_caller_rpath` with `OCERZ_NO_DLOPEN_CALLER_RPATH=1`, `dmac_syscall_low_stack` with `OCERZ_NO_MACSYS_XLATE=1`. A test that passes whether or not the fix is present pins nothing.

CHAPTER 16

Performance

AArchX is measured against Rosetta, on the same machine, running the same x86-64 binary, with the output compared byte for byte. On the project's fifteen-kernel benchmark its translated code is now faster than Rosetta's on most kernels, and its remaining losses are a handful of percent with known causes. Real applications tell a different story: what costs time there is translating code for the first time, Wine's memory layout, and a long tail of system-level work. This chapter covers the method, the results, the techniques that produced them, and the gaps.

0.72x

 MEMCPY, LINKED, VS
ROSETTA

13 / 15

 KERNELS UNDER
1.00X, LINKED BUILD

17 ns

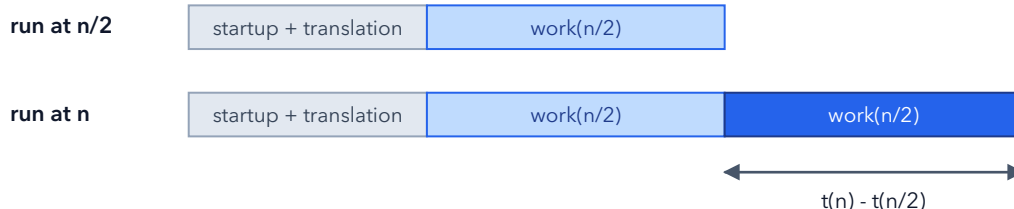
 A NATIVE-MODE
CROSSING

45-65 fps

 R.E.P.O. MENU UNDER
WINE (ROSETTA ~178)

How it is measured

`tests/xbench_compare.py` runs the fifteen kernels of `xbench`, a single x86-64 program compiled from one source in two builds: a freestanding static binary that makes no library calls at all, and `xbench_dyn`, the same source linked against `libSystem`. For each kernel the harness first calibrates a size n at which Rosetta takes about a third of a second. It then times both engines at n and at $n/2$ and reports the difference of the two.



The difference keeps only steady-state work: process startup, mapping, dyld and the first translation of every block appear in both runs and cancel. Each kernel is run five times (REPS=5) and the median is reported, separately for AArchX and for Rosetta.

Figure 16.1. The paired-delta method. Subtracting the half-size run removes everything that does not scale with the work, so the ratio compares the engines' translated code rather than their startup.

Both engines' output is compared byte for byte. Each kernel prints a checksum of its own result, so a kernel that is fast because it computed the wrong answer fails rather than scores, and the harness doubles as a correctness check (Chapter 15). The commands:

```
python3 tests/xbench_compare.py # static build, plain
memory
OCERZ_NO_PLAIN_MEM=1 python3 tests/xbench_compare.py # ordered memory
XB=tests/guest/benchbin/xbench_dyn python3 tests/xbench_compare.py # linked against libSystem
XB=tests/guest/benchbin/xbench_dyn OCERZ_MODE=native python3 tests/xbench_compare.py
```

Two rules govern how the numbers are read. A ratio within a couple of percent of 1.00x is a tie, which lands on either side from run to run. And the machine must be idle: the project's notes record benchmark sessions spoiled by a stuck background process that held a core for two days, and a loaded machine moves every ratio by several percent.

The kernels

Kernel	What it does	What it stresses
<code>icall</code>	indirect calls through a table	indirect branch lookup, return-address stack
<code>jtab</code>	a switch compiled to a jump table	indirect jumps, site caches
<code>depchain</code>	a dependent chain of arithmetic	register pinning, flag elision
<code>brmiss</code>	unpredictable branches	conditional branch layout, flag fusion
<code>memcpy</code>	copies of random small sizes	the system's copy routine, alignment
<code>str</code>	<code>strlen</code> and <code>strcmp</code> on short strings	call overhead into tiny routines
<code>hash</code>	multiply-shift hashing	multiply latency (a floor on both sides)
<code>idiv</code>	integer division	division and its flag rules
<code>fpsse</code>	scalar SSE floating point	scalar FP, NaN exactness
<code>fpvec</code>	packed single-precision arithmetic	packed FP, NaN checks
<code>chase</code>	pointer chasing through a shuffled list	dependent loads (a cache-latency floor)
<code>qsort</code>	sorting with a comparator	calls, callbacks, branches
<code>leafcall</code>	a tight loop of small function calls	call and return bookkeeping
<code>mixed</code>	struct updates, branches and small loops	everything at once
<code>vm</code>	a bytecode interpreter loop	indirect dispatch, loads and stores

Results

The current table was taken on an Apple M5 under macOS 27 on 2026-09-21, five repetitions per kernel. The **static** columns measure a binary in which every instruction is the program's own translated code. The **dynamic** columns are the same source linked against libSystem, which is what a real program looks like; its `memcpy` and `str` kernels spend their time in the system's own routines. **Ordered** is `OCERZ_NO_PLAIN_MEM=1`, the stricter memory model every program with a second thread runs under. **Native** binds the dynamic build against the Mac's own arm64 frameworks (Chapter 12).

Kernel	static	static, ordered	dynamic	dynamic, ordered	native
<code>icall</code>	0.92x	0.98x	0.93x	0.82x	0.84x

Kernel	static	static, ordered	dynamic	dynamic, ordered	native
jtab	0.89x	0.86x	0.98x	0.92x	0.97x
depchain	0.90x	0.90x	0.90x	0.90x	0.90x
brmiss	0.97x	0.99x	0.97x	0.98x	0.96x
memcpy	1.04x	1.22x	0.72x	0.73x	0.73x
str	1.05x	1.16x	0.77x	0.96x	0.82x
hash	1.00x	1.00x	0.99x	0.99x	0.99x
idiv	0.99x	0.99x	0.99x	0.99x	0.99x
fpsse	0.86x	0.88x	0.86x	0.86x	0.85x
fpvec	0.82x	0.82x	0.80x	0.83x	1.05x
chase	1.00x	1.23x	1.00x	1.22x	1.00x
qsort	0.98x	1.04x	0.97x	1.04x	0.97x
leafcall	1.03x	1.02x	1.03x	1.03x	1.03x
mixed	0.88x	0.88x	0.83x	0.80x	0.90x
vm	0.85x	0.83x	0.84x	0.76x	0.73x

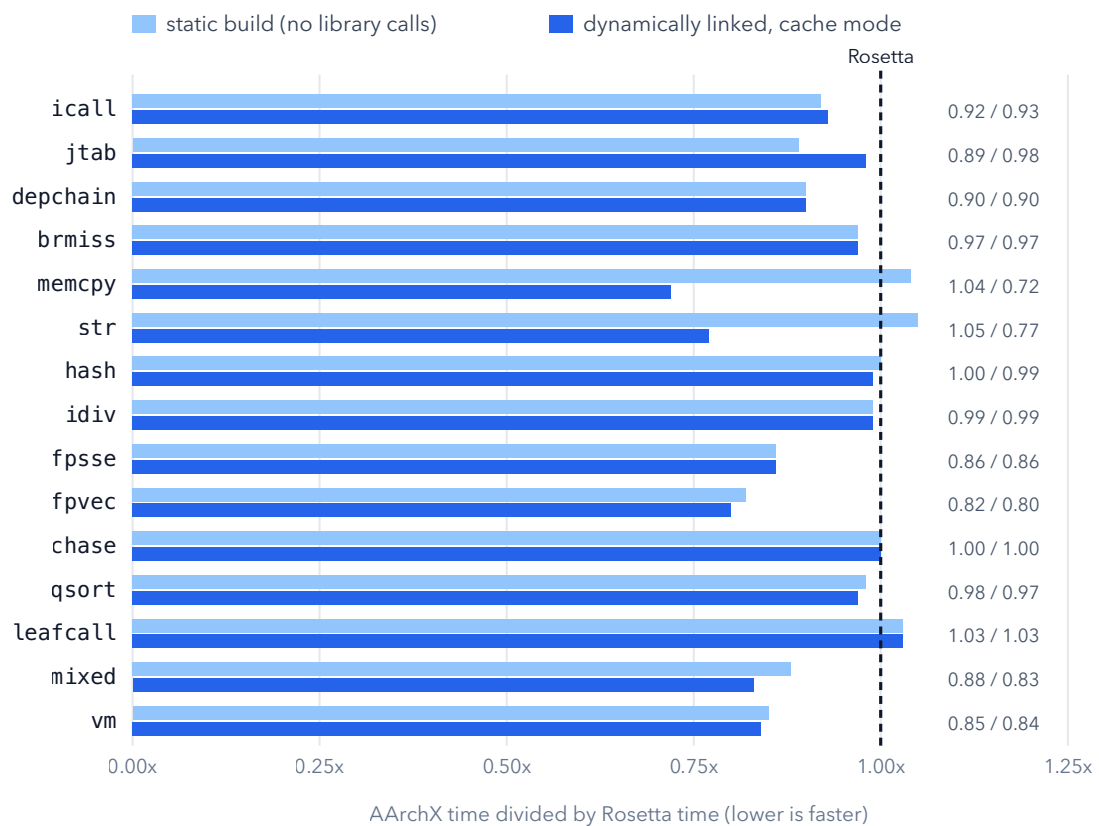


Figure 16.2. The plain-memory columns of the table: static build (light) and dynamically linked build in cache mode (dark), each as AArchX time over Rosetta time. Bars left of the dashed line are faster than Rosetta.

Rounded to two decimals, the dynamically linked build has thirteen kernels under 1.00x, `chase` at parity and `leafcall` 3% behind; the commit that produced the table counted fourteen as faster than Rosetta, so `chase` sits just below parity before rounding. The static build has ten kernels under 1.00x, `hash` and `chase` at parity and three behind by at most 5%. The history is worth knowing: the first commit to claim rough parity with Rosetta was the plain-memory tier of 2026-07-23 (1527008); on 2026-08-17 the static build had eight wins and three ties; on 2026-09-04, on an Apple M2 Max, thirteen wins and two ties.

What makes it fast

The techniques are described in depth in Chapters 6 and 7. Seen from the benchmark table, each one moves particular kernels.

- **The guest's registers stay in registers.** All sixteen general registers and all sixteen vector registers live in fixed arm64 registers for a whole block, with one layout shared by every block, so control passes from one block's body straight into another's without touching memory. This is most of `depchain`'s 0.90x and much of `mixed`.
- **Flags are computed where they are read.** x86 writes six flags on almost every arithmetic instruction and reads them rarely. AArchX records what produced them and materialises only what a later instruction needs, and a compare followed by a branch becomes one arm64 flag-setting operation and a conditional branch.
- **Blocks are chained and grown.** Direct branches replace dispatch; a host return stack keeps the processor's return predictor aligned with the guest's calls; a block continues past a conditional branch with the rare side out of line, and is retranslated the other way round when the profile says so. `icall`, `jtab` and `vm` are the beneficiaries.
- **The processor produces x86's NaNs.** Where the processor reports `FEAT_AFP`, setting `FPCR.AH` makes add, subtract, multiply, divide and square root follow SSE's NaN rule, and the checks that otherwise guard every floating-point result disappear. In `fpvec` the end-of-batch check had been three vector operations on top of the eight doing the work, with the vector pipelines the limit: the kernel went from 1.06x to 0.82x on 2026-09-21, and the x86-64-v3 build's from 0.96x to 0.58x.
- **Hot library routines do not go through the library.** `strlen`, `strnlen`, `strcmp`, `strncmp`, `memcmp`, `bcmp`, `strchr`, `memchr`, `memcpy`, `memmove` and `memset` have hand-written arm64 versions in `src/leaf.s` that read their arguments from the registers that already hold `rdi`, `rsi` and `rdx` and leave the result in the one that holds `rax`, so translated code reaches one with a single direct branch and spills nothing. On 2026-09-21 that took the linked build's `str` from 1.08x to 0.77x and `memcpy` from 0.98x to 0.72x.
- **A program pays for ordering only when it can observe it.** A process that never starts a thread, forks or maps shared writable memory runs with plain loads and stores (Chapter 7). The benchmark kernels never do, which is why the plain columns are the headline; the ordered columns show the cost when the switch has happened.
- **Translations are kept.** Translated blocks are stored on disk and loaded by every later process that runs the same code. This does nothing for the steady-state table and a great deal for startup, discussed below.

What is still slower

`hash` and `chase` are ties no translation can move. `hash` is a chain of multiply, shift and or per step, and both engines are bound by the multiplier's latency; `chase` is a chain of dependent loads, and both wait on the cache. The losses that remain have known causes:

- **`leafcall`, about 3% behind everywhere.** Both calls in the loop are already inlined into one translated block. What is left is the frame bookkeeping the x86 calling convention asks for.
- **`chase` under ordered memory, 1.22x.** A dependent load through a shifted index is one instruction on x86. Under the ordered model it becomes an acquiring load, and arm64's acquiring loads have no register-index form, so the address is computed first. That is an instruction-set floor, not a missing optimisation.
- **The static `memcpy` and `str` under ordered memory, 1.22x and 1.16x.** These are the program's own byte loops at arbitrary alignments, and an ordered access that straddles a 16-byte boundary pays a barrier.
- **`fpvec` in native mode, 1.05x.** Native mode keeps the software NaN checks, because the host's own code runs on guest threads there, and clearing and restoring FPCR.AH around every crossing would cost about as much as the crossing itself.

SIMD: AVX2, FMA and MMX

Until 2026-09-13 every VEX-encoded instruction left translated code for the interpreter, so AVX2 and FMA loops ran up to a hundred times slower than under Rosetta. Then the JIT learned the instructions these kernels spend their time in. Timings are the best of five on an Apple M5 under macOS 26.6.2, taken on 2026-09-14 and, for nbody and mandelbrot, 2026-09-15; "before" is the build at `31bff03`.

Kernel	Before	Now	Rosetta
<code>memclr 32 MB, AVX2 vmovdqu</code>	24.12 ms	0.57 ms	0.56 ms
<code>indexbyte 32 MB, AVX2</code>	71.98 ms	0.85 ms	1.48 ms
<code>memeq 32 MB, AVX2</code>	64.79 ms	0.86 ms	1.72 ms
<code>int32 loop 4M, clang AVX2</code>	135.35 ms	0.44 ms	1.30 ms
<code>int32 loop 4M, clang SSE4.1</code>	29.48 ms	0.56 ms	0.78 ms
<code>saxpy 4M, clang AVX2+FMA</code>	37.62 ms	0.33 ms	0.48 ms
<code>nbody 200k steps, scalar SSE2</code>	72.90 ms	6.13 ms	5.92 ms
<code>nbody 200k steps, scalar AVX2</code>	1125.37 ms	6.97 ms	10.57 ms
<code>nbody 200k steps, scalar AVX2+FMA</code>	904.45 ms	6.31 ms	9.47 ms
<code>mandelbrot 400x400, scalar SSE2</code>	19.97 ms	12.07 ms	11.26 ms
<code>mandelbrot 400x400, scalar AVX2</code>	828.24 ms	10.95 ms	11.44 ms

The first three are hand-written loops shaped like Go's runtime routines, which matter because Go turns its AVX2 paths on under Rosetta without checking CPUID; the rest are C loops that clang vectorises, except nbody and mandelbrot, which stay scalar. Scalar floating-point loops now take 0.9 to 1.2 times Rosetta's

time whether built for SSE2 or for x86-64-v3, because scalar results stay in host lane registers across a loop instead of being merged into the guest register after every operation, and 256-bit loops keep the upper halves of their `ymm` registers in host registers too. The whole fifteen-kernel suite built for x86-64-v3 used to lose ten kernels, three of them by 4x to 13x; on 2026-09-15 it won twelve and lost none by more than 7%.

MMX followed on 2026-10-04 (`74e4300`). UnityPlayer's video decoder is written in MMX, and in R.E.PO.'s menu it cost about 4% of the process in the interpreter. A loop of the decoder's moves, adds, multiplies and packs now runs in 0.142 s instead of 1.395 s. Rosetta takes 0.053 s: AArchX keeps the MMX registers in the CPU structure, so every MMX instruction loads and stores its operands, which is the next step if it matters.

Exactness and its price

Bit-exact x86 floating point is not free on arm64. The two architectures disagree about NaNs: an invalid operation gives a negative default NaN on one and a positive one on the other, and when both operands are NaNs x86 returns the first where arm64 lets a signalling one win. Programs do notice. `mixed` was a 1.20x loss for a long time, and the whole gap was the cost of checking every packed result before anything could use it.

The fix came in stages. First the check was deferred to the compares that read a value. Then it became one batch per run of floating-point work (2026-09-15, `f5919cc`): moves, unpacks, stores and `ucomisd` with its branch stay inside the batch, a stored value is checked just before the store, a branch out of the loop carries its check in the exit stub, and only the registers the loop still reads are checked at the end. nbod's SSE2 pair loop went from 107 to 86 host instructions per iteration, 42 of which had been NaN bookkeeping. A store that might alias an earlier load of the batch used to end it, because the replay re-executes the loads; the batch now keeps the memory it is about to overwrite in a spare vector register, so the replay can restore it. Finally the hardware rule above removed the checks altogether on processors with `FEAT_AFP`, except for fused multiply-add, whose negated forms negate a NaN operand that x86 returns as it came.

Memory ordering

A program that has created a second thread, forked or mapped shared memory retires plain mode for good and pays for x86-TSO ordering on every scalar load and store; Wine is always in that mode. Scalar accesses use acquire and release forms, because flags, locks and atomics are scalar, and a release store orders every earlier vector store. SSE loads and stores are left plain, which is also what FEX ships, because ordering them cost 3.3x on `memcpy` and 3.0x on `fpvec`. That one decision, on 2026-09-05 (`800bb00`), took ordered `memcpy` from 3.55x to 1.35x of Rosetta and `fpvec` from 3.67x to 0.99x on an M2 Max.

The narrow SSE moves were left plain too, and on 2026-10-04 a test (`dtso_narrow`) showed that a pointer published through `movq` can be seen out of order hundreds to over a thousand times in three million handoffs, where x86 never shows it. `OCERZ_TSO_NARROW=1` orders `movd` and `movq` without a barrier, but it is not the default: it took R.E.PO.'s menu from 65 frames a second to 44, and no program is known to depend on it.

Native-mode crossings

In native mode a call from x86 code into a system library crosses into arm64 code (Chapter 12). Measured as a guest loop making ten million calls and timing itself, on the M5:

Guest loop, ns per iteration	through the trap	default	arm64 build
<code>getpid</code>	40.5	17.0	0.7
<code>strlen</code> of a 12-byte string	42.0	2.0	0.9
<code>memcpy</code> of 48 bytes	45.6	3.0	2.7
<code>-[NSString length]</code>	51.3	28.3	3.6
a <code>qsort</code> comparator, per call back into guest code	426	430	3.1

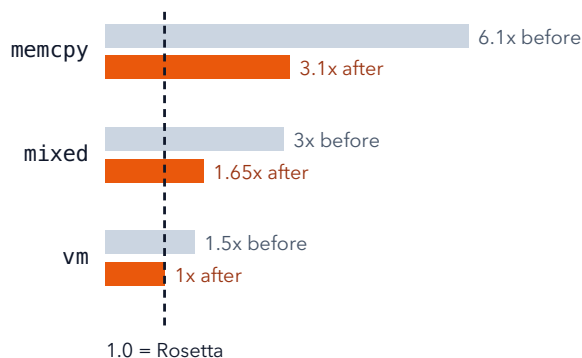
"Through the trap" is the original path: the stub traps, the dispatcher finds the bridge, the call is made and the run loop resumes. "Default" calls the bridge from inside the translated block and returns with a real `ret`, keeping the return predictor in step. `strlen` and `memcpy` are low because they do not cross at all; with `OCERZ_NO_LEAF_INPLACE=1` they read 17.3 and 20.8 ns. The callback direction is the expensive one, about 430 ns, and most of it is three system calls that save and clear the signal mask and alternate-stack state for the callback's fault-recovery point. Against cache mode the cost shows only in code that crosses in a tight loop: the linked build's `str` reads 1.03x of cache mode, while kernels that make no calls are at parity.

Wine's layout

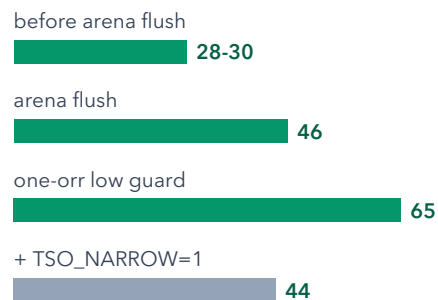
Wine's main image insists on a low address, so a Wine process runs in the low shadow (Chapter 8): guest addresses below 12 GB live at their own host base, and every memory access in translated code needs a check to find out which base applies. Until 2026-10-04 that check built three 64-bit constants and tested three ranges, about twenty instructions per load, and the kernels built into that layout ran up to six times slower than under Rosetta. The low shadow also keeps the fast memory forms, base hoisting, paired moves and the batch undo log off, because they need a host address formed without any check.

16a28b1 made the common case short. When the low window's host base has no bits below 16 GB and is encodable as an arm64 logical immediate, an address below 12 GB becomes a host address with a shift, a compare and one `orr`, and an address between 12 GB and the top strip is recognised as identity with two shifts and an `add`; only the top strip, which holds the commpage, still takes the general form. A debug build that ran both forms at every access and compared the results never saw them differ.

xbench in Wine's layout, x Rosetta



R.E.P.O. main menu, frames per second



The last bar is a diagnostic setting, not the default.

Figure 16.3. Left: three kernels built into Wine's address layout, before and after the short low-window check of 2026-10-04. Right: R.E.P.O.'s main menu under Wine, after the arena flush and then the short check; the grey bar is `OCERZ_TSO_NARROW=1`, which is correct in a case no program is known to need and costs a third of the frame rate.

Before the arena flush of the same day (9226846) existed, the long checks filled the 1 GB code arena while R.E.P.O. was still loading, and the game crashed at its first scene change in every run. The arena flush matters for speed in its own right: Unity's Mono JIT keeps generating code, and once the arena filled every new block ran interpreted. Flushing it and starting again at its front, about once a minute, took the menu from 28 to 30 frames a second to 46.

Startup: where real applications lose

The benchmark table measures steady state. Real applications are dominated by something the table deliberately subtracts: translating code for the first time. Windows Steam under Wine starts a dozen processes that each run the same Wine and system code, and on 2026-09-27 the sampling profiler (Chapter 17) split one startup as follows: of 54 s sampled, translation was 12.6 s across 2.18 million blocks with almost no churn, Windows code 14 s in translated code, and 3.3 s in `libswiftCore` walking every protocol conformance record because the shared cache's precomputed Swift tables are not offered to it.

The response came in two kinds. The first made translation itself cheaper:

- cross-block flag liveness memoises its successor decodes, which had been a fifth of translation time: 5.78 to 4.85 μ s per block over a Steam startup (64f0602);
- a logical-immediate encoding is refused by counting bit transitions before searching, where the search had been a tenth of translation time (bc694dc);
- a block retranslated with its branch flipped leaves the live array by a recorded index instead of a search and a multi-megabyte `memmove`, which had been about 330 ms of every busy process's startup (8380676);
- retiring invalidated blocks walks only their recorded predecessors instead of every live block's edges, which had been 18% of a CEF process's samples (a604f08).

The second avoided translating at all. Since 2026-09-27 (20465b4, on by default from 71b3d28) translated blocks are stored on disk and shared between processes. A record names the guest byte ranges the translation read and their hash, which a load checks first; code is emitted so it can move, with every pro-

cess-specific value a fixed instruction sequence or literal with a relocation record; and only code that sits at the same address every run is kept, which is all of a Wine process and the shared cache otherwise. Records are LZ4-compressed and written by a background thread.

Measurement	Value
Windows Steam to its main window, empty store	46.9 s
the same, warm store (two runs)	35.2 s, 33.7 s
the same under Rosetta	11-12 s
<code>hostname.exe</code> cold, before / after the store's overheads were cut	0.78 s / 0.61 s
<code>hostname.exe</code> with nothing kept	0.58 s
store size after a Steam session	2.1 GB

Other startup costs that were found and removed along the way show how much of the gap is ordinary system work done slowly. The shared cache's selector table was being indexed by hand, thirteen seconds of startup for every process that canonicalised a selector, until it was read from the cache header (2026-09-21). Electron Framework's 2,560 weak-coalescing binds each walked all 3,600 cache images at 1.8 ms apiece, 4.6 s in every Discord process, until the walk was limited to the 300 images that define weak symbols and backed by a bloom filter (2026-09-23). A `dlsym` benchmark of 4,500 framework look-ups took 8.17 s, against Rosetta's 0.21 s, until symbol lookup moved onto a per-process index; it now takes 0.12 s (2026-09-27). With those and the JIT forms of the same week, the two busiest Steam web helpers three and a half minutes after launch went from 197% and 167% of a core to 31% and 10%, and the whole settled Wine process tree uses about a tenth of a core.

Games

Brawlhalla, an Adobe AIR game launched from Steam, was the first game measured against Rosetta, on 2026-09-22. Its first frame took 5.3 s against 0.54 s, and its main thread saturated a core where Rosetta used a quarter to half of one. Three changes that evening are instructive because the profile pointed at each one:

- A return whose host return-stack entry had been lost left translated code through the epilogue, and every trip out wiped all entries still outstanding, so the whole call chain then missed on its way up: 3.2 billion misses in 88 s. A miss now stays in the body through the inline hash probe an indirect call uses. Throughput went from 2.6-2.9 to 3.85 billion guest instructions a second.
- A chain is a `b` instruction patched into the exiting block, and it reaches 128 MB. Script code generated late could not reach engine helpers translated early, and the patch was dropped silently, so a quarter of the thread's time was in prologues and epilogues. A 256 KB veneer pool every 64 MB of arena now catches every chain that does not reach; the thread fell from a saturated core to 71% of one while executing 60% more blocks a second.
- A rip-relative or absolute memory operand has a constant address, so its commpage guard and alignment test are answered at translation time: four host words instead of fourteen or eighteen, and 13% more blocks a second.

R.E.P.O., a Unity game, is the current workload under Wine with DXMT. Its Mono JIT patches call sites in code it has already run more than a thousand times a minute, each a write to a protected code page. Invalidation used to round each write out to a 64 KB granule and wipe every thread's return stack and every site cache; since 2026-10-04 only the blocks overlapping the written range are retired, found through per-granule lists, and only the cached entries pointing into them are cleared. A benchmark that rewrites one small function at a time among 300,000 translated ones went from 37.9 to 24.7 μ s per rewrite (Rosetta: 14.7).

Native mode on a real application

On 2026-09-21 the x86-64 macOS Steam client came up in native mode on macOS 27. From the web helper starting to the main window took 23 s in native mode and 34 s in cache mode on the same machine, 44 s from launch; the arm64 Steam takes 2 to 4 s. In native mode the system frameworks run as arm64 code and only Steam's own code is translated, which is the likely reason native mode was the faster of the two there.

Reading these numbers honestly

The kernels are microbenchmarks. Each isolates one thing, which is what makes it useful for finding a cost and what makes it a poor predictor of how long an application takes to start or how smoothly it runs. The real-application figures are single measurements on one machine, recorded in commit messages on the day; they show direction and size, not a reproducible score. And everything here is relative to Rosetta on the same hardware: an M5 that runs both engines faster moves no ratio.

CHAPTER 17

The debugging toolkit

A translator's bugs are rarely where they surface. A miscompiled block in a browser engine shows up minutes later as "out of memory" from SQLite; a wrong decode shows up as windows 74 pixels high; an initializer run in the wrong order shows up as a stack overflow in `malloc`. AArchX is built for that. It has a fixed method for narrowing any failure to one cause, and some four hundred switches that each turn one optimisation off, print one kind of event, or select one range. This chapter is the method, the instruments, and four failures traced with them from symptom to fix.

410OCERZ_VARIABLES
READ AT HEAD**124**OCERZ_NO_*
SWITCHES**87**

*LOG DIAGNOSTICS

~1 hTO FIND ONE BAD
BLOCK IN LIBCEF

The method

The contribution guide states the workflow the tree is built for, and it is worth following in order, because each step either names the cause or halves the space it can be in.

1. **Run it under `-no-jit`**. The interpreter is the reference implementation. If the program works interpreted and fails translated, the bug is in the JIT. If it fails both ways, the cause is in the decoder, the interpreter's semantics, the loader, the system-call layer, or the libraries' environment.
2. **Get a control**. Run the same binary under Rosetta, or natively if it has an arm64 slice. A failure that also happens there is not AArchX's. A value that differs is the first concrete fact.
3. **Shrink it**. Reproduce the failure in the smallest program that shows it. Several of the fixes in this book were found with probes that run in five seconds instead of Steam launches that take six minutes.
4. **Narrow with switches**. For a JIT bug, bisect addresses first and optimisations second. For everything else, turn on the log family for the subsystem and read what it says before forming a theory.

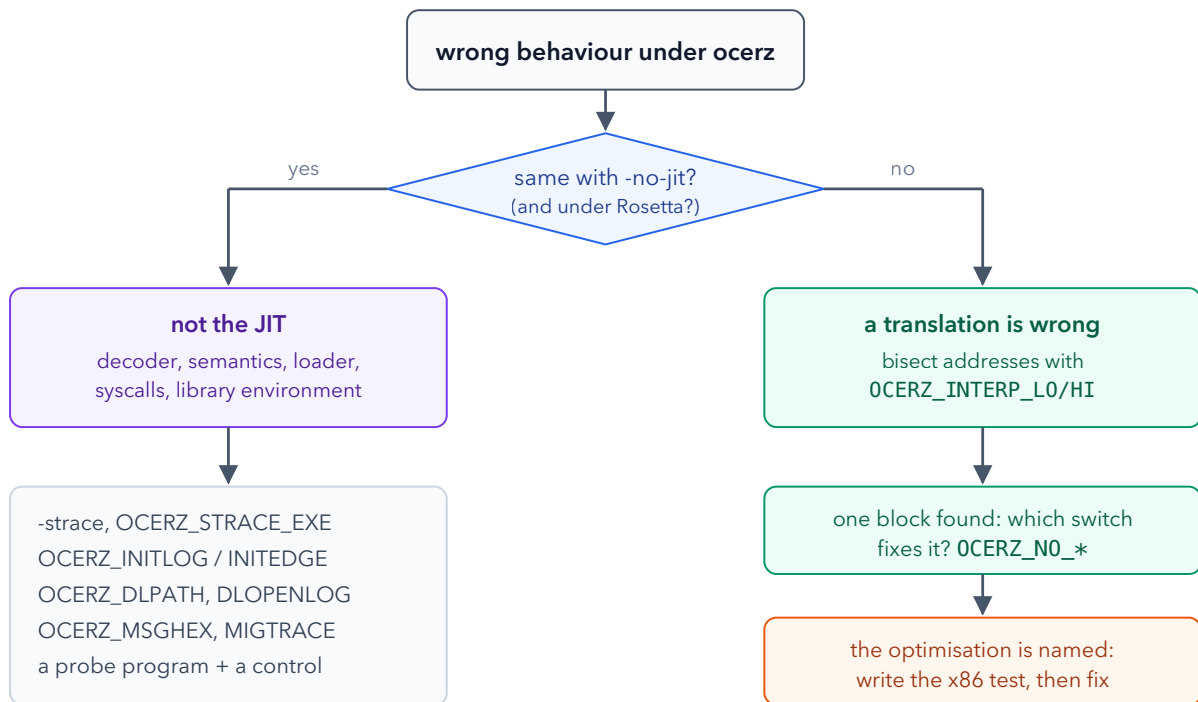


Figure 17.1. The narrowing procedure. The first branch decides which half of the system to look at; the right half ends in a named optimisation and a test written from the instruction semantics, the left half in the subsystem's own log.

Bisecting a JIT bug

Addresses first

When a program works under `-no-jit` and fails translated, some block is wrong, and the first job is to find which one. `OCERZ_INTERP_LO` and `OCERZ_INTERP_HI` name a guest address range whose blocks the JIT declines, so they run in the interpreter; `OCERZ_INTERP_LO2 / HI2` name a second range. Start with the range of the image under suspicion, typically one library's text, and confirm that interpreting it fixes the program. Then halve the range and keep the half that still fixes it. A few dozen runs take a range of tens of megabytes down to a single block. `OCERZ_INTERP_RIP` then keeps named block starts interpreted, which confirms the block and lets every other block run translated at full speed.



Figure 17.2. An address bisection. The range whose blocks are interpreted shrinks by half each run, keeping whichever half still fixes the program, until a single translated block is left to blame.

The same idea applies to features that are not "translate or not". Several fixes added their own range switch so the bisection could work on that one decision: the bug that painted every Wine window black was found with `OCERZ_RETFL_L0 / HI`, which kept flags live across returns only inside the range, until one return remained.

Optimisations second

With the block known, the next question is which optimisation miscompiles it. The JIT reads 78 `OCERZ_NO_*` switches at `9169680`, each turning off exactly one transformation: chaining, side-exit fusion, flag forwarding, branch flips, deferred NaN checks, `mov` fusion, lane caches, the in-place string routines, and so on. Running the failing program once per switch, or bisecting the set, names the one that fixes it. A switch that fixes the program names the bug's location; it is not a workaround to ship.

For Wine and Steam, where one failing process sits among a dozen healthy ones and turning a feature off for all of them changes timing enough to hide the bug, the switches can be scoped to one process. `OCERZ_EXE_ENV="cs2.exe:OCERZ_NO_CHAIN=1"` sets variables only in processes whose command line contains `cs2.exe`, and the setting is inherited through every exec in Wine's chain. `OCERZ_NOJIT_EXE` and `OCERZ_STRACE_EXE` do the same for the interpreter and the system-call trace.

Looking at the translation

`OCERZ_JITDIS=<path>` writes every translated block to `<path>.<pid>` as the arm64 words ocerz emitted, labelled by the guest instruction each group came from; `OCERZ_JITDIS_L0 / HI` limit it to a range and `OCERZ_JITDIS_BRIEF=1` leaves out the hex words. The dump has been fully buffered since 2026-09-22, because the line-buffered version made a write per line, starved the main thread on the JIT lock, and could not be profiled under. `OCERZ_CHAINCHECK`, `OCERZ_INVMAP_CHECK` and `OCERZ_TCACHE=verify` are validators: they check chaining, the code-region map and stored translations against their own invariants as the program runs.

BLACK-BOX BY RULE

When the block a bisection finds is inside one of Apple's libraries, the investigation stops at the block. The bisection has already named the faulty translation by address and by the optimisation whose switch fixes it, and that is enough to write a test from the x86 semantics alone (Chapter 15) and fix the translator. Reading the library's instructions is neither needed nor done.

Reading a crash report

When a guest thread faults and no guest handler takes the signal, ocerz prints a crash report and exits 139. Each line answers one question.

```

ocerz: guest crash[<pid>] cpu#0x0 SIGBUS host_addr=... host_pc=... host_insn=... host_lr=...
slide=... ocerz_base=...
  host-x: ...                               the arm64 registers at the fault
host_bt= guest_rip=... guest_addr=... icount=... cur_rip=... interp_once=... exec_state=...
regs: rax=... rcx=... rdx=... rbx=... rsi=... rdi=... rbp=... r8=...
gs_base=... fs_base=... insn@rip-24= ... [bytes at the faulting rip]
r12=... r13=... r14=... r15=...
blockhist: ...                               the last guest blocks entered
fault-page: UNCOMMITTED si_code=... host_prot=... region=(...)
rip-region=(...) prot=...
rbp-chain: ...
bt: ...
  frame 0x... /usr/lib/system/libsystem_malloc.dylib+0x...

```

- **The first line** is the host's view: the signal, the host address that faulted, the host instruction and its word, the link register, and `ocerz_base`, the address ocerz itself is loaded at. Subtracting it from `host_pc` gives an offset that `atos -l` turns into a function in ocerz, which says whether the fault was in translated code, the interpreter, or ocerz's own C.
- **guest_rip and guest_addr** are the guest's view: the instruction that faulted and the address it touched. `icount` counts interpreted instructions only, so a fully translated loop never advances it.
- **regs and r12 - r15** are the full guest register file, recovered from the arm64 registers the JIT keeps them in. The callee-saved registers are printed deliberately: "a register the ABI says survives a call did not" is a whole class of emulation bug.
- **insn@rip-24** shows the bytes around the faulting instruction, with the instruction itself in brackets; a decode bug shows here.
- **blockhist** is the guest addresses of the last blocks entered, the path that led to the fault.
- **fault-page** says whether the faulting address was committed guest memory, uncommitted, or outside the arena altogether, with its host protection. "UNCOMMITTED" at an address just below a stack is a stack overflow; "outside-arena" is a wild pointer.
- **rbp-chain, bt and the frame lines** are the guest backtrace, each frame named by image and offset. A short repeating pair of frames is unbounded recursion.

Other fatal lines name their own cause. `decode failed (... mode) at rip=...` with the bytes means the decoder met an instruction it does not know. `guest breakpoint/interrupt` is an `int3` or `ud2` in guest code, usually an assertion in the program; the report then shows where it was raised. `fatal on guest thread cpu#N`; `exiting process <pid> "<command line>"` names the process, because under Wine a dozen processes share one standard error. `BRIDGE-FAULT ... inside a bridged call`, not in guest code is native mode's own report (Chapter 12): it names the bridged function, its signature, and whether the faulting address was guest memory (the guest passed a bad pointer) or not (ocerz marshalled the call wrongly). `IMAGE-CLOBBER` is printed, always, when a guest mapping lands on a loaded image or a protection change takes execute away from one; a library unmapped under running code otherwise surfaces only much later as a wild jump. The `WILD-` family (`WILD-LOOKUP`, `WILD-STACK`, `WILD-FAULT-AV`, `WILD-WORKER-TERMINATE`) reports indirect branches and faults whose target lies outside anything the guest mapped.

A hung process

Hangs are harder than crashes, because nothing prints. Three instruments cover them.

kill -INFO <pid>. SIGINFO is always armed. It dumps every guest thread: the host thread id, the guest `rip` and `rsp`, the system call it is blocked in, and, in a Wine process, its Windows-side stack. Reading that stack means knowing where Wine keeps things: the TEB is found through a thread-specific-data slot, and the PE caller's registers sit in the syscall frame Wine saves in the TEB, because Wine runs its Unix-side code on a separate stack. A hung Wine process can be inspected this way without attaching a debugger, which matters because the debugger often cannot attach to it.

OCERZ_JITLOCKLOG=1 reports any wait for the translation lock longer than three seconds, with the holder's thread, where it took the lock, its guest `rip` and its kernel thread state. It found the stalls in Steam's web helper where a thread holding the lock had been suspended. **OCERZ_FLUSHLOG=1** logs every arena flush and names the threads a flush is waiting on.

Mach conversations. A guest thread parked in `mach_msg` or `kevent` is usually waiting for a reply that never came, not stuck in translation. **OCERZ_MACHMSG** prints Mach messages as they pass; **OCERZ_MSGHEX** prints a round trip exactly as the guest saw it, which separates "the kernel replied wrongly" from "the guest read the reply wrongly"; **OCERZ_PORTDUMP** names the conversation behind a lost-wakeup wedge; **OCERZ_ULOCKLOG** traces `unlock` waits and wakes with their owners.

Where the time goes

Switch	What it reports
OCERZ_PERF STAT=1	at exit: blocks translated and executed, instructions executed in translated code and in the slow path, the hottest blocks, the return-address-stack line (misses, slot-pool use, call sites without a slot) and the chain line (chains activated, veneered, out of range)
OCERZ_GUES TPROF=<usec >	a sampler that suspends each running guest thread at a jittered interval, reads its host pc and credits the place with the CPU time used since the last look; every OCERZ_GUESTPROF_PERIOD seconds (default 10) and at exit it prints the hottest guest addresses with their image, the hottest host symbols, the hottest interpreted instruction forms, the translator's own counts and time, and CPU per image
OCERZ_TRIP STAT=1	exits from translated code to the dispatcher per second, and the twenty commonest destinations with the state of the block at each
OCERZ_FPS=1	frames per second in cache mode, counted at the block that begins <code>CGLFlushDrawable</code>
OCERZ_BLAC KLOG=1 , OCE RZ_INVSRC=1	pages refused translation because they keep changing, and which code invalidated them
OCERZ_XLAT PAGES=1	every distinct 4 KB page a process translates
OCERZ_TCAC HE_LOG=1	per process, blocks loaded from and stored to the translation cache

GUESTPROF deserves a note on why it weights by CPU time rather than counting samples. Wine's device service process has a poller that sleeps a millisecond at a time and woke on the sampler's own timer ticks: a plain sample count billed it 39% of a core while `ps` measured 4.7%. Crediting each sample with the CPU the thread actually used since the last look removes that bias. The per-form output, the opcode, operand kinds and size of each instruction the interpreter executes, is how the Wine-only JIT gaps of 2026-09-27 were found: Steam's web helper was interpreting `pinsrw` from memory for 15% of its samples, and after those forms were translated the process spent 78-81% of its samples in translated code instead of 47-67%.

Loader and library diagnostics

Switch	What it shows
<code>OCERZ_DLPATH=1</code>	every <code>dlopen</code> and <code>dlsym</code> failure with its reason; unlike <code>-v</code> , inherited by child processes
<code>OCERZ_DLOPENLOG</code> , <code>OCERZ_DLSYMLOG=1</code>	each <code>dlopen</code> request and result with its caller; each <code>dlsym</code> with handle, result and caller
<code>OCERZ_INITLOG</code> , <code>OCERZ_INITTRACE</code> , <code>OCERZ_INITEDGE</code>	every initializer run, every image the initializer walk enters or skips, and every dependency edge it follows, upward edges marked
<code>OCERZ_ALLMISS=1</code> , <code>OCERZ_DYNLOKUPLOG=1</code>	every unresolved import rather than the first 24; symbols imported with <code>-undefined dynamic_lookup</code> that nothing defines
<code>OCERZ_DYLDAPI_TRACE=1</code>	every call into the dyld API table, with its slot, arguments and caller (Chapter 11)
<code>OCERZ_EXCLOG=1</code>	every Objective-C and C++ exception thrown, with the throw site and the exception's reason

System calls, Mach and IOKit

`-strace` traces every guest system call of one process; `OCERZ_STRACE_EXE=<text>` traces only the processes whose command line matches. `OCERZ_MACSYSLOG=1` prints each `__mac_syscall` with its policy, number and argument words. `OCERZ_FAULTLOG=1` prints the mapping state around every guest fault, and `OCERZ_WINEFAULTLOG` adds the faulting thread's first thread-specific-data slots next to its TEB, which tells a missing TEB from a wrong `gs`. `OCERZ_MODELLOG=1` prints every far transfer with its selector and target mode, the WoW64 32/64-bit switches. For IOKit, `OCERZ_MIGTRACE` traces the translation of Mach interface messages, `OCERZ_IOKITSEL=<selector>` (or `all`) prints every `io_connect_method` call with that selector, its scalar inputs, the first words of the reply and the guest's `rip`. `OCERZ_IOKITERR=1` prints every such call that comes back with `KIOReturnVMError` or `KIOReturnBadArgument`, with its selector, structure input and out-of-line buffers, which is what names the field a translation missed, and `OCERZ_MSGNEEDLE=<value>` reports every outgoing Mach message that carries a given 64-bit value, which is how the untranslated address in Metal's no-copy buffers was found.

For a value that is wrong without anything crashing, `OCERZ_WATCH` and `OCERZ_WATCHBT` watch a guest address and print who writes it, `OCERZ_REGTRAP` stops at a guest address and prints the registers and the frame-pointer return chain (`OCERZ_REGTRAP_MAX=N` stops after N hits, since a trap in a recursing function otherwise prints until the stack runs out), and `OCERZ_PEEK` answers the first question worth asking when a guest assertion fires on a constant: was this global already wrong before the guest ran?

Two utilities make Wine's process tree manageable. `OCERZ_STDERR_FILE=<file>` appends every process's standard error to one file, and every fatal line carries the process id and command line, so the interleaved output can be separated afterwards.

Case studies

The four failures below were each traced with the instruments above. They are told from the outside, as the instruments showed them.

One miscompiled block in a browser engine (2026-09-06)

Symptom. Steam's Windows client, running under Wine with Chromium's embedded browser, could not open any of its browser profile databases: SQLite reported "out of memory" for every one, the browser re-tried forever, and no renderer ever started. Under Rosetta none of this happened.

Narrowing. Under `-no-jit` the browser worked, so a translation was wrong. An address bisection with `OCERZ_INTERP_LO / HI` over the browser library's text halved the range until one block remained, in SQLite's B-tree cell walk inside the browser library. A bisection over the switches then found two that each fixed it: `OCERZ_NO_JCCFUSE`, which stops fusing a compare with its branch, and `OCERZ_NO_CHAIN`, which stops chaining. The whole hunt took about an hour.

Cause. The block ended in a fused compare and conditional branch, `cmp ebp,0xb` then `jbe`, and the branch target began with a `ja` that read the same flags. The fused branch recorded the producer's flags on its taken side in a stub, and when the edge was chained the branch was retargeted straight at the successor, past the stub. The successor's `ja` read a stale record. The bug had come in with conditional-branch retargeting on 2026-08-15 (4ace0b7) and was fixed on 2026-09-06 (264e647): a stub that records flags, replays lane flushes or checks a floating-point batch must stay on the path. The regression test, `jcc_chain_rec`, puts four producer shapes behind side exits with alternating inputs so that a stale record shows; it fails 30 of its 48 checks without the fix.

Windows 74 pixels high (2026-09-08)

Symptom. Steam's login window logged its size as 700 by 74, and every titled Cocoa window in every emulated process was squashed to the same height.

Narrowing. Steam takes six minutes to reach that point, so the first step was a probe: a small AppKit program that prints `NSScreen.visibleFrame`, run under `ocerz` in five seconds. It printed a 74-pixel-high frame, under the JIT and under the interpreter alike, so this was not a JIT bug. `OCERZ_MSGHEX` then showed the window server's reply as the guest received it, and the reply was correct. A correct reply turned into a wrong value by both engines points at the one thing they share: the decoder.

Cause. Opcode `66 0F 6D` is `punpckhqdq`, but a default arm in the decoder had mapped both `0F 6C` and `0F 6D` to `punpcklqdq` since the initial commit. The high quadword of a rectangle never moved down, so width and height came back as copies of x and y. Fixed in `0f19d4c`; `punpck_qdq` covers every form. Three five-second probes had replaced a day of six-minute Steam runs.

A stack overflow in `malloc` (2026-10-03)

Symptom. `sw_vers`, a two-line Objective-C tool, crashed on macOS 26.7.1 under `ocerz`. The crash report's backtrace was two frames in `libsystem_malloc` repeating, `malloc` and `malloc_zone_malloc` calling each other, and the fault page was uncommitted memory just below the stack: unbounded recursion.

Narrowing. `-no-jit` crashed the same way, so the JIT was not involved. None of the initializer-order switches changed it. `OCERZ_INITLOG` showed which initializers had run before the crash: `libc++abi`, `libc++`, `libswiftCore`, `libswift_Concurrency`, `DebugSymbols`, `CoreDisplay` and `SkyLight`, and not `CoreFoundation`, which `sw_vers` links directly. `OCERZ_INITEDGE` printed every dependency edge the walk followed, and following them back from `SkyLight` gave the chain: `CoreFoundation` depends on `libobjc`; `libobjc` links `libswiftCore` upward; `libswiftCore` links `Foundation` upward; and `Foundation`'s dependencies lead to `SkyLight`. So `SkyLight` was being initialized while `CoreFoundation` was still unfinished on the walk's stack, and its first allocation went through `CoreFoundation`'s allocator before `CoreFoundation` had set it up.

What real dyld does. AArchX followed each upward link as soon as the library declaring it had been initialized. To learn the right rule without reading Apple's loader, the fix was an experiment: three small libraries of the project's own, `core` depending on `mid`, `mid` linking `top` upward and `top` depending on `core`, each initializer logging its name. Under Rosetta the order was `mid core main top`: the upward target runs last, after the program's own initializer. The old ocerz printed `mid top core main`. Upward targets now wait until the whole walk from the program is done (`e705796`), and the experiment became the test `dupward_defer`. `sw_vers` prints what the native tool prints, and `tools/apptest.sh cli` went from 15 to 16 of 16 on that machine.

A nil drawable in a Unity game (2026-10-03)

Symptom. R.E.P.O., a Unity game running under Wine with DXMT, aborted on a nil Metal drawable in about two launches of three.

First cause. Metal frees a buffer and maps the next one at the same host address. Under Wine the guest sees such memory through an alias in the low shadow, and the alias kept the freed buffer's memory object alive: the guest read the previous compute job's results, Metal's completion state went stale, and a job could wait forever. Each alias is now recorded with the memory object it shows, and an IOKit reply that points into an alias whose host memory has changed re-aliases it (`0115ead`; `0CERZ_NO_ALIAS_REFRESH=1` turns it off). The game rendered, but the abort still came intermittently.

Second cause. `0CERZ_IOKITSEL` printed the IOKit calls that fetch a new drawable's properties. The buffer size in the failing calls was garbage: four gigabytes, with the right size in its low bits, and the kernel refused it with `kIOReturnVMError`. Creating an IOSurface returns two records inside memory the kernel shares with the process, at addresses that are 8-byte aligned but not page aligned. The IOKit reply scan aliased only page-aligned values into the low shadow, so under Wine those records were read through whatever the shadow held at that number. An 8-byte-aligned value inside IOKit or shared host memory is now a candidate too, skipped when the guest has its own memory at that number (`28e237b`). A companion change keeps the kernel from placing its mappings in the low 12 GB at all, by reserving the free host ranges there, so they land where guest and host addresses are the same and need no alias (`3cc5842`). In fifteen launches after the change, none failed.

WHAT THE FOUR HAVE IN COMMON

None of them was found by reading the code that misbehaved. Each was found by asking one question at a time of an instrument that could answer it: does the interpreter agree, does the control agree, which range, which switch, which reply, which order. The switches exist because each of these questions was once asked without one.

CHAPTER 18

Limitations and roadmap

AArchX is experimental software at version 0.3-dev. It runs a long list of real programs, and it also has gaps that stop others, deviations from x86 that a careful program could notice, and costs that make some workloads slower than under Rosetta. This chapter is the honest ledger as of 9169680 (2026-10-04), grouped by how each item bites, followed by the direction the project is taking now that Rosetta's end is in sight.

HOW TO READ THIS LIST

The gates compare the JIT with the interpreter, and the interpreter with goldens taken partly from Rosetta. A deviation the two engines share is invisible to a differential by construction, which is why several items below were found only by running real software. The list is what is known; it is not a guarantee that nothing else is wrong.

Programs that stop

- **Counter-Strike 2 under Wine** has loaded D3DMetal and opened its own window since 2026-09-26, then dies on a fatal "Multiple entity classes have the same designer name" error that Rosetta never reaches. The cause is believed to be a JIT feature, and `OCERZ_EXE_ENV` exists so that features can be switched off inside `cs2.exe` alone while the rest of the Wine tree runs at full speed. The same runs print hundreds of `dropping stale oolslow arm` warnings whose source is not known yet; the first forty now say which block and instruction the out-of-line arm belonged to.
- **Photoshop** stops during startup when an initializer runs while the Objective-C runtime holds its lock and the recursive acquisition aborts (2026-09-22). That is a re-entrancy problem in how initializers are driven and wants its own change. Three earlier causes found on the way were fixed: upward dependencies never initialized, dylibs with no export trie, and `-undefined dynamic_lookup` imports treated as fatal.
- **Photos** has not been run since the last of the three causes of its graphics failure, the invisible GPU, was fixed on 2026-09-22.
- **Ollama** runs its command line, its HTTP server and its menu-bar app, but nobody has run a model under AArchX yet.

The GPU and Steam

- The macOS Steam client in cache mode still runs with `-cef-disable-gpu`, rendering through software, and has not been retried with acceleration since translated programs began to see the GPU on 2026-09-22. In native mode on macOS 27 its GPU process runs hardware-accelerated graphics. Some cache-mode launches stall before the web helper starts its child processes, with the translation lock held inside translation; relaunching gets past it.

- Under MacNdCheese's Wine, DXMT's swap chain on a window owned by another process is refused by the Wine Mac driver ("Cross-process child window Metal swapchains are not implemented"), the same under Rosetta, which is why the Windows Steam client's browser was brought up with GPU compositing off. R.E.P.O., a single-process Unity game, renders through DXMT under AArchX since 2026-10-03.
- Steam's overlay frame counter relies on dyld interposing, which AArchX does not implement; `OCERZ_FPS=1` counts frames instead.

Slower than Rosetta

- **Wine startup.** Windows Steam reaches its main window in about 33 s under AArchX against 11-12 s under Rosetta, using about 58 s of CPU against 17-21 s (2026-09-27). With the translation cache warm it took 33.7 and 35.2 s in two runs, against 46.9 s from an empty cache. A short Wine process spends about 70% of its startup translating, most of it while the Wine Mac driver brings in AppKit and Foundation.
- **Wine's memory layout.** The low shadow still keeps the fast memory forms, base hoisting, paired moves and the floating-point batch undo log off, because they need a host address formed without a check. The short check of 2026-10-04 cut `memcpy` in that layout from 6.1x to 3.1x of Rosetta's time, `mixed` from 3.0x to 1.65x and `vm` to parity; the rest of the gap is these forms.
- **Swift conformance lookups.** `libswiftCore` spent 3.3 s of one Steam startup walking every protocol conformance record, because the shared cache's precomputed Swift tables are not offered to it.
- **Games.** Brawlhalla's first frame took 5.3 s against 0.54 s on 2026-09-22. MMX, translated since 2026-10-04, still keeps its registers in memory, so an MMX loop is 0.142 s against Rosetta's 0.053 s.
- **Self-modifying code.** A program that rewrites code it has run pays a page fault per write and a retirement per block hit. After the targeted retirement of 2026-10-04 a rewrite costs 24.7 μ s among 300,000 translated functions, against Rosetta's 14.7.
- **Callbacks in native mode.** A call from native code back into guest code costs about 430 ns, against 17 ns for a call out and 2 ns for the eleven string and memory routines answered in place.

Semantic deviations

These are places where AArchX does not do exactly what an x86 processor does. Each is deliberate or known, and each is the kind of thing a careful program could detect.

Area	Deviation
x87	The 80-bit registers are held as 64-bit doubles, so extended precision and range are lost: a product that overflows a double but fits an 80-bit value becomes infinity. <code>FXSAVE</code> uses a double-backed layout, and the MMX registers are kept apart from the x87 stack, so <code>FXSAVE</code> and signal frames do not carry them.
Approximations	<code>rcpps</code> , <code>rcpss</code> , <code>rsqrtps</code> and <code>rsqrtss</code> return exact results, not x86's 12-bit approximations.
Interpreted SIMD	Correct but slow: 256-bit byte and integer shuffles, unpacks, permutes and lane inserts, <code>vptest</code> , the immediate blends, mask-producing compares, gathers, <code>bextr</code> , <code>pdep</code> , <code>pext</code> , and the MMX forms not yet translated (<code>pmaddwd</code> , <code>pmovmskb</code> , <code>emms</code> , the SSSE3 MMX forms, conversions to and from SSE, shifts by a register count).

Area	Deviation
Not decoded	AVX-512 and its EVEX encoding, and privileged and I/O instructions, which no user program executes.
NaN payloads	A NaN produced inside a floating-point batch that faults before its check keeps the arm64 payload. On processors with <code>FEAT_AFP</code> , cache mode leaves add, subtract, multiply, divide and square root to the hardware's x86 NaN rule; fused multiply-add and native mode use the translated checks.
Narrow SSE ordering	In the ordered memory model, SSE loads and stores stay plain, the 8- and 4-byte <code>movq</code> and <code>movd</code> included, so a counter published through <code>movq</code> can become visible before data written earlier with a scalar store. <code>tests/dynamic/tso_narrow.c</code> sees it hundreds of times in three million handoffs. <code>OCERZ_TSO_NARROW=1</code> fixes it at a third of R.E.P.O.'s frame rate; no program is known to depend on it.
Signal timing	An asynchronous signal reaches a guest thread only at its next system call (or bridged call in native mode), so a thread spinning in its own code never sees one. A signal aimed at another thread reaches its guest handler only for the signals AArchX mirrors onto the host, such as <code>SIGUSR1</code> , <code>SIGTERM</code> and <code>SIGALRM</code> , not for fault signals.
Protection granularity	Guest protection changes are resolved on 16 KB host page boundaries, though guest pages are 4 KB.
<code>mach_vm_region</code>	Answered from the guest's own map since 2026-09-22, with each segment's declared protection, but <code>max_protection</code> always reads 7, because AArchX keeps no per-segment maximum.
<code>ioctl</code>	The third argument is always translated as a pointer. In the identity map every dynamic program uses that is the identity, but in the static arena and Wine's low shadow a scalar argument gets a base added. <code>fcntl</code> translates only for the commands that take a pointer.
Process identity	<code>proc_pidpath</code> and <code>proc_name</code> name the guest executable only when a process asks about itself in cache mode; other AArchX processes appear as <code>ocerz</code> .

Loader gaps in cache mode

- The dyld API object answers 59 slots, each identified by running code (Chapter 11). Every other slot answers zero in both result registers. Three that real programs reach have no handler: `+0x30`, which `_dyld_register_func_for_add_image` reaches first, so add-image callbacks are not called in cache mode; and `+0x420` and `+0x250`, which the macOS Steam client calls and tolerates a zero from.
- Upward links are followed during a `dlopen` only to Foundation, CoreFoundation and libobjc. Following all of them was tried and rejected: it pulled about 360 more initializers into every Wine process and killed Steam's browser.
- Not rechecked since the September work on `dlopen`: whether `dlopen(NULL)`'s handle finds the program's own symbols, `RTLD_LOCAL`, and `dlderror` after a failed `dlsym`. Native mode does all of these the way dyld does.
- dyld interposing is not implemented.

Translator internals still open

- **Arena overflow.** A translation that overflows the 1 GB code arena left the translator in a state that damaged later translations, reproducible in a second with a 512 KB arena, and why is not understood. Since 2026-10-04 the arena is flushed before it can overflow, which removes the symptom; the cause is still open.

- **Arena flush and stuck threads.** A flush needs every thread out of translated code. One that cannot be got out in 500 ms, such as a thread stopped by `thread_suspend` while it spins, makes the flush back off and retry two seconds later.
- **Stale out-of-line arms.** The dropping stale oolslow arm warnings seen under CS2 have no known source yet.

Native mode

Native mode stops a program it cannot run correctly with a named message, exit 71 for an import that cannot bind and 72 for an export with no crossing (Appendix B), rather than running it wrongly. Its gaps, in order of how much software they stop:

1. **Swift.** Swift classes and the frameworks' Swift overlays are refused. Calculator and most newer system applications are written in Swift. This gap grows by itself as Apple moves more system software to Swift.
2. **Libraries without a database.** Fifty-nine libraries have one, generated from the builder's SDK by `make apis`: `libSystem`, `libobjc`, `CoreFoundation`, `Foundation`, `AppKit`, `CoreGraphics`, `CoreServices`, `Security`, `IOKit`, `Metal`, `OpenGL`, `QuartzCore`, the audio and video frameworks and the small dylibs programs pick up. WebKit and GameKit are the gaps that matter for larger applications, and unsupported exports inside the 59 remain stubs.
3. **C++ across the boundary.** Guest C++ works with the optional LLVM guest runtime, exceptions between guest frames included, but guest C++ objects are not interchangeable with native arm64 ones, and no exception may unwind through a bridge or callback frame. Native Objective-C or C++ exceptions cannot be caught by guest handlers.
4. **Calls with no veneer.** The `scanf` family other than `scanf`, `fscanf`, `sscanf` and their `v` forms, `err`, `errx` and `asl_log` are stubs; so are functions taking a `long double`, a `va_list` (apart from the `printf` family) or a union or bitfield structure by value, and `forkpty`, `daemon` and `wordexp`.
5. **Replacing methods.** `class_replaceMethod`, `method_setImplementation`, the bulk method APIs and `imp_implementationWithBlock` are stubs, and a guest cannot call an `IMP` the native runtime returns, because it is arm64 code.
6. **Smaller items.** A `longjmp` out of a native callback is refused; an XPC service inside an application bundle is not reached, most likely because `launchd` looks it up for the executable that asks, which is `oerz`; a block that crosses as a plain object can be called only by the side that made it; after a bridged call from translated code, the xmm registers the signature does not use hold what native code left in them, which System V allows.

The direction: after Rosetta

Apple ends general-purpose Rosetta after macOS 27, and with it the `dyld_shared_cache_x86_64` that cache mode binds every program against. macOS 27 already moved that cache out of the operating system's own cryptex into a separate Rosetta cryptex. Cache mode works on both macOS 26 and 27 since 2026-10-03, looking in the Rosetta cryptex first and the old location second, but it cannot outlive the file.

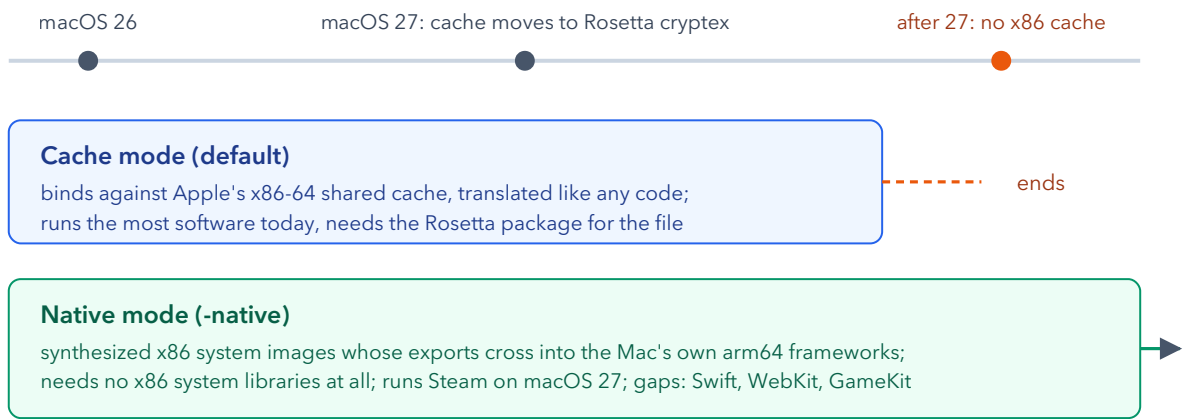


Figure 18.1. The two ways AArchX supplies a program's system libraries, against the end of Rosetta. Cache mode depends on a file Apple ships only while Rosetta does; native mode depends only on the Mac's own frameworks and on databases generated from its SDK.

Native mode is the answer, and the code says so: `include/ocerz/mode.h` describes it as "the mode that survives" the end of Apple's Intel frameworks, "selected explicitly until it is ready to be the default". Built from 2026-09-16, it ran the x86-64 Steam client on macOS 27 by 2026-09-21. Making it the default is a matter of closing the list above, in roughly that order: Swift, because it stops the most software and the share grows; the remaining frameworks, which is mostly a data change through `tools/sdkgen.sh` and `make apis` as long as their calls fit the ABI engine; C++ and exceptions across the boundary; and the cost of calling back into guest code, which today is three system calls saving signal state for each callback's fault-recovery point.

The second direction is visible in the tree rather than stated. Since 2026-09-23 the project is under the LGPL-2.1-or-later again and carries its own documentation, contribution rules, code of conduct, security policy and issue templates; the commit that brought the documentation in says it moved so the tree would be "self-contained when it becomes public". The README was cut to a short front door on 2026-10-03 with the long material in `docs/`.

Open fronts

Front	Where it stands
Games under Wine	R.E.P.O. renders through DXMT. After the arena flush and the short low-window check of 2026-10-04 its menu runs at roughly 45-65 frames a second, varying from run to run, and a singleplayer level at about 50; Rosetta draws the same menu at about 178, so the gap is still large. CS2 reaches its window and stops on the designer-name error.
Wine speed	Translation, the remaining low-shadow restrictions on fast memory forms, and the Swift conformance scan are the named costs. The translation cache, cheaper translation and the short check have taken the largest pieces.
Steam with the GPU	The macOS client in cache mode has not been retried with acceleration since the GPU became visible.
Application breadth	Photoshop's re-entrancy, Photos untried since its fixes, a model run under Ollama.

Front	Where it stands
Native-mode coverage	Swift, WebKit, GameKit, C++ across the boundary, method replacement, the remaining variadics.
Semantic gaps	80-bit x87, approximate reciprocals, the SIMD forms still interpreted, add-image callbacks and interposing in cache mode, three unanswered dyld API slots.

The June plan, scored

In June the project wrote down a seven-phase plan from a static-binary translator to Wine and Steam. Every phase has landed or been superseded, and several went differently from the plan in instructive ways.

June phase	Planned	What happened
1. dyld	run Apple's <code>/usr/lib/dyld</code> as a second image	Done differently: AArchX never runs Apple's dyld. It wrote its own loader and cache mapper and answers libdyld's API table itself (Chapter 11).
2. Signals	host trampolines, x86 contexts, <code>sigreturn</code>	Done, plus mask enforcement, delivery at block entry, mirroring onto the host, frames carrying the upper YMM halves.
3. Threads	host pthreads, per-thread <code>gs</code> , <code>unlock</code> and <code>psynch</code>	Done, plus the host workqueue bridge, emulated thread suspension, the unstick monitor, fork and spawn.
4. Memory model	acquire/release everywhere	Done differently: plain until a second observer appears, then ordered scalars and plain vectors, as FEX ships.
5. JIT tier 2	chaining, inline flags, pinned registers	Exceeded: full register pinning, deferred flags, superblocks and trace inversion, FP batches, AVX and MMX, in-place routines, a translation cache. Faster than Rosetta on most kernels.
6. Wine and Steam	wineserver, Notepad, then D3DMetal	Mostly done: Wine boots, WoW64 runs, Windows Steam reaches its main window, a Unity game renders; CS2 open.
7. Polish	80-bit x87, MXCSR rounding, exact reciprocals, AVX	MXCSR rounding and AVX done; 80-bit x87 and approximate reciprocals open.

Nothing in the June plan anticipated i386 and WoW64, native macOS applications, native mode, the re-name to AArchX, or the license's round trip through a proprietary text and back. A plan is a list of what is known to be needed; the software decides the rest.

APPENDIX A

Environment variables

AArchX is configured by its command line and by environment variables named `OCERZ_*`, after the project's former name. The first tables below are the variables meant to be used; the later ones are the switches added with particular fixes, and a census of the several hundred diagnostic knobs that exist for narrowing bugs. Every variable is read once, at first use, and is inherited by child processes unless noted.

Command line

```
usage: ocerz [-v] [-trace] [-strace] [-no-jit] [-native|-cache] [-path file] [--] program
[args...]
    ocerz version
```

Option	Effect
<code>-v</code>	more logging: the mode, the arena, the shared cache, image loads, the JIT arena
<code>-trace</code>	trace guest instructions
<code>-strace</code>	trace guest system calls
<code>-no-jit</code>	interpret this process only, without translating
<code>-native</code>	bind against the Mac's own arm64 frameworks (Chapter 12)
<code>-cache</code>	bind against Apple's x86-64 shared cache; the default
<code>-path file</code>	load <code>file</code> , and pass every following argument to the guest unchanged
<code>--</code>	end of AArchX's own options
<code>version</code>	print the name and version, AArchX 0.3-dev

The program may be an executable or an application bundle, in which case the executable its `Info.plist` names is run with that path as `argv[0]`.

Choosing what runs

Variable	Effect
<code>OCERZ_MODE=native cache</code>	pick the mode when no flag does. This is how a program's children inherit it, even when started with an empty environment. An unrecognised value is refused (exit 64).
<code>OCERZ_NOJIT=1</code>	interpret the whole process tree
<code>OCERZ_APIDB=<dir></code>	native mode: read the API databases from this directory instead of <code>runtime/apis</code> beside the binary
<code>OCERZ_GUEST_ROOT=<dir></code>	native mode: a directory mirroring absolute guest library paths; an x86-64 image found there is used in place of its database. Empty disables the default <code>runtime/guest</code> .

Process trees

Wine and Steam start many processes, and a setting is usually wanted in one of them. Each of these selects processes by a substring of their command line, and the setting is inherited through every exec in the chain.

Variable	Effect
<code>OCERZ_EXE_ENV="a.exe:K=V,K=V;b.exe:K=V"</code>	set variables only in the processes whose command line matches
<code>OCERZ_STRACE_EXE=<text></code>	trace system calls only in matching processes
<code>OCERZ_NOJIT_EXE=<text></code>	interpret only matching processes
<code>OCERZ_STDERR_FILE=<file></code>	append every process's standard error to one file

Keeping translations

Translated blocks are stored under `~/Library/Caches/ocerz`, one directory per build of ocerz and set of `OCERZ_` variables, and loaded by any later process that needs the same code. Only code at the same address in every run is kept: all of a Wine process, the shared cache otherwise. A store that reaches its limit is emptied by the next process that finds no other using it; stores of other builds are removed after a day, or sooner while they add up to more than 2 GB. The test runners set `OCERZ_TCACHE=off`.

Variable	Effect
<code>OCERZ_TCACHE=off</code>	translate everything afresh and keep nothing
<code>OCERZ_TCACHE_DIR=<dir></code>	keep the stores under this directory
<code>OCERZ_TCACHE_MAX_MB=<n></code>	the size at which a store stops growing (4096 by default)
<code>OCERZ_TCACHE_LOG=1 <file></code>	per process, blocks loaded and stored
<code>OCERZ_TCACHE=verify</code>	a check: translate everything anyway and compare each block with the stored one
<code>OCERZ_TCACHE=roundtrip</code>	a check: store nothing, move every translation to a new address and report anything still pointing at the old one

Memory ordering

Variable	Effect
<code>OCERZ_NO_PLAIN_MEM=1</code>	use ordered forms from the start, rather than from the first thread, fork or shared mapping
<code>OCERZ_TS0_STRICT=1</code>	order stack-relative accesses too
<code>OCERZ_TS0_VECTOR=1</code>	order every SSE load and store, at a large cost
<code>OCERZ_TS0_NARROW=1</code>	order <code>movd</code> and <code>movq</code> ; <code>=2</code> every SSE access of 8 bytes or fewer

Turning off what makes it fast

Each of these slows AArchX down and is useful for narrowing a suspected miscompilation: if one fixes a program, the bug is in what it controls (Chapter 17).

Variable	Effect
<code>OCERZ_NO_AFP=1</code>	check NaN results in translated code even where the processor produces x86's NaNs itself
<code>OCERZ_NO_JIT_MMX=1</code>	run MMX instructions in the interpreter
<code>OCERZ_NO_FAST_LOW_GUARD=1</code>	in a Wine process, map every guest address with the general three-range check
<code>OCERZ_NO_LEAF_INPLACE=1</code>	reach <code>strlen</code> , <code>memcpy</code> and the other nine routines the ordinary way
<code>OCERZ_NO_MEMFN_PLAIN=1</code>	translate the system's string and memory routines with ordered accesses when the process has ordering on
<code>OCERZ_NO_FPB_DEFER=1</code>	check every floating-point result immediately instead of once per batch
<code>OCERZ_NO_FLIP=1</code>	no profile-driven retranslation of superblocks
<code>OCERZ_NO_MOVFUSE=1</code>	do not fold a <code>mov</code> into a following shift
<code>OCERZ_NO_CHAIN=1</code> , <code>OCERZ_NO_JCC FUSE=1</code>	no block chaining; no compare-and-branch fusion
<code>OCERZ_NO_HOST_RAS=1</code>	use the software return-address stack in the CPU structure instead of the host stack
<code>OCERZ_NO_XLIVE_MEM0=1</code>	do not memoise cross-block flag liveness
<code>OCERZ_NO_LOW_RAS=1</code> , <code>OCERZ_NO_LOW_SPLICE=1</code>	in Wine's layout, no host return stack, no spliced calls
<code>OCERZ_NO_BRIDGE_FASTCALL=1</code>	native mode: reach every bridged call through the trap and dispatcher
<code>OCERZ_NO_COMPACT=1</code>	keep every block's decoded instructions after translation
<code>OCERZ_NO_JIT_FLUSH=1</code>	when the code arena fills, interpret everything new instead of flushing the arena

Behaviour

Variable	Effect
<code>OCERZ_STRICT_SYSCALL=1</code>	stop on an unimplemented system call instead of returning <code>ENOSYS</code>
<code>OCERZ_NO_HOSTWQ=1</code>	turn off the host workqueue bridge, which is on by default
<code>OCERZ_NO_UNSTICK=1</code>	never interrupt a guest thread out of a long wait
<code>OCERZ_UNSTICK_ALL=1</code>	let the unstick monitor interrupt every blocking call, <code>read</code> , <code>recvmsg</code> and <code>poll</code> included
<code>OCERZ_NO_THREADACT=1</code>	hand <code>thread_suspend</code> , <code>thread_resume</code> and <code>thread_get_state</code> on guest threads to the kernel
<code>OCERZ_NO_FILEMAP=1</code>	read every private file mapping into anonymous memory

Variable	Effect
OCERZ_PRELOAD_OBJC=<paths>	put matching shared-cache Objective-C images into the startup batch; @cat does it for every image that defines categories
OCERZ_NO_LOADMAP=1	do not map a cache image's Objective-C classes before its +load runs
OCERZ_NO_LATE_CATLIST=1	do not report category lists for cache images loaded after startup
OCERZ_NO_VMMAP_STEER=1	let mach_vm_map place mappings anywhere in host space
OCERZ_REFAULT_INVAL=1	invalidate again on every repeated alignment or commpage fault
OCERZ_NO MOCK_KEYCHAIN=1	launch Steam's helper without --use-mock-keychain
OCERZ_NO_NATIVE_CHILDRN=1	native mode: run the system tools a guest starts under AArchX too
OCERZ_NO_TSD_DTORS=1	native mode: do not run the guest's thread-specific-data destructors at thread exit

Switches that restore old behaviour

Most fixes that change a rule add a switch that puts the old rule back. They exist for A/B measurement and to prove the regression test fails without the fix, not for daily use.

Variable	Restores
OCERZ_NO_UPWARD_INIT=1	upward dependencies never initialized (before 2026-09-22)
OCERZ_NO_CLOSURE_LOADS=1	no +load phase in a dlopen closure
OCERZ_NO_DLOPEN_UPWARD=1	no following of upward links to Foundation, CoreFoundation and libobjc during a dlopen
OCERZ_NO_DLOPEN_CALLER_RPATH=1	@-paths in dlopen not resolved against the caller
OCERZ_NO_DLSYM_DEPS=1 , OCERZ_DLSYM_UPWARD=1	image-only dlsym on disk images; upward links followed by dlsym in the cache
OCERZ_NO_BULK_NOTIFY=1	bulk image-load callbacks called once, at registration
OCERZ_NO_MACSYS_XLATE=1	no rewriting of stack pointers in __mac_syscall arguments
OCERZ_NO_THREAD_SIGNALS=1	signals delivered from a shared pool, not kept on the receiving thread's cpu
OCERZ_NO_PEER_VM=1	no translation of mach_vm_* calls aimed at another task
OCERZ_NO_VMMAP_TAKEN=1	a fixed mach_vm_map may move over guest pages in use
OCERZ_NO_ALIAS_CLIP=1 , OCERZ_NO_ALIAS_REFRESH=1	host-memory aliases not clipped at guest data; not refreshed when the host maps new memory there
OCERZ_NO_IOKIT_INNER_PTR=1	only page-aligned values in IOKit replies aliased
OCERZ_NO_LOW_HOLE_FILL=1	the kernel free to map below 12 GB in a Wine process
OCERZ_NO_GPU_NOCOPY_XLATE=1	Metal no-copy buffers passed with guest addresses

Variable	Restores
OCERZ_INV_SCAN=1 , OCERZ_INV_GRANULE=1 , 0 CERZ_RETIRE_SCAN=1 , OCERZ_FAULT_INV_RANGE=1	the older invalidation strategies: walk every block, round to 64 KB granules, scan every block's edges, range-invalidate on a first fault
OCERZ_RET_FLAGS_DEAD=1	flags treated as dead at every <code>ret</code> ; <code>ret_flags</code> fails under it
OCERZ_NO_LAZY_REMAP=1 , OCERZ_EAGER_SLIDE =1	in-place rebasing of shared-cache pages; rebasing all of them at startup

The diagnostic knobs, counted

At 9169680 the source reads 410 distinct `OCERZ_*` names through `getenv` , `OCERZ_ENV_ON` or `jit.c`'s own `ENV_ON` . Each is documented where it is read, in that file's explanation block or beside the code. They fall into families by name:

Family	Names	Purpose
OCERZ_NO_*	124	turn one optimisation or behaviour off; 78 of them in <code>jit.c</code>
*LOG	87	print one kind of event as it happens
*TRACE	18	finer-grained tracing of one subsystem
*DUMP	13	dump a structure or state on a trigger
*_LO / *_HI	14	address ranges for bisection
*CHECK / *VERIFY	9	run-time validators of internal invariants
*_EXE	6	select processes by command line
OCERZ_TCACHE*	5	the translation cache
*STAT	5	counters printed periodically or at exit
OCERZ_TSO_*	3	memory-ordering strength

By file, `jit.c` reads 162 of them, `syscall.c` 113, `vm.c` 55, `dyld.c` 25, `dyldapi.c` 23, `mem.c` and `main.c` 12 each. The count grows by a few with most fixes, because the instrument that found a bug usually stays in the tree afterwards, inert when its variable is unset.

APPENDIX B

Exit codes and messages

When a program runs to completion under AArchX, the process exits with the program's own status. When AArchX itself cannot go on, it prints at least one line on standard error beginning `ocerz:` and exits with one of a small set of codes. The first such line is nearly always the one that matters. This appendix lists the codes, then the messages by where they come from, with what each means and what to do about it.

Exit codes

Code	Meaning	Typical first line
guest's	the program ran and exited	none
64	AArchX's own arguments were wrong; <code>OCERZ_MODE</code> held an unknown value; the file is not an Intel program; native mode was given a statically linked program	has no x86_64 slice: it is not an Intel program, so there is nothing to translate
65	the program could not be read or, on the static path, loaded	cannot read <path>
70	static path only: the guest arena or the guest stack could not be set up	cannot build guest stack
71	native mode: imports could not be bound, before the guest ran an instruction	native: no bridge for <symbol> in <library>
72	native mode: a bound export with no crossing was called	bridge: <library> <symbol> not implemented
125	a call AArchX made into guest code, such as an initializer, hit a fatal error outside tolerant mode	initializer call to <addr> aborted after <n> instructions
126	the diagnostic instruction cap <code>OCERZ_ICAP</code> was reached	ICAP hit at <n> instructions
128 + n	the guest sent itself signal <i>n</i> and had no handler for it	guest self-signal <n>, no handler; exiting <128+n>
139	a fatal fault: a guest crash with no guest handler, or a fault inside a bridged native call	guest crash[...] or BRIDGE-FAULT[...]
250-252	the dynamic loader returned an internal error before the program started; the status is the low byte of a negative code: 252 for an I/O failure such as an unmappable shared cache, 251 for a malformed image, 250 for an arena that could not be reserved	cannot map shared cache for dynamic loading

Codes 64, 65 and 70 come from `src/main.c`; 71 from the loader at the end of a native-mode program's fixups; 72 is `OCERZ_BRIDGE_UNIMPL_EXIT` in `include/ocerz/vdylib.h`; 125, 126 and 139 from `src/vm.c`; 128 + *n* from `src/syscall.c`. The 250-252 range is derived from the code rather than documented: `main` returns the loader's status, and the loader's error codes are small negative numbers.

Starting up

unknown OCERZ_MODE value '<v>', want native or cache (64)

The variable is refused rather than ignored, so a typo cannot silently select the wrong mode.

<path> has no x86_64 slice: it is not an Intel program, so there is nothing to translate (64)

An arm64-only binary was given at the top level. A guest that *spawns* such a binary is not refused: the child runs natively, as it would under Rosetta. Many bundled applications are arm64-only on macOS 27.

cannot read <path> (65)

The path is wrong or the file is not a Mach-O AArchX can parse.

native mode cannot run the static image <path> (64)

Native mode stands in for the libraries a program links against, so a statically linked program has nothing for it to do. Run it in cache mode.

cannot find the x86_64 shared cache in <Rosetta cryptex> or <OS cryptex>

Cache mode found no `dyld_shared_cache_x86_64` in either location. This normally means Rosetta is not installed; `softwareupdate --install-rosetta` installs it. Native mode needs no cache. It is followed by `cannot map shared cache for dynamic loading` and exit 252.

Loading and binding

Library not loaded: <path> (no such file), Symbol not found

dyld's own wording, used by AArchX's loader for a dependency that cannot be found or an import nothing exports. In a `dlopen` the load fails cleanly and leaves nothing mapped; at startup it names the image that wanted it. `OCERZ_DLPATH=1` prints every such failure with its reason.

native: no bridge for <symbol> in <library> ... native: N unresolved imports, which no virtual library exports (71)

Native mode could not bind imports: the library has no API database, or its database does not list that export. The message suggests supplying an x86-64 build of the library through `OCERZ_GUEST_ROOT` or `runtime/guest`. Cache mode will usually run the program.

native library without an API database

A native-mode `dlopen` of a host library no database describes; the call returns NULL with this as its `dlerror`.

init-skip: initializer <addr> faulted at rip=<addr> (skipped, process continues)

An initializer run for a `dlopen` faulted. Those run in a tolerant mode, so the process continues; a framework whose initializer was skipped may misbehave later.

initializer call to <addr> aborted after <n> instructions (125)

The same outside tolerant mode, preceded by `initabort-bt` and a backtrace with each frame named by image and offset.

Running

guest crash[<pid>] cpu#N <signal> host_addr=... host_pc=... ocerz_base=... (139)

The guest faulted and had no handler that took the signal, which is what would happen on an x86 Mac too. The full report is described in Chapter 17: guest registers, the last blocks entered, the state of the faulting page and a symbolised backtrace.

fatal: decode failed (<n>, <mode> mode) at rip=<addr> followed by bytes:

The decoder met an instruction it does not know, or a byte sequence that is not an instruction at all. The bytes say which; a wild jump into data looks like the second.

guest breakpoint/interrupt

An `int3` or `ud2` in guest code, almost always an assertion in the program or a library. The report shows where; compare with the program under Rosetta.

fatal on guest thread cpu#N; exiting process <pid> "<command line>"

A guest thread hit a fatal condition and takes the whole process with it. The process id and command line are printed because under Wine a dozen processes share one standard error.

guest self-signal <n>, no handler; exiting <128+n>

The guest raised a signal at itself, typically `abort`, and had no handler. The frame-pointer chain printed before it shows who raised it.

unimplemented BSD syscall num=<n> -> ENOSYS rip=...

A system call `AArchX` does not implement; the guest gets `ENOSYS` and continues. With `OCERZ_STRICT_SYSCALL=1` it stops instead.

unknown Mach trap: class=1 num=<n> name=<name> ...

A Mach trap with no handler and no safe default.

IMAGE-CLOBBER[<pid>] guest <call> addr=... len=... lands on <image>

Always printed, never fatal by itself: a guest mapping landed on a loaded image, or a protection change took read or execute away from one. A library unmapped under running code otherwise surfaces only much later as a wild jump.

WILD-LOOKUP, WILD-STACK, WILD-FAULT-AV, WILD-WORKER-TERMINATE

An indirect branch or fault whose target lies outside anything the guest mapped, with the source instruction and the guest registers. `OCERZ_WILDLOG=1` reports such branches as they happen.

Native mode

bridge: <library> <symbol> not implemented (72)

The program called an export that binds but has no crossing: a stub, because its signature cannot cross correctly (a variadic without a veneer, a `long double`, a union by value) or because nobody has written the crossing yet. An exit 72 often runs the program's own exit handlers on the way out and produces a second, confusing report; the line naming the stub is the real one.

BRIDGE-FAULT[<pid>] <signal> inside a bridged call, not in guest code (139)

A fault inside the Mac's own arm64 code while it ran on a guest thread. The following lines name the call, its signature, the native function and the nesting depth (`call=<library>:<symbol> sig='...' host_fn=... depth=N`), the fault address and guest `rip`, and a `cause:` line saying which side of the guest boundary the faulting address was on: inside guest space means the guest passed a bad pointer; outside means `AArchX` marshalled the call wrongly. The process stops because a native frame can be neither resumed nor unwound. A fault inside one of the eleven in-place string routines is not reported this way: it is delivered to the guest as its own fault at the calling instruction.

APPENDIX C

Glossary

Terms used in this book and in AArchX's source, with the meaning they have there. Where a term names a mechanism, the chapter that explains it is given in parentheses.

ABI engine

`src/abi.c` and `src/abicall.s`: moves arguments and results between the System V x86-64 calling convention and Apple's arm64 one, in both directions, from a function's signature. Native mode's crossings are built on it (Chapter 12).

Acquire and release forms

The arm64 loads and stores with ordering guarantees (`ldapur / stlur` in AArchX's ordered mode) used to give scalar accesses x86's total store order (Chapter 7).

AFP (FEAT_AFP , FPCR.AH)

An arm64 feature whose control bit makes add, subtract, multiply, divide and square root produce x86's NaN results. Where the processor has it, cache mode sets it and drops the software NaN checks.

Alias (of host memory)

In Wine's low shadow, a host region a service or the kernel handed the guest, remapped into the shadow at the same number so guest code can read it. Since 2026-10-03 each alias records the memory object it shows and is refreshed when the host maps new memory there (Chapter 8).

API database

One text file per library under `runtime/apis/macos/<sdk>/`, generated from the builder's own SDK by `make apis`, listing every export native mode synthesizes and how it crosses. Not committed, because it is derived from the SDK (Chapter 12).

Arena

Either the guest arena, the reserved region guest memory lives in, or the code arena, the 1 GB region translated code is emitted into. Context says which.

Arena flush

Since 2026-10-04, when less than an eighth of the code arena (at most 8 MB) is left, the next translation miss retires every block, empties every table that points into the arena and starts translating again at its front, once every thread has left translated code (Chapter 7).

Batch (floating-point batch)

A run of floating-point work whose NaN checks are deferred to its end, with a checkpoint and an undo log so that it can be replayed exactly through the interpreter if a NaN appears.

Block

The unit of translation: guest code from an entry point to the instruction that ends it. A superblock continues past conditional branches.

Bridge, bridged call, crossing

In native mode, the path from an x86 stub to the host's real arm64 function: the arguments are read from the guest's registers by signature, the native function is called, and the result is put back in `rax` or `xmm0`.

Cache mode

The default mode: guest programs bind against Apple's x86-64 dyld shared cache, which AArchX maps and translates like any other code. Needs the Rosetta package installed for the file, but never runs Rosetta.

Callback trampoline bank

A fixed, pre-assembled bank of 65,536 arm64 trampolines. A guest function handed to native code is bound to one slot, and native code calls the slot's address; the same function always gets the same address.

Chaining

Patching a block's exit into a direct branch to its successor's translated body once the successor exists, so control passes without the dispatcher.

Churn reprieve

A page whose translations are invalidated repeatedly is left to the interpreter for a while; since 2026-09-12 it is retranslated after 4,096 refusals even if it never goes quiet, because hot library pages never did.

Commpage

The page the kernel maps at a fixed address for fast user-space reads of values such as the time base. AArchX emulates the x86-64 guest's commpage at `[0x7fffffe00000, 0x7fffffe04000)`.

Cryptex

A signed, separately updatable disk image macOS mounts under `/System/Volumes/Preboot/Cryptexes`. macOS 26 keeps the x86-64 shared cache in the OS cryptex, macOS 27 in a separate Rosetta cryptex; AArchX looks in the Rosetta one first.

Deferred flags

Instead of computing x86's six status flags after every instruction, the JIT records what produced them and materialises only what a later reader needs (Chapter 6).

Differential gate

A test that runs the same input through the interpreter and the JIT and requires identical results. No golden is needed, so it checks every instruction a test happens to execute (Chapter 15).

Dispatcher

The loop that finds or creates the translation for a guest address and enters it. A "trip" is an exit from translated code to the dispatcher.

dyld API table

The object through which libdyld's exported functions reach the dynamic loader, called by byte offset into a table of methods. AArchX installs its own and answers 59 slots, each identified by running code (Chapter 11).

dyld shared cache

A single file in which Apple pre-links every system library at fixed addresses. The x86-64 one is what cache mode maps.

DXMT

A Direct3D 11 implementation over Metal used by MacNdCheese's Wine builds; R.E.P.O. renders through it under AArchX.

D3DMetal

Apple's Direct3D-to-Metal layer from the Game Porting Toolkit, which Counter-Strike 2 uses under Wine.

Explanation block

The prose comment at the top of every AArchX source file, which says what the file owns and why it is shaped as it is. The tree has no inline comments; the blocks are the specification (Chapter 3).

Flag record

The saved description of a deferred flag producer (operation, operands, result) from which any flag can later be computed.

Flip, trace inversion

Retranslating a superblock with a conditional branch the other way round when the profile shows the side exit is the hot path.

Golden

A test's stored expected output, preferably recorded by running the same binary under Rosetta.

gs base, TEB

On macOS the `gs` segment base points at the thread's pthread data; under Wine, Windows code expects it to point at the Thread Environment Block. Wine switches between the two, and AArchX keeps the base as guest CPU state (Chapter 14).

Guest, host

The guest is the x86 program being run and everything it sees; the host is the real arm64 Mac and AArchX itself.

Identity map

The layout used for dynamically linked programs, where guest addresses are host addresses, so a translation needs no address arithmetic. The identity arena is 256 GB, starting at 12 GB.

In-place routines (leaf routines)

Hand-written arm64 versions of `strlen`, `strnlen`, `strcmp`, `strncmp`, `memcmp`, `bcmp`, `strchr`, `memchr`, `memcpy`, `memmove` and `memset` in `src/leaf.s`, called from translated code with the guest's registers left where they are.

Initializer

Code a library runs before `main`: C and C++ constructors, and Objective-C `+load` methods. AArchX runs them in dyld's order, dependencies first and upward dependencies after the whole walk (Chapter 11).

i386 differential corpus

The 20,000 generated 32-bit instruction sequences, plus 33 hand-written ones, that the i386 differential gate runs through both engines and compares in full architectural state (Chapter 15).

Image list

The ordered list of loaded images that `_dyld_image_count` and its siblings walk. A shared-cache image reached by `dlopen` joins it since 2026-09-22; that is what made the GPU visible to translated programs.

Interpreter

AArchX's reference engine, which executes decoded instructions one at a time in C. Everything the JIT declines runs here, and every JIT result is checked against it.

JIT lock

The lock held while translating. It never parks a waiting thread, because a lost wakeup once wedged translation for a whole process.

Lane cache

Inside a translated loop, scalar SSE results and the upper halves of `ymm` registers are kept in spare host vector registers instead of being merged back after every instruction.

Low shadow

The third memory layout, used by Wine processes and other non-PIE programs with low addresses: guest addresses below 12 GB and a strip at the top live at their own host bases, and every memory access is checked to choose the base (Chapter 8).

Low-shadow guard

The check a translated memory access performs in the low shadow. Since 2026-10-04 an address below 12 GB becomes a host address with a shift, a compare and one `orr`.

MacNdCheese

A distribution of Wine with graphics back ends for macOS, whose Wine builds run under AArchX.

KUSER_SHARED_DATA

The Windows page of shared system values at a fixed low address, which Wine maps in every process. Under AArchX its 16 KB host page is shared by the whole Wine prefix, which is why Wine processes reserve their arena at a fixed base.

Mach trap, Mach message

The Mach half of macOS's kernel interface: traps are direct kernel entries, messages carry data and port rights between tasks. Both often carry pointers AArchX must translate.

MIG

The Mach Interface Generator, whose message formats IOKit and many system services use; AArchX translates their in-line and out-of-line pointers.

Mini-dyld

AArchX's own dynamic loader in `src/dyld.c`, which loads, binds and initializes a dynamically linked program without running Apple's dyld (Chapter 11).

Native mode

The mode selected by `-native`: no shared cache is mapped, and the guest's system libraries are synthesized x86 images whose exports cross into the Mac's own arm64 frameworks (Chapter 12).

Non-PIE

A program linked to load at a fixed address rather than anywhere. A non-PIE main with a low base activates the low shadow; the test suite builds guest tests that way to reach Wine's layout.

NZCV forwarding

Emitting an x86 compare, test or arithmetic instruction as a flag-setting arm64 instruction when its consumer is a branch or conditional select that can read arm64's NZCV flags directly.

Ordered mode, plain mode

Plain mode emits ordinary loads and stores; ordered mode uses acquire and release forms for scalar accesses to give x86's ordering. A process starts plain and switches to ordered for good at its first thread, fork or shared writable mapping.

Out-of-line arm (oolslow)

A rarely taken path, such as an unaligned access or a NaN fix-up, emitted after a block's body instead of inline.

Paired delta

The benchmark method: time a kernel at size n and at $n/2$ and report the difference, which removes startup and first translation (Chapter 16).

Pinning, pin class

Keeping guest registers permanently in fixed host registers. In pin class 3, guest `rax - rdi` live in `x21 - x28`, `r8 - r13` in `x3 - x8`, `r14 - r15` in `x1 - x2`, and `xmm0 - xmm15` in `v16 - v31` (Chapter 6).

PE

Portable Executable, the Windows binary format. Under Wine, "PE code" is Windows code as opposed to Wine's own Unix-side code.

RAS (return-address stack)

A stack of guest return addresses and their translated continuations, kept on the host stack so a guest `ret` becomes a real `ret` and the processor's return predictor stays accurate.

Retirement

Marking a translated block dead when the guest code it came from changed or its translation must be redone, and unchaining branches into it.

Rosetta 2

Apple's x86-64 translator for Apple silicon, the baseline every AArchX measurement and most goldens are taken against, and the source of the x86-64 shared cache package.

Selector

Either an x86 segment selector, which WoW64 uses to switch between 32- and 64-bit code and which AArchX keeps as CPU state, or an Objective-C method selector. Context says which.

Side exit

The out-of-line stub through which a superblock leaves on its rarely taken conditional branch.

Slot (4 KB slot)

Guest pages are 4 KB and host pages 16 KB, so AArchX tracks protection and ownership per 4 KB slot inside each host page and asks the kernel for the union.

SMC, write-arming

Self-modifying code. Pages that hold translated code are write-protected (armed); a guest write faults, and the blocks overlapping the written range are retired.

Splicing

Translating a small called function into its caller's block, with matched pushes and pops renamed into registers.

Static arena

The layout for statically linked programs: a fixed guest region `[0x100000000, 0x900000000)` at a host base.

Stub, trap window

Each export of a synthesized library is twelve bytes of x86: load the export's id into `r11d`, jump to an address inside `[0xdda00000, 0xdda10000)`. Jumping there traps into AArchX, which performs the call.

Superblock

A translated block that continues past forward conditional branches along the likely path, with the other path in side exits.

Synthesized image (vdylib)

An x86-64 Mach-O built in memory by `src/vdylib.c` for a system library that has no file in native mode: real headers, an export trie and a stub per export.

Thread-specific data (TSD)

macOS's per-thread slots reached through the `gs` base. AArchX maps a host thread's TSD back to the guest thread it belongs to when answering thread queries.

Tolerant initializer mode

Initializers run for a `dlopen` may fault without ending the process: the faulting initializer is skipped with an `init-skip` message.

Translation cache (tcache)

Translated blocks stored on disk under `~/Library/Caches/ocerz` and loaded by later processes, on by default since 2026-09-27 (Chapter 7).

TSO

Total store order, x86's memory model, stronger than arm64's.

Two-level namespace

Mach-O's rule that an import names the library it comes from, so two libraries can export the same name without conflict.

Unstick monitor

A thread that interrupts guest threads parked for more than 800 ms in waits that tolerate a spurious `EINTR`, so a lost wakeup does not hang a process forever.

Upward dependency

A library link marked upward, the back edge of a dependency cycle. It orders nothing, but the target is still loaded and initialized, after the whole initializer walk.

Veneer

A small stub in one of the 256 KB pools placed every 64 MB in the code arena, through which a chain reaches a target more than 128 MB away.

Workqueue bridge

Forwarding libdispatch's kernel workqueue to the host and running each work item on a guest personality, so Cocoa applications do not deadlock. On by default since 2026-09-10.

WoW64

Windows-on-Windows 64, Wine's way of running 32-bit Windows code in a 64-bit process, switching between 32- and 64-bit code segments (Chapter 14).

xbench

The fifteen-kernel benchmark in `tests/guest`, in a static and a dynamically linked build, compared against Rosetta by `tests/xbench_compare.py`.

xlive

Cross-block flag liveness: an exit's flags are dead when the successor, decoded up to three blocks deep, overwrites them before reading them.

x86-64-v3

The x86 feature level with AVX2, FMA, BMI1/BMI2, F16C, LZCNT and MOVBE, which Rosetta runs since macOS 15 and AArchX implements.

APPENDIX D

Timeline

Four months from an initial commit to a translator that runs Steam, Wine and a Unity game. The milestones below are taken from the git history and the project's notes; each hash is a commit that can be read for the full story. The repository's history was rewritten once, between 2026-09-19 and 2026-10-02, and every hash in this book belongs to the rewritten history.

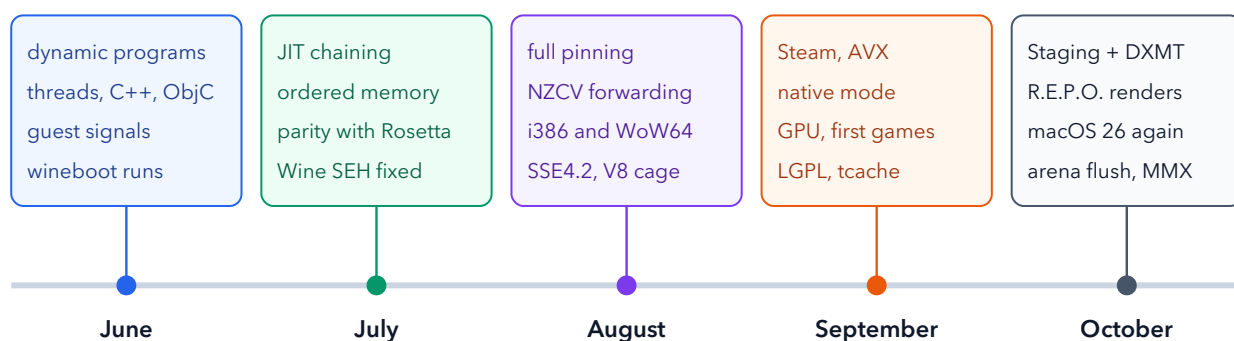


Figure D.1. The four months at a glance. September carried the largest share of the work: the macOS Steam client, AVX, native mode, the GPU and the first games all arrived in it.

June: from static binaries to Wine

Date	Commit	Milestone
2026-06-03	d4d3cae	Initial commit; LGPL-2.1 license file the same day.
2026-06-05	4f4d973	Real Objective-C programs run end to end against the x86-64 shared cache.
2026-06-06	fb0801a, 87b28df	Real guest threads, Swift metadata and the C++ standard library; C++ exceptions through the dyld unwind-sections slot.
2026-06-08	5b8072b	@rpath expansion and thread-local variables: Homebrew binaries and C++ thread_local work.
2026-06-11	741bdf0	Guest signals, and the start of Mach thread semantics.
2026-06-13	3169ede	Wine's wineboot runs end to end into the GUI layer.
2026-06-14	f040e12	Kernel-allocated Mach messages bridged to the guest's libdispatch: Notepad builds its GUI.

July: a JIT that keeps up

Date	Commit	Milestone
2026-07-16	c9ad4bf	x86-TSO memory ordering.

Date	Commit	Milestone
2026-07-17	378b4a9	JIT: lazy flags, call and return inlining, chaining infrastructure.
2026-07-18	ca84db3 , 3caf38f	Block chaining and two-way conditional-branch linking.
2026-07-23	1527008	The plain-memory tier: rough parity with Rosetta for the first time.
2026-07-25	11a4e4f	Ordered memory 37% faster with <code>ldapr</code> and relocated unaligned arms.
2026-07-26	a9460e8	<code>FXSAVE / FXRSTOR</code> save the XMM registers: <code>wineboot</code> stops crashing.
2026-07-30	37410f6 , 7f66fde	Host memory regions aliased into Wine's low shadow; <code>dlopen_preflight</code> implemented for Metal's device discovery.
2026-07-31	78dee57	The Windows exception blocker resolved: <code>vm_region</code> replies describe the guest's memory, not the host's.

August: the pinned JIT and 32-bit code

Date	Commit	Milestone
2026-08-15	4a1fe83	The JIT rewrite begins: all sixteen <code>xmm</code> registers pinned in every block, value-based conditions, in-place SSE on pinned registers.
2026-08-16	ea79f8a , b8f4390 , 82efd17	NZCV forwarding; plain memory until a second observer appears, so dynamic programs run at static speed; shared-cache pages rebased lazily per page.
2026-08-17	2ac3ae0 , eb4b26d	Cheap ordered memory with <code>ldapur / stlur</code> ; superblocks past backward branches.
2026-08-20	c7bfa4c	i386: WoW64 host support, 32-bit JIT code generation and the i386 differential gate.
2026-08-25	218918d , ba913bf , d0d0349	SSE4.2 implemented and advertised (Steam checks for it); <code>rsp</code> held as a host pointer in pin class 3; Chromium's V8 cage lands.
2026-08-26	7efddef	<code>os_unfair_lock</code> made robust against owner death: a Steam crash fixed.

September: Steam, AVX, native mode and games

Date	Commit	Milestone
2026-09-04	0dd72c5 , abb3a13	The project is called AArchX; <code>ocerz version</code> says AArchX 0.1.
2026-09-05	0c03488 , c23de6f , 65c6e74	Flags survive a return after a compare: Wine windows stop painting black. Notepad stays up. 32-bit exceptions reach the 32-bit dispatcher, ending Steam's restart storm.
2026-09-05	800bb00 , bbaca41	Vector accesses left plain in ordered mode; the license changed to a proprietary text.
2026-09-06	264e647 , a9f1c20	The SQLite miscompile fixed; misaligned locked atomics under an address-striped lock.
2026-09-07	cbff144	Plain guest stores into translated code are trapped: self-modifying code handled.

Date	Commit	Milestone
2026-09-08	<code>0f19d4c</code> , <code>7ece4f2</code>	<code>punpckhqdq</code> decoded correctly, ending the 74-pixel windows; VEX decoded and interpreted. Windows Steam's login window renders with software compositing.
2026-09-10	<code>d66221e</code>	Initializers run in dyld's order.
2026-09-11	<code>f4e4cc0</code> , <code>05d42d6</code>	Chess plays and eight more system applications open their windows; System V IPC lets Steam's bootstrapper run.
2026-09-12	<code>47d311c</code> , <code>ee8cbd1</code> , <code>ee581ec</code>	One explanation block per file, no inline comments; Steam's <code>ipcserver</code> runs translated; the macOS Steam client comes up with its full UI.
2026-09-13	<code>e4bfd1c</code> , <code>62edd5c</code> , <code>358a003</code>	AVX2 and FMA translated; the 256 GB guest arena; Ollama's command line works.
2026-09-15	<code>f5919cc</code>	One NaN batch per run of floating-point work.
2026-09-16	<code>d79e2a1</code>	Native mode: a process that maps no shared cache.
2026-09-19	<code>231fae0</code>	macOS 27 fixes; the shared cache is looked for in the Rosetta cryptex.
2026-09-21	<code>0f810e8</code> , <code>e860528</code>	Native mode runs Steam; the in-place string routines and hardware NaNs; Safari works.
2026-09-22	<code>69590ca</code> , <code>5681be5</code> , <code>815cbd9</code>	Translated programs see the GPU; chains through veneers; <code>shmat</code> . Brawlhalla reaches its menus.
2026-09-23	<code>ca02b38</code> , <code>f430371</code> , <code>d8b9325</code>	LGPL-2.1-or-later again, with the documentation in the tree; games launched from Steam run under AArchX; every dyld API slot identified by running code.
2026-09-26	<code>dff4f4a</code> , <code>c86567d</code> , <code>d82e1e7</code>	Wine processes reserve their arena at a fixed base; Counter-Strike 2 loads D3DMetal; OpenGL works from Wine threads.
2026-09-27	<code>812a5fb</code> , <code>20465b4</code> , <code>71b3d28</code>	The sampling profiler across a process tree; translations kept on disk, on by default.

October: games under Wine

Date	Commit	Milestone
2026-10-01	<code>654a81a</code> , <code>3ca606a</code>	Steam on stock Wine Staging 11.16 with DXMT; dlopened frameworks reach their XPC services.
2026-10-03	<code>0115ead</code> , <code>28e237b</code> , <code>3cc5842</code>	Host-memory aliases refreshed, IOKit records aliased, the low 12 GB kept for the guest: R.E.PO. renders under Wine through DXMT.
2026-10-03	<code>33bb4ae</code> , <code>f467f63</code>	Guest tests restated in x86 terms; the README cut to a front door with the long material in <code>docs/</code> .
2026-10-03	<code>9198b58</code> , <code>e705796</code>	Cache mode on macOS 26 and 27; upward dependencies initialized after the whole walk, fixing <code>sw_vers</code> on macOS 26.7.1.
2026-10-04	<code>9226846</code> , <code>16a28b1</code>	The code arena flushes instead of filling; a Wine address below 12 GB becomes a host address with one <code>orr</code> . R.E.PO.'s menu goes from 28-30 frames a second to roughly 45-65 (single runs reached 65).

Date	Commit	Milestone
2026-10-04	74e4300, ef59a1d	MMX translated; 0CERZ_TSO_NARROW for narrow SSE ordering.